

Sistemas Embebidos en FPGA

RICARDO CAYSSIALS

Apoyo en la



```
a = r;
b = m;
p = 0;
low_m = 0;
for (i = 0, i < n, i++) begin
    case {b(1) low_m}
        "01":
            p = p + a
        "10":
            p = p - a
    endcase
    p >> 1
    b >> 1
end
result = p;
```



Alfaomega

Sistemas Embebidos en FPGA

Ricardo Cayssials

Sistemas Embebidos en FPGA

Ricardo Cayssials



Buenos Aires • Bogotá • México DF • Santiago de Chile

Cayssials, Ricardo
Sistemas Embebidos en FPGA. - 1a ed. - Ciudad Autónoma de Buenos Aires : Alfaomega
Grupo Editor Argentino, 2014.
340 p. ; 17x23 cm.
ISBN 978-987-1609-69-7
1. Informática. 2. Electrónica. I. Título
CDD 005.3

Queda prohibida la reproducción total o parcial de esta obra, su tratamiento informático y/o la transmisión por cualquier otra forma o medio sin autorización escrita de Alfaomega Grupo Editor Argentino S. A.

Edición: Damián Fernández

Diseño y armado de interiores: Ricardo Cayssials

Corrección: Vanesa García

Diseño de tapa: Iris Biaggini

Revisión de armado: Vanesa García

Internet: <http://www.alfaomega.com.mx>

Todos los derechos reservados © 2014, por Alfaomega Grupo Editor Argentino S. A.

Paraguay 1307, PB, oficina 11

ISBN -987-1609-69-7

Queda hecho el depósito que prevé la ley 11.723

NOTA IMPORTANTE: La información contenida en esta obra tiene un fin exclusivamente didáctico y, por lo tanto, no está previsto su aprovechamiento a nivel profesional o industrial. Las indicaciones técnicas y programas incluidos han sido elaborados con gran cuidado por el autor y reproducidos bajo estrictas normas de control. Alfaomega Grupo Editor Argentino S. A. no será jurídicamente responsable por: errores u omisiones; daños y perjuicios que se pudieran atribuir al uso de la información comprendida en este libro, ni por la utilización indebida que pudiera dársele.

Los nombres comerciales que aparecen en este libro son marcas registradas de sus propietarios y se mencionan únicamente con fines didácticos, por lo que Alfaomega Grupo Editor Argentino S. A., no asume ninguna responsabilidad por el uso que se dé a esta información, ya que no infringe ningún derecho de registro de marca. Los datos de los ejemplos y pantallas son ficticios, a no ser que se especifique lo contrario.

Los hipervínculos a los que se hace referencia no necesariamente son administrados por la editorial, por lo que no somos responsables de sus contenidos o de su disponibilidad en línea.

Empresas del grupo:

Argentina: Alfaomega Grupo Editor Argentino, S. A.

Paraguay 1307 P.B. "11", Buenos Aires, Argentina, C.P. 1057

Tel.: (54-11) 4811-7183/0887

E-mail: ventas@alfaomegaeditor.com.ar

México: Alfaomega Grupo Editor, S. A. de C.V.

Pitágoras 1139, Col. Del Valle, México, D.F., México, C.P. 03100

Tel.: (52-55) 5575-5022 - Fax: (52-55) 5575-2420/2490. Sin costo: 01-800-020-4396

E-mail: atencionalcliente@alfaomega.com.mx

Colombia: Alfaomega Colombiana S. A.

Calle 62 N° 20-46, Bogotá, Colombia

Tel. (57-1) 7460102 - Fax: (57-1) 2100415

E-mail: cliente@alfaomega.com.co

Chile: Alfaomega Grupo Editor, S. A.

Av. Providencia 1443, Oficina 24, Santiago de Chile, Chile

Tel.: (56-2) 235-4248/2947-5786 - Fax: (56-2) 235-5786

E-mail: agechile@alfaomega.cl

Mensaje del editor

Los conocimientos son esenciales en el desempeño profesional, sin ellos es imposible lograr las habilidades para competir laboralmente. La universidad o las instituciones de formación para el trabajo ofrecen la oportunidad de adquirir conocimientos que serán aprovechados más adelante en beneficio propio y de la sociedad; el avance de la ciencia y de la técnica hace necesario actualizar continuamente esos conocimientos. Cuando se toma la decisión de embarcarse en una vida profesional, se adquiere un compromiso de por vida: mantenerse al día en los conocimientos del área u oficio que se ha decidido desempeñar.

Alfaomega tiene por misión ofrecerles a estudiantes y profesionales conocimientos actualizados dentro de lineamientos pedagógicos que faciliten su utilización y permitan desarrollar las competencias requeridas por una profesión determinada. Alfaomega espera ser su compañera profesional en este viaje de por vida por el mundo del conocimiento.

Alfaomega hace uso de los medios impresos tradicionales en combinación con las tecnologías de la información y las comunicaciones (TIC) para facilitar el aprendizaje.

Libros como éste tienen su complemento en una página Web, en donde el alumno y su profesor encontrarán materiales adicionales.

Esta obra contiene numerosos gráficos, cuadros y otros recursos para despertar el interés del estudiante, y facilitarle la comprensión y apropiación del conocimiento. Cada capítulo se desarrolla con argumentos presentados en forma sencilla y estructurada claramente hacia los objetivos y metas propuestas.

Los libros de Alfaomega están diseñados para ser utilizados dentro de los procesos de enseñanza-aprendizaje, y pueden ser usados como textos para diversos cursos o como apoyo para reforzar el desarrollo profesional.

Alfaomega espera contribuir así a la formación y el desarrollo de profesionales exitosos para beneficio de la sociedad.

Acerca del autor

Ricardo Cayssials

Ricardo Cayssials es Ingeniero Electrónico, Doctor en Ingeniería y Profesor Titular. También es responsable de las cátedras que cubren “Diseño de Hardware Digital”, “Lenguajes de Descripción de Hardware y dispositivos FPGA” en la Universidad Nacional del Sur y en la Facultad Regional Bahía Blanca de la Universidad Tecnológica Nacional. Fue Profesor Visitante de la Universidad Federal Fluminense en Brasil e Investigador Visitante de la Universidad de York en Inglaterra.

Es responsable y Director de la Especialización de Posgrado Profesional en Tecnologías Digitales Configurables de la Universidad Nacional del Sur y Director de Proyectos de Investigación y Desarrollo en sistemas embebidos utilizando dispositivos FPGA.

Introducción

El desarrollo de sistemas embebidos modernos demanda la implementación de funciones sofisticadas en plazos de diseño cortos. Actualmente, es muy frecuente que la implementación de un sistema embebido, además de realizar las funciones específicas deseadas, requiera, por ejemplo, comunicación con dispositivos externos cableados e inalámbricos, dispositivos de visualización y de almacenamiento masivo, compatibilidad con diferentes formatos de datos, entre otros. Es por esta razón que la flexibilidad en el diseño e implementación debe ser considerada al momento de emprender el desarrollo de un sistema embebido.

Los dispositivos FPGA permiten la implementación de todo el hardware de un sistema digital en un circuito integrado configurable, permitiendo desarrollos conocidos como *Sistemas-en-Chip-Programable* (SOPC, *System-on-Programmable-Chip*, en inglés). La especificación de todo el sistema se realiza en forma flexible debido a que su diseño e implementación involucran herramientas de desarrollo que permiten un elevado nivel de abstracción.

Por otro lado, es necesario contar con un soporte de software que nos permita disponer de todas las funcionalidades que requieren los desarrollos modernos. Generalmente, este soporte está considerado y desarrollado en diversos sistemas operativos. Existen algunos que permiten ser ejecutados en diferentes plataformas de hardware y otros desarrollados para sistemas embebidos específicos. Sin embargo, la mayoría de ellos satisfacen el estándar POSIX lo que permite relativamente una migración sencilla de aplicaciones de software a diferentes plataformas de hardware.

Los fabricantes de dispositivos FPGA brindan diversas herramientas para la realización de SOPC y plataformas de desarrollos (*kits*) que incrementen la eficiencia y flexibilidad de diseño que utilicen sus dispositivos.

En este libro, se introducen los conceptos y metodologías necesarios para la realización de Sistemas Embebidos en dispositivos FPGA. Las metodologías de diseño, implementación y desarrollo estarán realizadas utilizando las plataformas y dispositivos de Altera, pero los fundamentos conceptuales permitirán su utilización en plataformas y dispositivos de otros fabricantes.

Dr. Ing. Ricardo Cayssials

Antes de comenzar a leer

En este libro, se utiliza la tipografía `courier new` en los casos en que se hace referencia a código o acciones que se han de realizar en la computadora, ya sea en un ejemplo o cuando se refiere a alguna función mencionada en el texto. También se utiliza para indicar teclas o direcciones URL.

El texto **resaltado** es utilizado para indicar acciones que deben ejecutarse mediante la selección de comandos de la barra de menús o para indicar el nombre de componentes.

Los términos o definiciones cuyos significados están muy asociados al inglés se expresan en dicho idioma en *cursiva*. Por ejemplo, el archivo *netlist* para indicar el archivo que posee la descripción del sistema como interconexión de componentes.

Podrá acceder al código fuente y a los proyectos que se desarrollan en esta obra desde la página Web: <http://libroweb.alfaomega.com.mx>

TABLA DE CONTENIDOS

CAPÍTULO 1 Diseño de sistemas embebidos	1
Introducción	1
Sistemas embebidos en lógica programable	2
Desafíos del diseño digital	3
Modelo de rebalse (Waterfall Model)	3
Modelo de prototipo (Prototype Model)	5
Métricas de diseño	7
Velocidad	7
Energía	8
Tamaño	8
Costo	9
Otras métricas de diseño	9
Conclusiones	10
CAPÍTULO 2 Lógica programable y lenguajes de descripción de hardware	11
Circuitos digitales con lógica programable	11
Circuito combinacional	11
Circuito secuencial	12
Dispositivos lógicos programables: CPLD y FPGA	14
Recursos adicionales en dispositivos FPGA	17
Lenguajes de descripción de hardware	18
Niveles de abstracción en lenguajes de descripción de hardware	20
Síntesis y simulación de archivos de descripción de hardware	21
Verilog HDL	22
Estructura module	23
Sección <i>port list</i>	23
Sección Declaraciones de <i>ports</i>	24
Sección Declaración de tipos de datos	25
Descripción de la funcionalidad del circuito	25
Instanciación de module	26
Conexión por orden y por nombre	26
Representación de valores en Verilog HDL	27
Operadores en Verilog HDL	27
Circuitos combinacionales y secuenciales en Verilog HDL	28
Bloque <i>always</i>	29
Sentencia <i>assign</i>	29
Estructuras de control	30
Sentencia <i>if-else</i>	30
Sentencia <i>case</i>	30
Sentencia <i>for</i>	31
Ejemplo	31
Multiplicador de Booth	31
Conclusiones	36

CAPÍTULO 3 Herramientas de diseño de hardware 37

Herramientas de diseño de hardware	37
Flujo de diseño con dispositivos lógicos configurables	38
Plataformas y licencias de Quartus II	40
Proyectos en Quartus II	40
Interfaz gráfica de Quartus II	41
Creación de proyecto	43
Creación del archivo de diseño	49
Entrada del diseño (<i>Design Entry</i>)	50
Compilación del diseño	51
Etapa fitter (place & route)	52
Configuración	53
Asignación de pines (Pin Planner) 	53
Programador del dispositivo “  ”	54
Simulación del diseño	55
Simulación de diseños digitales con lenguajes de descripción de hardware: <i>testbench</i>	56
Simulación en el flujo de diseño de Quartus II	57
Flujos de simulación en Quartus II	59
Configuración de la simulación en Quartus II (opción NativeLink)	59
Inicialización de ModelSim a Nivel RTL (opción NativeLink)	62
Inicialización de ModelSim a Nivel <i>Gate</i> (opción NativeLink)	62
Simulación con ModelSim	63
Configuración de los estímulos de la simulación	65
Conclusiones	67

CAPÍTULO 4 Soft-processors en dispositivos FPGA 69

Introducción a soft-processors	69
Arquitectura del procesador Nios II	70
Características del procesador Nios II	70
Versiones de Nios II	73
Instrucciones personalizables (<i>Custom Instructions</i>)	74
Síntesis de software a hardware	75
Licencia de Nios II	75
Recomendaciones para diseños con Nios II	76
Concepto de configuración de soft-processors	76
Flexibilidad de periféricos y mapa de direcciones	77
Bus Avalon	77
Características del <i>bus</i> Avalon	77
Interfaz Avalon Memory-Mapped	78
Transferencias en Avalon-MM	80
Típica transferencia de lectura y escritura	80
Transferencias de lectura y escritura con wait-states fijos	80
Transferencia con pipeline	80
Transferencias <i>burst</i>	80
Interfaz Avalon Interrupt	80
Generador de interrupción	81
Receptor de interrupción	81
Interfaz Avalon Streaming	81
Características de la interfaz Avalon Streaming	81

Interfaz Avalon Conduit	81
Interfaz Avalon Tri-state Conduit	81
Componentes de biblioteca	82
Conclusiones.....	82

CAPÍTULO 5 Diseño de sistemas embebidos en lógica programable..... 83

Sistemas-en-chip-programable (SOPC)	83
Flujo de Diseño de Sistemas en Chips Programables	83
Qsys y SOPC Builder	84
Directorio de proyecto en Qsys	85
Diseño de SOPC en Qsys	86
Realización del flujo de diseño de un SOPC	88
Creación del proyecto de máxima jerarquía	88
Creación del diseño del SOPC	89
Configuración del SOPC	89
Interfaz gráfica de Qsys.....	89
Definición de la fuente de reloj externa	90
Configuración de los componentes.....	91
Selección y configuración del procesador Nios II	92
Selección y configuración de la memoria RAM interna del dispositivo FPGA	94
Selección y configuración del temporizador del sistema (Interval Timer)	97
Selección y configuración de un puerto de entrada/salida (PIO Parallel IO)	101
Selección y configuración de un puerto de comunicaciones JTAG (JTAG UART)	103
Configuración del SOPC	106
Asignación automática de direcciones de memoria	107
Asignación automática de interrupciones	107
Configuración de los vectores de <i>reset</i> y <i>exception</i> del procesador Nios II del sistema.....	108
Configuración de las entradas y salidas del SOPC	109
Generación del sistema en Qsys.....	110
Solapa del generador del sistema (Generation).....	110
Instanciación del SOPC en Quartus II	114
Agregado de lógica adicional y asignación de pines.....	116
Agregado de lógica adicional	116
Asignación de nombres a los puertos	117
Conexión de la lógica externa	118
Asignación de pines	119
Conclusiones.....	119

CAPÍTULO 6 Generación del software..... 121

Generación de software para Nios II	121
Capa de abstracción de hardware (HAL) y Sistemas Operativos (OS)	122
Capa de abstracción de hardware (HAL)	123
Sistema operativo (OS)	124
Estructura monolítica	125
Estructura micrónúcleo	126
Estructura híbrida	126
Sistemas operativos para SOPC	127
Estructura de la HAL de Altera	128
Modelo genérico de los controladores de dispositivos de la HAL de Altera	129

Desarrollo de software embebido para el procesador Nios II de Altera	131
Ambiente de desarrollo de software para el procesador Nios II	131
Flujo de diseño con Nios II EDS	131
Makefile y Nios II SBT	132
Creación de un proyecto de software en Nios II SBT	133
Ejecución de Eclipse para Nios II SBT y la inicialización del espacio de trabajo	133
Estableciendo el espacio de trabajo del proyecto (Workbench)	134
Creación del proyecto en Nios II SBT	135
Programación del código de la aplicación	138
Configuración del proyecto BSP: editor de BSP (BSP Editor)	139
Solapa Main	141
Categoría Common	141
Categoría Advanced	142
Solapa Software Packages	142
Solapa Drivers	143
Solapa Linker	143
Solapa Enable File Generation	145
Solapa Target BSP Directory	145
Propiedades del proyecto BSP	145
Configuración del proyecto de aplicación	146
Construcción de los proyectos	148
Conclusiones	149
CAPÍTULO 7 Implementación del SOPC	151
Introducción	151
Implementación del hardware y software	152
Ejecución del software desde memoria interna del dispositivo FPGA	152
Creación de los archivos de inicialización de memoria interna	153
Inclusión automática de los archivos de inicialización de memoria al proyecto Quartus II	153
Inclusión manual de los archivos de inicialización de memoria al proyecto Quartus II	154
Ventajas y desventajas de la utilización de memoria interna del dispositivo FPGA	155
Ejecución del software desde la memoria <i>flash</i> de configuración del dispositivo FPGA	156
Aplicación <i>boot loading</i>	156
Configuración por defecto de boot loading en la HAL de Altera	157
Configuración de opciones de la aplicación boot loading	157
Inclusión de memoria <i>flash</i> en el SOPC	158
Selección y configuración de memoria <i>flash</i> en SOPC	158
Programación de la <i>flash</i> del sistema	160
Creación del archivo de configuración de <i>Flash Programmer</i>	161
Especificación de la configuración de <i>Flash Programmer</i>	163
Opciones de <i>Flash Programmer</i>	163
Ejecución del software desde memoria externa de almacenamiento masivo	164
Características de las memorias SD	164
Modo de funcionamiento de las memorias SD	165
Selección y configuración de memoria SD en SOPC	165
Conclusiones	165

CAPÍTULO 8 Verificación del funcionamiento de sistemas embebidos en dispositivos FPGA..... 167

Introducción	167
Simulación de SOPC.....	168
Generación de un modelo de simulación en Qsys	168
Depuración en placa de desarrollo de SOPC	173
Nios II Software Build Tools for Eclipse	173
Componente Nios II System ID	174
Generación del archivo OBJdump.....	174
Consola de Nios II y funciones de la biblioteca stdio	177
Desbordamiento de la pila.....	177
Puntos de quiebre (Breakpoints) y ejecución paso-a-paso	177
In-System Memory Content Editor.....	178
SignalTap II Logic Analyzer.....	178
Conclusiones.....	179

CAPÍTULO 9 El lenguaje de comandos Tcl 181

Introducción	181
El lenguaje de comandos Tcl	181
Sintaxis básica del lenguaje Tcl	182
Intérprete Tcl	182
Creación de archivos de comandos Tcl	183
Ejecución de archivos de comandos Tcl	183
Creación y sustitución de variables en Tcl.....	184
Creación de variables.....	184
Sustitución \$	184
Operaciones aritméticas con variables: comando <code>expr</code>	185
Sustitución anidada []	185
Sustitución backslash.....	185
Evaluación de cadenas de caracteres: comando <code>eval</code>	185
Listas	186
<code>llength</code>	186
<code>lindex</code>	186
<code>lsearch</code>	186
<code>lappend</code>	187
<code>foreach</code>	187
Estructuras de control en Tcl	187
Comando <code>if/then/else/elseif</code>	188
Comando <code>switch</code>	188
Comando <code>for</code>	189
Comando <code>foreach</code>	190
Comando <code>while</code>	190
Comando <code>break</code> y <code>continue</code>	191
Procedimientos.....	191
Alcance de las variables en procedimientos (Scope)	192
Comandos de entrada/salida	192
Comando <code>open</code>	192
Comando <code>close</code>	193
Comando <code>puts</code>	193

Comando gets.....	194
Comentario.....	194
Ejecución de archivos Tcl por línea de comando.....	195

CAPÍTULO 10 Lenguaje de comandos Tcl en herramientas de diseño: Quartus II

..... 197

Lenguaje de comandos Tcl en herramientas de diseño	197
Soporte de archivos de comandos Tcl en Quartus II	198
Paquetes Tcl (<i>Tcl Packages</i>)	198
Ejecutables de línea de comandos de Quartus II	200
Utilidad para ayuda de paquetes y comandos Tcl en Quartus II	202
Creación y manipulación de proyectos en Quartus II con comandos Tcl.....	203
Paquete : : quartus : : project	204
Comando project_exist	206
Comando project_new.....	206
Comando project_open	207
Comando project_close	208
Comando create_revision.....	209
Comando export_assignments	209
Creación de proyectos en Quartus II mediante comandos Tcl.....	210
Ejemplo: creación de proyecto en Quartus II mediante comandos Tcl.....	211
Asignación de pines	214
Compilación de proyectos en Quartus II con comandos Tcl.....	214
Paquete : : quartus : : flow	214
Comando execute_flow	215
Compilación de Proyectos en Quartus II mediante comandos Tcl	216
Ejemplo: compilación de proyecto en Quartus II mediante comandos Tcl	216
Conclusiones.....	217

CAPÍTULO 11 Utilidades y archivos de comandos para la generación de SOPC

..... 219

Introducción	219
Utilidades en Qsys y Nios II SBT.....	219
Utilidades y archivos de comandos en Qsys.....	220
Creación y manipulación de SOPC con qsys-script	220
Ejemplo de arquitectura de SOPC en Qsys.....	222
Generación del SOPC en Qsys con ip-generate.....	223
Generación de los archivos de simulación del SOPC con ip-make-simscript	225
Utilidades y archivos de comandos en Nios II SBT	225
Archivos makefile en Nios II SBT	226
Archivos de comandos y utilidades de línea de comandos de Nios II SBT	226
Comandos Tcl para la configuración del proyecto BSP	228
Generación del proyecto BSP y comandos Tcl “Callbacks”	228
Herramientas de línea de comando GNU para Nios II (Consola “bash”)	229
Utilidades GNU para Nios II de Altera	230
nios2-elf-gcc y nios2-elf-g++	230
nios2-elf-objdump	231
Creación de archivos de inicialización de memoria.....	231
Conclusiones.....	231

Referencias	233
Anexo 1 Instalación de Linux CentOS-5.....	235
Anexo 2 Instalación de Quartus II Web Edition versión 13	253
Pasos para la instalación	253
Configuración del menú de aplicaciones en CentOS	260
Anexo 3 Flujo de diseño con utilidades de línea de comandos	265
Introducción	265
Flujo de diseño del proyecto en Quartus II mediante línea de Comandos	266
Flujo de diseño de proyecto SOPC en Qsys mediante línea de comandos.....	267
Flujo de diseño de proyecto BSP mediante línea de comandos.....	268
Flujo de diseño de proyecto de aplicación mediante línea de comandos.....	269
Anexo 4 Comandos Tcl para la utilidad “qsys-script”	271
Ejemplo.....	272
Comandos Tcl de qsys-script.....	272
Anexo 5 Comandos Tcl para la configuración de proyectos BSP	283
Anexo 6 Referencia de configuraciones de BSP, paquetes de software y controladores.....	291
Anexo 7 Funciones y estructuras de la HAL de Altera.....	301
Funciones de la API de la HAL de Altera.....	301
Estructuras estándar de la HAL de Altera.....	310
Anexo 8 Biblioteca de código abierto newlib	313
Introducción	313
Tipos de datos en C.....	313
Funciones de utilidades estándar (stdlib.h)	314
Macros y funciones para caracteres (ctype.h)	315
Funciones de entrada/salida (stdio.h).....	316
Manipulación de memoria y string (string.h).....	318
String del tipo <i>wide character</i> (wchar.h).....	319
Manejo de señales: signal handling (signal.h).....	319
Funciones de tiempo (time.h).....	320
Configuración Local (locale.h).....	321
Reentrancia.....	321
Llamadas al sistema (<i>System Calls</i>).....	321
Definiciones para la interfaz con el sistema operativo (<i>stubs</i>)	321

CAPÍTULO 1

DISEÑO DE SISTEMAS EMBEBIDOS

INTRODUCCIÓN

Existen diferentes definiciones de *sistemas embebidos* (*Embedded Systems*, en inglés), pero, en general, se define en términos de lo que pueden o no pueden realizar o de las aplicaciones en las cuales pueden ser utilizados.

“Un sistema embebido es un sistema basado en microprocesador que es construido para controlar una o varias funciones y que no es diseñado para que sea programado por el usuario final en la misma manera que, por ejemplo, lo es una Computadora Personal ((1))”.

Generalmente, el usuario puede realizar cambios en la configuración de la funcionalidad del sistema embebido, pero no lo puede efectuar mediante el cambio o remplazo del software del mismo. Tal tarea de programar y realizar el software del sistema embebido está a cargo del diseñador/programador quién evaluará las diferentes alternativas de configuración que estarán disponibles para el usuario final.

Un sistema embebido se contrapone en funcionalidad con una Computadora Personal, en la cual el usuario final puede seleccionar el software que desea ejecutar para modificar la funcionalidad de la misma. De esta manera, una Computadora Personal puede utilizarse como, por ejemplo, un procesador de texto, una base de datos, una consola de videojuegos, etc. Por supuesto que un diseñador podría configurar una Computadora Personal para una determinada aplicación y ejecutar un software específico para tal tarea, dejando al usuario final las opciones de solo configurar tal aplicación. En este último caso, se trataría, por su aplicación, de un sistema embebido por más que se utilice una Computadora Personal para su implementación.

Por lo visto, un sistema embebido generalmente queda definido por las aplicaciones a la que está destinado y, por consiguiente, define metodologías de diseño e implementación específicas.

Actualmente, las áreas de aplicación de sistemas basados en microprocesadores son de las más variadas y, en consecuencia, las de utilización de sistemas embebidos. Automóviles, electrodomésticos, componentes industriales, equipamiento para la comunicaciones de voz y datos, instrumental de medicina, consolas de entretenimiento, dispositivos de audio y video son solo algunas de las áreas en donde se aplican los sistemas embebidos.

La utilización de sistemas embebidos fue originada por la flexibilidad que provee un sistema basado en microprocesador para adaptarse a diferentes aplicaciones. De esta manera, se pudo remplazar diseños electrónicos, mecánicos o eléctricos específicos por sistemas programables que, sin mayores cambios que el software, pudieron ser utilizados tanto para el control de un ascensor como para el de un horno de microondas o un control de acceso de personal, por nombrar solo algunos ejemplos.

Es de esta forma que aparecieron diferentes microcontroladores que permitían al diseñador pequeños sistemas embebidos en muy pocos pasos. Los microcontroladores de Microchip (de la familia PIC), de AVR y de Motorola abarcaron muchas aplicaciones. Posteriormente, la función que se requiere de un sistema embebido no solo se restringe a las tareas que debe cumplir según la aplicación, sino que es casi obligatorio incluir más funciones para satisfacer las exigencias de los usuarios. Por lo tanto, ya no es suficiente que una máquina fotográfica saque fotos, sino que es necesario que: almacene las fotos en una memoria compatible con una Computadora Personal, transfiera los archivos de fotos y video mediante un cable USB, detecte la sonrisa del objetivo, etc. Este tipo de exigencias en el diseño de sistemas embebidos requieren recursos de hardware y software adecuados para satisfacer los requerimientos de diseño modernos. En este sentido, las plataformas de lógica programable y el soporte de sistemas operativos permiten abarcar estas nuevas exigencias en el desarrollo de sistemas embebidos actuales.

SISTEMAS EMBEBIDOS EN LÓGICA PROGRAMABLE

Los sistemas embebidos están en continuo desarrollo, ofreciendo nuevas posibilidades día a día mediante la incorporación de nuevos dispositivos y tecnologías. Existen diferentes plataformas para la implementación de estos sistemas. Muchos de ellos poseen plataformas de desarrollo en las cuales se pueden implementar todo el software necesario para un hardware específico. Dichas plataformas presentan una solución para el desarrollador que generalmente abarca un determinado grupo de aplicaciones (automatización, video, procesamiento de señales, etc.). Por lo general, el soporte de sistemas operativos es restringido a las particularidades del hardware del sistema embebido y muy difícilmente puedan ser utilizadas en otros sistemas o aplicaciones.

Los dispositivos de lógica programable modernos, especialmente los dispositivos FPGA (*Field Programmable Gate Array*, en inglés), permiten la implementación de diferentes arquitecturas de procesamiento en hardware. El diseño y la implementación se pueden realizar mediante herramientas de desarrollo que permiten una gran flexibilidad al momento de adaptar las arquitecturas de hardware a las particularidades propias de una determinada aplicación. La capacidad de las FPGA modernas para la implementación de hardware permite la implementación de todo el sistema digital en un solo chip, lo que se denomina *Sistema-en-Chip-Programable* (SOPC, *System-on-Programmable-Chip*, en inglés). Esta flexibilidad en la

implementación del hardware posibilita cubrir una gran diversidad de aplicaciones, pero debe ser soportada al mismo tiempo por una “capa de abstracción de hardware” (*Hardware Abstraction Layer*, en inglés) que pueda ser adaptada fácilmente a las particularidades del software de aplicación. Es en este sentido que debe tenerse un conocimiento adecuado tanto del hardware como del software del sistema para permitir un diseño, desarrollo e implementación adecuada.

DESAFÍOS DEL DISEÑO DIGITAL

Las características propias de los sistemas embebidos han impuesto el desarrollo de filosofías de diseño flexibles y eficientes. El desafío de diseño digital ha sido tratado por diferentes autores.

Muchas de las filosofías de diseño consideran a todo el proceso como un procedimiento que posee un comienzo y un final en el que se entrega un producto terminado. Todo este proceso se define como “ciclo de vida del diseño” (*Design Life Cycle*). Existen diferentes maneras de implementarlo, todas ellas pudiendo producir el mismo resultado.

Una de las metodologías más tradicionales para la realización de diseños digitales es el “modelo de rebalse” (*Waterfall Model*, en inglés). La industria de software, debido a sus características específicas, ha adoptado metodologías de diseño más flexibles como ser el “modelo de prototipo” (*Prototype Model*, en inglés). Actualmente, el diseño de hardware digital basado en dispositivos digitales configurables (CPLD y FPGA) ha permitido incorporar estas metodologías de diseño puesto que poseen características similares a las del diseño de software.

Puede decirse que, independientemente del modelo de diseño seleccionado, el manejo de un proyecto debe comprender la revisión de cada etapa del proceso de diseño antes de continuar con la siguiente. Por esta razón, a medida que el proyecto avanza, existe más compromiso de recursos financieros y humanos afectados al mismo y las fallas introducidas en etapas tempranas del modelo repercuten adversamente en las siguientes.

MODELO DE REBALSE (WATERFALL MODEL)

Este modelo es un desarrollo tradicional en la industria electrónica y de hardware. Se basa en un proceso secuencial en el que cada etapa produce resultados para el comienzo de la siguiente en manera análoga al llenado de recipientes mediante el rebalse de un fluido. Este modelo (2) tiene sus orígenes en las industrias de manufactura y construcción, las cuales están fuertemente restringidas, por los costos o posibilidades, a efectuar modificaciones una vez que el producto ha sido realizado.

La Figura 1-1 muestra la representación típica del “modelo de rebalse” simulando una cascada de agua entre las diferentes etapas del proceso de diseño.

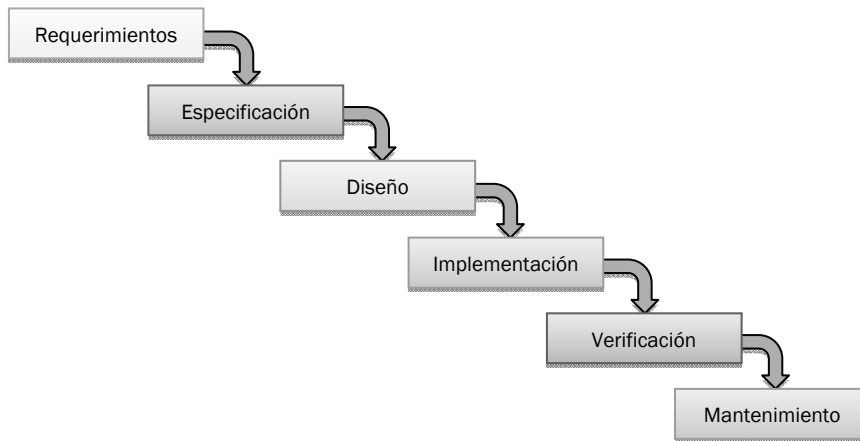


Figura 1-1: Ciclo de vida del diseño del modelo de rebalse (Waterfall Model).

Las etapas que generalmente se consideran en este modelo son:

- **Requerimientos:** en esta etapa, se describen los requerimientos que deberá satisfacer el producto final del diseño. Esta etapa podrá comenzar con una descripción abstracta y general para finalizar en una descripción detallada de los requerimientos que debe satisfacer el producto final del diseño. La etapa de requerimientos tiene una muy fuerte interacción con el usuario del producto final.
- **Especificación:** a diferencia de la etapa de requerimientos, esta etapa tienen una muy fuerte interacción con el desarrollador. La mayoría de las especificaciones puede provenir de la determinación técnica para la resolución de los requerimientos, pero otras pueden proceder de las alternativas técnicas introducidas por un preplanteo para su implementación. Es posible también especificar métricas que permitan verificar la correcta implementación del producto.
- **Diseño:** si bien el diseño abarca todas las etapas descritas, esta es la etapa específica en la que el diseño es formalizado. La división entre módulos, la selección de las técnicas y metodologías, la adopción de metodologías y las estrategias de planificación y demás detalles del diseño son detalladas en esta etapa.
- **Implementación:** en esta etapa, se comienza a desarrollar todos los componentes que constituirán el diseño y se integran para su correcto funcionamiento. Siguiendo el detalle realizado en la etapa de diseño se llega al resultado final que cumple con los requisitos y especificaciones.
- **Verificación:** en esta etapa, se comprueba que la implementación cumple con los requerimientos de acuerdo a las métricas especificadas para tal fin. En caso

de no satisfacerlas, podrá ser necesario realizar nuevamente la implementación, el diseño, la especificación o los requerimientos.

- **Mantenimiento:** generalmente, luego de la verificación el producto está listo para ser entregado al usuario o para ser producido masivamente. En muchas cadenas de manufactura o de construcción, hay pocas alternativas de diseño para realizar posteriormente. En diseño de software o basados en FPGA, la etapa de mantenimiento permite la corrección de errores o fallas de diseño que no fueron detectadas anteriormente.

El modelo de rebalse estructura todo el proceso de diseño, forzando a todas las personas intervinientes a definir completamente cada una de las etapas. Esto permite una interacción más eficiente entre todos los actores del proceso y reduce el riesgo de que el producto final no cumpla con las expectativas del usuario.

Con el avance de la industria del software, este modelo comenzó a recibir algunas críticas por su metodología. La principal de ellas es que, en el proceso de diseño de software, el usuario puede no tener una idea acabada del producto final al inicio del diseño y que esto requiera de una interacción permanente con el diseñador durante cada una de las etapas del proceso. Por otro lado, debido a las características en el desarrollo de software, es muy frecuente que el diseño no finalice solamente en el mantenimiento del producto final, sino que sea permanente un rediseño del mismo para producir nuevas versiones del producto que cumpla con nuevos requerimientos.

La realización de hardware digital mediante dispositivos CPLD y FPGA, ha permitido que el diseño se realice mediante la utilización de Lenguajes de Descripción de Hardware para la implementación de productos de Propiedad Intelectual (IP de *Intellectual Property* en inglés) en forma similar al desarrollo de software. Por este motivo, metodologías utilizadas en el desarrollo de software, y que permiten interacción entre las diferentes etapas del proceso de diseño, puedan ser aplicadas para el desarrollo de hardware digital en dispositivos digitales configurables.

MODELO DE PROTOTIPO (PROTOTYPE MODEL)

El concepto principal del modelo de prototipo (3) es permitir una interacción durante todo el proceso de diseño entre el usuario y el diseño en la especificación del producto. Como ocurre en el desarrollo de software, el cliente puede establecer determinados requerimientos que sería conveniente para el diseñador comprobar su posibilidad de implementación mediante prototipos. De esta manera, el cliente y el diseñador podrían interactuar para redefinir las especificaciones de acuerdo a las posibilidades tecnológicas con las que cuentan.

Asimismo, las diversas iteraciones producirán el diseño de varios prototipos. Puede llegar el momento en que se considere que el prototipo está listo para ser calificado como producto terminado y ser entregado, al mismo tiempo que las tareas de rediseño en otros prototipos continúa como trabajo interno de la compañía. Este proceso es muy frecuente en software en donde un producto posee diversas versiones.

La realización de un prototipo permite mejorar el conocimiento sobre las capacidades de implementación, los alcances de las funcionalidades y el refinamiento de los requerimientos

que pueden establecerse para el diseño final. La Figura 1-2 muestra una metodología de implementación del “ciclo de vida del diseño” de un modelo de prototipo en la que puede observarse que los requerimientos y especificación son refinados mediante el desarrollo de prototipos ligeros que permiten una interacción fluida entre el usuario y el diseñador. Este refinamiento puede continuarse aun cuando se decidiera que un determinado prototipo estuviera listo para su implementación, permitiendo el mejoramiento de futuras versiones del producto.

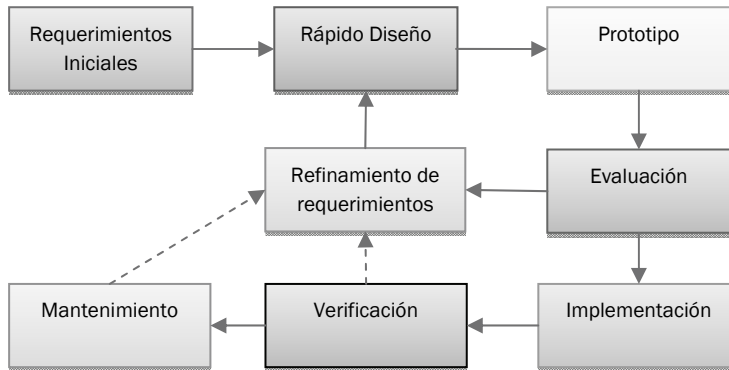


Figura 1-2: Ciclo de vida del diseño del modelo de prototipo (*Prototype Model*).

El desarrollo de un prototipo puede involucrar solamente alguna implementación parcial del diseño con el fin de evaluar la factibilidad o alcances que se puede lograr y redefinir los requerimientos y especificaciones. Algunos de los beneficios que pueden lograrse mediante este tipo de prototipos preliminares son:

- Análisis del alcance del diseño, permitiendo redefinir dicho alcance de acuerdo a la capacidad y complejidad de cada implementación.
- Demostración del funcionamiento general del sistema, permitiendo al desarrollador y al usuario obtener conocimientos para determinar las mejores alternativas para su implementación.
- Evaluación preliminar de los costos del producto y tiempo de desarrollo, dado que los cambios en etapas más avanzadas del proceso involucran costos y tiempos exponencialmente mayores.
- Obtención de volúmenes de información y procesamiento a considerar en el proyecto, para determinar las plataformas necesarias para un manejo adecuado y eficiente.

La utilización de dispositivos digitales configurables, como CPLD y FPGA, ha permitido la utilización del modelo de prototipo y otras metodologías de diseño de software al desarrollo de sistemas embebidos en dispositivos FPGA, mejorando la eficiencia y flexibilidad de los

procesos de diseño que se adapten a los cambios del ambiente que impone el mercado actual.

MÉTRICAS DE DISEÑO

Desde sus comienzos, las comparaciones entre los diferentes sistemas digitales de procesamiento fueron realizadas basadas en su velocidad: en cuán rápido podían ejecutar una determinada aplicación o software. A medida que estos sistemas fueron siendo utilizados para la implementación de sistemas embebidos, la velocidad pasó a ser solo una más de las métricas de comparación y evaluación de la eficacia y eficiencia de un determinado diseño.

Los requerimientos y las especificaciones detallan las condiciones que el diseño debe satisfacer, mientras que las **métricas** son las medidas que determinan si el producto final efectivamente satisface dichos requerimientos y especificaciones. En otras palabras, mientras que los requerimientos y especificaciones guían el proceso de diseño, las métricas verifican su cumplimiento.

Existe otro tipo de métricas que determina un grado de eficiencia de la implementación y que puede servir como criterio de comparación. Por ejemplo, si el diseño debe satisfacer una determinada velocidad de procesamiento, una implementación más veloz será más eficiente, según esta métrica, que otra que solo la cumple estrictamente. Por ejemplo, un diseño portable que pesa 300 gramos será considerado mejor que el que pesa 600 gramos. Sin embargo, esta mejor métrica de peso debe ser considerada conjuntamente con otras métricas de diseño, como por ejemplo el costo que ello implica.

Existen diversas métricas que pueden ser tenidas en cuenta en el diseño de sistemas embebidos. A continuación, se detallan solo algunas métricas de diseño cuantificables técnicamente como son: velocidad, energía, tamaño y costo.

VELOCIDAD

Gran parte de la literatura de sistemas de procesamiento basan la medición de la eficiencia de un sistema en su velocidad de procesamiento. Cuanto más rápido sea el procesamiento de un sistema, mayor su eficiencia. De esta manera, se proponen diferentes arquitecturas para incrementar la velocidad promedio en la ejecución de un determinado software. Este “criterio de eficiencia” está basado en que, generalmente, si un procesador es “más rápido” que lo requerido el sistema funcionará correctamente, mientras que si es “más lento” degradará la funcionalidad del sistema, pudiendo traer consecuencias inesperadas en la aplicación.

Características como coprocesamiento, memorias “caché”, *branch prediction*, etapas de *pipeline* son algunas de las metodologías y mecanismos propuestos para incrementar la velocidad de procesamiento de los procesadores modernos. En la mayoría de los casos, el incremento de la velocidad de procesamiento en un sistema embebido podría demandar un incremento en el costo unitario del producto. Por otro lado, la utilización de un procesador más veloz podría simplificar el desarrollo del software y, en consecuencia, el costo de diseño,

pero podría demandar un mayor consumo de energía. Todas las influencias deben ser evaluadas para determinar la conveniencia en la selección de la velocidad de procesamiento.

La velocidad de procesamiento de un sistema embebido no se restringe solo a la frecuencia de reloj con que este funciona. Existen diversos factores que intervienen en la velocidad final de un determinado sistema como ser la diagramación de las tareas en tiempo-real y la utilización de los recursos de hardware para la ejecución del software.

ENERGÍA

Todos los sistemas requieren energía para su funcionamiento. En especial, todos los sistemas digitales modernos están contruidos con componentes electrónicos que requieren energía eléctrica para su funcionamiento. En sistemas embebidos alimentados por baterías, puede ser necesario reducir el consumo de energía para incrementar la autonomía de los mismos.

Desde el punto de vista energético, puede verse un circuito digital como un sistema que transforma energía eléctrica en calor ya que toda esta energía que consume el circuito es disipada en forma de calor. El control del calor generado puede requerir la utilización de disipadores o sistemas de refrigeración forzada que pueden influir negativamente en la confiabilidad, tamaño o costos del diseño.

La disipación de energía, y por consiguiente el calor generado, está asociada con la **potencia** consumida por el dispositivo que es definida por la energía requerida en un determinado periodo de tiempo. Eléctricamente, la potencia (*power*, en inglés) es la tasa a la cual la **energía es transferida al circuito** eléctrico. La unidad de potencia es **watt** que equivale a un **Joule por segundo**. En un circuito digital, la energía consumida es proporcional al calor disipado al medio ambiente. Es necesario aplicar técnicas de disipación del calor generado para evitar que el circuito incremente su temperatura a valores perjudiciales para el sistema o el medio ambiente que lo rodea.

Otro factor importante es determinar el perfil del consumo de potencia, siendo conveniente, generalmente, tender a un consumo constante de la potencia evitando picos de consumo. En circuitos alimentados con baterías, los picos de potencia pueden acortar considerablemente la duración de la carga de la mismas afectando perjudicialmente la autonomía del sistema.

TAMAÑO

En sistemas embebidos modernos, el tamaño de los mismos es un requerimiento de diseño establecido por la portabilidad o espacio definido para su implementación. Eventualmente, el requerimiento de tamaño podrá limitar la velocidad del procesador para evitar un consumo que requiera un sistema de disipación voluminoso o un tamaño de baterías inadecuado.

Por otro lado, restricciones de tamaño podrían limitar la modularidad de la implementación que eventualmente imposibilitarían adaptaciones posteriores, su confiabilidad o dificultarían o afectarían las etapas de mantenimiento o la vida útil del proyecto.

COSTO

Las métricas anteriores definen la “eficiencia técnica” del diseño. En oposición a estas métricas, el costo determina la factibilidad económica o comercial del diseño. Existen diferentes costos que deben considerarse en el desarrollo de un determinado producto:

- Costo de ingeniería no retornable: es el costo que involucra el diseño y lograr un producto listo para producción. Principalmente, este costo debe considerar los costos del personal involucrado en el diseño y el de la infraestructura necesaria para su realización.
- Costo de fabricación: es el costo de fabricar el producto para su comercialización. Este costo debe considerar el costo de los componentes que conforman el producto y los costos involucrados en su manufactura.
- Costo de comercialización: es el costo económico y financiero requerido para la distribución y venta del producto y la administración del proceso de comercialización.

Todos estos costos deben ser considerados en el “costo unitario” final del producto. Los costos de ingeniería no retornable y los de comercialización debieran ser prorrateados en el cálculo del costo unitario de acuerdo a la cantidad de productos fabricados.

Los dispositivos digitales configurables permiten disminuir el tiempo necesario del ciclo de vida del diseño y consecuentemente el costo de ingeniería no retornable. La reducción de estos costos es importante cuando la cantidad de productos fabricados es limitada y cuando el tiempo para la entrada en el mercado es reducido.

OTRAS MÉTRICAS DE DISEÑO

En los sistemas embebidos, en los cuales el diseño involucra componentes de software como hardware, puede resultar difícil o imposible obtener una implementación que optimice cada una de las métricas de diseño especificadas. Si bien las detalladas anteriormente pueden resultar las más frecuentemente especificadas, existen diversas métricas de diseño, algunas de ellas de compleja cuantificación, entre las que se pueden listar:

- Tiempo al prototipo (*Time-to-prototype*, en inglés): es el tiempo requerido para construir una versión del sistema que pueda ser utilizada para verificar su funcionalidad.
- Tiempo a la comercialización (*Time-to-market*, en inglés): es el tiempo requerido para diseñar y manufacturar el sistema hasta el momento en que se encuentre en condiciones de que pueda ser comercializado a los consumidores.
- Conformidad (*Correctness*, en inglés): es el grado de satisfacción de que el sistema cumple con las especificaciones de diseño.
- Mantenimiento (*Maintainability*, en inglés): es la posibilidad de realizar modificaciones o mejoras al producto luego de su comercialización.

- Seguridad (*Safety*, en inglés): es generalmente expresado por la probabilidad de que una falla en el sistema no causará daño.
- Violabilidad (*Security*, en inglés): es el grado de protección contra sabotajes que posee el producto, el desarrollo o su diseño.
- Confiabilidad (*Reliability*, en inglés): es la evaluación de que el sistema realizará las funcionalidades para las que ha sido diseñado, durante un determinado periodo de tiempo y bajo condiciones establecidas.

El desarrollo de sistemas embebidos basados en dispositivos FPGA y la aplicación de metodologías flexibles de diseño pueden contribuir a mejorar significativamente estas métricas, aun cuando la implementación final se realice en otro tipo de tecnología.

CONCLUSIONES

Los dispositivos digitales configurables, como son los dispositivos CPLD y FPGA, han permitido la utilización de metodologías más eficientes en el diseño de sistemas embebidos. Mientras que en un principio el hardware digital debía, por su naturaleza, ser manufacturado, actualmente puede ser descrito y concebido como una propiedad intelectual factible de ser conceptualmente rediseñada. Bajo el punto de vista de diseño, no existen mayores diferencias entre el software referido al programa ejecutado por el procesador del sistema y el software referido al código que permite configurar el hardware de un dispositivo digital configurable. Los dispositivos CPLD y FPGA permiten la implementación de hardware físico mediante un código que puede ser concebido en ciclos de vida de diseño con metodologías solo aplicables anteriormente al diseño de software.

CAPÍTULO 2

LÓGICA PROGRAMABLE Y LENGUAJES DE DESCRIPCIÓN DE HARDWARE

CIRCUITOS DIGITALES CON LÓGICA PROGRAMABLE

Los dispositivos lógicos programables pueden definirse, en principio, como circuitos integrados que permiten la implementación de circuitos digitales. Los circuitos digitales electrónicos son ampliamente utilizados en diferentes áreas (procesamiento de señales, automatización, entretenimiento, comunicaciones, medicina, entre otras).

Conceptualmente, se puede afirmar que cualquier circuito digital puede resumirse en un circuito **combinacional** o un circuito **secuencial**. En este capítulo, se describen los principales conceptos de estos diferentes tipos de circuitos digitales y las capacidades que brindan los diferentes tipos de dispositivos de lógica programable (CPLD y FPGA). Al final del capítulo, se introducen los lenguajes de descripción de hardware como alternativa de representación de circuitos digitales complejos.

CIRCUITO COMBINACIONAL

Un circuito combinacional es aquel en el que la salida depende solo de la entrada actual. La Figura 2-1 muestra un diagrama de bloques el cual representa a la lógica combinacional como una “caja negra” en la cual el valor de las salidas depende de los valores que poseen las entradas.

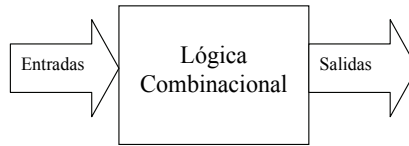


Figura 2-1: Diagrama de bloques de circuito combinacional.

Un circuito combinacional puede describirse mediante la especificación en una tabla del valor de la salida para cada una de las combinaciones de las entradas. Este tipo de tabla, que representa el funcionamiento de un circuito combinacional, se denomina **tabla de verdad**.

Por ejemplo, la expresión de una lógica combinacional de, por ejemplo, 3 entradas (**A**, **B** y **C**) y 2 salidas (**X** y **Z**), mediante una tabla de verdad, es de la forma:

TABLA 2-1: CIRCUITO COMBINACIONAL REPRESENTADO POR TABLA DE VERDAD

Entradas			Salidas	
A	B	C	X	Z
0	0	0	1	0
0	0	1	1	0
0	1	0	1	1
0	1	1	1	1
1	0	0	0	0
1	0	1	0	0
1	1	0	1	1
1	1	1	1	1

El circuito combinacional representado por una tabla de verdad puede ser implementado mediante compuertas electrónicas digitales que implementen funciones lógicas (compuertas NOT, AND, OR, NAND, etc.). Existen diferentes métodos para obtener las funciones lógicas de un circuito combinacional a partir de su especificación.

CIRCUITO SECUENCIAL

Un circuito secuencial es aquel cuya salida es función de la *secuencia* en la que se fueron dando las entradas. Un ejemplo de esto es un sistema de teclado de una alarma. En este tipo de circuito, se debe ingresar un determinado número para desactivar la alarma. Si los números son ingresados, pero en diferente orden, la alarma no se desactivará. Por otro lado, si los números son ingresados en la *secuencia* correcta, la alarma se desactivará.

En los circuitos secuenciales, no es práctico, y generalmente imposible, describir el funcionamiento del circuito mediante una tabla que enumere todas las secuencias posibles de las entradas. Es por ello que se utiliza el **estado** para expresar la condición en la que está

el circuito, producto de las secuencias de entradas. Hoy en día, una de las definiciones citadas más frecuentes del “estado” en un circuito digital es la realizada en (4):

“El estado de un circuito secuencial es una colección de variables de estado cuyos valores en cualquier tiempo contienen toda la información acerca del pasado necesaria para explicar el comportamiento futuro del circuito”.

En un circuito electrónico digital, como los que generalmente son implementados en dispositivos FPGA, se utilizan elementos de memoria que se denominan *flip-flops* para almacenar el estado del sistema.

Habiendo definido al estado como la información del pasado del sistema necesaria para el comportamiento futuro, podemos decir que: *en un circuito secuencial, el valor de la salida depende del estado y de las entradas*. La Figura 2-2 muestra un diagrama de bloques de un circuito secuencial.

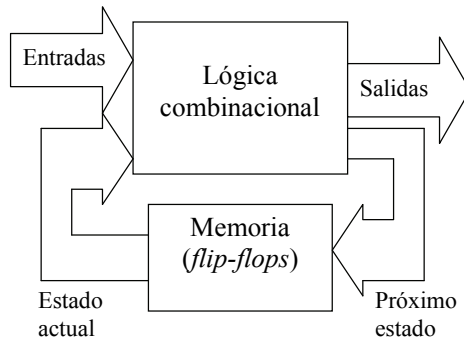


Figura 2-2: Diagrama de bloques de circuito secuencial.

Del diagrama de bloques puede observarse que la lógica combinacional, además de determinar las salidas del sistema, también determina el próximo estado al que va a evolucionar el sistema. El próximo estado es función del estado actual y de las entradas del sistema.

El circuito secuencial de la Figura 2-2 representa un circuito secuencial **modo fundamental**. En este modo, existen diversos factores a tener en cuenta para un correcto funcionamiento (carreras críticas, riesgo y realimentación analógica) originados en la realimentación analógica y en el funcionamiento en tiempo continuo del circuito. Para evitar los inconvenientes del modo fundamental, en electrónica digital, se realizan circuitos en **modo reloj**. En este modo, la rama de realimentación del circuito es cortada mediante una señal de reloj. Si bien un circuito secuencial en modo reloj no funciona a tiempo continuo sino a tiempo discreto, el diseño e implementación del mismo es más sencillo y justifica su elección en la mayoría de los circuitos electrónicos digitales.

Los dispositivos lógicos configurables, como CPLD y FPGA, están concebidos principalmente para la implementación de circuitos secuenciales en modo reloj, como así también los lenguajes de descripción de hardware Verilog HDL y VHDL y las herramientas de síntesis asociadas.

La Figura 2-3 muestra un diagrama de bloques de un circuito secuencial modo reloj.

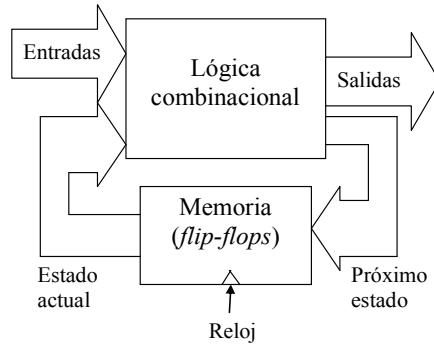


Figura 2-3: Diagrama de bloques de circuito secuencial.

De lo expuesto, se puede concluir que la implementación de circuitos digitales requiere, en forma general, la capacidad de implementación de lógica combinacional y de elementos de memoria (*flip-flops*). La funcionalidad del circuito dependerá de la forma en la que estos componentes se interconecten entre sí. Los dispositivos lógicos programables (CPLD y FPGA) poseen recursos para la implementación de lógica combinacional y *flip-flops* con capacidad de interconectarlos entre sí de acuerdo a la configuración que el diseñador determine.

DISPOSITIVOS LÓGICOS PROGRAMABLES: CPLD Y FPGA

El avance tecnológico ha posibilitado capacidades de integración electrónica impensadas pocos años atrás. Esto ha beneficiado indudablemente a los circuitos digitales que, por sus características, pueden lograr escalas de integración cada vez mayores ante el avance tecnológico.

Este avance tecnológico ha permitido desarrollar dispositivos integrados con gran capacidad de implementación de circuitos combinacionales y *flip-flops* que también cuentan con gran flexibilidad para la interconexión de tales recursos. Actualmente, existen dos grandes ramas de estos dispositivos lógicos programables, bien diferenciados por: sus características tecnológicas, sus funcionalidades y sus aplicaciones. Estos dispositivos son: CPLD y FPGA.

Dentro de cada uno de estos dispositivos existen variantes de diseño elegidas por cada uno de sus fabricantes. En este libro, utilizaremos, a modo de ejemplo, los dispositivos fabricados por Altera, sin impedimento de que los mismos conceptos pueden ser aplicados a los dispositivos de los demás fabricantes.

La Figura 2-4 se muestra la arquitectura de una celda básica, denominada **macrocelda**, de una CPLD de la familia MAX7000S de Altera:

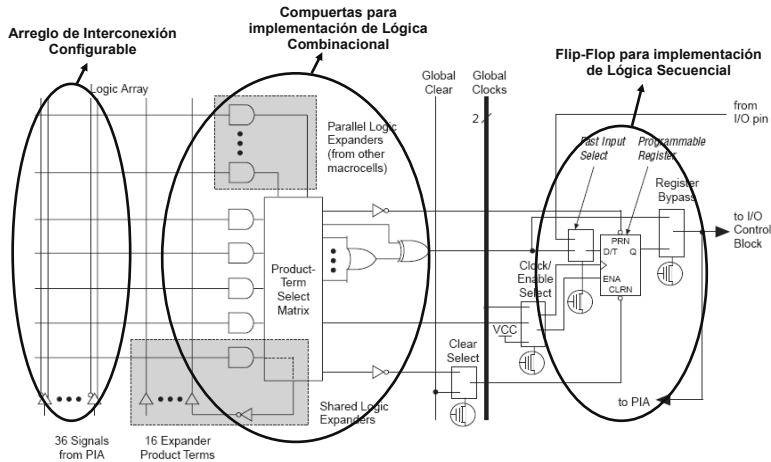


Figura 2-4: Macrocelda de una CPLD MAX7000S detallando las diferentes capacidades de interconexión y de implementación para lógica combinacional y secuencial (figura de (5)).

Esta macrocelda es solo un bloque básico que es implementado varias veces en un mismo dispositivo, conjuntamente con más recursos para la interconexión configurable de las diferentes salidas y entradas de dichas macroceldas. La Figura 2-5 muestra la arquitectura de un dispositivo CPLD de la familia MAX7000S de Altera.

Por otro lado, la arquitectura de un dispositivo FPGA es diferente, pero con las mismas funciones básicas referidas a lógica combinacional, secuencial y de interconexión. La Figura 2-6 muestra el diagrama de bloques de un **elemento lógico** (similar a la macro celda en CPLD) de un dispositivo FPGA de Altera de la familia Cyclone III:

A diferencia de las macroceldas de los dispositivos CPLD, en los cuales los dispositivos de configuración son transistores E²PROM, en los dispositivos FPGA, la tecnología utilizada para los recursos configurables son de tecnología RAM. La utilización de tecnología RAM permite la implementación de dispositivos FPGA de muy alta escala de integración y, por consiguiente, poseen generalmente mayor capacidad de implementación lógica que los dispositivos CPLD. Como contrapartida, la tecnología RAM de los dispositivos FPGA pierde su configuración cada vez que el dispositivo se desenergiza y debe ser reconfigurado cada vez que el dispositivo FPGA entra en funcionamiento. Para este propósito, cuando se utilizan dispositivos FPGA, se disponen conjuntamente con una memoria *flash* en la cual se almacena la configuración que el dispositivo FPGA lee automáticamente cuando es energizada para autoconfigurarse. Existe una alternativa que permite configurar el dispositivo FPGA desde una computadora personal que ejecuta el software del fabricante y que es muy útil al momento de diseño, pues permite reconfigurar el dispositivo FPGA muy fácilmente.

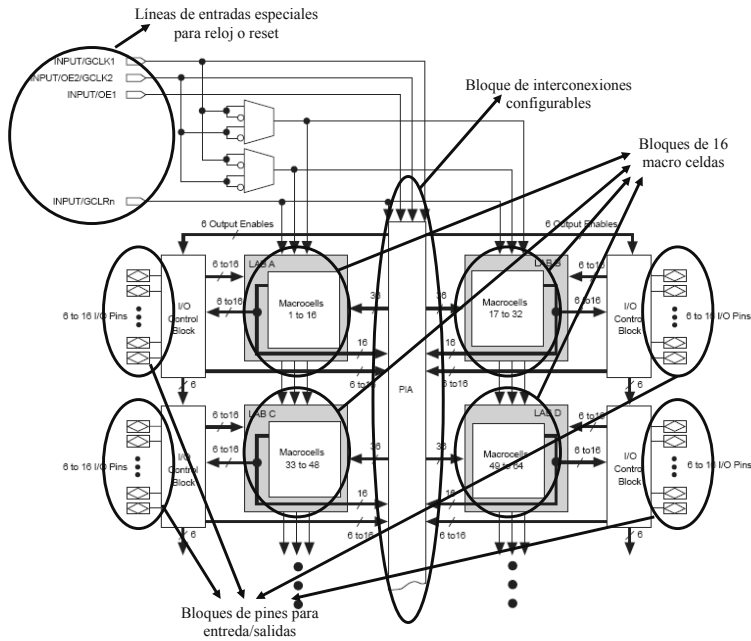


Figura 2-5: Arquitectura en bloques de un CPLD de la familia MAX7000S detallando los diferentes bloques de interconexión, lógica y conexión de entradas/salidas (figura de (5)).

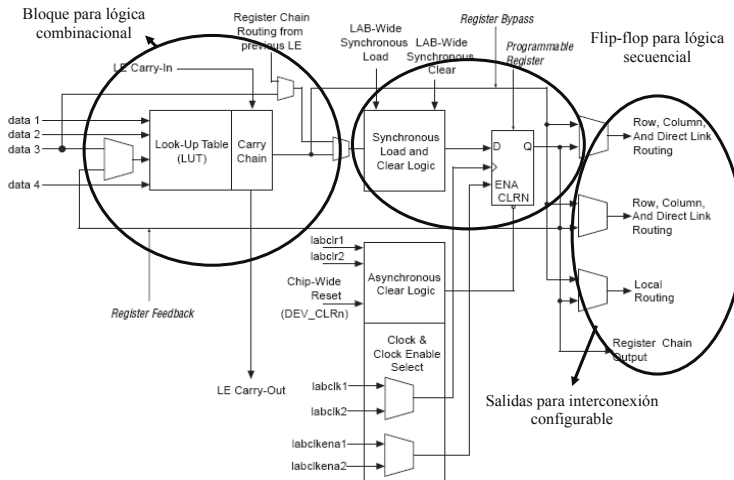


Figura 2-6: Elemento lógico de un dispositivo FPGA Cyclone III detallando las diferentes capacidades de interconexión y de implementación para lógica combinacional y secuencial (figura de (6)).

La Figura 2-7 muestra un bloque esquemático parcial de una FPGA de Altera de la familia Cyclone III. La arquitectura no permite realizar un mayor detalle debido a la gran densidad de lógica y de posibilidades de interconexión. Sin embargo, puede apreciarse la gran densidad de bloques para la implementación de lógica (denominados LAB) y la gran disponibilidad de recursos de interconexión dispuestos en forma vertical y horizontal.

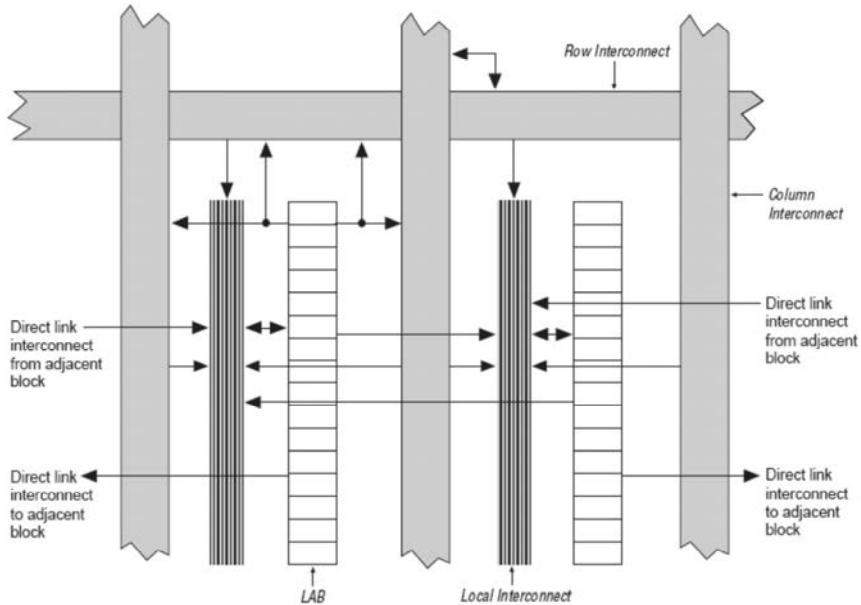


Figura 2-7: Arquitectura parcial en bloques de un FPGA de la familia Cyclone III detallando los diferentes bloques de interconexión y lógica (LAB) (figura de (6)).

RECURSOS ADICIONALES EN DISPOSITIVOS FPGA

La implementación de circuitos digitales frecuentemente requiere recursos específicos que, o bien escapan a la capacidad de ser implementados mediante los *elementos lógicos* de los dispositivos FPGA, o bien no es eficiente su implementación mediante tales elementos. Cabe señalar que los dispositivos FPGA, son utilizados en diferentes campos de aplicaciones y por consiguiente cada uno de ellos podrá requerir de especiales especificaciones de consumo, frecuencia de funcionamiento o capacidad de almacenamiento.

Es por esta razón, y debido a la gran capacidad de integración que permiten los procesos tecnológicos actuales, que los fabricantes de dispositivos FPGA ponen a disposición del diseñador, diversos recursos según las necesidades que puedan requerir sus aplicaciones. Por este motivo, los dispositivos FPGA modernos pueden contener recursos integrados tales como:

- **Bloques de memoria:** estos bloques pueden ser configurados de diversas maneras según la necesidad (RAM, Dual-Port RAM, *Content Address Memory* (CAM), FIFO, etc.).
- **Bloques sumadores-multiplicadores:** permiten la implementación más eficiente de algoritmos de procesamiento de datos que si las mismas operaciones se implementaran mediante elementos lógicos.
- **Bloques de sincronización de reloj (PLL):** estos bloques permiten sincronización de la frecuencia de un reloj externo para la obtención de diferentes temporizados, adecuados para la sincronización de los diferentes componentes del circuito digital implementado.
- **Interfaces de entrada/salida de alta velocidad (*transceivers*):** permiten la transferencia de datos a gran velocidad entre circuitos digitales (PCI, PCI Express, SATA, etc.).
- **Procesadores embebidos:** existen algunas versiones de FPGA que poseen núcleos de procesadores (ARM, POWER PC) embebidos entre los recursos del dispositivo FPGA. Sin embargo, son pocas las versiones de FPGA que cuentan con núcleos de procesadores embebidos, siendo generalmente la implementación de un procesador en un FPGA realizada mediante la configuración de los elementos lógicos de tal manera que el hardware se comporte como el procesador que se desea implementar.

De la descripción de los diferentes dispositivos lógicos programables, tales como CPLD y FPGA, se puede deducir que los mismos posibilitan la implementación de diversos circuitos digitales cuya complejidad solo está limitada por la capacidad de implementación lógica de dichos dispositivos. Es por esta razón que los dispositivos FPGA modernos permiten la implementación de sistemas completos en un solo circuito integrado. Tales niveles de integración requieren herramientas y metodologías de especificación, descripción, diseño, simulación y documentación adecuadas que permitan la abstracción necesaria para poder abarcar toda la complejidad del sistema. Por tal motivo, se proponen los lenguajes de descripción de hardware en la especificación de circuitos digitales.

LENGUAJES DE DESCRIPCIÓN DE HARDWARE

En diversos cursos y libros de electrónica digital, se introduce al diseño de circuitos digitales y se realizan diferentes implementaciones mediante la utilización de compuertas o elementos digitales (NOT, AND, OR, *flip-flops*, etc.). Tales circuitos se representan mediante esquemáticos en los cuales se interconectan gráficamente los símbolos de cada una de las compuertas de acuerdo a los circuitos que se desean implementar. Sin embargo, este tipo de representación se vuelve dificultosa cuando el circuito o sistema sobrepasa la decena de componentes digitales.

Para sistemas más complejos, se comienza a utilizar una representación por bloques. En este tipo de representaciones, existe un diagrama que contiene un esquemático de las unidades funcionales más importantes del circuito y luego otros diagramas que mostrarán en detalle cada una de dichas unidades funcionales. En otras palabras, existe una

representación de **mayor jerarquía** que solo es un diagrama de bloques. Luego existirá una representación con mayor detalle por cada uno de los bloques. Cada una de las representaciones dará más detalle y podrá también requerir de una representación de **menor jerarquía**. Este tipo de representaciones posibilita al diseñador poder abarcar todo el diseño sin desviar su atención debido a la excesiva información necesaria para la implementación.

Sin embargo, cuando los sistemas se vuelven aún más complejos, los tiempos de diseño se vuelven cada vez más exigentes al igual que las etapas de simulación y se necesita aumentar la reutilización de diseños anteriores en nuevos productos. Es entonces cuando los **Lenguajes de descripción de hardware** (HDL, *Hardware Description Languages*, en inglés) se vuelven de uso obligatorio.

Un lenguaje de descripción de hardware permite describir un circuito digital mediante un código almacenado generalmente en un archivo de texto. Este archivo de texto es procesado por la herramienta de síntesis brindada por el fabricante de dispositivos lógicos configurables (CPLD o FPGA) y genera un archivo de configuración para tal dispositivo. Los lenguajes de descripción de hardware permiten la descripción de los circuitos digitales en diferentes niveles jerárquicos para permitir abarcar sistemas complejos.

De esta manera, la utilización de lenguajes de descripción de hardware posibilita la reutilización de los diseños puesto que solo es el procesamiento de diferentes archivos. Es más, pueden incluirse en un diseño hardware diseñado por otros usuarios puestos a disposición, gratuitamente o no, mediante archivos compartidos. Tales archivos pueden ser agrupados de acuerdo a diferentes criterios para formar bibliotecas de diseño similares a como se acostumbra a realizar en proyectos de software.

Actualmente, existen dos lenguajes de descripción de hardware ampliamente utilizados y soportados por las herramientas de diseño y síntesis de los diferentes fabricantes: **VHDL** y **Verilog HDL**. Ambos lenguajes son abiertos y estandarizados por el IEEE; VHDL en el estándar IEEE 1076 y 1164 y Verilog en el estándar IEEE 1364.

Cualquier lenguaje de descripción de hardware debe ser capaz de permitirle al diseñador especificar, mediante dicho lenguaje, la arquitectura y funcionamiento de cualquier hardware digital. Por este motivo, es que el lenguaje debe ser capaz de describir actividades concurrentes debido a que en un hardware, todos los componentes que lo integran están “funcionando” de manera concurrente (al mismo tiempo). Esto es una diferencia muy importante con respecto a un lenguaje de programación, en donde se programa un procedimiento que será ejecutado instrucción por instrucción en forma totalmente secuencial.

Otra característica que debe poseer un Lenguaje de Descripción de Hardware es la capacidad de descripción de hardware a diferentes niveles jerárquicos. Esta posibilidad debe permitirle al usuario poder diseñar hardware más complejo a partir de la interconexión de componentes más simples.

Por último, una posibilidad que brindan los lenguajes como VHDL y Verilog HDL (y que carecen muchos de los lenguajes propuestos por fabricantes) es la especificación de tiempo. Esta característica es muy adecuada al momento de realizar verificaciones y simulaciones utilizando el mismo lenguaje.

La utilización de lenguajes de descripción de hardware y de dispositivos lógicos programables permite una flexibilidad sin precedentes en el diseño de hardware digital. Hoy en día, es posible concebir el hardware digital mediante la adecuada interrelación de descripciones contenidas en archivos de computadoras y, sintetizadas mediante las herramientas de software adecuadas, configurar dispositivos lógicos programables para implementar físicamente el hardware digital deseado.

Las descripciones realizadas en diseños anteriores pueden ser reutilizadas o adquiridas a terceros para realizar nuevas implementaciones, reconfigurar dispositivos ya utilizados para modificar sus prestaciones o utilizarlos en diferentes aplicaciones.

Este tipo de metodología de diseño y síntesis nos permite diseñar todo un sistema de hardware digital con la sola selección de los componentes que queremos utilizar. Dichos componentes, que incluyen procesadores, periféricos de comunicación (Ethernet, serial, etc.), controladores de video, entre otros, pueden ser brindados por Altera, Xilinx o diferentes compañías en forma de bibliotecas de componentes compatibles entre sí. Esta posibilidad de implementación de todo un sistema de hardware digital en un dispositivo lógico programable (FPGA) se denomina generalmente SOPC (Sistemas-en-Chip-Programable, del inglés *System-on-Programmable-Chip*).

En este capítulo, introduciremos las principales características del lenguaje Verilog HDL el cual, conjuntamente con VHDL, es uno de los lenguajes más ampliamente utilizados para la descripción de hardware digital.

NIVELES DE ABSTRACCIÓN EN LENGUAJES DE DESCRIPCIÓN DE HARDWARE

Un lenguaje de descripción de hardware debe tener una estructura adecuada para la especificación de cualquier hardware digital. Dicha estructura debe ser capaz de describir el hardware digital para que sea sintetizado en recursos lógicos reales (en nuestro caso recursos de CPLD o FPGA). Una vez cumplido este requisito fundamental, podría utilizarse dicho lenguaje para diferentes funciones tales como: documentación, especificación de comportamiento o simulación. Es obvio que no podría sintetizarse a hardware un archivo de documentación, aun cuando dicho archivo estuviera escrito en un lenguaje de descripción de hardware.

En algunos lenguajes de hardware, como VHDL y Verilog HDL, se refieren a estos distintos “objetivos” de la utilización del lenguaje como diferentes **niveles de abstracción** (*Abstraction Levels*, en inglés). De esta manera, en el diseño digital, se definen cuatro diferentes *niveles de abstracción*:

- **Comportamiento (*Behavioral Level*):** una descripción a nivel de comportamiento especifica la funcionalidad del circuito o sistema sin dar mayores detalles de la manera en que logra dicha funcionalidad. Una descripción a nivel de comportamiento difícilmente pueda sintetizarse a hardware, salvo muy específicos casos particulares.

- **Transferencia de registros (RTL, *Register Transfer Level*, en inglés):** una descripción a nivel RTL permite el mayor grado de abstracción sintetizable. El hardware es descrito mediante componentes implementables en hardware tales como registros, multiplexores, decodificadores, entre otros, necesarios para determinar el funcionamiento real del circuito.
- **Compuerta (*Gate Level*):** es un nivel de abstracción menor que el RTL y especifica la arquitectura del circuito a nivel de compuertas lógicas y *flip-flops*. Debido a su bajo nivel de abstracción, el nivel de compuerta puede ser inadecuado cuando se realizan descripciones de circuitos medianamente complejos.
- **Físico (*Physical Level*):** este es menor nivel de abstracción y contempla la implementación física del circuito tanto en los recursos utilizados como su interconexión. Por su bajo nivel de abstracción, es muy extraña su utilización para la descripción por el diseñador de un hardware digital.

La mayoría de las descripciones realizadas por un diseñador en Verilog HDL o VHDL son efectuadas a nivel de comportamiento, para simulación o a nivel RTL para síntesis. En este libro, abordaremos principalmente la utilización de Verilog HDL a nivel RTL.

SÍNTESIS Y SIMULACIÓN DE ARCHIVOS DE DESCRIPCIÓN DE HARDWARE

Los archivos de descripción de hardware realizados en Verilog HDL, a nivel RTL, pueden ser procesados para simulación o para síntesis.

Para realizar la síntesis de una descripción, se utilizan herramientas de desarrollo que, cuando se aplican a FPGA, se denominan **compiladores de síntesis**. Un compilador de síntesis toma como entrada un archivo Verilog HDL y entrega como salida un **archivo de interconexión de componentes** (*Netlist File*). Entonces, la función del compilador de síntesis es tomar la descripción del hardware realizada en el archivo Verilog HDL y expresar la estructura que describe en los recursos lógicos específicos que contiene el dispositivo configurable destino (CPLD o FPGA). En otras palabras, el compilador de síntesis transforma una descripción Verilog HDL en una estructura de hardware digital que puede ser implementada en un dispositivo lógico configurable real.

Por otro lado, la simulación es un proceso por el cual puede determinarse cómo se comportará un determinado hardware ante un cierto **escenario de simulación**. Este escenario es especificado por el valor de las entradas del circuito, generalmente producido mediante: (1) una descripción Verilog HDL denominada *testbench* (o banco de pruebas) o (2) un archivo en donde se especifica el valor de las entradas y su variación en el tiempo denominado *Test Vectors* (o vectores de prueba). El proceso de simulación puede ser útil entonces para comprobar que el circuito se comportará según las especificaciones cuando se implemente en la aplicación final.

La Figura 2-8 muestra el flujo de síntesis y de simulación de hardware. La simulación de una descripción de hardware puede realizarse tanto a nivel RTL, si se realiza sobre el archivo Verilog HDL, o a nivel de compuerta si se realiza sobre el archivo de interconexión de componentes. Mientras que la primera opción permite simulaciones más rápidas, la segunda

permite simulaciones con mayor detalle debido a que se realizan a diferentes niveles de abstracción.

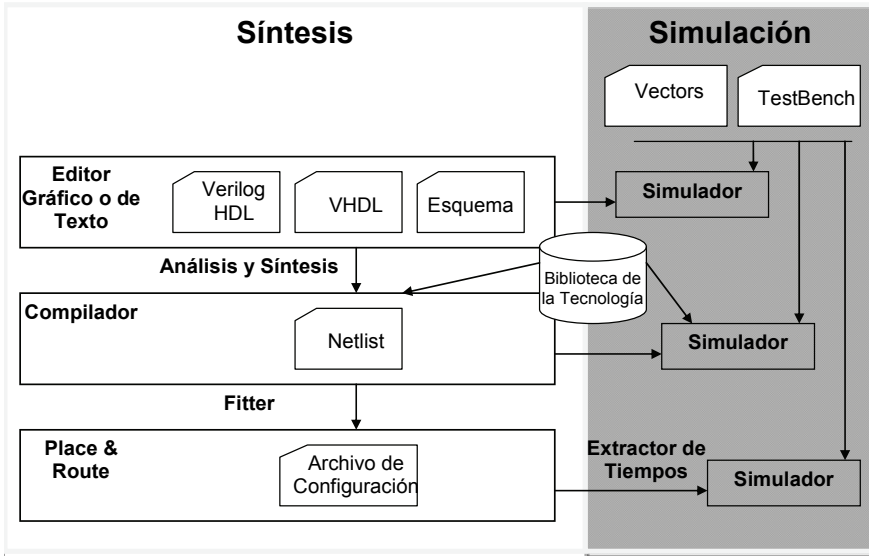


Figura 2-8: Flujo de los procesos de síntesis y simulación de hardware.

VERILOG HDL

Verilog HDL es un lenguaje basado en texto que permite describir hardware tanto para síntesis como para simulación. Sus orígenes se remontan al año 1984 cuando es introducido por la compañía *Gateway Design Automation*. En 1989, la compañía Cadence adquiere a *Gateway Design Automation* y hace público el lenguaje Verilog HDL en 1990, conformando el grupo de trabajo *Open Verilog International (OVI)* para controlar las especificaciones del lenguaje.

En 1993, OVI lanza la versión 2.0 del lenguaje. En 1995, el Instituto de Ingeniería Eléctrica y Electrónica (IEEE) acepta como el estándar **Verilog 1364** la especificación realizada por OVI. Nuevas revisiones del estándar son realizadas en los años 2001 y 2005 (7).

Verilog HDL es un lenguaje que se escribe en un archivo de texto y distingue entre minúsculas y mayúsculas (*case sensitive*, en inglés). Todas las palabras reservadas del lenguaje deben escribirse en minúsculas. Como ocurre en el lenguaje C y VHDL, cada sentencia del lenguaje debe ser finalizada con un punto y coma (;).

Los comentarios deben ser realizados comenzando la línea con "--", mientras que comentarios multilínea pueden comenzarse con "-*" y finalizados en la última línea con "*-".

ESTRUCTURA MODULE

Habiendo realizado una introducción a la historia de Verilog HDL y al proceso de síntesis y simulación, se realizará un detalle de la sintaxis y las estructuras básicas del lenguaje.

Toda la descripción realizada en Verilog HDL debe ser realizada dentro de la estructura *module*. La estructura *module* comienza con la palabra reservada *module* seguida por el nombre de *module* y finaliza con la palabra reservada “*endmodule*”. La Figura 2-9 muestra la estructura *module* y sus principales secciones constitutivas.

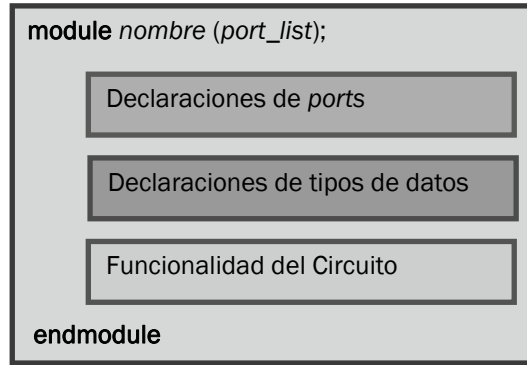


Figura 2-9: Estructura básica de *module*.

Sección *port list*

La sección *port list* es la primera sección de la estructura *module*, contiene una lista de todas las conexiones de entrada, salida y bidireccional al circuito y se encuentra entre paréntesis a continuación del nombre del *module*.

Ejemplo: para ilustración de cada una de estas secciones, utilizaremos un circuito contador como el mostrado en la Figura 2-10.

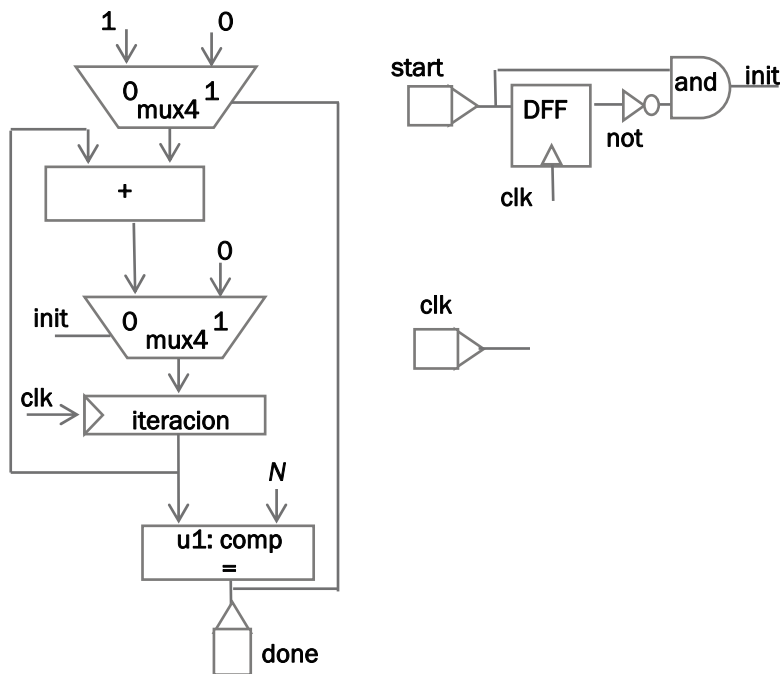


Figura 2-10: Ejemplo contador para descripción en Verilog HDL.

En el circuito de la Figura 2-10, las entradas *start*, *clk* y la salida *done* definirán la sección *port list* de la siguiente manera, nombrando al *module* “contador”:

```
module contador (start, clk, done);
```

Sección Declaraciones de ports

La sección Declaraciones de *ports* asocia cada una de las conexiones de la sección *port list* con su tipo. Existen tres tipos de conexiones: **input** (para conexiones de entrada), **output** (para conexiones de salida) y **inout** (para conexiones bidireccionales). Los canales (*buses*, en inglés), especifican sus índices entre corchetes ('[]'). Si se considera el circuito de la Figura 2-10, la sección Declaraciones de '*ports*' estará descrita como:

```
input clk, start;
output done;
```

En la versión Verilog HDL 2001, existe la opción de describir la conexiones y sus tipos en la definición del *module*, haciendo la declaración más concisa. De esta manera, con la versión Verilog HDL 2001, podría declararse ambas secciones como:

```
module contador
(
  input clk, start;
  output done;
);
```

Sección Declaración de tipos de datos

Existen dos declaraciones de tipos de datos fundamentales en Verilog HDL: **Net** y **Variable**.

- **Net**: representa una conexión física entre procesos o bloques funcionales. Estas conexiones están activas todo el tiempo y no almacenan ningún valor.
- **Variable**: representa un elemento de almacenamiento de dato temporario. El tipo de dato variable puede ser sintetizado a hardware como *flip-flops* o registros según su utilización. Las variables necesitan ser asignadas dentro de un procedimiento, función o tarea que son detalladas más adelante.

Los cuatro tipos de datos más comunes, sintetizables, para declaraciones **Net** son:

- **wire**: representa un nodo o una conexión.
- **tri**: representa un nodo de tres-estados” (*tri-state*).
- **supply0**: representa una constante lógica 0.
- **supply1**: representa una constante lógica 1.

Los tipos de datos para declaraciones **Variable** sintetizables son:

- **reg**: es una variable de cualquier tamaño de bits con representación sin signo. Incorporándole la cláusula *signed*, la representación será en complemento a dos.
- **integer**: representa una variable de 32 bits con signo.

DESCRIPCIÓN DE LA FUNCIONALIDAD DEL CIRCUITO

Una vez que se declaró la lista de conexiones y los tipos de dato de las conexiones en el *module*, debe describirse la funcionalidad del circuito. Una de las maneras de descripción que soporta todo lenguaje de descripción de hardware, incluido Verilog HDL, es la jerárquica. En este tipo de descripción, se utilizan módulos definidos previamente para especificar el funcionamiento de un circuito más complejo mediante la conexión de componentes más simples. La utilización de la descripción de un componente en un *module* de mayor nivel jerárquico se realiza mediante la **instanciación** de dicho componente.

Instanciación de module

La instanciación de un componente en un *module* de mayor nivel jerárquico se realiza mediante la siguiente sintaxis:

```
<nombre_del_componente> <nombre_de_la_instancia> (port_list)
```

donde:

<nombre_del_componente> es el nombre que se le dio al *module* del componente que se desea instanciar,

<nombre_de_la_instancia> es el nombre que se le dará a la instancia del componente en el *module* de mayor jerarquía, y

(port list) es una lista de señales que se conectarán con la conexiones del módulo instanciado.

Cuando se conectan las señales a un *module* de menor jerarquía, se puede elegir una de dos maneras: (1) por orden, o (2) por nombre.

Conexión por orden y por nombre

En el ejemplo de la Figura 2-10, se instancia un *module* comparador para realizar la comparación entre las entradas *iteración* y *N*. Supóngase que la declaración del *module* del comparador se especificó de la siguiente manera:

```
module comp (entrada1, entrada2, salida);
```

La instanciación del componente comparador (nombrada *u1*) en la sección de funcionalidad del circuito en el *module* del contador, puede ser realizada por orden mediante la sentencia:

```
comp u1 (iteracion, N, done);
```

Por otro lado, la conexión por nombre permite referenciar la señal con el nombre del puerto independientemente del orden con que haya sido declarada en el *module* del componente. La instanciación del componente “comparador” mediante la conexión por nombre puede ser realizada mediante la sentencia:

```
comp u1 (.salida(done), .entrada1(iteracion), .entrada2(N));
```

Debe tenerse en cuenta que todas las señales que se conectan a la salida de un *module* deben ser declaradas como tipo *net*. En forma similar, las entradas de un *module* deben ser declaradas de tipo *net* dentro del *module*.

Representación de valores en Verilog HDL

En Verilog HDL, los valores pueden ser representados de dos maneras: (1) indicando el número de bits (*sized*) o (2) sin indicar el número de bits por lo que el sintetizador tomará por defecto 32 bits (*unsized*).

El formato para expresar un número en la asignación es:

```
<size>'<base format><number>
```

donde:

<size> indica la cantidad de bits que requerirá la representación del número en binario. Si se omite este parámetro, se tomará por defecto 32 bits.

<base format> indica la base en que está indicado el número. Las alternativas del formato pueden ser:

- Decimal - indicada con 'd o 'D,
- Hexadecimal - indicada con 'h o 'H,
- Binario - indicado con 'b o 'B,
- Octal - indicada con 'o o 'O,
- Número con signo - indicado con 's o 'S y seguido del formato del número

<number> indica el número que se asignará en el formato especificado por el campo <base format>.

Para especificar número negativos, se debe indicar anteponiendo el signo menos ('-') al campo <size>. Por ejemplo: -4'd3 indicará la representación numérica en complemento a 2 de -3.

OPERADORES EN VERILOG HDL

El lenguaje Verilog HDL posee, como otros lenguajes, operadores que permiten realizar diferentes funciones aritméticas, lógicas, relacionales o de manipulación de datos. Los operadores en Verilog HDL poseen mucha similitud a los utilizados en el lenguaje de programación C. La Tabla 2-2 lista los operadores más comúnmente utilizados en Verilog HDL.

TABLA 2-2: OPERADORES DE VERILOG HDL

Tipo de operador	Símbolo	Operación realizada
Aritméticos	*	Multiplicación
	/	División
	+	Suma
	-	Substracción
	%	Módulo
	++	Incremento
	--	Decremento
Lógicos	!	Negación lógica
	&&	AND lógica
		OR lógica
Relacionales	>	Mayor que
	<	Menor que
	>=	Mayor o igual
	<=	Menor o igual
Igualdades	==	Igual
	!=	Distinto
Reducción	~	Not
	~&	nand
		or
	~	nor
	^	xor
	^~	xnor
	~^	xnor
Desplazamiento	>>	Desplazamiento a la derecha
	<<	Desplazamiento a la izquierda
Concatenación	{ }	Concatenación
Condicional	?	Condicional

CIRCUITOS COMBINACIONALES Y SECUENCIALES EN VERILOG HDL

En este libro, orientamos principalmente la utilización del lenguaje Verilog HDL para la síntesis de hardware digital. Como se ha descrito, los circuitos digitales pueden ser circuitos combinacionales o circuitos secuenciales. En particular, todo circuito secuencial destinado a su implementación en dispositivos lógicos configurables, como pueden ser CPLD y FPGA, son circuitos secuenciales **con reloj** (o “modo reloj” de la teoría de sistemas digitales).

Bloque *always*

Un bloque *always* es una estructura que permite describir el comportamiento de un hardware digital concurrente (como es el funcionamiento de cualquier hardware digital) en forma de procedimiento. De esta manera, si bien el hardware será concurrente, la evaluación del bloque podrá realizarse en forma de procedimiento según el orden de las sentencias en su interior. La estructura de un bloque *always* es la siguiente:

```
always @(sensitive list)
begin
    .....
    sentencias
    ....
end
```

La lista sensible (*sensitive list*) es una lista de variables cuyos cambios producirán que el bloque se evalúe. En otras palabras, el bloque no se evalúa permanentemente, sino cuando existe un cambio en el valor de alguna de las variables de la lista sensible (denominado **evento**).

Si se desea implementar un circuito combinacional, entonces todas las variables de entrada al circuito deben ser incluidas en la lista sensible.

Si se desea implementar un circuito secuencial modo reloj, entonces solo la señal de reloj y el o los *reset* deben ser incluidos en la lista sensible.

Sentencia *assign*

Un circuito secuencial puede ser tan simple como una compuerta. Para tales descripciones, el bloque *always* puede resultar en un código complejo y ser más adecuada la sentencia *assign*. Esta posee la siguiente sintaxis:

```
assign <variable> = function;
```

Por ejemplo, si se desea que la variable A sea la salida de la compuerta cuya función lógica es la *and* entre la variable B y C, entonces se podría describir como:

```
assign A = B and C;
```

De esta manera, con el bloque *always* y con la sentencia *assign* es posible la descripción de circuitos combinacionales y secuenciales para ser implementados en lógica digital.

ESTRUCTURAS DE CONTROL

Cuando se describe un hardware digital mediante un lenguaje, es conveniente contar con sentencias que permitan controlar la evaluación del proceso descrito dentro de un bloque *always*. En esta sección, se describirán las principales estructuras de control del lenguaje Verilog HDL.

Sentencia *if-else*

La sentencia *if-else* es evaluada de arriba hacia abajo, lo que resulta en una “priorización”. La primera condición verdadera determina la secuencia de sentencias que son evaluadas.

La sintaxis general de la sentencia *if-else* es:

```
if <condición 1> then
    {secuencia de sentencias}
else if <condición 2> then
    {secuencia de sentencias}
else if <condición 3> then
    {secuencia de sentencias}
else
    {secuencia de sentencias}
end
```

La priorización de las condiciones puede resultar en una implementación más compleja. Siempre que no se requiera una priorización de las condiciones, es conveniente la utilización de la sentencia *case*.

Sentencia *case*

La sentencia *case* evalúa todas las condiciones al mismo momento lo que resulta en una no priorización de las condiciones. Las condiciones tienen que ser excluyentes (solo una puede ser verdadera a la vez) y la cláusula *default* abarca todos los casos no explícitamente cubiertos.

La sintaxis general de la sentencia *case* es:

```
case {expresión}
    <condición 1>:
        {secuencia de sentencias}
    <condición 2>:
        {secuencia de sentencias}
    <condición 3>:
        {secuencia de sentencias}
    default:
        {secuencia de sentencias}
endcase
```

La condición *default* es opcional y cubre todos los casos que no son abarcados por las demás condiciones.

Sentencia *for*

La sentencia *for* es adecuada para la descripción de operaciones repetitivas. Hay que considerar que la sentencia *for* **no es implementada en hardware**, sino que es resuelta por el compilador antes de realizar la síntesis del circuito. Cuando se utiliza la sentencia *for* para sintetizar, no pueden especificarse rangos de iteración variables ni modificar arbitrariamente la variable de iteración dentro del ciclo de iteración.

La sintaxis general de la sentencia *for* es:

```
for (<valor inicial>,<expresión>,<incremento>) begin
    {secuencia de sentencias}
end
```

La sentencia *for* permite una estructura de control más adecuada, pero no reduce el hardware necesario para la implementación.

Con estas estructuras, se finaliza una somera descripción de las principales características del lenguaje Verilog HDL para síntesis. Para una revisión más profunda del lenguaje, se sugiere textos destinados a tal fin (7). A continuación, se utilizará el lenguaje Verilog HDL para la descripción, a modo de ejemplo, de un circuito digital que implementa la arquitectura de un multiplicador de Booth.

EJEMPLO

MULTIPLICADOR DE BOOTH

Uno de los más populares algoritmos de multiplicación es el algoritmo de Booth (8). Este algoritmo permite la multiplicación de números binarios en representación con signo a complemento a 2.

El algoritmo multiplica dos números binarios en complemento a 2 de n -bits, que denominaremos **r** para el multiplicando y **m** para el multiplicador, mediante la acumulación iterativa de los productos parciales. La acumulación parcial se almacena en un registro que llamaremos **p**. El algoritmo, como fue propuesto en (8), se puede expresar en pseudocódigo como:

```
a = r;
b = m;
```

```

p = 0;
low_m = 0;
for (i = 0, i < n, i++) begin
    case {b(1) low_m}
        "01":
            p = p + a
        "10":
            p = p - a
    endcase
    p >> 1
    b >> 1
end
result = p;

```

Este procedimiento puede implementarse como un *datapath* en hardware, requiriendo señales adicionales de control como ser:

- **start** una señal de entrada que indica con un cambio de nivel de 0 a 1 que las entradas tienen los factores que se desean multiplicar y que puede comenzar el proceso de multiplicación.
- **done**: una señal de salida que indica con 1 que la multiplicación ha finalizado y que la salida tiene el producto final. La señal “done” toma el valor 0 luego de que la señal *start* cambia de nivel de 0 a 1.

La Figura 2-11 muestra una arquitectura de hardware digital que implementa el algoritmo de Booth.

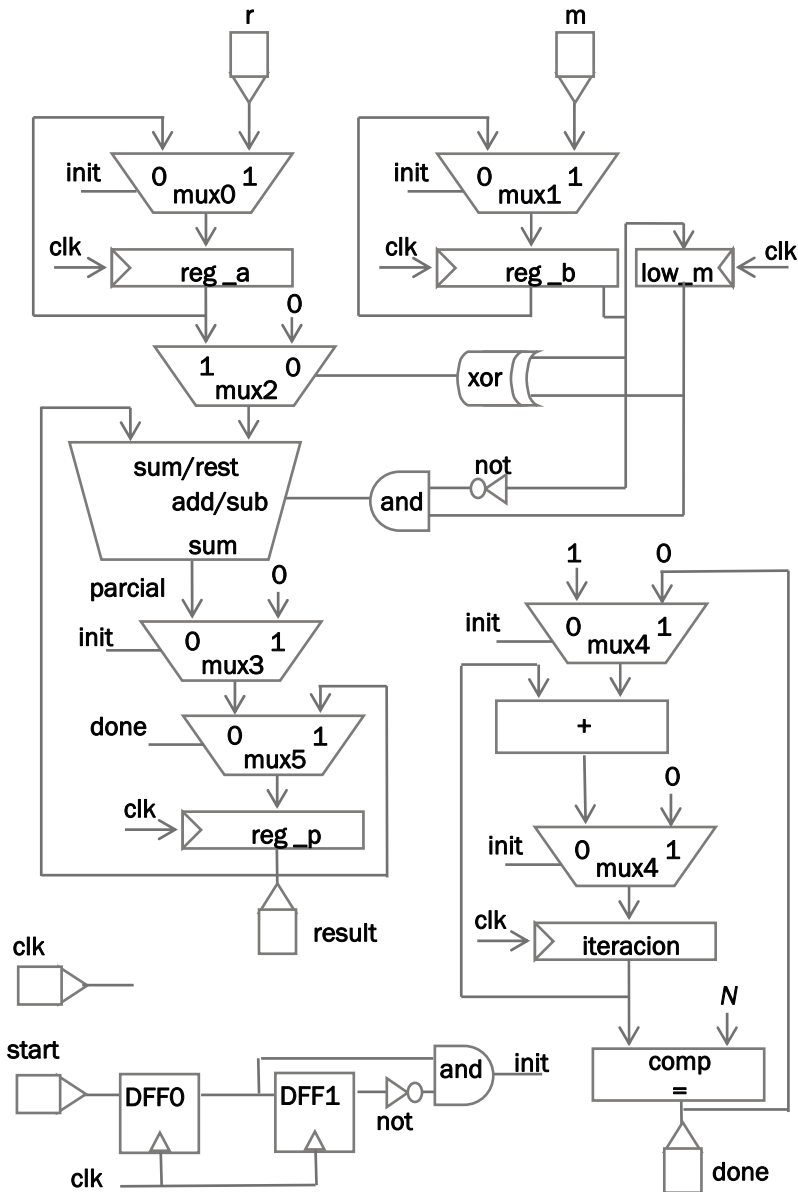


Figura 2-11: Arquitectura multiplicador de Booth.

NOTA: la arquitectura propuesta fue elegida por su facilidad para comprender la implementación del algoritmo, pero existen alternativas mucho más eficientes para la implementación del algoritmo de Booth que escapan al objetivo de este libro.

El multiplicador podría realizarse incorporando un parámetro en la descripción para un ancho genérico n de los factores. El producto tendría un ancho $2*n$. La inclusión de un parámetro es opcional en la definición del *module* con la palabra clave *parameter*. El código Verilog HDL podría ser:

```
module Booth

#(
    // Parameter Declarations
    parameter n = 8
)

(
    // Input Ports
    // Multiplicando r y Multiplicador m
    input [n-1:0] r,
    input [n-1:0] m,
    // reloj clk y senal start
    input clk, start,

    // Output Ports
    // Producto result
    output [2*n-1:0] result,
    // salida done
    output done
);

// Variables
reg [2*n:0] reg_p;
reg [n:0] reg_a;
reg [n-1:0] reg_b;
reg low_m;
reg DFF0;
reg DFF1;
reg donei;
reg [n:0] suma;
integer iteracion;
reg [n:0] out_mux2;
reg [n:0] out_mux3;
wire [n:0] parcial;

// Nodos
wire init;
```

```
wire cout;

// Logica de DFF0, DFF1
always@(posedge clk)
begin
    DFF0 <= start;
    DFF1 <= DFF0;
end
// Logica init
assign init = !DFF1 && DFF0;

// mux0 + reg_a
always@(posedge clk)
begin
    if (init == 1'b1)
        reg_a <= {r[n-1] ,r};
    else
        reg_a <= reg_a;
end

// mux1 + reg_b + low_m
always@(posedge clk)
begin
    if (init == 1'b1) begin
        reg_b <= m;
        low_m <= 1'b0;
    end
    else begin
        reg_b <= {reg_b[n-1],reg_b[n-1:1]} ;
        low_m <= reg_b[0];
    end
end

// mux2
always@(reg_a or reg_b or low_m)
begin
    if (reg_b[0] ^ low_m) out_mux2 = reg_a;
    else out_mux2 = 'b0;
end

assign parcial = reg_p[2*n : n];

// Sumador Restador
always@(*)
begin
    if (!reg_b[0] && low_m) suma = parcial + out_mux2;
    else suma = parcial - out_mux2;
end
```

```

//mux3
always@(*)
begin
    if (init) out_mux3 = 'b0;
    else out_mux3 = suma;
end

// mux5 y reg_p
always@(posedge clk)
begin
    if (done) reg_p <= reg_p;
    else reg_p <= {out_mux3[n],out_mux3[n:0],reg_p[n-1:1]};
end
assign result = reg_p[2*n-1:0];

// Contador de iteraciones
always@(posedge clk)
begin
    if (init) iteracion = 0;
    else if(!done) iteracion <= iteracion + 1;
end

always@(*)
begin
    if (iteracion >= 8) donei = 1'b1;
    else donei = 1'b0;
end
assign done = donei;
endmodule

```

CONCLUSIONES

Los dispositivos FPGA modernos ofrecen una muy alta capacidad de implementación digital, incluyendo el desarrollo de sistemas de procesamiento y de comunicaciones digitales completos en uno solo de estos dispositivos. La complejidad del diseño de tales sistemas requiere de lenguajes de descripción de hardware y de herramientas de diseño que brinden al diseñador niveles de adecuados de abstracción.

En este capítulo se describieron las principales características de dispositivos CPLD y FPGA para la implementación de circuitos digitales combinacionales y secuenciales. También se detalló la estructura básica del lenguaje de descripción de hardware Verilog HDL y se ilustró su utilización para síntesis mediante la codificación de una arquitectura de multiplicador de Booth.

CAPÍTULO 3

HERRAMIENTAS DE DISEÑO DE HARDWARE

HERRAMIENTAS DE DISEÑO DE HARDWARE

Todo diseño e implementación de hardware en dispositivos FPGA, requiere de herramientas adecuadas para su especificación, simulación y síntesis. Los fabricantes de dispositivos FPGA ofrecen plataformas de software para el soporte de sus productos y permiten ampliar sus prestaciones mediante paquetes de otras compañías tales como Mentor Graphics, Cadence y Synopsis, entre los más importantes.

Altera provee el software Quartus II (9) como soporte de sus dispositivos lógicos programables (CPLD y FPGA). La herramienta de desarrollo provista por Xilinx se denomina ISE (actualmente también Vivado) y brinda prestaciones similares a las de Quartus II. Estas herramientas de desarrollo se referencian generalmente como “*herramientas de diseño totalmente integradas*”, ya que abarcan todas las aplicaciones necesarias para realizar un proyecto en CPLD o FPGA, sin requerir ninguna otra aplicación adicional.

En lo que sigue, se hará referencia solo a la herramienta Quartus II de Altera, pero haciendo la aclaración que los demás fabricantes de dispositivos lógicos configurables ofrecen herramientas de desarrollo con prestaciones similares. Se realiza la elección de esta herramienta de diseño solo con el objetivo de facilitar el desarrollo de las metodologías detalladas en este libro y sin que ello implique una valoración comparativa de las prestaciones que una herramienta ofrece por sobre otra.

FLUJO DE DISEÑO CON DISPOSITIVOS LÓGICOS CONFIGURABLES

La realización de un diseño digital moderno involucra diferentes etapas que deben realizarse metodológicamente para llegar a una implementación exitosa. Todo diseño debe partir de una adecuada **especificación**, la cual determina todos los requerimientos que la implementación final debe satisfacer. Considerando la especificación, el **diseño** es creado, típicamente con alguna forma de representación gráfica o mediante un lenguaje de descripción de hardware. El diseño describe la estructura lógica del hardware a un Nivel de Transferencia de Registros (RTL) capaz de ser implementada en un hardware digital.

Con el diseño especificado, la **síntesis** puede llevarse a cabo. En este proceso, el diseño es expresado en recursos disponibles en el dispositivo lógico configurable. De esta manera, la etapa de la síntesis entrega como salida la expresión de la estructura lógica descrita por el diseño, pero en recursos propios del dispositivo configurable en que será implementado.

Por supuesto que deberá seleccionarse un dispositivo CPLD o FPGA que contenga tantos recursos lógicos como los necesarios para implementar la lógica obtenida por la síntesis. La síntesis expresa que para una determinada descripción se requiere determinado recurso, pero no determina qué recurso específico de todos los que posee el dispositivo CPLD o FPGA debe ser utilizado. El proceso de asignación de cada uno de los recursos lógicos expresados en la síntesis a recursos específicos del dispositivo CPLD o FPGA se denomina *place*. Debido a que la síntesis no solo determina los recursos lógicos necesarios, sino también su interconexión, es necesario interconectar los elementos lógicos asignados en el dispositivo CPLD o FPGA para que la implementación sea correcta. El proceso de asignación de recursos de interconexión del dispositivo FPGA para cumplir con la estructura lógica expresada en la síntesis se denomina *route*. Luego del proceso *place and route*, se posee un archivo de configuración del dispositivo FPGA que puede ser “bajado” al dispositivo en el proceso de **programación y configuración** para que pueda implementarse el diseño en el dispositivo FPGA.

Durante todo este flujo de diseño puede realizarse **simulación** para comprobar la correcta descripción del diseño y verificar que cumple con las especificaciones.

La Figura 3-1 muestra las etapas básicas del proceso de diseño cuando se utiliza el software Quartus II (9) de Altera y puede resumirse entonces en:

- **Diseño:** abarca la descripción del hardware a implementar mediante la utilización de lenguajes de descripción de hardware, esquemáticos, gráficos, etcétera.
- **Síntesis:** permite indicar las características del dispositivo a utilizar para generar un archivo que expresa el diseño en función de los recursos disponibles en dicho dispositivo.
- **Place and route:** especifica los recursos de dispositivo que se van a utilizar y la manera en que éstos se interconectarán. En diseños complejos, puede ser necesario que el diseñador realice cambios manuales a la especificación realizada por el software con el fin de cumplir con las restricciones temporales o de área.

- **Análisis temporal:** el software analiza si las frecuencias especificadas por el diseñador son satisfechas por la implementación sugerida. En caso de no satisfacerse, será necesaria la ejecución nuevamente de la etapa *place and route*. En caso de continuar sin satisfacerse, podrá ser necesario la intervención manual del diseñador o en último caso el rediseño, parcial o total del desarrollo.
- **Simulación:** permite que el diseñador proponga diferentes escenarios en un ambiente virtual para comprobar el comportamiento del desarrollo ante dichos escenarios. La simulación permite la búsqueda de errores de diseño.
- **Programación y configuración del dispositivo:** esta es la etapa final del proceso que permite transferir desde la computadora personal la configuración del dispositivo programable (dispositivo CPLD o FPGA). La configuración es almacenada como un archivo en la computadora personal que contiene toda la información necesaria para configurar un dispositivo lógico y que este se comporte como el hardware que el diseñador especificó en la etapa de diseño. Los archivos **.pof** y **.sof** son los archivos de correspondientes si se desea la configuración mediante la memoria *flash* o directamente a la FPGA.

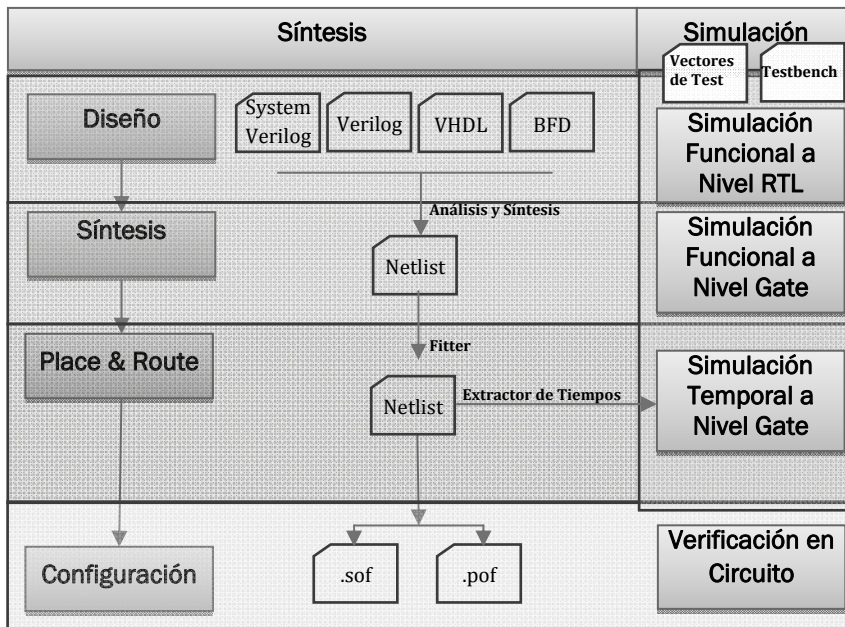


Figura 3-1: Flujo de diseño en Quartus II.

Resumiendo, el objetivo del software de desarrollo es poder compilar un archivo de descripción de hardware (ya sea en forma de esquemático o archivo de descripción de

hardware (escrito, por ejemplo, en Verilog HDL o VHDL) a un archivo que permita la configuración del dispositivo CPLD o FPGA de manera tal que se comporte como el hardware descrito. La herramienta deberá abarcar posibilidades para la verificación y simulación del diseño y utilidades para transferir la configuración al dispositivo FPGA.

Muchas de las funciones realizadas por el software Quartus II pueden ser realizadas por software ofrecidos por diversas empresas (Altera, al igual que sucede con Xilinx, hace referencia a los productos de estas compañías como *third party products*). Es por este motivo que Altera hace compatible a Quartus II con diversas herramientas provistas por Mentor Graphics, Cadence, Synopsis, entre otras.

Sin embargo, Quartus II es generalmente imprescindible, cuando se trabajan con dispositivos de Altera, en la etapa del proceso *place and route*.

PLATAFORMAS Y LICENCIAS DE QUARTUS II

Quartus II está disponible para diversos sistemas operativos, entre ellos Windows y Linux, con varias formas de licencias para su utilización. Por ejemplo, Altera ofrece una **versión gratuita** denominada **Quartus II Web Edition**, disponible para dos plataformas: una para Microsoft Windows y otra para Linux. La versión Quartus II Web Edition **no requiere licencia**. En este libro, utilizaremos la versión Web Edition para Microsoft Windows, pero en el apéndice se detallará la instalación de la versión Linux para aquellos lectores que no cuentan con una licencia válida de Microsoft Windows.

La versión con licencia es denominada **Quartus II Subscription Edition** y posibilita dos alternativas principales: (1) *nodo fijo* en la cual el software Quartus II solo puede ser ejecutado en el computador en el que se instaló la licencia y (2) *nodo flotante* en la cual existe un servidor de licencias que es consultado por los computadores que solicitan oportunamente licenciamiento para ejecutar el software Quartus II instalado localmente.

En (10), se listan las diferencias entre las versiones *Web Edition* y la *Subscription Edition* siendo las más importantes: el licenciamiento de las propiedades intelectuales (IP, del inglés *Intellectual Property*) de diferentes componentes de biblioteca y el soporte de todos los dispositivos de Altera (CPLD y FPGA) en la versión *Subscription Edition*.

PROYECTOS EN QUARTUS II

Quartus II utiliza el concepto de **proyecto** para contener toda la información referente a un determinado diseño. Un proyecto es definido como todo el conjunto de archivos de diseño y de bibliotecas requeridos para crear un archivo de configuración del dispositivo CPLD o FPGA. Generalmente, todos los archivos son almacenados en una carpeta o directorio destinado al proyecto, aunque también pueden referenciarse archivos fuera de dicha carpeta. En un proyecto, existe un archivo de **máxima jerarquía** y diferentes archivos de diseño de menor jerarquía, archivos de simulación, de documentación, de reporte, de configuración, de asignación de pines, etc. El proyecto también debe especificar el dispositivo CPLD o FPGA destino en el que se implementará el diseño. Tanto el archivo de máxima jerarquía como el dispositivo destino pueden modificarse en cualquier etapa del proceso de diseño.

Todas las configuraciones del proyecto son almacenadas en un único archivo denominado **Quartus II settings files** con extensión **.QSF**.

Los procesos de síntesis y *place and route* almacenan la información que generan en archivos ubicados en una subcarpeta con el nombre “db”, dado que contiene la base de datos (*database*, en inglés) de la compilación del proyecto.

La creación de un proyecto, como muchas otras tareas, puede realizarse mediante la utilización de comandos y *scripts* Tcl. Sin embargo, la manera más simple, y utilizada en este libro, es mediante la opción de “Guía para la creación de nuevo proyecto” (*New Project Wizard*). Esta opción está disponible cuando se utiliza la interfaz gráfica de usuario (GUI) del Quartus II, la cual será descrita a continuación.

INTERFAZ GRÁFICA DE QUARTUS II

La interfaz gráfica de Quartus II de Altera, puede parecer compleja en un principio debido a la variedad de vistas e información que despliega en pantalla. Este entorno tiene la ventaja de poder ser reconfigurado según las necesidades o preferencias del usuario.

Como los demás entornos de desarrollo, presenta una pantalla principal donde se despliegan todo el resto de las herramientas que posee, las cuales incluyen editores de texto, editores de diagramas de bloque, distintas visualizaciones del proyecto y sus archivos, consolas de comandos, de mensajes y de errores, y herramientas de síntesis, análisis, simulación y descarga a los dispositivos.

Al iniciar el software Quartus II, se visualiza una pantalla de bienvenida la cual permite seleccionar un proyecto que se ha desarrollado en corridas del Quartus II anteriores o abrir la guía para la creación de un proyecto nuevo. La pantalla de bienvenida puede cerrarse mediante el botón a tal fin ubicado en la esquina superior derecha (Figura 3-2).

Al cerrar la pantalla de bienvenida, queda activa la pantalla de interfaz con el usuario de Quartus II. La Figura 3-3 muestra la pantalla principal de Quartus II con la definición de cada uno de los sectores que la integran.



Figura 3-2: Pantalla de bienvenida de Quartus II.

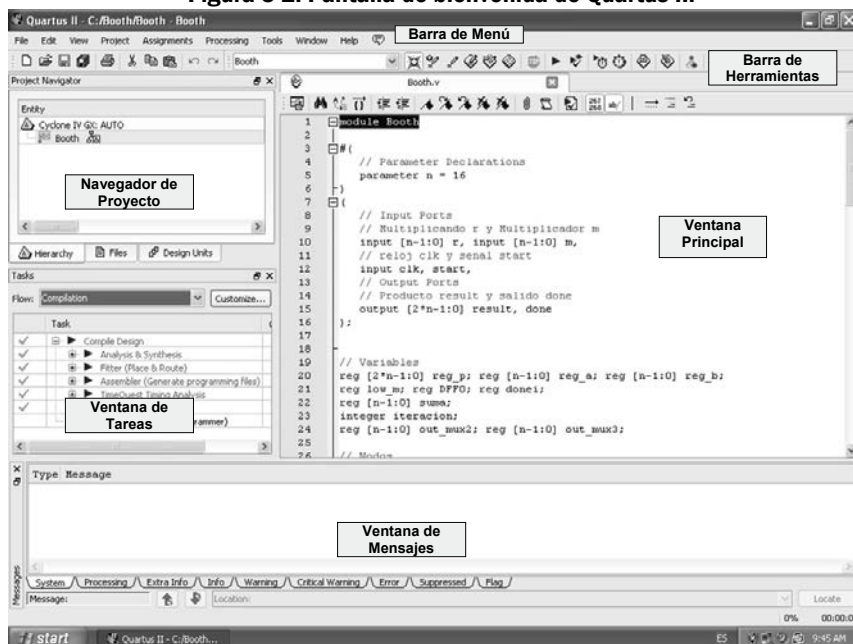


Figura 3-3: Interfaz gráfica de Quartus II.

Las principales barras y ventanas de la interfaz de usuario de Quartus II son:

- **Ventana principal:** es la que ocupa mayor espacio de la interfaz y es utilizada para visualizar o editar los archivos de diseño y visualizar los archivos de reporte de las diferentes herramientas del Quartus II.
- **Navegador de proyecto:** provee información de la estructura del proyecto que está en diseño y de los archivos asociados.
- **Ventana de tareas:** permite un rápido acceso a las herramientas de diseño más frecuentemente utilizadas y características del software, al mismo tiempo que ofrece una guía del típico proceso de diseño.
- **Ventana de mensajes:** lista los mensajes producidos durante el proceso de compilación.
- **Barra de menú:** permite el acceso a menús desplegables para la selección de diferentes comandos o aplicaciones. Es una barra de menú típica de una aplicación de Microsoft Windows (o Linux) cuyas opciones pueden modificarse de acuerdo a la herramienta que se esté seleccionado en Quartus II.
- **Barra de herramientas:** permite un rápido acceso a diferentes alternativas de la herramienta que se está desplegando en la pantalla principal.

Las ventanas o barras pueden ser reubicadas o personalizadas como tradicionalmente se realiza en todas las aplicaciones de Microsoft Windows o Linux.

Durante el transcurso del libro, se identificarán diversas herramientas con el ícono que aparece en la barra de herramientas, el cual permite un acceso rápido, pero se indicará también la selección que deba realizarse para llegar a tal herramienta en la barra de menú.

CREACIÓN DE PROYECTO

La manera más adecuada de comenzar el primer proyecto es mediante la guía de nuevo proyecto. Se puede acceder a esta opción a través de la barra de menú mediante **File -> New Project Wizard**, o la opción **Open New Project Wizard** en la ventana de tareas (Figura 3-4).

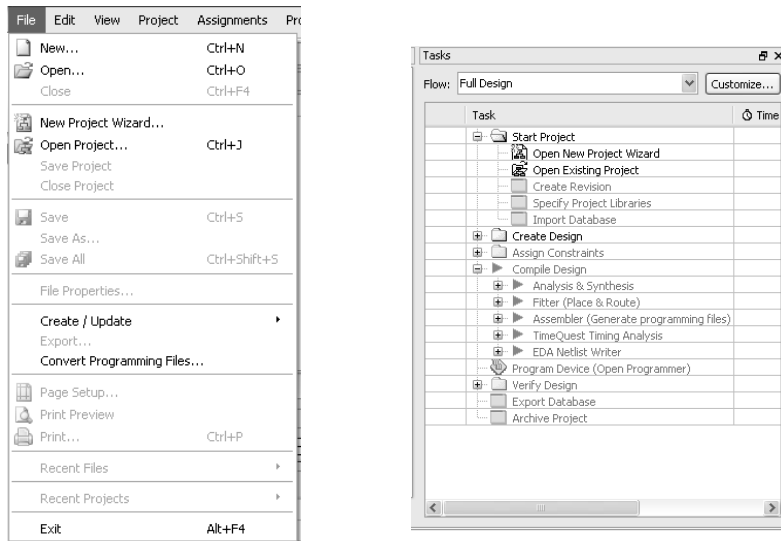


Figura 3-4: Alternativas para la creación de un proyecto nuevo mediante la guía.

Una vez seleccionada la opción Guía de nuevo proyecto, se habilitará una secuencia de pantallas que requerirán los datos principales para la creación del mismo. La primera pantalla es una introducción (Figura 3-5) que informa las diferentes etapas del proceso de creación. Debe seleccionarse el botón **Next >**.

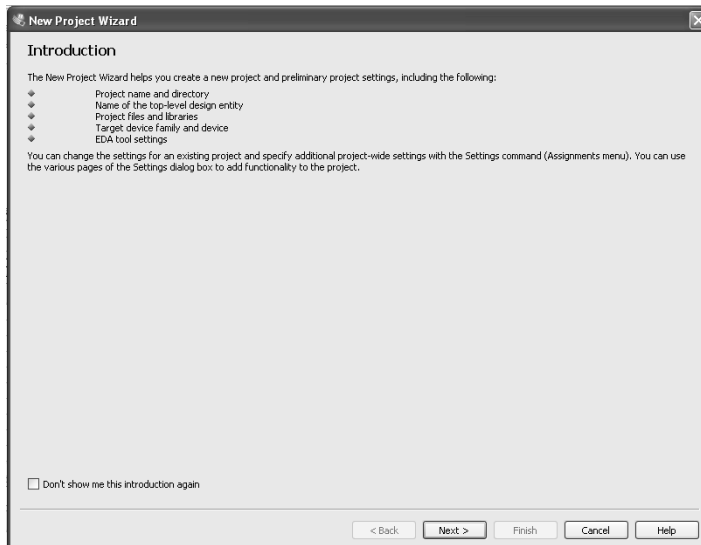


Figura 3-5: Pantalla de introducción a la creación de proyecto.

La siguiente pantalla solicita la definición del directorio, del nombre del proyecto y de la entidad de mayor jerarquía (debe tenerse en cuenta mayúsculas y minúsculas en el nombre). Los botones “...” permiten la navegación para la selección adecuada de los directorios o archivos. En la Figura 3-6, se define el directorio “C:/Booth” como directorio de trabajo, el nombre del proyecto “Booth” y el nombre de la entidad de mayor jerarquía como “Booth”. Luego de realizar todas estas definiciones, se debe seleccionar el botón **Next >**. En caso de no existir el directorio, consultará si se desea crear, a lo que debe seleccionarse **Yes**.



Figura 3-6: Definición de directorios y estructura del proyecto.

La siguiente pantalla solicitará incorporar todos los archivos que deben ser incluidos en el proyecto. En caso de contar los archivos de descripción del diseño en el disco, pueden seleccionarse mediante el botón de navegación “...” e incorporados mediante el botón **Add All**. En el ejemplo propuesto, al no existir un archivo creado, se ingresará el nombre “Booth.v” en el campo “File Name” y se seleccionará el botón **Add**. En la ventana de texto inferior, aparecerá el nombre de archivo y descripción de tipo (Figura 3-7). Luego de esto, seleccionar el botón **Next >**.

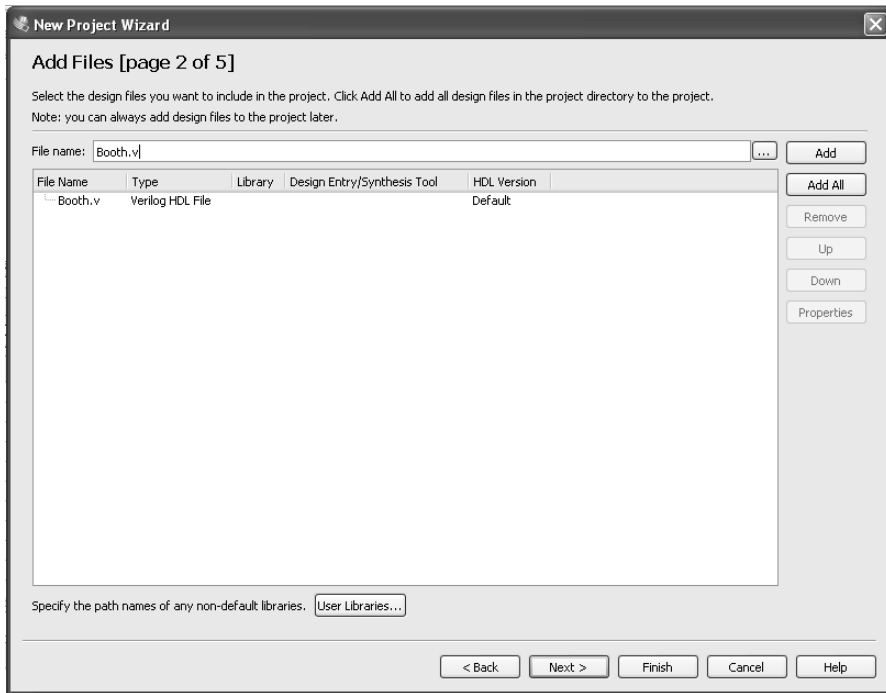


Figura 3-7: Inclusión de archivos del proyecto.

Una vez ingresados los archivos del proyecto, la guía de creación del nuevo proyecto permitirá definir el dispositivo FPGA destino de la implementación (Figura 3-8). La pantalla permite especificar la familia y el modelo de todos los dispositivos de la familia Altera. Por ejemplo, si se utiliza la placa **Bemicro SDK**, entonces debe seleccionarse la familia **Cyclone IV E** y el dispositivo **EP4CE22F17C7**. Puede darse el caso de que no se cuente con la definición del dispositivo en esta etapa del diseño, pudiendo dejar que Quartus II lo seleccione automáticamente de acuerdo al resultado de la síntesis del diseño. Esta selección puede ser modificada en cualquier momento posterior. Luego de esto, se puede pulsar el botón **Next >**.

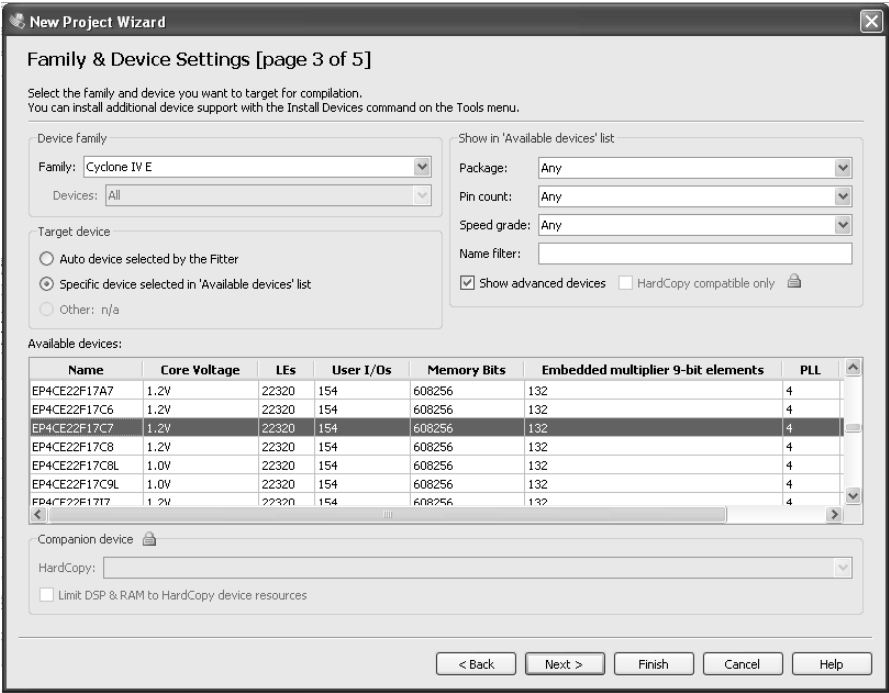


Figura 3-8: Selección de la familia y dispositivo.

La cuarta pantalla de la guía de creación de proyecto nuevo permite especificar herramientas de otros fabricantes para la realización de diferentes etapas del proceso de diseño. Para el ejemplo propuesto, se puede especificar la simulación mediante el software ModelSim para dispositivos Altera, brindado gratuitamente por Altera y denominada **ModelSim-Altera**. Seleccionar el formato Verilog HDL y que ejecute la simulación automáticamente después de la compilación (Figura 3-9). Al realizar esto, se puede seleccionar el botón **Next >**.

La última pantalla de la guía de creación de proyecto nuevo presenta un resumen de las diferentes configuraciones que se realizaron en las pantallas previas (Figura 3-10). Solo resta seleccionar el botón **Finish** para finalizar la creación del proyecto.

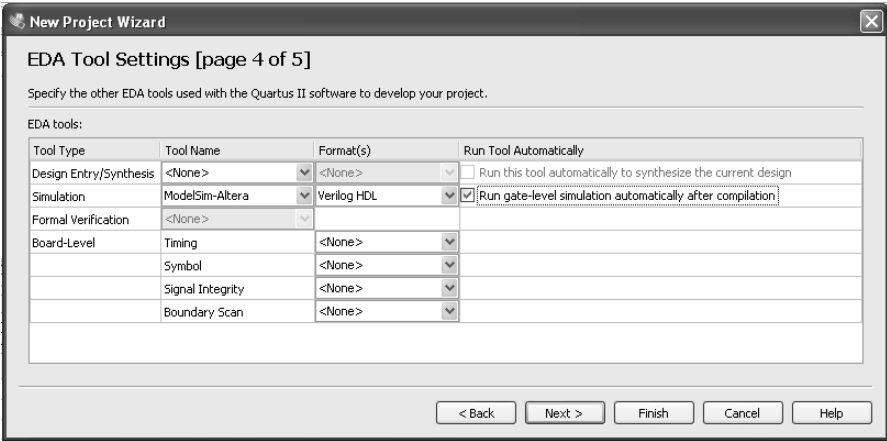


Figura 3-9: Definición de herramientas de diseño de otros fabricantes a utilizar en el proyecto.

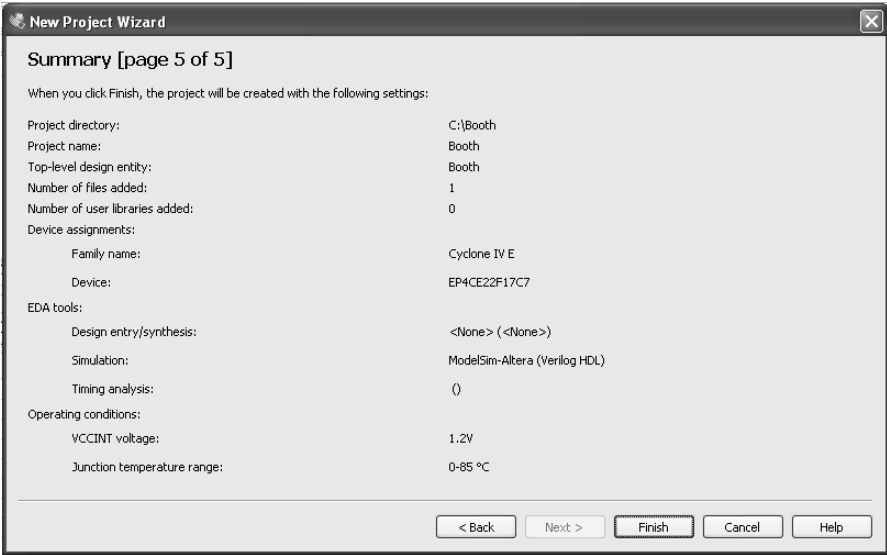


Figura 3-10: Pantalla de resumen de la creación de proyecto.

Finalizada la creación del proyecto, se regresa a la interfaz de usuario de Quartus II con la ventana del navegador de proyecto especificando el nombre del proyecto y del dispositivo CPLD o FPGA seleccionado (Figura 3-11).

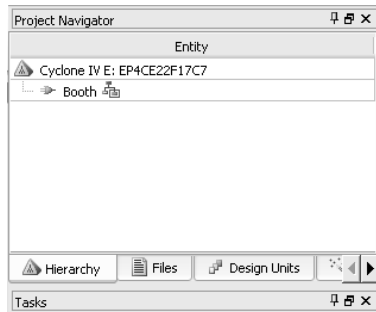
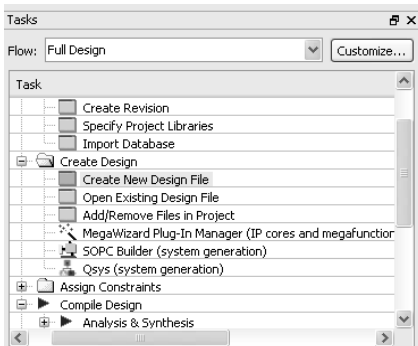


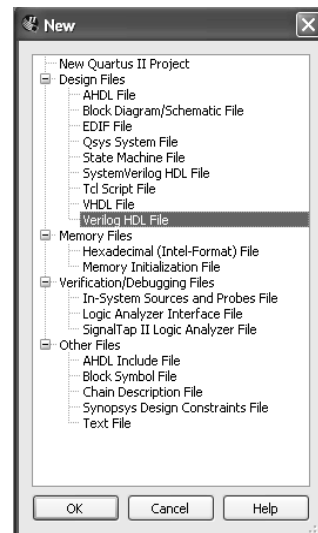
Figura 3-11: Pantalla de navegación con el proyecto creado.

CREACIÓN DEL ARCHIVO DE DISEÑO

Definida la estructura del proyecto, es necesaria la incorporación del archivo de descripción de hardware que contenga el diseño a implementar. Quartus II posee un editor de texto sensitivo que permite la entrada de diseño y remarca las palabras o estructuras del lenguaje para facilitar su edición. La creación de un nuevo archivo de diseño se realiza mediante la selección de **Create New Design File** de la ventana de tareas (Figura 3-12a) o seleccionado en la barra de menú el comando **File > New**. Ambas opciones abren la ventana para seleccionar el tipo de archivo que se desea crear (Figura 3-12b). Para la implementación del multiplicador Booth propuesto en el Capítulo 2, seleccionar la opción **Verilog HDL File**.



a) Selección de Creación de nuevo archivo de diseño.



b) Selección de tipo de nuevo archivo de diseño a crear.

Figura 3-12: Creación de nuevo archivo de diseño.

La creación del archivo de diseño abre en la ventana principal un archivo de texto sin nombre. Es conveniente nombrar y guardar este archivo con el nombre del archivo que se desea para la mayor jerarquía del diseño (Booth.v). Para ello, de la barra de menú debe seleccionarse el comando **File > Save As...** que abrirá una pantalla de diálogo de Windows que permitirá asignarle el nombre **Booth.v** al archivo de diseño.

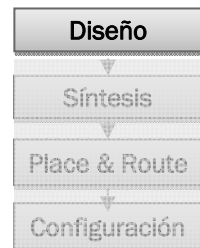
La Tabla 3-1 describe las diferentes extensiones que pueden tener, según su tipo, los archivos de diseño que soporta Quartus II.

TABLA 3-1: EXTENSIONES DE LOS ARCHIVOS DE DISEÑO DE HARDWARE SOPORTADO POR QUARTUS II

Extensión	Tipo	Descripción
.bdf	Archivo de diseño por bloques	Archivo de diseño esquemático creado con el editor de bloques.
.edf .edif	Archivo de entrada tipo EDIF	Un archivo de <i>netlist</i> EDIF generado por cualquier escritor estándar de <i>netlist</i> EDIF.
.gdf	Archivo de diseño gráfico	Un archivo de diseño esquemático creado con el editor gráfico de MAX+PLUS II (software anterior al Quartus II).
.tdf	Archivo de diseño AHDL	Un archivo de diseño escrito en <i>Altera Hardware Description Language</i> (AHDL).
.v .vlg .verilog	Archivo de diseño Verilog HDL	Un archivo de diseño que contiene lógica definida con Verilog HDL.
.vh .vhd .vhdl	Archivo de diseño VHDL	Un archivo de diseño que contiene un diseño de lógica definida mediante VHDL.
.vqm	Archivo de mapeo de Verilog HDL Quartus	Un archivo de formato <i>netlist</i> Verilog HDL generado por el software Synplicity Synplify o por Quartus II.

ENTRADA DEL DISEÑO (*DESIGN ENTRY*)

La entrada del diseño se realiza mediante la edición del archivo de texto de descripción de hardware en la ventana principal. El editor permitirá visualizar las palabras reservadas del lenguaje en forma resaltada para facilitar la edición y comprensión del código. Para implementar el ejemplo planteado en el Capítulo 2, se debe ingresar el



código del multiplicador de Booth (Figura 3-13). Es recomendable guardar periódicamente el archivo mediante el comando de la barra de menús **File > Save** o mediante el ícono de acceso directo “”.

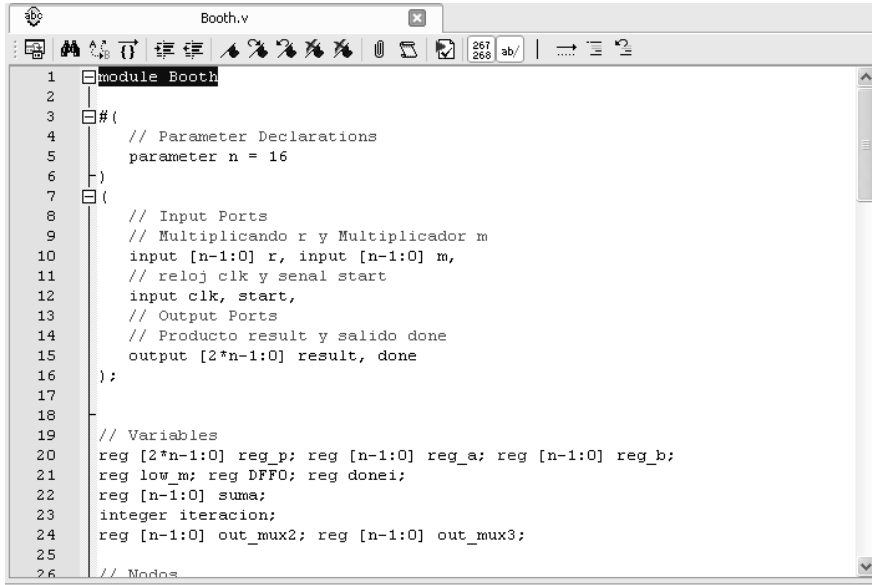


Figura 3-13: Entrada de diseño en Quartus II.

Una herramienta que ofrece Quartus II para facilitar la correcta especificación de la estructuras de diversos lenguajes, es **Insert Template** (“”). Esta herramienta permite insertar estructuras y sentencias de diversos lenguajes, indicando los campos que deben ser especificados o definidos por el diseñador.

COMPILACIÓN DEL DISEÑO

Conforme al flujo de diseño de Quartus II, la siguiente etapa es la compilación del proyecto (Figura 3-1). Debido a que la compilación abarca varias etapas del proceso de diseño, pueden seleccionarse, en principio, dos alternativas principales:

- **Start Compilation** “”: realiza todas las etapas del diseño. Este proceso puede demorar algunos minutos y es conveniente realizarlo cuando la entrada de diseño fue depurada de los errores de código. Puede ejecutarse desde la



ventana de Tareas, el ícono en la ventana principal (“▶”), del comando en la barra de menús mediante **Processing > Start Compilation** o presionando las teclas **Ctrl+L**.

- **Start Analysis & Synthesis** “▶” : realiza solo una verificación de la consistencia del código de diseño y sintetiza el diseño a recursos del dispositivo CPLD o FPGA. Esta es una de las etapas que se ejecuta cuando se selecciona la opción “**Start Compilation**” por lo que se realiza en menos tiempo y por consiguiente es más adecuada cuando se depuran los errores de código. Puede ejecutarse desde la ventana de tareas, el ícono de la ventana principal (“▶”), del comando en la barra de menús mediante **Processing > Start > Start Analysis & Synthesis** o presionando las teclas **Ctrl+K**.

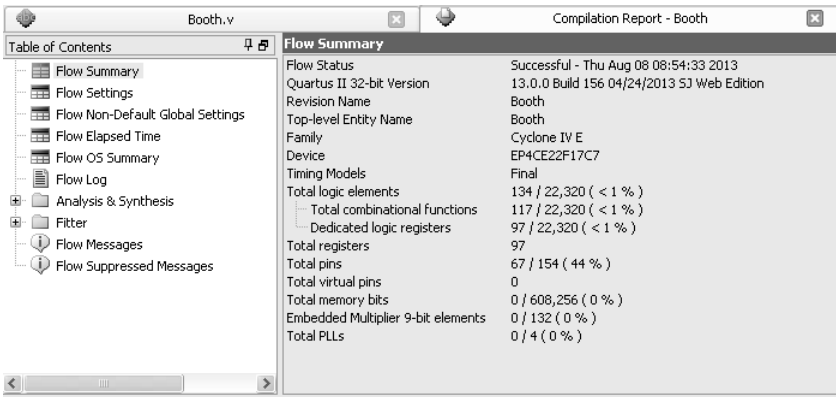
La opción **Start Compilation** producirá también las etapas de **Fitter (Place & Route)** y la extracción de los archivos de simulación temporal del diseño implementado.

ETAPA FITTER (PLACE & ROUTE)

La síntesis del diseño realiza una base de datos expresada en recursos (macroceldas o elementos lógicos) del dispositivo destino (CPLD o FPGA). Estos recursos deben ser asignados a los que el dispositivo realmente posee y deben ser también interconectados adecuadamente para implementar el hardware descrito en el diseño. Este proceso es realizado por la etapa **Fitter**. Para la ejecución de esta etapa, se debe ejecutar el comando de la barra de menús: **Processing > Start > Start Fitter**, o desde la ventana de tareas.

Sin embargo, es conveniente que este proceso sea realizado en una compilación completa para incluir todas las demás etapas de análisis y extracción temporal. En este libro, se realizarán comandos **Start Analysis & Synthesis** hasta depurar el código de diseño de errores. Luego, se realizará una compilación completa **Start Compilation** para abarcar todas las etapas de diseño (incluyendo **Fitter**). Dicha compilación finaliza con un reporte de compilación (“▶”) con los recursos del dispositivo utilizados y los requerimientos temporales que satisface (Figura 3-14).





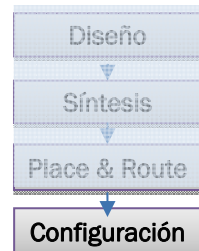
Flow Summary		
Flow Status	Successful - Thu Aug 08 08:54:33 2013	
Quartus II 32-bit Version	13.0.0 Build 156 04/24/2013 5J Web Edition	
Revision Name	Booth	
Top-level Entity Name	Booth	
Family	Cyclone IV E	
Device	EP4CE22F17C7	
Timing Models	Final	
Total logic elements	134 / 22,320 (< 1 %)	
Total combinational functions	117 / 22,320 (< 1 %)	
Dedicated logic registers	97 / 22,320 (< 1 %)	
Total registers	97	
Total pins	67 / 154 (44 %)	
Total virtual pins	0	
Total memory bits	0 / 608,256 (0 %)	
Embedded Multiplier 9-bit elements	0 / 132 (0 %)	
Total PLLs	0 / 4 (0 %)	

Figura 3-14: Reporte de compilación en Quartus II.

CONFIGURACIÓN

Asignación de pines (Pin Planner)

Antes de realizar la programación del dispositivo CPLD o FPGA, es necesario especificar las señales que estarán disponibles en cada uno de los pines del dispositivo. Este proceso de asignación puede realizarse en diferentes etapas del diseño, pero es conveniente efectuarlo luego de una compilación para que la herramienta liste todas las señales de entradas y salidas que deben ser asignadas a los pines del dispositivo. La aplicación utilizada para este proceso es el *Pin Planner* (Figura 3-15). Se puede acceder a la aplicación mediante el comando de la barra de menú **Assignments > Pin Planner**, o a través del ícono de la ventana principal “”.



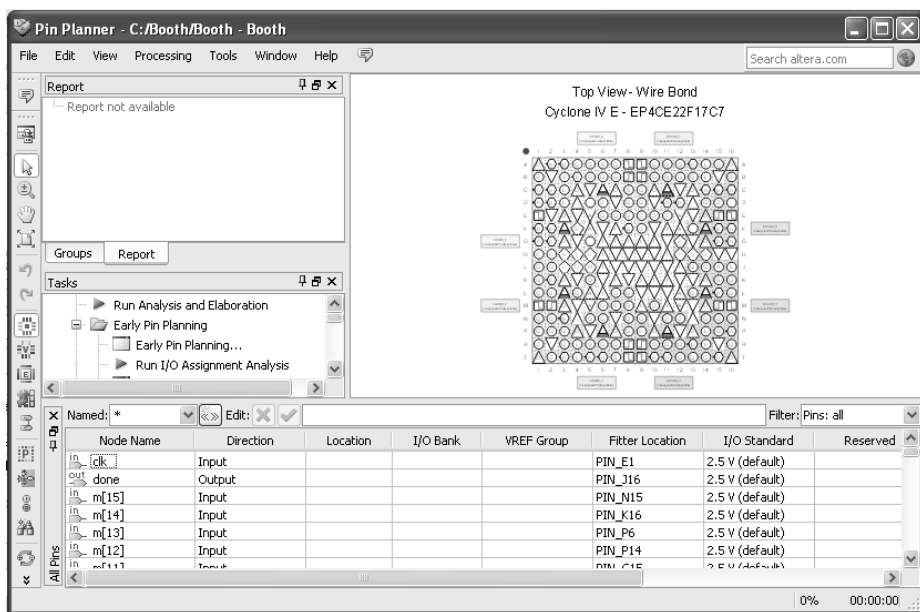


Figura 3-15: Pantalla Pin Planner de Quartus II.

La asignación se realiza mediante la selección en la tabla inferior del campo “Location” del pin correspondiente. La asignación de pines dependerá de la placa de desarrollo que se posea y de la conexión que el fabricante realizó en dicha placa. Generalmente, todos los kits de desarrollo cuentan con un archivo brindado por el fabricante que incluye los comandos para la configuración automática de la asignación de pines. Debe tenerse la precaución, al utilizar estos archivos de comandos, nombrar los pines del diseño efectuado con el mismo nombre que el fabricante le dio a cada uno de los pines correspondientes.

Es necesaria la conexión de la lógica implementada a los pines adecuados para un correcto funcionamiento del sistema. Por ejemplo, en el kit de desarrollo Bemicro SDK, la entrada de reloj está conectada al pin PIN_E1.

Programador del dispositivo “”

La programación del dispositivo se realiza mediante el programador de Quartus II (denominado *programmer*). Se puede acceder al programador mediante la barra de menú **Tools > Programmer**, o a través del ícono de la ventana principal “”.

La pantalla principal (Figura 3-16) mostrará los dispositivos FPGA o CPLD conectados al computador personal a través de la interfaz JTAG. En la parte superior de la pantalla, indicará el nombre del archivo de configuración (en el ejemplo propuesto, **Booth.sof**). En caso de que se requiera modificar el archivo, puede realizarse mediante la navegación habilitada por el

botón **Add File**. Los archivos con extensión **.sof** son necesarios para la configuración directa de los dispositivos CPLD o FPGA desde el computador personal, mientras que los de extensión **.pof** son archivos para programar la memoria EPROM de configuración de dispositivos FPGA, generalmente incluida en las placas de desarrollo (*kits*).

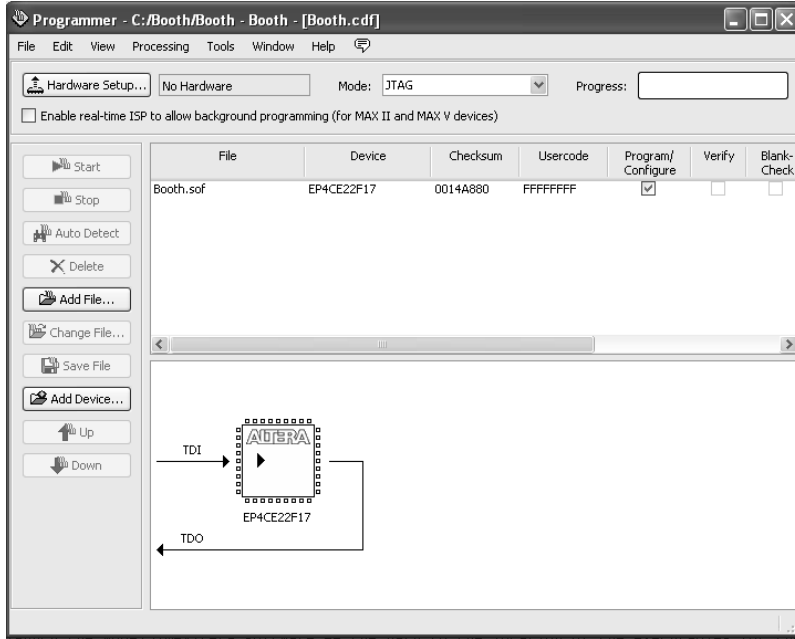


Figura 3-16: Pantalla Programmer de Quartus II.

La configuración del dispositivo CPLD o FPGA se realizará mediante la selección del botón **Start**. Finalizado el proceso de configuración, cuyo progreso es indicado mediante la barra **Progress**, el dispositivo habrá sido configurado de no haber existido ningún inconveniente.

SIMULACIÓN DEL DISEÑO

La simulación de un diseño permite analizar su comportamiento ante determinadas condiciones de las entradas. De esta manera, se puede comprobar si el diseño producirá las salidas adecuadas según especificación bajo diferentes condiciones de las señales de entrada. En este libro, definiremos **escenario** a la combinación de valores de las entradas y sus cambios en el tiempo a los que se somete el diseño para determinar el valor de las salidas. Entonces, el proceso de simulación estará destinado a comprobar si el diseño produce las salidas correctas ante diferentes escenarios de simulación.

Puede ocurrir que, luego de simular el diseño, se decida implementarlo en hardware y notar que el circuito no se comporta correctamente. El error ahí no es un problema del simulador, sino que no se consideró adecuadamente el escenario que, posteriormente en la realidad, produce una salida incorrecta.

La simulación permite determinar si existe algún inconveniente en el diseño, pero no garantiza que el hardware implementado funcionará correctamente según la especificación. Con la simulación, se puede decir cuándo existe una falla de diseño, pero no su ausencia.

La simulación del diseño se realiza mediante la utilización de un software simulador. Este genera todos los escenarios en forma virtual y determina el comportamiento del diseño de acuerdo al **modelo** que lo caracteriza.

El modelo es una representación que permite determinar, con cierto grado de precisión, la respuesta que puede tener la implementación real ante determinados escenarios del ambiente.

Modelo: Esquema teórico, generalmente en forma matemática, de un sistema o de una realidad compleja que se elabora para facilitar su comprensión y el estudio de su comportamiento.

De esta manera, no es necesario, en principio, generarle un escenario real a la implementación del diseño para comprobar la ausencia de fallas, sino que puede generarse en forma virtual considerando el modelo del diseño. La generación virtual de la simulación permite “controlar” todo el desarrollo de la depuración del diseño, mejorando su eficiencia.

Como puede observarse en la Figura 3-1, el proceso de simulación puede realizarse en las diferentes etapas del diseño. A medida que se avanza en estas etapas, el modelo será más preciso y, en consecuencia, la simulación también, aunque el tiempo requerido por el simulador puede incrementarse considerablemente.

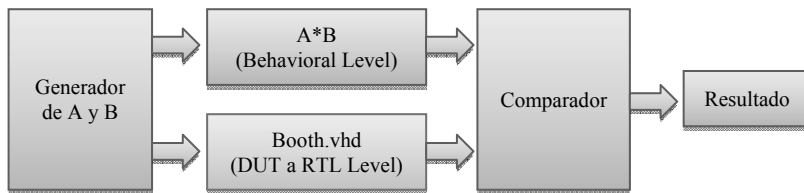
SIMULACIÓN DE DISEÑOS DIGITALES CON LENGUAJES DE DESCRIPCIÓN DE HARDWARE: *TESTBENCH*

Como se ha descripto en la sección anterior, un diseño digital puede representarse mediante la utilización de lenguajes de descripción de hardware. De manera similar, todo el ambiente en el cual funcionará el diseño puede representarse mediante un lenguaje adecuado. Específicamente, lenguajes de descripción de hardware como VHDL y Verilog HDL permiten la representación a nivel de comportamiento de sistemas útiles para la generación de estímulos a los que estará sometido un determinado diseño. Existen otros lenguajes adecuados para la generación de escenarios para la simulación, como puede ser el lenguaje “SystemC” (11).

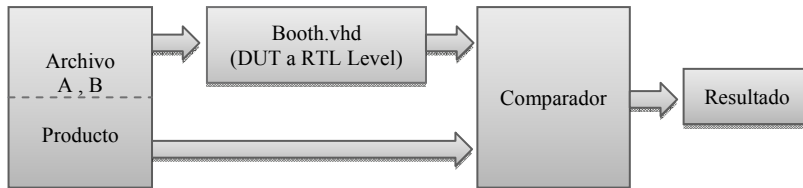
Un *testbench* (banco de pruebas, en español) es una descripción del ambiente que genera los diferentes escenarios que son aplicados al diseño bajo prueba. Los escenarios pueden ser producidos por un modelo del ambiente en el cual funcionará el diseño o por la especificación de estímulos que serán inyectados al diseño. Un *testbench* puede también tener una descripción del diseño a nivel de comportamiento, que permita comparar el funcionamiento correcto del diseño a sintetizar (denominado generalmente DUT, *Design Under Test* en inglés, **Diseño bajo prueba** en español).

Por ejemplo, para simular el algoritmo de Booth propuesto en el capítulo anterior, podrían utilizarse diferentes alternativas:

- Comparación de la arquitectura con la función multiplicación descrita a nivel de *Behavioral* y resuelta por la computadora personal durante la simulación.



- Generación de estímulos almacenados en archivo con las correspondientes salidas.



Cuando existe una forma de comparar la respuesta de un DUT, entonces el *testbench* puede realizar la simulación y verificación de las salidas e indicar al diseñador solo cuando la comparación no es exitosa.

SIMULACIÓN EN EL FLUJO DE DISEÑO DE QUARTUS II

La Figura 3-1 muestra las diferentes etapas del proceso de diseño en las cuales se puede realizar la simulación. A partir de la versión 10.0, el software Quartus II no tiene incorporada una aplicación para simular diseños. Versiones anteriores tenían un simulador con capacidades restringidas que solo era adecuado para sistemas simples.

A partir de la versión 10.0 de Quartus II, Altera decidió discontinuar la incorporación del simulador propio y sugerir para el proceso de simulación el software ModelSim de la firma Mentor Graphics. Existe una versión gratuita del software que puede bajarse desde el sitio oficial de Altera que se denomina **ModelSim-Altera Starter Edition** que no requiere licencia. En

este libro, se utilizará dicha versión para los desarrollos propuestos, aun cuando existen otros simuladores soportados por Quartus II (Tabla 3-2).

TABLA 3-2: SIMULADORES SOPORTADOS POR QUARTUS II DE ALTERA

Fabricante	Simulador	Plataforma
Aldec	Active-HDL	Windows
Aldec	Riviera-PRO	Windows, Linux
Cadence	Incisive Enterprise	Linux
Mentor Graphics	ModelSim-Altera	Windows, Linux
Mentor Graphics	ModelSim PE	Windows
Mentor Graphics	ModelSim SE	Windows, Linux
Mentor Graphics	QuestaSim	Windows, Linux
Synopsys	VCS/VCS MX	Linux

De acuerdo a la precisión que se desee lograr, la simulación puede realizarse a diferentes niveles de abstracción (Figura 3-1):

- **Nivel RTL:** En un primer momento, podría sugerirse realizar una simulación del código del lenguaje de descripción de hardware como lo podría realizar el diseñador, es decir, ir evaluando las diferentes sentencias del código de acuerdo a diferentes valores de las entradas. Este nivel de simulación se denomina, en el flujo de diseño de Quartus II, **Nivel de Simulación RTL**.
- **Nivel Gate Functional:** se realiza después de que el código del lenguaje de descripción de hardware fue compilado a recursos del dispositivo CPDL o FPGA. Este nivel de simulación es más preciso debido a que el sintetizador debió reconocer el hardware descrito y expresarlo en recursos lógicos reales contenidos en los dispositivos lógicos configurables.
- **Nivel Gate Temporal:** se realiza luego de que el proceso de *place and route* asignó todos los recursos lógicos y de interconexión a los disponibles en el dispositivo CPLD o FPGA. Para realizar el modelo temporal del diseño, es necesario extraer todos los parámetros temporales surgidos como consecuencia de la asignación de los recursos. Para la extracción del archivo de simulación a nivel de *Gate Temporal*, Altera brinda el software **TimeQuest Timing Analyzer**.

Flujos de simulación en Quartus II

En la Figura 3-1, se muestran diferentes niveles de simulación soportados en el flujo de diseño de Quartus II. La Tabla 3-3 numera las diferentes alternativas de flujos de simulación que soporta Quartus II (9).

TABLA 3-3: FLUJOS DE SIMULACIÓN SOPORTADOS POR QUARTUS II

Flujo de simulación	Descripción
NativeLink	Es una aplicación que automatiza el proceso de simulación. No es recomendado cuando se desean realizar simulaciones con el control total sobre los escenarios de simulación. NativeLink genera <i>scripts</i> de compilación de las librerías necesarios para la simulación.
Custom	Es un proceso manual de control de los aspectos de simulación tales como: Compilación manual de los <i>testbench</i>, bibliotecas de componentes y la escritura manual de los <i>scripts</i> de simulación.
Specialized	Es un proceso que permite la simulación de aplicaciones particulares como sistemas embebidos que incluyen procesadores Nios II de Altera.

Como puede observarse, la simulación de un diseño puede involucrar mucho más que el código incluido en el archivo realizado por el diseñador. Entre otras, pueden abarcar funciones definidas por el lenguajes (por ejemplo, un sumador a partir de la función '+'), bibliotecas de recursos propios de las FPGA (memorias embebidas, multiplicadores, *transceivers*, etc.) u otros modelos especiales (por ejemplo, el procesador Nios II diseñado por Altera para ser implementado en sus dispositivos FPGA).

CONFIGURACIÓN DE LA SIMULACIÓN EN QUARTUS II (OPCIÓN NATIVELINK)

Es necesario configurar correctamente las opciones de Quartus II para ejecutar la simulación mediante NativeLink. Para ello, deben realizarse los siguientes pasos:

Con un proyecto abierto, seleccionar de la barra de menús **Tools > Options** y especificar el directorio correcto del simulador en la categoría **EDA Tool Options** (ej.: "c:/altera/<versión_de_quartus>/modelsim_ase/win32aloem/") y seleccionar el botón **OK**.

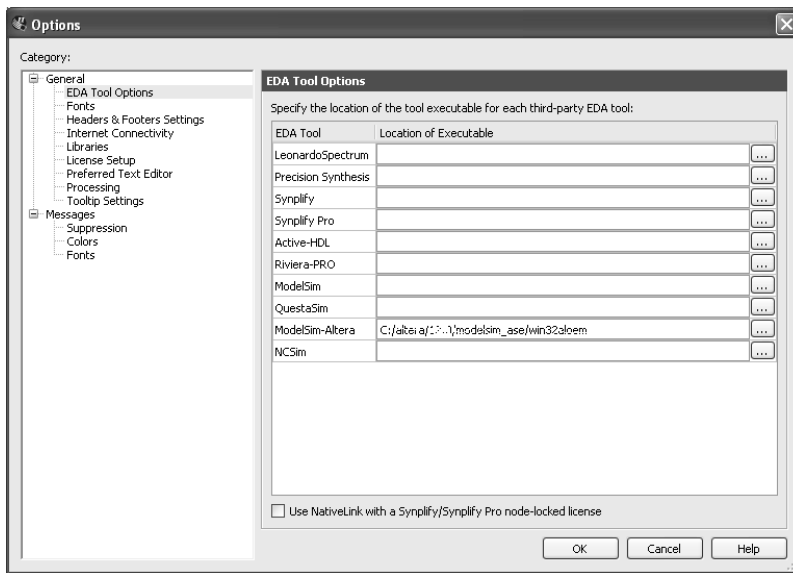


Figura 3-17: Configuración del directorio de ModelSim para NativeLink.

En caso de no encontrar el directorio del simulador, puede buscarse la aplicación *modelsim.exe* en el Explorador de Windows.

Posteriormente, se debe seleccionar de la barra de menú **Assignments > Settings**, categoría **EDA Tool Settings -> Simulation**.

Al seleccionar el botón **More NativeLink Settings...**, se pueden especificar opciones adicionales (Figure 3-19). Presionar **OK** sin realizar ninguna modificación para continuar la configuración.

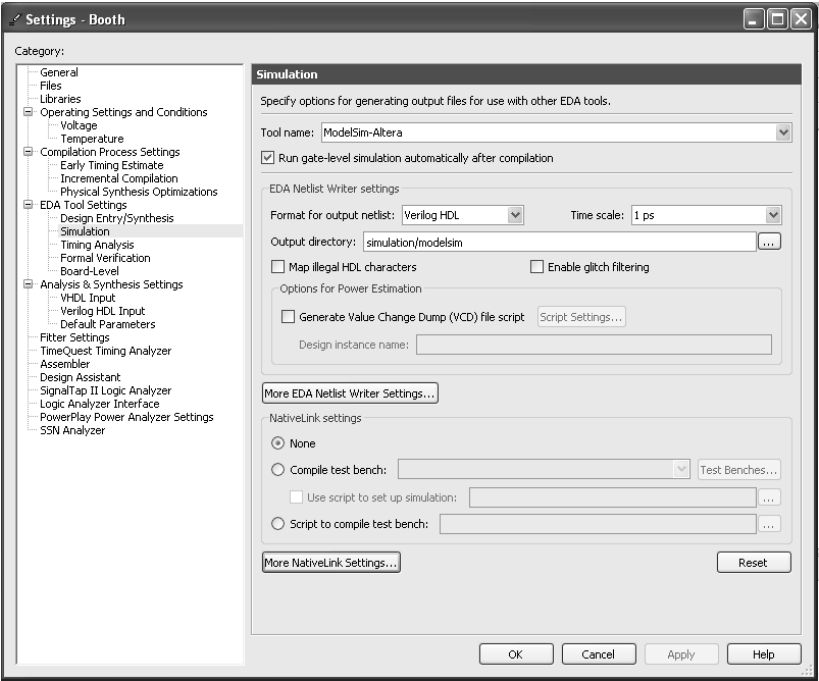


Figura 3-18: Configuración de Settings de NativeLink en Quartus II.

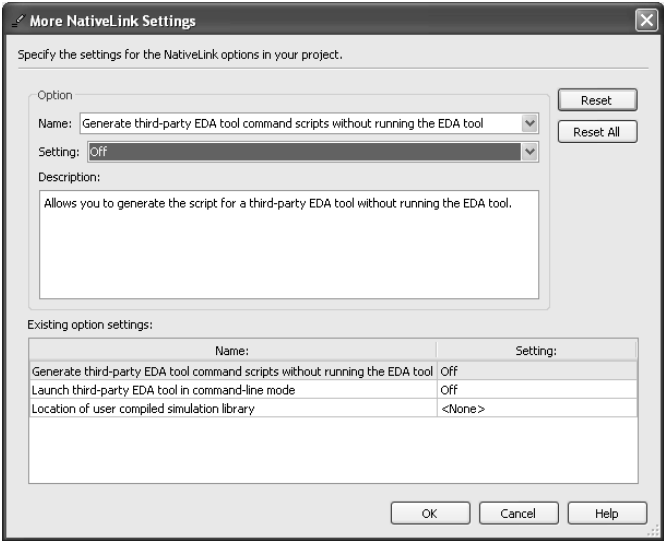


Figura 3-19: Pantalla de More NativeLink Settings.

En caso de querer simular a nivel RTL, se debe de-seleccionar la opción **Run gate-level simulation automatically after compilation**.

INICIALIZACIÓN DE MODELSIM A NIVEL RTL (OPCIÓN NATIVELINK)

Una vez configuradas las opciones para la simulación NativeLink, se puede realizar la simulación a Nivel RTL, seleccionado la compilación **Start Analysis and Elaboration** ("🔍").

Al finalizar el análisis y la elaboración de la base de datos, se ejecuta el simulador mediante el comando de la barra de menús **Tools > Run Simulation Tool > RTL Simulation**. Luego de unos instantes, el software ModelSim comenzará a ejecutarse configurando como diseño de simulación al proyecto creado en Quartus II.

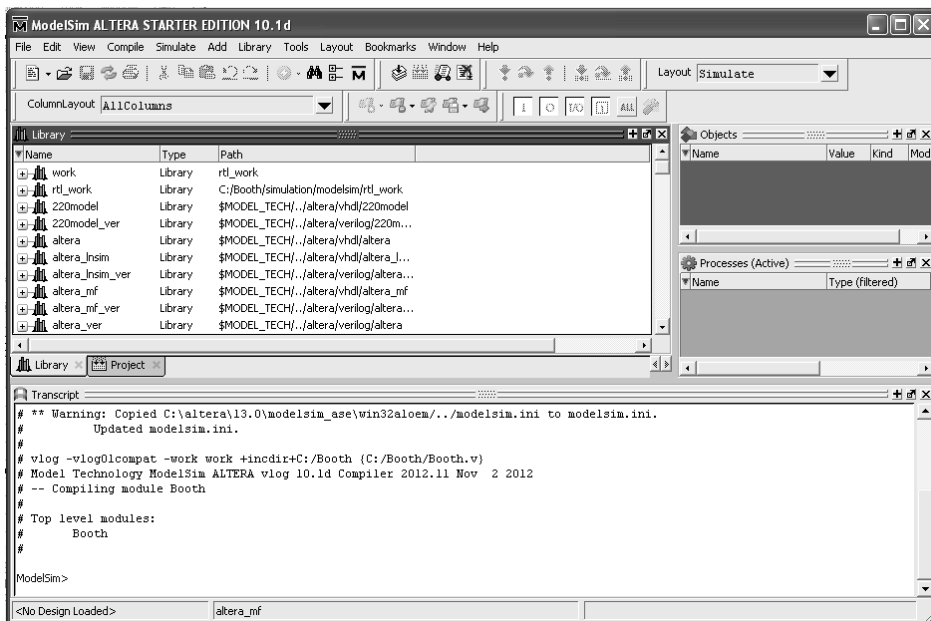


Figura 3-20: Pantalla de inicio de ModelSim.

INICIALIZACIÓN DE MODELSIM A NIVEL GATE(OPCIÓN NATIVELINK)

La simulación a Nivel de Gate puede realizarse en forma funcional o temporal. Para dicha configuración, debe seleccionarse o deseleccionarse, respectivamente, la opción **Generate netlist for functional simulation only** en la pantalla **More EDA Netlist Writer Settings** de la categoría **EDA Tool Settings -> Simulation** accedida mediante el comando de la barra de menú **Assignments > Settings**.

SIMULACIÓN CON MODELSIM

ModelSim es un ambiente de simulación digital que soporta diferentes lenguajes de descripción de hardware. Todo el proceso de simulación de un diseño es controlado mediante la ejecución de diferentes comandos. Estos comandos pueden ser ejecutados de tres maneras diferentes:

- **Barra de menú:** accediendo a las diferentes opciones o a través de los íconos de acceso directo en la barra de menú.
- **Línea de comando:** mediante el tecleo del comando correspondiente en el *prompt* ("**>**") de la consola de comandos.
- **Archivo de comandos (*script*):** mediante la invocación de un archivo de texto que contiene los comandos que se desean ejecutar.

La integración con Quartus II y su configuración con NativeLink permiten acceder a ModelSim con el proyecto ya creado y los archivos de diseño anexados a la biblioteca *work* del diseño y compilados.

En caso de requerir una recompilación de un archivo del proyecto, se puede hacer clic con el botón de derecho de *mouse* sobre el archivo en la Ventana "Library" y seleccionar la opción **Recompile** del menú desplegable.

La compilación en ModelSim crea la base de datos necesaria para la realización de la simulación y contiene todas las señales internas del diseño. Luego de una compilación exitosa, puede realizarse la simulación del diseño mediante la selección del comando de la barra de menú **Simulate > Start Simulation...** o desde el ícono (""). Este comando abre la aplicación de simulación del ModelSim denominada **vsim**.

Antes de abrir el simulador, la aplicación solicita que se defina el archivo a simular, debiendo seleccionar en el ejemplo propuesto, el archivo "Booth" de la biblioteca *work* (Figura 3-21). Luego, se debe seleccionar el botón **OK** y la aplicación invocará al simulador.

La pantalla del simulador (Figura 3-22) presenta diferentes ventanas (**Objects**, **Processes**, **Wave**) que permiten observar los diferentes valores de las señales o procesos que incluye el diseño. Cada una de estas ventanas es expandible y desvinculable de la pantalla principal del simulador (mediante los botones "  "), para facilitar la depuración del diseño. La visualización o no de las diferentes ventanas puede realizarse mediante la selección correspondiente en la opción **View** de la barra de menús.

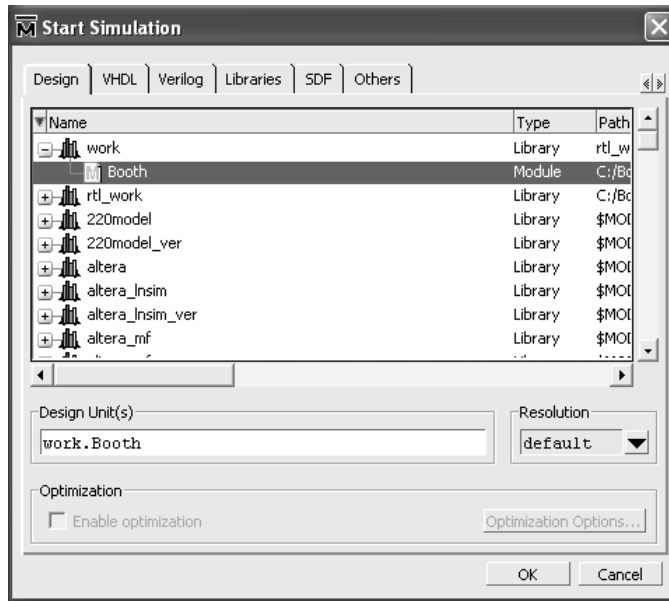


Figura 3-21: Pantalla de inicio de simulador.

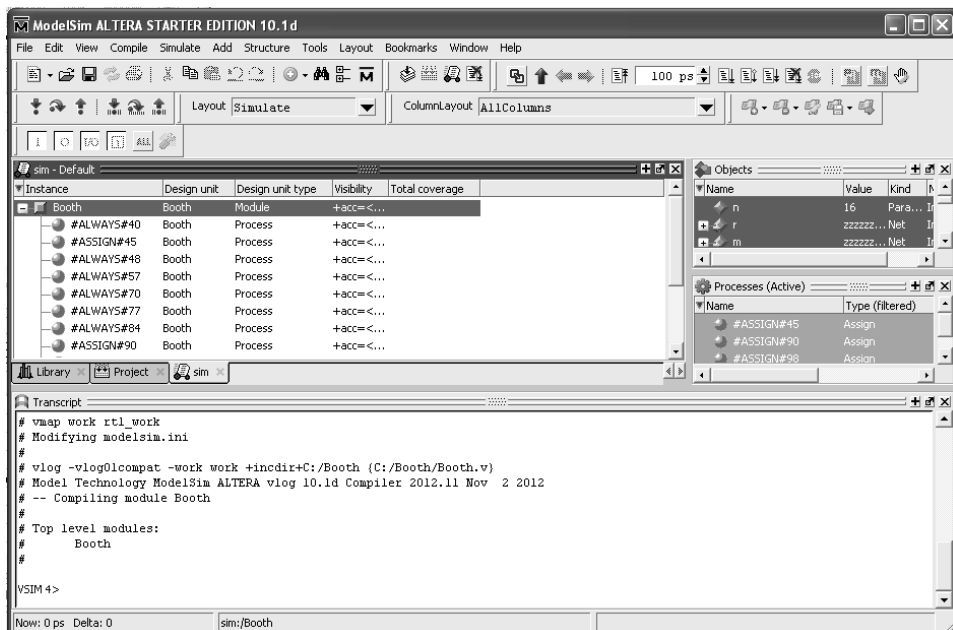


Figura 3-22: Pantalla de vsim.

Configuración de los estímulos de la simulación

Para realizar la simulación de un diseño digital, es importante proveer los estímulos necesarios a las entradas del diseño para generarle el escenario que se desea comprobar su funcionamiento.

El comando **force** permite asignar un valor a la entrada del diseño en el tiempo de corrida. Por ejemplo, para el diseño del multiplicador de Booth, es necesario generar una señal de reloj en la señal “clk”. Dicho estímulo puede realizarse mediante el siguiente comando en la consola de comandos (en el *prompt* `VSIM x>`) para generar un reloj de 200ns de periodo:

```
VSIM 4> force clk 1 0, 0 100ns -repeat 200ns
```

ModelSim interpreta este comando como:

- Forzar la señal “clk” a 1 a partir del tiempo actual.
- Forzar la señal “clk” a 0 luego de 100ns.
- Repetir luego cada 100ns.

En forma similar, se debe generar los estímulos para el multiplicando (**r**), el multiplicador (**m**) y la señal de comienzo (**start**). Los siguientes son los comandos para comprobar la multiplicación de -14×4 ($=-56$). Se puede notar que el primer comando utiliza la notación “-10#14” para indicar que es un número decimal (14) negativo.

```
VSIM 5> force m -10#14
VSIM 6> force r 10#4
VSIM 7> force start 0, 1 400ns, 0 1000ns
```

Para visualizar la evolución de cada una de las señales durante el tiempo de simulación, deben agregarse a la ventana **Wave** mediante el comando **add wave** (o seleccionado con el botón derecho del *mouse* sobre la señal en la ventana **Objects** y eligiendo la opción **Add > To Wave > Selected Signals** de la barra de menú. Para agregar las señales de entrada y salida del diseño del multiplicador de Booth, deben ejecutarse los siguientes comandos:

```
VSIM 8> add wave r
VSIM 9> add wave m
VSIM 10> add wave clk
VSIM 11> add wave start
```

En caso de desear ver las diferentes señales o registros del diseño, se deben agregar cada uno de ellos, por ejemplo:

```
VSIM 12> add wave result
VSIM 13> add wave done
VSIM 14> add wave reg_p
VSIM 15> add wave reg_a
VSIM 16> add wave reg_b
VSIM 17> add wave low_m
```

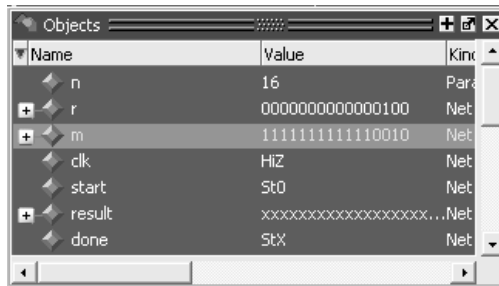


```
VSIM 18> add wave DFF0
```

Con los estímulos definidos, puede comenzarse la simulación del diseño mediante el comando **run**. El comando **run** avanza el tiempo de simulación. Si no se especifica ningún tiempo, el simulador avanza una unidad de tiempo predefinida que puede modificarse en la barra de menú. Caso contrario, puede indicarse el tiempo de simulación en el comando **run**. Por ejemplo, para simular por 1ms, debe ejecutarse el siguiente comando:

```
VSIM 19> run 1ms
```

Esta simulación permitirá ver cómo los valores de las entradas toman los estímulos forzados con los comandos anteriores. Si se desea ver una señal en otro formato diferente al binario, entonces se debe seleccionar dicha señal en la ventana **Wave** con el botón derecho del *mouse* y seleccionar **Radix > Decimal**.



Name	Value	Kind
n	16	Para
r	0000000000000100	Net
m	111111111110010	Net
clk	HiZ	Net
start	St0	Net
result	xxxxxxxxxxxxxxxx...	Net
done	StX	Net

Figura 3-23: Objetos forzados.

El resultado final de la multiplicación estará dado en tantos flancos ascendentes de la señal de reloj como el ancho del multiplicador (16 bits en el caso propuesto). Para ello, se debe correr la simulación por 4ms. El resultado final de la multiplicación puede observarse cuando la señal de **done** cambia de 0 a 1.

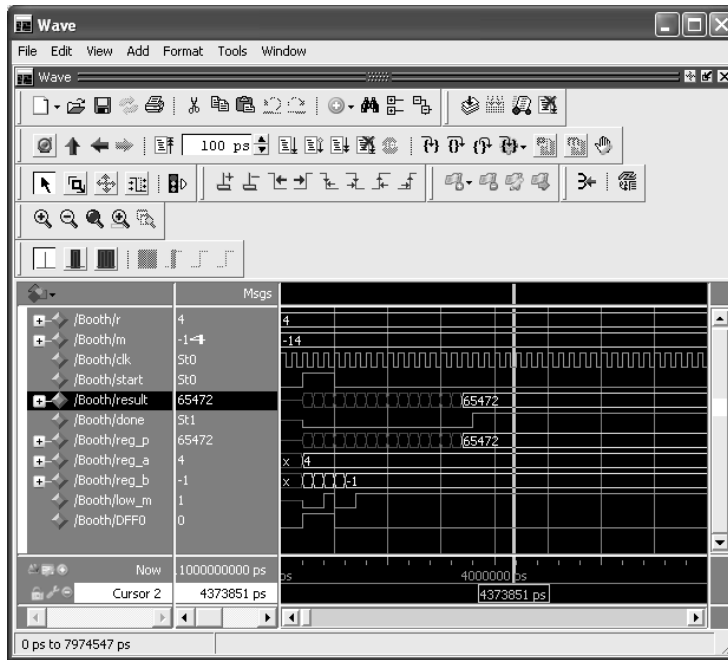


Figura 3-24: Resultado de la simulación.

CONCLUSIONES

En este capítulo, se ha descrito el flujo de diseño para la síntesis y para la simulación utilizando las herramientas brindadas por Altera y Mentor Graphics. Se describieron los principales conceptos para la realización de diferentes alternativas de simulación y las distintas configuraciones y etapas que involucran todo el proceso de diseño de hardware digital.

Quartus II está destinado a servir como herramienta de desarrollo para todo proyecto que se realice utilizando los dispositivos lógicos programables de Altera. Quartus II permite el soporte de herramientas de simulación, síntesis, verificación de otros fabricantes y también la incorporación de herramientas propias de Altera como complementos (*plug-ins*, en inglés). Una de estas herramientas adicionales que pueden incorporarse al ambiente de Quartus II es **Qsys**, el cual permite el diseño de sistemas basados en *soft-processors* y, en especial y en forma nativa, el procesador Nios II.

En los próximos capítulos, se describirán alternativas para la implementación de sistemas embebidos en dispositivos FPGA utilizando componentes y herramientas que brindan los fabricantes. Si bien el proceso de especificación y diseño de sistemas embebido será utilizando plataformas adecuadas para ello, la síntesis y simulación de los mismos serán realizadas con los procedimientos expuestos en este capítulo.

CAPÍTULO 4

SOFT-PROCESSORS EN DISPOSITIVOS FPGA

INTRODUCCIÓN A SOFT-PROCESSORS

Los lenguajes de descripción de hardware permiten contener todo el diseño de sistemas digitales en archivos que son compilados por las herramientas que brinda el fabricante. El resultado de la compilación es un archivo de que posee la información para configurar un dispositivo CPLD o FPGA para implementar el hardware digital descrito. De manera similar a lo ocurre con el software, los diferentes diseños son almacenados en archivos y éstos pueden ser agrupados de acuerdo a algún criterio para conformar **bibliotecas de componentes**.

Dichas bibliotecas de componentes pueden contener un procesador y diferentes periféricos compatibles con dicho procesador. Las bibliotecas de componentes que brinda Altera están basadas en el procesador **Nios II**, mientras que la firma Xilinx están centradas en el procesador **Microblaze**.

Las bibliotecas que cada fabricante brinda están preprocesadas (no brindan al usuario el archivo fuente de la descripción) por lo que deben implementarse los desarrollos en los dispositivos FPGA del propio fabricante. La disponibilidad del código fuente de los componentes de un fabricante requiere de licencias especiales, generalmente reservadas solo a clientes preferenciales.

Es importante notar que el procesador que brinda cada fabricante de dispositivos FPGA no es en sí un procesador “físico o real”, sino una descripción de dicho procesador que puede ser implementada en un dispositivo FPGA. Es por este motivo que se define para este tipo de procesadores la denominación *soft-processors*, puesto que no son procesadores “reales” (o *hard-processors*), sino que se implementan mediante la configuración de un dispositivo FPGA.

En este libro, utilizaremos las herramientas de desarrollo (Quartus II y Qsys) y las bibliotecas de Altera, por lo que los diseños estarán basados en el procesador Nios II.

ARQUITECTURA DEL PROCESADOR NIOS II

El procesador Nios II es un *soft-processor* específicamente diseñado por Altera para ser implementado en sus dispositivos FPGA. La versión Nios II incorpora diferentes mejoras de la versión original Nios y haciéndola más adecuada para diferentes aplicaciones como procesamiento de señales, sistemas de control y sistemas embebidos. La posibilidad de incorporar el procesador Nios II en dispositivos diferentes a los del fabricante Altera requiere de licencias particulares solo disponibles para usuarios especiales.

Xilinx, por su parte, brinda para sus dispositivos FPGA el *soft-processor* Microblaze, con prestaciones similares a las de Nios II. Lattice Semiconductor ofrece el procesador LatticeMico32 (12). Todas las herramientas brindadas por los fabricantes ofrecen prestaciones similares; sin embargo, no son compatibles entre sí. Es por ese motivo que se seleccionó el procesador Nios II de la firma Altera para los desarrollos realizados en este libro, sin perjuicio que los conceptos descriptos puedan ser utilizados en desarrollos realizados con herramientas de otros fabricantes.

Características del procesador Nios II

El procesador Nios II es un núcleo de propósito general de tipo RISC con una arquitectura de memoria de tipo Harvard, con caché de datos e instrucciones configurables y separadas. El procesador tiene diferentes características que pueden ser configuradas por el diseñador para mejorar su eficiencia de procesamiento o requerir menos recursos de hardware en la FPGA. El procesador requiere para su implementación en el dispositivo FPGA de elementos lógicos y bloques de memoria. La cantidad necesaria de estos recursos dependerá de la configuración que el diseñador establezca.

La Figura 4-1 (13) muestra la arquitectura general del núcleo del procesador Nios II. Como puede observarse, existen módulos del núcleo que son necesarios, mientras que otros son optativos y su selección puede realizarse de acuerdo a los requerimientos que presente la aplicación.

La arquitectura del *soft-processor* Nios II define las siguientes unidades funcionales:

- **Registros de propósito general y de control (*Register file*):** Nios II posee un mapa de registros de 32 registros de 32-bits de propósito general y hasta 32 registros de 32-bits de control. La arquitectura soporta modos de supervisor y usuario y permite la protección de los registros de control para las aplicaciones que se ejecutan en modo usuario.
- **Unidad aritmético-lógica (*Arithmetic Logic Unit (ALU)*):** esta unidad opera con los datos almacenados en los registros de propósito general, tomando los datos de uno o dos registros y almacenando el resultado nuevamente en el mismo. Las operaciones soportadas por la ALU se muestran en la Tabla 4-1. Las operaciones que no son implementadas por hardware se denominan *instrucciones no-implementadas (Unimplemented Instructions)* y el procesador genera una excepción para invocar la rutina de software que la implementa.

- **Instrucciones personalizables (*Interface to custom instruction logic*):** los *soft-processor* permiten ampliar las funciones que implementan a través de la ALU mediante la especificación de hardware adicional que implementan instrucciones personalizables (***Custom Instructions***). El agregado de hardware específico para la incorporación de instrucciones personalizables debe satisfacer la especificación de la interfaz definida por el *soft-processor*.

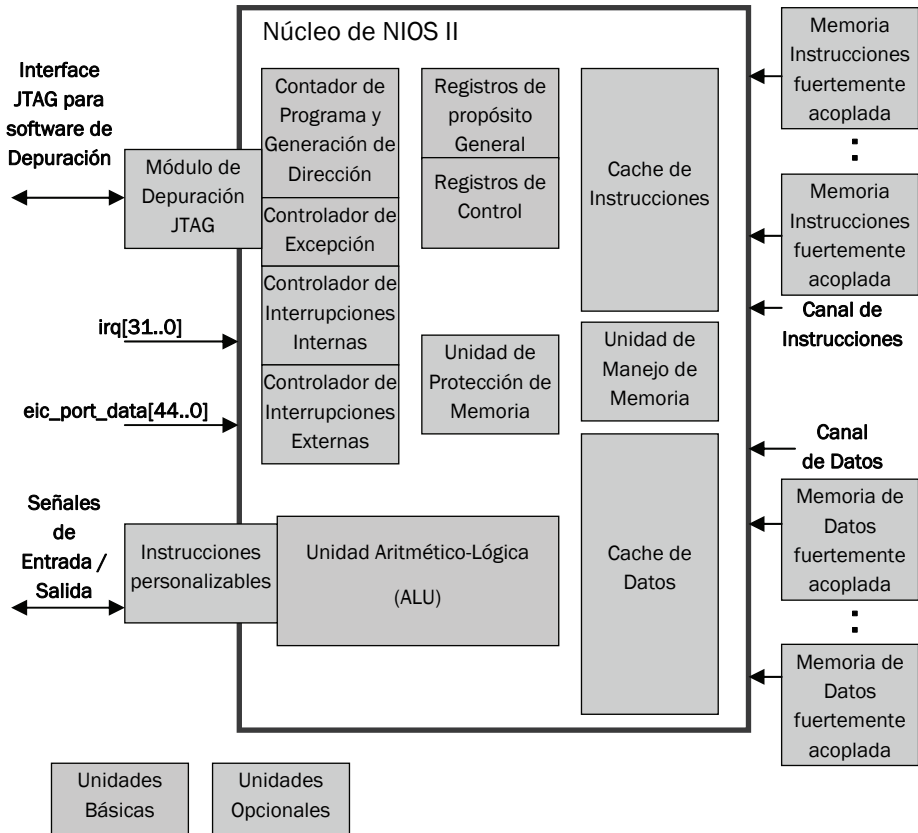


Figura 4-1: Arquitectura general del procesador Nios II.

- **Controlador de interrupciones internas y excepción (*Internal interrupt and Exception controller*):** el procesador Nios II produce una sola excepción para manejar todas las excepciones e interrupciones. Cada excepción, al igual que las interrupciones internas, causan que el procesador transfiera su ejecución a una dirección de excepción. El manejador de excepción es el que determina la causa de la excepción y despacha la rutina apropiada. El controlador de interrupciones interno soporta 32 interrupciones.

- **Controlador de interrupciones externas (*External interrupt controller*):** el procesador Nios II permite la conexión de un controlador de interrupciones externo periférico para mejorar la eficiencia de las interrupciones de hardware. Generalmente, cuando existe un controlador de este tipo, entonces no se implementa el controlador de interrupciones interno.
- **Canal de datos e instrucciones (*Instruction and Data bus*):** el *soft-processor* Nios II soporta canales de dirección y datos separados, por lo que puede clasificarse como una arquitectura Harvard. Ambos canales son implementados adhiriendo a la especificación de Master de la arquitectura Avalon-MM que será detallada más adelante.
- **Unidad de manejos de memoria (*Memory Management Unit (MMU)*):** esta unidad permite la implementación de memoria virtual de instrucciones y datos y protección de memoria. El direccionamiento de memoria se realiza en 32-bits, lo que permite un espacio de memoria virtual y físico de 4 GB. Los primeros 512 MB de memoria física están disponibles para acceso directo, mientras que los restantes deben realizarse mediante los registros de conversión (TLB, *Translation Lookaside Buffers*, en inglés).
- **Unidad de protección de memoria (*Memory Protection Unit (MPU)*):** esta unidad permite definir 32 regiones de instrucción y 32 regiones de datos para su protección mediante permisos. Las unidades MMU y MPU no pueden ser habilitadas simultáneamente.
- **Memorias caché de datos e instrucciones (*Instruction and data cache memories*):** el procesador Nios II soporta, opcionalmente, caché de instrucciones y de datos. La memoria caché se implementa con recursos de memoria de la FPGA y mejora el tiempo de acceso a memoria en comparación con memorias SDRAM externas al dispositivo FPGA. La consistencia de la memoria caché es manejada por software y el Nios II provee instrucciones especiales para tal fin.
- **Memorias de datos e instrucciones fuertemente acopladas (*Tightly-coupled memory interfaces for instructions and data*):** el procesador Nios II posee la opción de interconexión de memorias de instrucciones y datos fuertemente acopladas. En estas memorias, pueden almacenarse código y datos en forma similar a las memorias de instrucciones y datos del procesador, pero la eficiencia es similar a la de la memoria caché. De esta manera, pueden ejecutarse instrucciones o acceder a datos de operaciones críticas, con una eficiencia similar a la de memoria caché. Físicamente, la memoria es interconectada de manera similar a memoria de instrucciones y datos externas, pero es implementada mediante recursos de memoria del dispositivo FPGA.
- **Módulo de depuración JTAG (*JTAG debug module*):** el módulo de depuración a través del puerto JTAG permite la emulación del procesador. Mediante este módulo, el *soft-processor* se comunica a un computador personal que ejecuta el software de depuración, el cual permite el control y monitoreo de las

actividades que el procesador realiza en tiempo de corrida. Entre algunas de las posibilidades que brinda este módulo, se encuentran: carga del programa a memoria, arranque y detención de la ejecución del programa, posibilidad de inserción de *breakpoint*, análisis del contenido de los registros del Nios II y memoria del sistema y seguimiento de la ejecución del programa en tiempo real.

TABLA 4-1: CATEGORÍA DE OPERACIONES SOPORTADAS POR LA ALU

Categoría	Descripción
Aritmética	Soporta suma, resta, multiplicación y división con operandos con y sin signo.
Relacional	Soporta comparaciones de operandos con y sin signo.
Lógica	Soporta operaciones AND, OR, NOR y XOR.
Desplazamiento y rotación	Soporta los operadores de desplazamiento y rotación, tanto en forma aritmética como lógica.

VERSIONES DE NIOS II

Altera posibilita 3 versiones configurables del procesador Nios II que el diseñador puede seleccionar según sus requerimientos o disponibilidad de recursos:

- **Fast** (/f core): esta versión está optimizada para altas eficiencias de procesamiento, contando con una unidad de manejo de memoria (MMU, *Memory Management Unit*, en inglés) opcional y unidad de protección de memoria (MPU, *Memory Protection Unit*, en inglés). La unidad de manejo de memoria posibilita la ejecución de programas aun cuando la memoria física disponible no es suficiente, transfiriendo el código ejecutable de unidades de almacenamiento masivo a medida que es requerido. La unidad de protección de memoria posibilita que la ejecución de varios programas no interfieran entre sí, evitando la corrupción en datos o instrucciones potencialmente peligrosas. Incluye 6 etapas *pipeline*.
- **Economy** (/e core): esta versión de Nios II es la que requiere la menor cantidad de elementos lógicos (alrededor de 700 elementos lógicos). Está optimizada para requerir la menor cantidad de recursos de la FPGA.
- **Standard** (/s core): esta versión mejora la eficiencia de procesamiento de la versión económica mediante la incorporación de 5 etapas de *pipeline* sin incrementar en demasía, con respecto a la versión *economy*, los recursos de FPGA requeridos.

Altera brinda un cuadro comparativo entre las versiones que se muestran en la Tabla 4-2.

TABLA 4-2: CUADRO COMPARATIVO DE VERSIONES DE NIOS II

	Fast	Standard	Economy
Ejecución de instrucciones (Máxima)	1 instrucción por ciclo de reloj	1 instrucción por ciclo de reloj	1 instrucción por cada 6 ciclos de reloj
Máximo número de instrucciones activas en <i>pipeline</i>	6	5	1
Tiempo de multiplicación	1 ciclo de reloj	3 ciclos de reloj	Emulada por software
Predicción de salto (<i>Branch Prediction</i>)	Dinámico	Estático	No soportado
Caché de instrucciones	Configurable	Configurable	No soportado
Caché de datos	Configurable	No soportado	No soportado
Elementos lógicos requeridos	1800 sin MMU 3200 con MMU	1200	600
Instrucciones personalizables	Hasta 256		

Todas las versiones de Nios II soportan un direccionamiento de memoria de al menos 2 GB, lógica para el soporte de depuración de software (*debug*), la incorporación de **instrucciones personalizables** y la posibilidad de implementación en hardware a partir de funciones programadas en software (**C2H compiler**). A continuación, se describen con mayor detalle estas dos últimas características, puesto que son nuevas posibilidades que brindan algunos *soft-processor* implementados en dispositivos FPGA.

Instrucciones personalizables (*Custom Instructions*)

Todos los procesadores “tradicionales” (*hard-processor*) poseen un conjunto de instrucciones (*Instruction Set*, en inglés) que pueden ejecutar. Este conjunto de instrucciones es definido por el fabricante de acuerdo al área de aplicaciones a la que estará orientado el procesador o a la característica propia del procesador. De esta manera, un procesador RISC (Procesador de conjunto de instrucciones reducido, *Reduced Instruction Set Controller*, en inglés) tiene por naturaleza un conjunto de instrucciones diferente a un procesador CISC (Procesador de conjunto de instrucciones complejo, *Complex Instruction Set Controller*, en inglés). El conjunto de instrucciones definirá las herramientas de desarrollo que serán necesarias para crear aplicaciones y en las aplicaciones que puedan ser utilizados. Por ejemplo, una aplicación desarrollada para un procesador Intel podrá ser ejecutada en un

procesador AMD si este último posee el conjunto de instrucciones que la aplicación requiere en el procesador Intel.

En procesadores “tradicionales”, implementados en hardware por el fabricante, el conjunto de instrucciones es fijo y el programador debe realizar su aplicación acotado por las instrucciones que brinda el procesador. Existen algunas excepciones en los cuales puede modificarse el microcódigo del procesador para introducir limitada flexibilidad al conjunto de instrucciones. En los *soft-processor*, existe un conjunto de instrucciones definido por el fabricante para dicho procesador, pero permite ampliar el conjunto de instrucciones mediante lo que se denomina **instrucciones personalizadas** (*Custom Instructions*, en inglés). El usuario puede definir un hardware determinado que realice una o varias funciones específicas (por ejemplo, procesamiento de video o audio) e incorporar dichas instrucciones al conjunto de instrucciones original del procesador. De esta forma, puede incrementarse y adaptarse la eficiencia del procesador mediante la implementación en hardware de funciones específicas de la aplicación que, de otra manera, debería realizarse en software adecuándose a las instrucciones que brinda el procesador.

Síntesis de software a hardware

Otra alternativa que permiten los sistemas implementados en dispositivos FPGA es que puede migrarse código escrito en ANSI C a hardware, transparentemente interconectado con el procesador del sistema. Mediante esta alternativa, las rutinas de software, que tradicionalmente son compiladas al conjunto de instrucciones del procesador, pueden seleccionarse para ser implementadas en hardware. De esta manera, la aplicación, al hacer una llamada a esta función o subrutina, no producirá la ejecución de la misma por el procesador, sino que leerá la salida de un hardware específico que produce el mismo resultado.

Altera ofrece esta posibilidad de síntesis de software a hardware mediante la opción de **C to hardware** o **C2H compiler**. Por supuesto, deben mantenerse determinadas estructuras en el código ANSI C que se desea implementar para que el sintetizador pueda realizar un hardware adecuado para ser implementado en un dispositivo FPGA (14).

Altera ha dejado de soportar el sintetizador C2H para ofrecer la compilación de OpneCL a hardware con el propósito de aceleración de software.

LICENCIA DE NIOS II

La única versión de Nios II que no requiere licencia es la versión *economy*. Esta versión se entrega en forma encriptada, lo que significa que no existe ningún archivo de descripción de hardware del código fuente del *soft-processor*.

Las demás versiones del procesador requieren licencia, pero pueden ser libremente evaluadas mientras estén conectadas a un computador personal que ejecute el software Quartus II. La licencia debe ser adquirida a Altera, pero siempre el código que entrega es encriptado. La licencia del Nios II es un archivo cuyo texto debe ser agregado al archivo de licencia de Quartus II.

Excepciones para obtener el código fuente del procesador Nios II deben ser tramitadas especialmente con Altera, generalmente basadas en diseños que son migrados a circuitos integrados de otros fabricantes.

Todas las versiones de Nios II son *royalty-free* lo que significa que no se debe pagar más por el número de sistemas que se producen utilizando el procesador Nios II.

Recomendaciones para diseños con Nios II

Existen diversas recomendaciones que brinda Altera (pero que es conveniente mantener con otros fabricantes) para evitar dificultades en el momento de diseño. Entre algunas de las recomendaciones que realiza Altera, se pueden citar:

- obtener la última versión de Quartus II y la última versión de Nios II que la acompaña para nuevos diseños. Para diseños anteriores es conveniente mantener la versión de Quartus II y la versión de Nios II originales.
- utilizar directorios, nombre de archivos y proyectos que no contengan espacios. Esta recomendación también debe tenerse en cuanto al momento de instalar Quartus II y demás plataformas de desarrollo.
- considerar la versión de Nios II para evaluar si debe estar conectado al computador personal en caso de que no se cuente con licencia. Esta posibilidad permite evaluar el sistema antes de adquirir la licencia correspondiente.

CONCEPTO DE CONFIGURACIÓN DE SOFT-PROCESSORS

Como se puede observar, un *soft-processor* permite muchas más posibilidades que la implementación en un dispositivo FPGA de un procesador “tradicional”. Permite flexibilidad en su arquitectura, tanto por la posibilidad de incorporar instrucciones personalizables como la integración con componentes compatibles, sintetizados para funciones particulares del diseño.

Además de la flexibilidad que brinda la implementación de procesadores en dispositivos FPGA, los *soft-processor* permiten particularizar su configuración de acuerdo a las características requeridas para cada uno de los sistemas en los que es utilizado. La configuración de los *soft-processor* permite adaptar también sus características según el dispositivo FPGA en el que será implementado. Esto se logra mediante la selección, o no, de los módulos opcionales de la arquitectura del *soft-processor* o la especificación de diferentes características de los módulos requeridos (Figura 4-1).

Es importante remarcar el concepto en que las unidades funcionales de la arquitectura del *soft-processor* Nios II (Figura 4-1) conforman la base del conjunto de instrucciones del Nios II y no significa que dichas unidades sean implementadas en hardware.

Específicamente, la arquitectura del procesador Nios II describe un conjunto de instrucciones y no un hardware en particular.

Una unidad funcional puede ser implementada en hardware, emulada en software u omitida completamente. La elección de la manera de implementación de cada unidad funcional estará de acuerdo a criterios de menor utilización de recursos del dispositivo FPGA o eficiencia en ejecución de la aplicación.

Flexibilidad de periféricos y mapa de direcciones

La flexibilidad en los dispositivos periféricos es una de las mayores diferencias entre los *soft-processor* y los procesadores tradicionales o microcontroladores. Debido a que el *soft-processor* es implementado en un dispositivo FPGA, los periféricos pueden ser elegidos de acuerdo a los estrictamente necesarios por la aplicación.

Como consecuencia de la flexibilidad en los periféricos, se encuentra la flexibilidad en el mapa de direcciones. Altera provee estructuras de software para acceder a memoria y a periféricos en forma genérica e independiente de la dirección. Esto permite alta flexibilidad en la configuración de los periféricos y en la realización del software de la aplicación.

BUS AVALON

Todos los sistemas basados en procesadores necesitan un medio de comunicación para transferir información entre los diversos componentes y periféricos del sistema. Para tal comunicación, los diseñadores especifican o adoptan diversas interfaces que se denominan **canales** (*bus*, en inglés). En el caso de Nios II, Altera especificó el **bus Avalon** (15) como soporte de interconexión entre el procesador y los periféricos del sistema. Es por esto, que cualquier periférico que presente una interfaz Avalon podrá integrar un sistema basado en el procesador Nios II de Altera. Existen algunos mecanismos de adaptación ofrecidos por Altera que permiten conectar periféricos diseñados para otros procesadores al procesador Nios II, pero la eficiencia en la transferencia puede reducirse significativamente.

Las herramientas de desarrollo provista por Altera (Quartus II y Qsys) hacen transparente al diseñador todos estos detalles de implementación. Mediante estas herramientas, el diseñador solo debe especificar los periféricos que el sistema necesita, siendo la herramienta la que determina las interconexiones más adecuadas para ellos.

CARACTERÍSTICAS DEL BUS AVALON

El *bus Avalon* define una interfaz apropiada para la interconexión de los diferentes componentes de un sistema, soportando la capacidad de alta transferencia de datos, lectura y escritura de registros y memoria e interfaz con dispositivos externos al dispositivo FPGA. El *bus Avalon* especifica siete diferentes categorías de interfaces:

- **Avalon Memory Mapped Interface (Avalon-MM):** es una interfaz basada en direcciones de lectura/escritura con conexiones del tipo Maestro-Esclavo (*Master-Slave*, en inglés).

- **Avalon Streaming Interface (Avalon-ST):** es una interfaz que soporta flujo de datos unidireccional, incluyendo flujos multiplexados, paquetes y datos de procesamiento de señales.
- **Avalon Conduit Interface:** es una interfaz que adapta señales individuales o grupo de señales que no son compatibles con otros tipos de interfaces Avalon.
- **Avalon Tri-state Conduit Interface:** es una interfaz que soporta conexiones con dispositivos externos al dispositivo FPGA. Múltiples componentes pueden compartir, a través de multiplexado, pines del dispositivo FPGA para reducir la cantidad necesaria de pines y líneas en el circuito impreso.
- **Avalon Interrupt Interface:** es una interfaz que permite a los componentes generar señales de eventos a otros componentes.
- **Avalon Clock Interface:** es una interfaz que maneja o recibe señales de reloj. Todas las interfaces Avalon son sincrónicas.
- **Avalon Reset Interface:** es una interfaz que provee la señal de inicialización (*reset*) al sistema.

Cada componente del sistema puede incluir varias de estas interfaces y también puede incluir múltiples instancias del mismo tipo de interfaz.

La interfaz Avalon es un estándar abierto y no se requiere licencia o *royalty* para desarrollar o comercializar productos que utilizan o son basados en la interfaz Avalon.

INTERFAZ AVALON MEMORY-MAPPED

La interfaz Avalon *Memory-Mapped* (Avalon-MM) es utilizada para leer y escribir en componentes maestros o esclavos en un sistema direccionado de memoria. Estos componentes pueden incluir procesadores, memorias, UART, controladores DMA y *timers* que cuentan con interfaces de maestro o esclavo.

La interfaz Avalon-MM soporta múltiple maestro (*Master*) lo que permite el acceso de diferentes componentes a componentes esclavos (*Slave*) comunes. La arbitración del *bus* es realizada automáticamente mediante el agregado por medio de la herramienta de diseño de un árbitro de canal. La Figura 4-2 muestra un sistema con diferentes componentes conectados al procesador Nios II. El SOPC de la Figura 4-2 será utilizado para ilustrar la utilización de la herramienta Qsys de Altera.

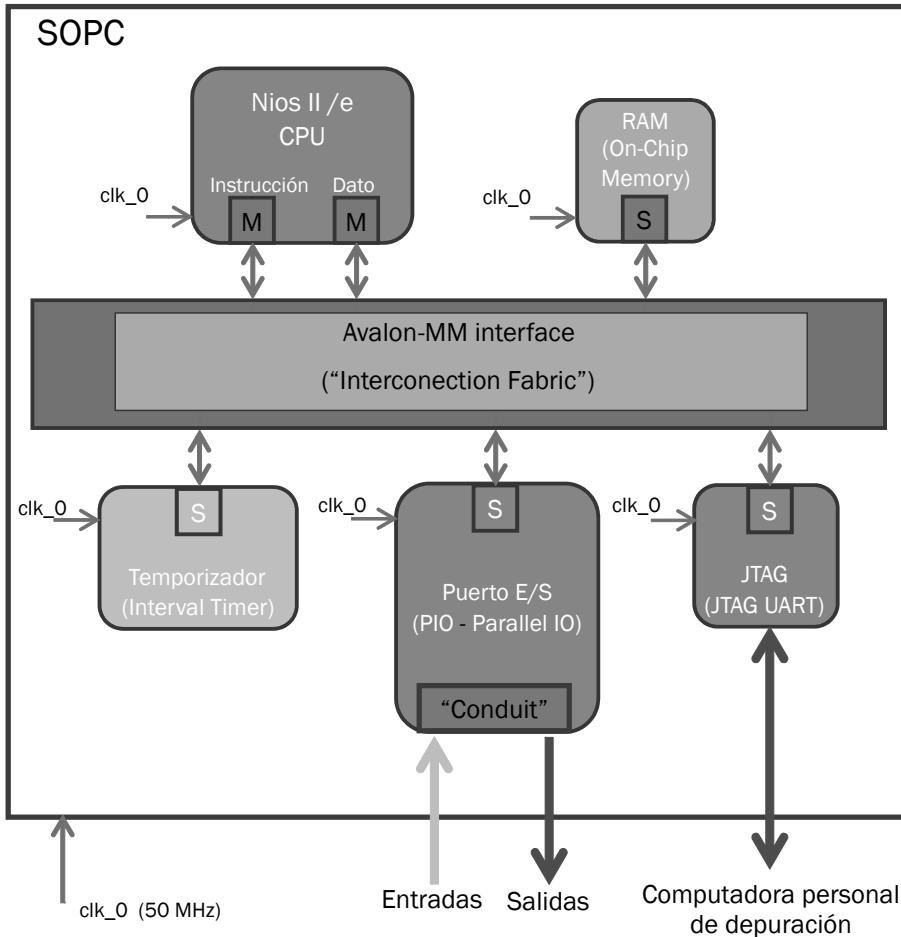


Figura 4-2: SoPC con componentes con interfaz Avalon-MM.

La interfaz Avalon-MM soporta diferentes modos de funcionamiento, adecuados para diversos componentes. Todos los componentes basados en interfaz Avalon-MM son interconectados a través de un *canal de interconexión (Interconnection Fabric)*.

Cada componente de la interfaz Avalon-MM incluye solamente las señales requeridas por la lógica del componente. Por ejemplo, un componente periférico esclavo con un registro de 16-bits que solo responda a requerimientos de escritura, incluirá las señales de sincronismo para la escritura del registro. De esta manera, puede reducirse el hardware necesario para la implementación de los periféricos mediante la selección adecuada de las unidades requeridas para el diseño y prescindiendo de las no necesarias.

La interfaz Avalon-MM es síncronica y todas las lógicas realizadas deben estar referenciadas a la señal de reloj de la interfaz Avalon-MM a la que está asociada.

TRANSFERENCIAS EN AVALON-MM

La interfaz Avalon-MM permite las transferencias de lectura o escritura entre un dispositivo *maestro* y un dispositivo *esclavo*. El tamaño de la transferencia puede ser entre 8 y 1024 bits, pudiendo necesitar entre 1 o más ciclos de reloj para completar la transferencia. Hay diferentes tipos de transferencias soportadas por la interfaz Avalon-MM adecuadas para diferentes aplicaciones.

Típica transferencia de lectura y escritura

La interfaz Avalon-MM soporta transferencias de lectura y escritura en las cuales el esclavo puede insertar ciclos de *waitrequest* en los cuales el *maestro* detiene la transferencia esperando la respuesta del esclavo. La transferencia finaliza cuando el esclavo retira la señal *waitrequest*. No hay límite de ciclos por los cuales un esclavo puede detener el procesador en una transferencia, por lo que debe tenerse en cuenta de no mantener la señal *waitrequest* activa indefinidamente.

Transferencias de lectura y escritura con wait-states fijos

Una alternativa a la transferencia anterior es que la transferencia tenga un valor por defecto que defina la cantidad de ciclos de reloj por los que el maestro Avalon-MM debe esperar por la respuesta del esclavo. Este tipo de transferencia es adecuada para cuando un maestro está conectado a múltiples esclavos y se desea homogenizar el tiempo de transferencia entre todas ellas.

Transferencia con pipeline

En las transferencias con *waitrequest*, el esclavo debe realizar la transferencia mientras que el maestro espera por la respuesta del esclavo. Si bien existen esclavos que pueden poseer rápidas respuestas, pueden requerir de un tiempo inicial para preparar el primer dato. En este caso, es posible que la transferencia con *pipeline* sea adecuada.

En la transferencia con *pipeline*, se considera los tiempos de accesos iniciales para incrementar la transferencia de múltiples ciclos de acceso.

Transferencias burst

La transferencia *burst* está concebida para múltiples transferencias como una unidad en lugar de realizar transferencias individuales. Varios tipos de componentes interconectados como esclavos soportan transferencias *burst* (por ejemplo, memorias SDRAM) mejorando considerablemente la tasa de transferencia.

INTERFAZ AVALON INTERRUPT

La interfaz Avalon permite que componentes esclavos generen señales de eventos para ser atendidos por el maestro.

Generador de interrupción

Un componente generador de interrupción es capaz de producir una señal de evento para ser enviada al maestro. La señal debe ser generada hasta que el dispositivo maestro la reconozca y es este último el que determina la manera en que el esclavo será atendido. La forma en que la interrupción es reconocida en el componente esclavo es específica para cada esclavo.

Receptor de interrupción

El receptor de la interrupción es un dispositivo maestro del sistema. El receptor de interrupciones puede aceptar una interrupción por cada dispositivo esclavo que lo requiera.

INTERFAZ AVALON STREAMING

La interfaz Avalon *Streaming* (Avalon-ST) es adecuada para componentes que requieren gran ancho de banda, baja latencia y transferencia unidireccional de datos. Las aplicaciones típicas incluyen el multiplexado de paquetes y procesamiento digital de señales. La interfaz Avalon-ST soporta flujos de datos por canales o por paquetes y protocolos más complejos capaces de transferir paquetes de datos en múltiples canales.

Características de la interfaz Avalon Streaming

La interfaz Avalon-ST tiene como principales características las siguientes:

- Latencia baja y alta tasa de transferencia punto a punto.
- Soporte de múltiples canales mediante el intercalado de paquetes.
- Soporte de transferencia por ráfagas (*burst*).
- Adaptación automática de la interfaz a las características de los componentes interconectados.

INTERFAZ AVALON CONDUIT

La interfaz Avalon *conduit* permite que un conjunto de señales estén disponibles para una descripción de hardware jerárquicamente superior. Todos los componentes e interconexiones que definen un sistema basado en el *soft-processor* Nios II son internas al sistema, siendo la manera de poder contar con estas señales en una descripción de hardware jerárquicamente superior es a través de la interfaz Avalon *conduit*. En la Figura 4-2, el componente puente de entrada/salida implementa una interfaz Avalon *conduit* que permite leer entradas y escribir salidas del exterior del SOPC.

INTERFAZ AVALON TRI-STATE CONDUIT

La interfaz Avalon *Tri-state Conduit* es similar a la anterior, pero utilizada para la conexión a componentes externos al dispositivo FPGA.

COMPONENTES DE BIBLIOTECA

Cada fabricante de dispositivos FPGA entrega junto con la especificación de su *soft-processor* una biblioteca de componentes periféricos compatibles con el canal de interconexión del procesador. Altera brinda diversos componentes conjuntamente con la especificación de su procesador Nios II. Existe la posibilidad de adicionar componentes mediante diseños propios o la adquisición de los mismos a diferentes compañías.

CONCLUSIONES

Los dispositivos lógicos configurables de alta capacidad han permitido la descripción e implementación de procesadores y componentes que conforman un sistema en uno solo de estos dispositivos. La especificación de procesadores, denominados *soft-processors*, no solo se restringe a la descripción de procesadores “tradicionales” en dispositivos FPGA sino que permiten una configuración con gran flexibilidad para adaptarse a las características de la aplicación. De esta manera, los *soft-processor* soportan un conjunto de instrucciones que puede ser expandido con hardware personalizado para mejorar la eficiencia del procesador y determinar las funciones que se desean ejecutar en software o implementar en hardware. La implementación de sistemas basados en *soft-processor* requiere de canales de transferencia adecuados a las características de estos y a las de los dispositivos FPGA en que son implementados. Este capítulo describió las características más importantes del *soft-processor* Nios II de la firma Altera y las diferentes características de *bus Avalon* especificado por Altera para mejorar la eficiencia en los sistemas que utilizan sus componentes.

CAPÍTULO 5

DISEÑO DE SISTEMAS EMBEBIDOS EN LÓGICA PROGRAMABLE

SISTEMAS-EN-CHIP-PROGRAMABLE (SOPC)

Se denomina SOPC (sistema-en-chip-programable, *System-On-A-Programmable Chip*, en inglés) a todo sistema basado en una o más CPU, memoria y otros periféricos incluidos en un dispositivo lógico programable (FPGA).

Los SOPC son implementados en dispositivos lógicos programables de alta capacidad como los dispositivos FPGA, existiendo en el mercado varios fabricantes que brindan soporte para su implementación, entre los que se destacan Altera, Xilinx y Actel. Cada fabricante brinda su propia herramienta de desarrollo, conjuntamente con las bibliotecas de componentes para la implementación de SOPC.

FLUJO DE DISEÑO DE SISTEMAS EN CHIPS PROGRAMABLES

El proceso de diseño de sistemas-en-chip-programable involucra generalmente el diseño del hardware y del software asociado. El hardware de un SOPC se realiza generalmente integrando los componentes de bibliotecas conforme a los requerimientos de la aplicación. A la vez, el desarrollo del software se debe realizar acorde a la arquitectura de hardware del sistema.

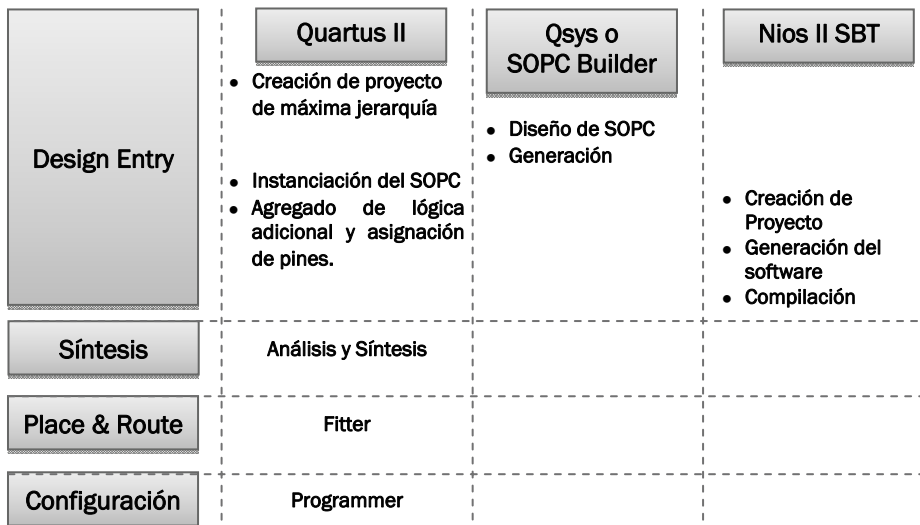


Figura 5-1: Flujo de diseño de SOPC de Altera.

La Figura 5-1 muestra las herramientas propuestas por Altera para el flujo de diseño de sistemas en chips programables. El proyecto se realizará en Quartus II al que se le incorporarán las diferentes descripciones de sistemas embebidos realizadas con el software Qsys de Altera o su aplicación anterior denominada *SOPC Builder*. El hardware del sistema embebido se genera con la herramienta de desarrollo Qsys y el software se realiza mediante la herramienta Nios II SBT. Todo el SOPC resultará en una descripción de hardware con la configuración del contenido de memoria según el software de la aplicación y que luego prosigue el flujo de diseño de Quartus II. A continuación, se detallará la utilización de la herramienta de diseño del Sistema-en-chip-programable.

QSYS Y SOPC BUILDER

Qsys (la versión anterior se denomina *SOPC Builder* (16)) es una herramienta de desarrollo, brindada por Altera, poderosa y versátil para la creación de sistemas basados en *soft-processor* a partir de una biblioteca de componentes. De esta manera, permite definir y generar sistemas-en-chips-programables (SOPC) de manera más rápida y con un conocimiento más acabado que con las herramientas tradicionales de edición en modo texto de archivos de descripción de hardware (VHDL o Verilog).

Qsys es accesible dentro del entorno Quartus II. Es necesario previamente crear o abrir un proyecto, ya que el resultado de la generación de Qsys se instancia en el proyecto activo de Quartus II.

Para acceder a esta herramienta, se debe:

- a. Abrir Quartus II.
- b. Abrir o crear el proyecto que contendrá el módulo del diseño del SOPC.
- c. Abrir Qsys mediante la barra de menú y el comando **Tools > Qsys**.

Toda la información utilizada por Qsys es almacenada en una estructura de directorio que contiene todo el proyecto.

DIRECTORIO DE PROYECTO EN QSYS

Al comenzar un proyecto en Qsys, se crea una carpeta que contendrá todos los archivos referidos al sistema creado en dicho proyecto.

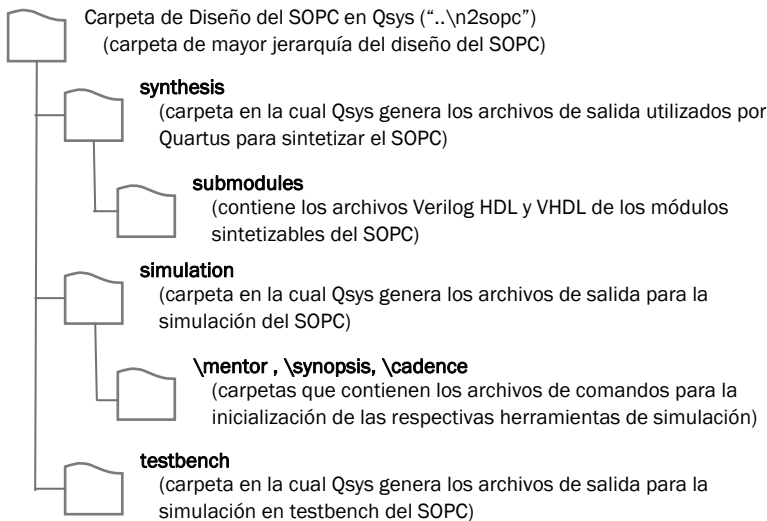


Figura 5-2: Estructura de carpetas de proyecto SOPC de Qsys.

La información de los componentes incluidos en el proyecto y la información configurada en Qsys es almacenada en un archivo de texto con el nombre del proyecto y con extensión **.SOPC**. El software Qsys almacena y edita este archivo de acuerdo a la configuración que el usuario realiza sobre el diseño. Cuando el sistema es generado, entonces Qsys crea un archivo con extensión **.SOPCINFO** (en versiones anteriores de *SOPC Builder*, era un archivo con extensión **.PTF**) que contiene toda la información de los componentes del sistema necesaria para las herramientas de generación del software que se ejecutará sobre la arquitectura generada.

DISEÑO DE SOPC EN QSYS

En principio, Qsys permite el diseño de sistemas basados en el procesador de Altera Nios II, aunque también puede ser utilizado tanto para la creación de sistemas con uno o más de estos procesadores, con procesadores de otros diseñadores (por ejemplo, ARM), o, eventualmente, diseños que no posean ningún procesador.

Qsys automatiza la tarea de integrar componentes de hardware en un sistema mayor. Si se utilizan las herramientas tradicionales, el diseñador debe escribir en los archivos HDL de mayor jerarquía las interconexiones entre los distintos componentes o debe especificarlas de manera gráfica uniendo manualmente los distintos puntos de conexión. Usando esta herramienta se especifican los componentes y sus características en una interfaz gráfica muy amigable al diseñador y Qsys genera un archivo HDL que contiene todos los componentes y sus interconexiones. Actúa, de esta manera, como sistema *front-end* permitiendo una visualización gráfica y una interacción más amigable con los distintos dispositivos y sus características.

Así, es posible describir a Qsys como dos subsistemas separados (Figura 5-3).

- **La interfaz gráfica de usuario (GUI)** para el listado y ordenamiento de los componentes del sistema. Dentro de esta GUI, cada componente puede tener su propia interfaz de configuración. La GUI crea una descripción del sistema en un archivo **.ptf** o **.sopcinfo** de acuerdo a la versión del Quartus II.
- **El programa generador** que convierte el sistema descrito en el archivo **.ptf** o **.sopcinfo** en una implementación de hardware. Este programa generador tiene como tarea principal crear la descripción del sistema en HDL y sintetizarla para el dispositivo lógico programable especificado.

Durante el proceso de generación, además de generarse los archivos de descripción de hardware anteriormente mencionados, también se crea un conjunto de archivos para la simulación del sistema así como, cuando los componentes lo indiquen, archivos de software los cuales incorporan instrucciones a nivel máquina para cada tipo de dispositivo sintetizado. En la Figura 5-3, se describen las relaciones entre los dos módulos del programa y las distintas clases de archivos utilizados en cada etapa.

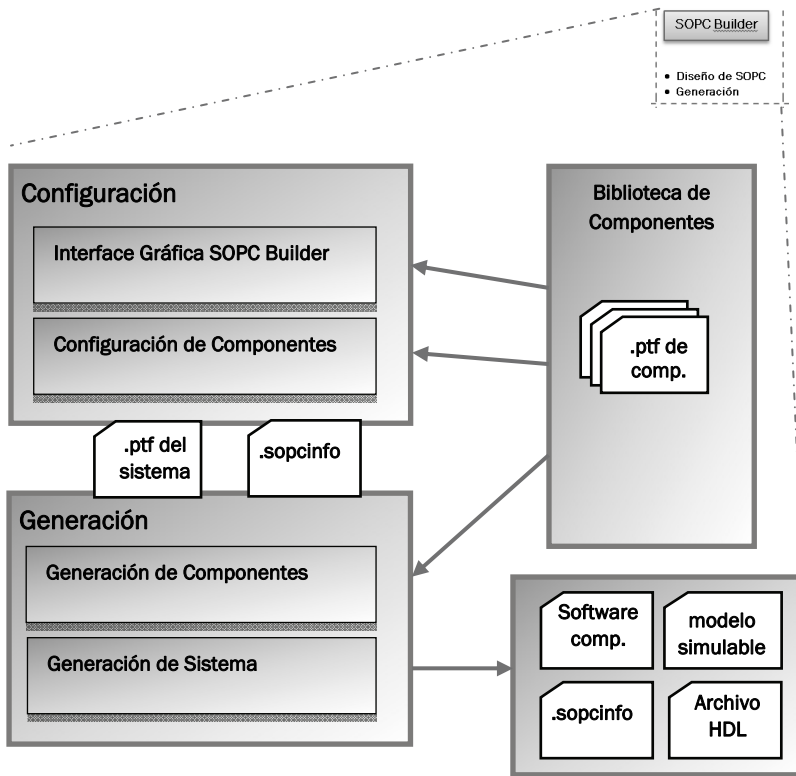


Figura 5-3: Esquema de funcionamiento y archivos asociados a Qsys.

Los componentes disponibles en Qsys, para la creación de un sistema, pueden provenir de diferentes orígenes como ser:

- Componentes automáticamente instalados con el Quartus II. (Capítulo 4).
- Componentes desarrollados por otras compañías y que son importados en Qsys.
- Componentes incluidos en *kits* de desarrollo de Altera para soporte de dispositivos específicos (controlador de video, códec de audio especiales, Ethernet de alta velocidad, etc.).
- Componentes desarrollados por el mismo usuario en algún lenguaje de descripción de hardware para componentes propios (por ejemplo, interfaces de entrada/salida para algún propósito particular).

La biblioteca de componentes brindada por Altera, denominada EDS (Desarrollo de Software Embebido, *Embedded Software Development*, en inglés), brinda una lista muy completa orientada a SOPC con componentes desarrollados por Altera y por otras compañías, pudiendo

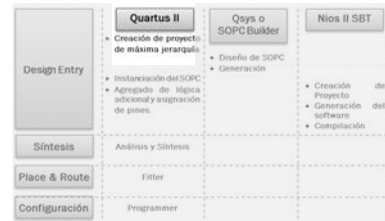
agregarse posteriormente más componentes según las necesidades. Los componentes pueden ser simples bloques de lógica o pueden ser subsistemas complejos que se generen dinámicamente mediante parámetros configurables. Muchos de estos componentes incluyen interfaces gráficas para su configuración y programas de generación de HDL que pueden entregar versiones sintetizables tanto en VHDL como en Verilog HDL.

REALIZACIÓN DEL FLUJO DE DISEÑO DE UN SOPC

El diseño de un SOPC debe realizarse dentro de un proyecto de diseño de hardware en Quartus II. Para ello, debe crearse primero un proyecto tal cual se detalló en el Capítulo 3.

Creación del proyecto de máxima jerarquía

La creación de un proyecto en Qsys debe realizarse dentro de un proyecto de mayor jerarquía en Quartus II (Figura 5-1). Como un ejemplo simple, crearemos un proyecto en Quartus II que denominaremos “n2sopc”. Para ello, realizaremos los pasos realizados en el Capítulo 3 de acuerdo a los siguientes pasos:



- 1) Ejecutar Quartus II, cerrando la página de bienvenida.
- 2) Acceder a la guía de nuevo proyecto (Figura 3-4).
- 3) Seleccionar **Next >** en la pantalla de introducción a la creación de proyecto (Figura 3-5).
- 4) Especificar el directorio “c:\n2sopc” y n2sopc para el nombre del proyecto y el nombre de la entidad de mayor jerarquía (Figura 3-6). Seleccionar **Next >** para continuar con el proceso de creación de proyecto. Seleccionar **Yes** para indicar que se desea crear el directorio del proyecto.
- 5) Introducir el nombre “n2sopc.bdf” en el campo “File name” y seleccionar el botón **Add** (Figura 3-7). Esta acción determinará que el archivo de mayor nivel jerárquico será una descripción esquemática para mejor representación del SOPC que se desarrollará. Seleccionar **Next>** para continuar con la creación del proyecto.
- 6) Seleccionar la familia y el dispositivo FPGA deseado. En caso de no contar con algún dispositivo FPGA en particular, la selección de la Figura 3-8 es conveniente para la realización de este ejemplo. Seleccionar **Next>** para continuar con la creación del proyecto.
- 7) Seleccionar **ModelSim-Altera** en el campo “Tool Name” de la fila “Simulation”, “Verilog HDL” en el campo “Format(s)” de la misma fila y deseleccionar la opción **Run gate-level simulation automatically after compilation** en el campo **Run Tool Automatically** de la misma fila, de igual manera que en la Figura 3-9. Seleccionar **Next>** para continuar con la creación del proyecto.
- 8) Seleccionar **Finish** para finalizar la guía de creación de proyecto nuevo (Figura 3-10).

- 9) Crear el archivo “nios2_sopc.bdf” mediante el comando de la barra de menú **File > New** y seleccionando la opción **Block Diagram/Schematic File** para indicar que el archivo nuevo es un esquemático. Seleccionar **OK** y, en la barra de menús, el comando **File > Save As** y guardar el archivo con el nombre “n2sopc.bdf” en la carpeta “c:\n2sopc\”. De esta manera, el archivo de mayor jerarquía fue creado y guardado en la carpeta del proyecto (Figura 3-12).

Creación del diseño del SOPC

Como todo proyecto de diseño, es necesario crear un archivo de diseño de la mayor jerarquía que Quartus II sintetizará posteriormente para producir el archivo de configuración del dispositivo FPGA correspondiente. Este archivo de diseño contendrá como uno de sus componentes la definición del SOPC que se creará mediante Qsys. Para lograr este objetivo, debe ejecutarse el Qsys mediante los siguientes pasos:

- 1) Abrir, en la ventana principal, el archivo “n2sopc.bdf” realizando un doble clic en el nombre del archivo en el navegador del proyecto (Figura 3-3).
- 2) Seleccionar de la barra de menú el comando **Tools > Qsys**. Este comando comenzará la ejecución de Qsys.

Las nuevas versiones de Quartus II recomiendan la utilización de Qsys para nuevos proyectos, siendo *SOPC Builder* la aplicación anterior que brindaba Altera. La plataforma Qsys presenta una interfaz similar a *SOPC Builder*, por lo que su utilización no presenta mayores inconvenientes para los usuarios acostumbrados a *SOPC Builder*.

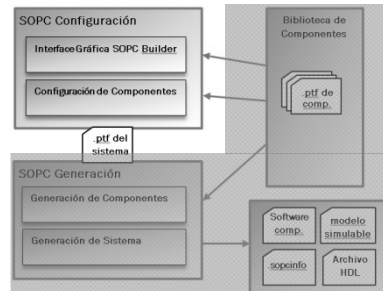
En la aplicación Qsys, es conveniente guardar el proyecto periódicamente. El comando **guardar** se realiza mediante la ejecución del comando **File > Save**. En caso de que el proyecto Qsys no haya sido guardado con anterioridad, aparecerá una pantalla de exploración de Windows. En el ejemplo propuesto, se almacenará en el directorio “c:\n2sopc\” con el nombre “sopcqsys”.



CONFIGURACIÓN DEL SOPC

INTERFAZ GRÁFICA DE QSYS

La interfaz gráfica de usuario (GUI) Qsys presenta un ambiente en el cual se especifican los componentes, sus características y cómo se interconectan, creando así el archivo .ptf o .sopcinfo. Este archivo de texto puede ser editado con cualquier editor de textos, pero debe tenerse la precaución de cerrar Qsys para evitar que dos programas modifiquen un mismo archivo



pudiendo provocar resultados indeseables. La Figura 5-4 muestra la interfaz gráfica de software Qsys y sus sectores principales.

Puede observarse que la ventana principal de Qsys posee diferentes solapas, siendo **System Contents** la que se encuentre activa por defecto y la que listará todos los componentes que contendrá el SOPC. La primera parte del diseño abarcará principalmente esta solapa.

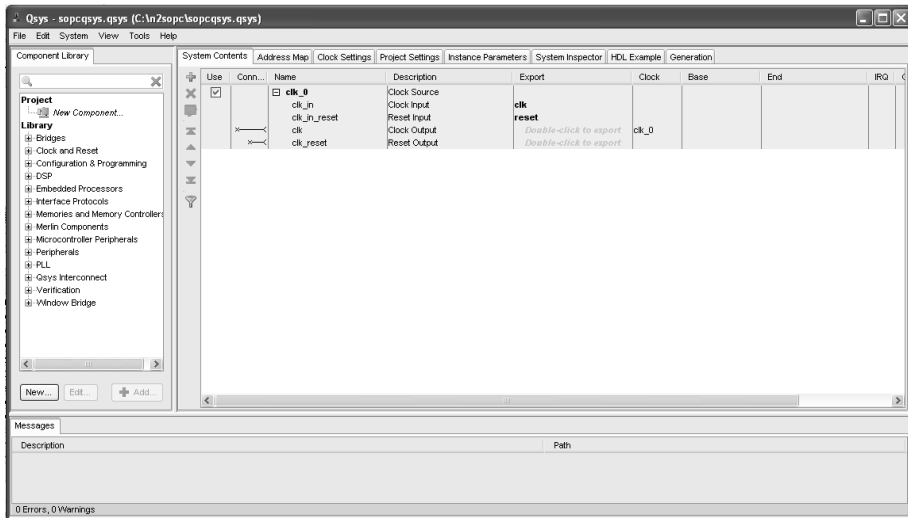
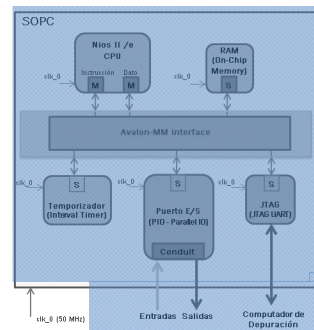


Figura 5-4: Pantalla de Qsys.

Definición de la fuente de reloj externa

Una de las especificaciones más importante que debe realizarse es la fuente de reloj que poseerá el sistema. Una de las señales de reloj debe ser externa, mientras que las restantes pueden ser generadas mediante la utilización de componentes PLL incluidos en el dispositivo FPGA. Qsys incluye por defecto una fuente de reloj y *reset* externa incluida como componente **Clock Source** en la ventana principal (Figura 5-5).



Use	Conn...	Name	Description	Export	Clock	Base	End	IRQ
☒		clk_0	Clock Source					
		clk_in	Clock Input	clk				
		clk_in_reset	Reset Input	reset				
		clk	Clock Output		clk_0			
		clk_reset	Reset Output					

Figura 5-5: Componente Clock Source agregado por defecto por Qsys.

Como opción por defecto, el cuadro muestra una señal externa de reloj a la que se le asignó el nombre *clk_0*. El nombre del componente **Clock Source** puede ser modificado seleccionando el campo con el *mouse* y escribiendo el nuevo nombre. La frecuencia de reloj externa puede configurarse realizando doble clic sobre el componente. Esta acción abrirá la pantalla de configuración del componente **Clock Source** (Figura 5-6).

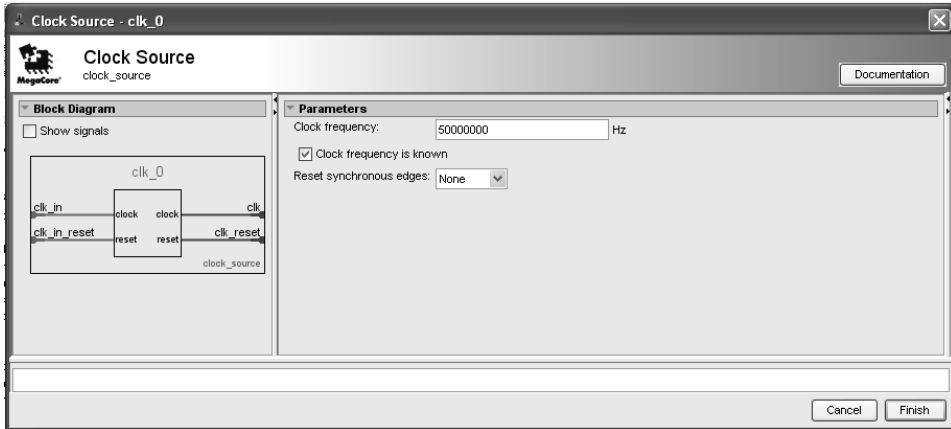


Figura 5-6: Configuración de componente Clock Source.

Para el ejemplo propuesto, se definirá en el campo “Clock frequency” el valor 50000000 para indicar una señal de reloj externa de una frecuencia de 50MHz. La inclusión de más señales de reloj externas se realiza mediante la incorporación de componentes **Clock Source** adicionales. Seleccionar el botón **Finish** una vez realizada la configuración.

CONFIGURACIÓN DE LOS COMPONENTES

La etapa de diseño del SOPC se realiza mediante la selección y configuración de los componentes de biblioteca que brinda Qsys. Todos los componentes de biblioteca se encuentran disponibles en la ventana **Component Library** de Qsys (Figura 5-7).

Los componentes están agrupados en diferentes categorías para una más eficiente organización. Un componente de biblioteca es un **módulo de diseño** que Qsys reconoce y puede integrar al sistema automáticamente. Es posible definir y utilizar componentes a medida en un diseño (mediante desarrollos propios o mediante la adquisición de diseños a otros diseñadores o empresas), además de los que se encuentran en la distribución brindada por Altera. Cuando se incluyen los componentes para generar un diseño, Qsys automáticamente los conecta entre sí y crea un archivo de mayor nivel HDL que lo denomina **módulo de sistema**. En Quartus II, este archivo será instanciado como un bloque que luego será compilado con toda la restante lógica incluida en el proyecto de Quartus II.

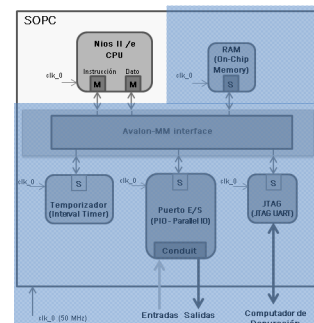


Figura 5-7: Ventana de componentes de biblioteca.

En los siguientes pasos, se detallará el procedimiento para diseñar el sistema de la Figura 4-2.

Selección y configuración del procesador Nios II

El diseño propuesto requiere la selección y configuración de un procesador Nios II. Para incorporarlo al diseño, se debe dirigir a los componentes de biblioteca (Figura 5-7), expandir la categoría **Embedded Processors** y seleccionar el componente **Nios II Processor**. Al hacer clic en el botón **Add...**, se abrirá la pantalla de configuración del procesador Nios II (Figura 5-8). Esta pantalla presenta las alternativas de configuración en diferentes solapas:



- Solapa Core Nios II:** permite seleccionar la versión de Nios II que se desea implementar (detalle en la página 73). Para el diseño propuesto, puede seleccionarse la versión *economy* mediante la opción **Nios II/e**. Puede observarse que, en la parte inferior de la pantalla de configuración, hay un mensaje indicando que los vectores de *Reset* y *Exception* no pueden especificarse hasta que se hayan incluido componentes de memoria al diseño. Este mensaje será resuelto más adelante en el proyecto. En la sección **Core Nios II**, puede configurarse si se implementarán las unidades de manejo

de memoria (MMU) o de protección de memoria (MPU), pero solo están disponibles para la versión *fast* (/f) del procesador Nios II. Las restantes solapas de configuración del procesador Nios II permiten la especificación de características avanzadas, no disponibles para la versión *economy* por lo que en esta etapa podría finalizarse la configuración seleccionando el botón **Finish**. Sin embargo, a continuación, se detallará cada una de las restantes solapas.

- **Solapa Caches and Memory Interfaces:** permite definir las características de las memorias caché del procesador. La versión **Standard** (/s) permite la especificación solo de la memoria caché de instrucciones, mientras que la versión *fast* (/f) permite la especificación de ambas memorias caché: de datos e instrucciones. La versión *economy* (/e) no permite la especificación de ninguna de las memorias cache.
- **Solapa Advanced Features:** permite especificar los mecanismos con los que se manejarán las interrupciones del sistema. La versión *fast* (/f) es la única versión que soporta el módulo de interrupciones externas. Las dos versiones restantes solo soportan interrupciones internas, las cuales pueden ser conectadas en Quartus II al módulo del sistema que genera el *SOPC Builder*.

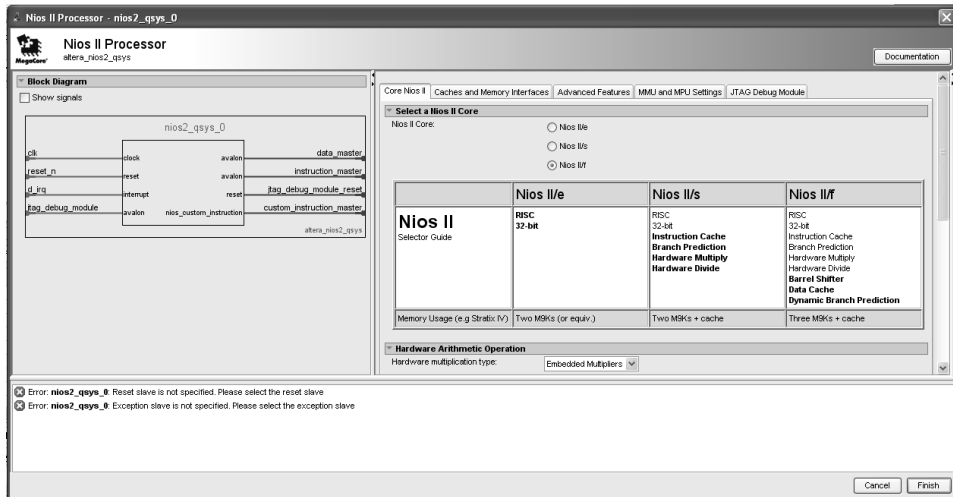


Figura 5-8: Pantalla 1 de configuración del procesador Nios II.

- **Solapa MMU and MPU Settings:** permite especificar las características de la unidad de manejo de memoria (MMU) o de la unidad de protección de memoria cuando se implementa la versión *fast* (/f) del procesador Nios II.
- **Solapa JTAG Debug Mode:** permite especificar los diferentes niveles de depuración soportados por el módulo de depuración JTAG. De acuerdo a la versión del procesador Nios II seleccionada, se habilitarán diferentes niveles

de depuración con consecuente incremento de los recursos de hardware del dispositivo FPGA necesarios para su implementación.

La característica de *soft-processor* permite una configuración flexible y adaptable a las necesidades de la aplicación y de los recursos con los que cuenta el dispositivo FPGA en el que se implementará.

Al finalizar, la configuración del procesador Nios II, la ventana de diseño contendrá una instancia del procesador Nios II que, por defecto, Qsys nombrará **nios2_qsys_0** (Figura 5-9). Si se desea renombrar la instancia; entonces, con el *mouse*, se debe seleccionar el nombre y con el menú desplegado mediante un clic en el botón derecho del *mouse*, elegir el comando **Rename**. Esto permitirá ingresar el nombre que se desea dar a la instancia del procesador Nios II.

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ
☒		clk_0	Clock Source					
		clk_in	Clock Input	clk				
		clk_in_reset	Reset Input	Double-click to export				
		clk	Clock Output	Double-click to export	clk_0			
		clk_reset	Reset Output	Double-click to export				
☒		nios2_qsys_0	Nios II Processor					
		clk	Clock Input	Double-click to export	clk_0			
		reset_n	Reset Input	Double-click to export	[clk]			
		data_master	Avalon Memory Mapped Master	Double-click to export	[clk]			
		instruction_master	Avalon Memory Mapped Master	Double-click to export	[clk]			
		jtag_debug_module_re...	Reset Output	Double-click to export	[clk]			
		jtag_debug_module	Avalon Memory Mapped Slave	Double-click to export	[clk]			
		custom_instruction_m...	Custom Instruction Master	Double-click to export	[clk]			

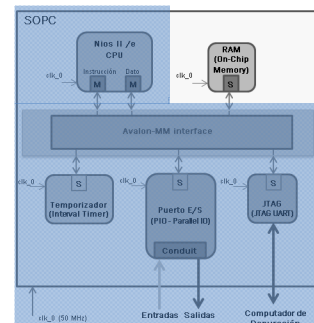
Figura 5-9: Especificación de la señal de reloj del procesador Nios II.

Hay que notar que el procesador Nios II requiere una señal de reloj y que puede seleccionarse mediante la lista que se despliega en la columna 'Clock'. Según el diseño propuesto, se debe elegir la señal de reloj **clk_0** (Figura 5-9) con una frecuencia de 50MHz.

También deben conectarse la señal de reset y los canales de datos e instrucciones del procesador Nios II como se muestra en la Figura 5-9. Para ello, se debe posicionar con el *mouse* sobre la intersección de las líneas de las señales hasta se habiliten los cruces y hacer clic con el *mouse* sobre la circunferencia correspondiente para que esta se rellene indicando que las líneas están conectadas.

Selección y configuración de la memoria RAM interna del dispositivo FPGA

Todo sistema embebido requiere de memoria RAM para poder almacenar la información de estado del sistema. Los dispositivos FPGA permiten implementar, hasta determinadas capacidades, memorias RAM mediante los bloques de memoria incluidos en el dispositivo. La utilización de bloques de memoria interna del dispositivo FPGA, denominados *on-chip memory blocks*, presenta ventajas frente a la implementación de memoria externa al dispositivo FPGA, como ser:



- Tiempo de acceso más rápido comparado con memorias externas al dispositivo FPGA.
- Instanciación automática por Qsys evitando que el diseñador deba especificar manualmente las conexiones del bloque.
- Inicialización del contenido de la memoria cuando el dispositivo FPGA es energizado. Esta posibilidad permite que una memoria, aunque sea especificada como RAM, se le asigne un archivo **‘.hex’** que inicialice el contenido de la memoria al momento de encendido. Cuando una memoria externa es utilizada, se debe inicializar su contenido mediante la ejecución de una rutina o procedimiento de carga (detalle en el Capítulo 7).
- Soporte para la implementación de memorias Dual-Port para acceso simultáneo de dos componentes maestros a su contenido.

Para agregar un módulo de memoria interna, se debe expandir la categoría **Memories and Memory Controllers** de los componentes de la biblioteca (Figura 5-7), expandir la subcategoría **On-Chip** y seleccionar el componente **On-Chip Memory (RAM or ROM)**. Al hacer clic en el botón **Add...**, se abrirá la pantalla de configuración de memoria interna del dispositivo FPGA (Figura 5-10).

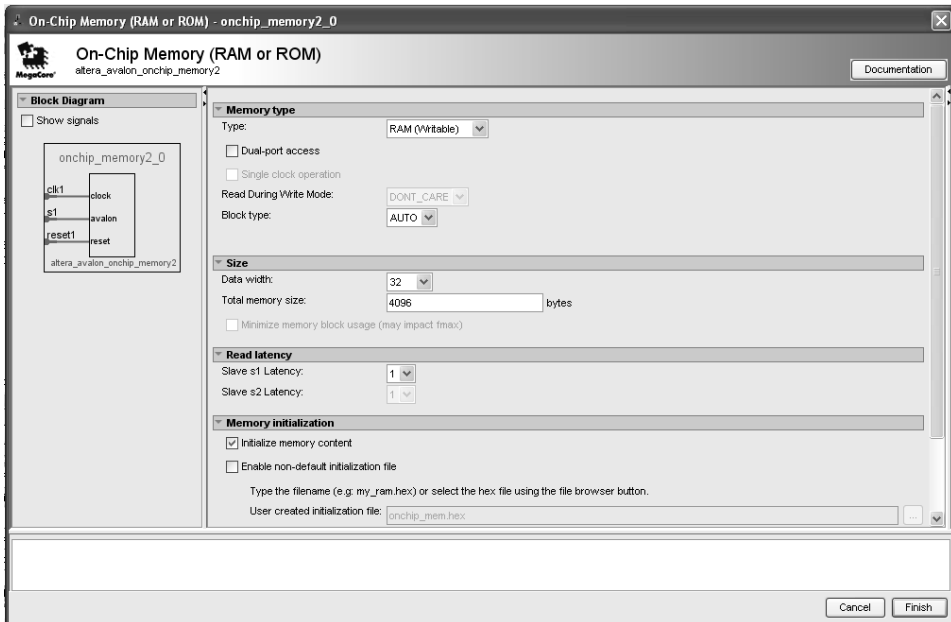


Figura 5-10: Pantalla de configuración memoria interna dispositivo FPGA.

La pantalla de configuración permite diferentes alternativas para la utilización de los bloques de memoria interna del dispositivo FPGA. Entre ellas, se encuentran:

- Tipo de Memoria (*Memory Type*)

Los bloques de memoria interna del dispositivo FPGA pueden ser configurados en tres tipos de estructuras (*Type*):

- RAM: permite crear un módulo de memoria que puede leerse y escribirse.
- ROM: permite crear un bloque de memoria que solo puede leerse y su contenido no puede modificarse durante el tiempo de corrida del sistema.
- Dual-port: permite crear un módulo de memoria al cual pueden acceder dos maestros simultáneamente.

En el ejemplo propuesto, debe seleccionarse **RAM (Writable)** sin seleccionar la opción **Dual-port Access**.

La implementación del módulo de memoria puede realizarse mediante la utilización de diferentes alternativas de recursos del dispositivo FPGA para tal fin. La opción **Block Type** permite definir los recursos del dispositivo FPGA que deben ser utilizados para la implementación del módulo de memoria interna. Generalmente, es aconsejable seleccionar **Auto** para permitir que Qsys determine la manera más eficiente de implementación.

- Capacidad de la memoria (*Size*)

La capacidad de la memoria se define por:

- **Data width**: determina el ancho de datos de la memoria. Las opciones disponibles son 8, 16, 32, 64, 128, 256, 512 y 1024 bits. Es conveniente definir el ancho de datos conforme al ancho de datos del dispositivo maestro que accede más frecuentemente. Cuando se conecta a un procesador Nios II, es conveniente especificarlo en **32 bits** para obtener la mejor eficiencia en la transferencia de datos.
- **Total memory size**: determina la capacidad de la memoria en *bytes*. Por supuesto, el dispositivo FPGA debe contener suficientes recursos de bloques de memoria para poder implementar la capacidad deseada.

En el ejemplo propuesto, especificar 32 bits en el campo **Data width** y 32.768 en el campo **Total Memory Size**.

- **Latencia de lectura (*Read Latency*)**: permite especificar la latencia de la lectura para transferencias que utilizan *pipeline*. El *pipeline* mejora la tasa de transferencia, pero puede perjudicar la latencia en la lectura.

En el ejemplo propuesto, especificar 1 en el campo **Slave 1s Latency**.

- **Inicialización de la memoria (*Memory Initialization*)**: permite establecer archivos de imagen de memoria (en formato **.hex**). Esta alternativa es imprescindible cuando se definen memorias tipo ROM y es de utilidad cuando se especifican memorias del tipo RAM.

En el ejemplo propuesto, seleccionar solo la opción **Initialize memory content** permitiendo a Qsys indicar si existe necesidad de inicialización de la memoria con el archivo de imagen de memoria por defecto.

Luego de realizar la configuración deseada (Figura 5-10), seleccione el botón **Finish**. El componente **onchip_memory2_0** estará incorporado en la ventana de diseño del sistema. Es conveniente renombrar el módulo de memoria a un nombre más adecuado. Para ello, se debe hacer clic sobre el nombre con el botón derecho del mouse y seleccionar el comando **Rename** del menú sensitivo que aparece (Figura 5-11). Posteriormente, ingresar el nombre **memoria_ram** en el campo de edición habilitado. Debe conectarse, en la columna "Connections", la señal "clk1" de entrada de la memoria a la señal de reloj "clk_0" del componente "Clock Source" y el puerto Avalon "s1" de la memoria a las salidas "instruction_master" y "data_master" del procesador "nios2_qsys_0" como muestra la Figura 5-11.

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Opcode
☒		clk_0	Clock Source						
		clk_in	Clock Input	clk					
		clk_in_reset	Reset Input	Double-click to export					
		clk	Clock Output	Double-click to export	clk_0				
		clk_reset	Reset Output	Double-click to export					
☒		nios2_qsys_0	Nios II Processor						
		clk	Clock Input	Double-click to export	clk_0				
		reset_n	Reset Input	Double-click to export	[clk]				
		data_master	Avalon Memory Mapped Master	Double-click to export	[clk]	IRQ 0	IRQ 31		
		instruction_master	Avalon Memory Mapped Master	Double-click to export	[clk]				
		jtag_debug_module_re...	Reset Output	Double-click to export	[clk]				
		jtag_debug_module	Avalon Memory Mapped Slave	Double-click to export	[clk]				
		custom_instruction_m...	Custom Instruction Master	Double-click to export	[clk]	0x800	0x7fff		
☒		memoria_ram	On-Chip Memory (RAM or ROM)						
		clk1	Clock Input	Double-click to export	clk_0				
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk1]	0x0	0x7fff		
		reset1	Reset Input	Double-click to export	[clk1]				

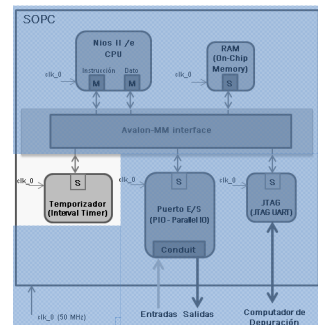
Figura 5-11: Pantalla de configuración memoria interna dispositivo FPGA.

Una vez insertada la memoria RAM del sistema, se procederá a incluir los restantes periféricos del sistema.

Selección y configuración del temporizador del sistema (Interval Timer)

La mayoría de los sistemas requieren un periférico que permita temporizar eventos en el procesador. La incorporación de un periférico temporizador permite al sistema mantener una base de tiempo y temporizar la ejecución de aplicaciones implementadas. La Biblioteca de Componentes de Altera brinda un periférico para la implementación de intervalos temporizados, denominado **Interval Timer**.

Para insertar el temporizador, se debe expandir la categoría **Peripherals** de los componentes de la biblioteca (Figura 5-7), expandir la sub-categoría **Microcontroller Peripherals** y seleccionar el componente **Interval Timer**. Al hacer clic en el botón **Add ...**, se abrirá la pantalla de configuración del temporizador de intervalos (Figura 5-12).



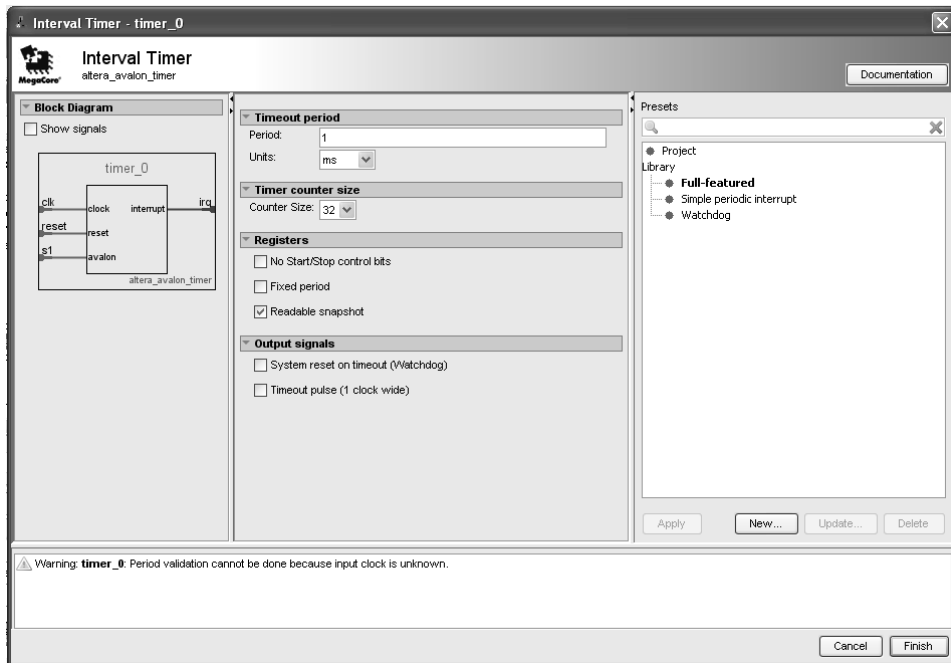


Figura 5-12: Pantalla de configuración del temporizador de intervalos.

La pantalla de configuración del **Interval Timer** presenta los siguiente parámetros:

- **Periodo de tiempo (Timeout Period):** especifica el valor inicial del registro de periodo que determina el intervalo de tiempo entre dos eventos producidos por el temporizador. Este valor puede ser modificado por el procesador durante el tiempo de corrida solo si la opción **Writeable period** es seleccionada. Si bien el campo de la pantalla de configuración establece este parámetro en unidades de tiempo, el registro del componente **Interval Timer** estará especificado en unidades del reloj conectado al temporizador, redondeando por exceso si el tiempo especificado no es múltiplo entero del periodo de la señal de reloj. Por ejemplo, si se especifica un periodo de tiempo de 1ms con un periodo de reloj de 30ns, entonces la cuenta del **Interval Timer** se inicializará con 33.334, la cual produciría un periodo de 1,00002ms.
- **Tamaño del contador (Timer Counter Size):** este parámetro determina la cantidad de bits con que se implementará el registro de cuenta de tiempo. Las opciones son 32 o 64 bits.
- **Opciones de hardware (Hardware):** estas opciones permiten especificar la estructura del hardware del componente **Interval Timer**. Las opciones que permite habilitar son:

- **Custom:** permite modificar el comportamiento del temporizador de acuerdo a la configuración que se realice durante el tiempo de funcionamiento del sistema.
- **Simple periodic interrupt:** permite configurar el temporizador para realizar una interrupción periódica al procesador. El periodo no puede ser modificado en tiempo de funcionamiento del sistema, pero la interrupción puede ser deshabilitada.
- **Full-feature:** con esta opción el temporizador permite modificar el periodo de cuenta y controlar el arranque y parada del temporizado.
- **Watchdog:** permite la implementación de un temporizador de *watchdog* que resetea el sistema cuando el contador llega a cero. La configuración y utilización de este tipo debe ser realizada por el diseñador sin soporte de los controladores de software que brinda Altera.
- **Opciones de registro (Register):** esta opción permite configurar la estructura hardware que poseerá el temporizador luego de sintetizado. Las opciones que brinda son:
 - **Fixed period:** si la opción no está seleccionada, entonces el periodo del temporizador puede ser modificado mediante una escritura en el registro de periodo. Si esta opción no está seleccionada, entonces el registro de periodo no se implementa en hardware.
 - **Readable snapshot:** si la opción está marcada, entonces se puede acceder para lectura al registro del contador. Si no está seleccionada, entonces el registro no es implementado en hardware y una lectura produce resultados inciertos.
 - **No Start/Stop control bits:** si la opción no está seleccionada entonces el contador puede dispararse y pararse mediante la escritura de los bits correspondientes en el registro de control. Si la opción está seleccionada, entonces el contador cuenta continuamente.
- **Opciones de Salida (Output signal):** estas opciones permiten definir el hardware requerido para implementar la función de salida del temporizador:
 - **Timeout pulse (1 clock wide):** si la opción se selecciona, la salida 'timeout_pulse' producirá un pulso en uno por un periodo de la señal de reloj cada vez que el contador llegue a cero. Si la opción no es seleccionada, entonces la salida no se implementa en hardware.
 - **System reset on timeout (watchdog):** esta opción permite especificar que el temporizador genere una señal de *reset* de la interfaz Avalon cuando la cuenta del registro contador llegue a cero. Esta opción no es válida cuando se seleccionó **Watchdog** en las opciones de hardware.

Para el ejercicio propuesto, puede configurarse el temporizador con los siguientes parámetros (Figura 5-12):

- *Timeout period*
 - Period: 1
 - Units: ms
- *Timer counter size*
 - Counter size: 32
- *Registers*
 - Fixed period: No
 - Readable snapshot: No
 - No Start/Stop control bits: Si
- *Output signals*
 - Timeout pulse (1 clock wide): No
 - System reset on timeout (Watchdog): No

Al finalizar la configuración, se debe seleccionar el botón **Finish** para que el módulo se inserte en el diseño del sistema.

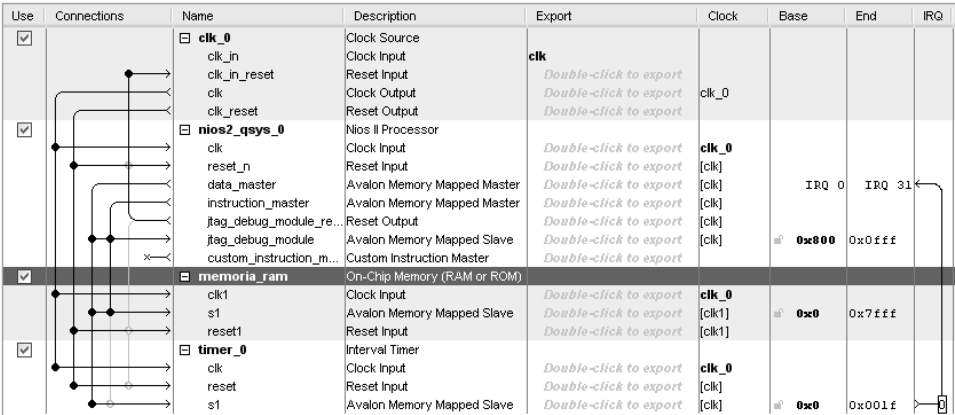
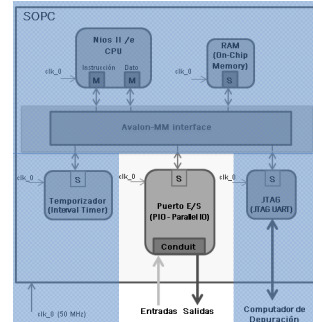


Figura 5-13: Detalle de conexión del temporizador al puente de adaptación de señales de reloj.

El temporizador debe conectarse al puerto **data_master** del procesador, por lo que se debe seleccionar, en la columna de 'Connections', la conexión del puerto esclavo **s1** del temporizador con la proveniente del puerto **data_master** del procesador. De la misma manera, se debe proceder para las señales de reloj, reset e interrupción. Debe notarse que la señal de interrupción debe conectarse a la derecha de la ventana como muestra la Figura 5-13.

Selección y configuración de un puerto de entrada/salida (PIO Parallel IO)

La mayoría de los sistemas requieren un periférico que permita manejar salidas y leer el estado de entradas digitales. La incorporación de un periférico de Entrada/Salida permite una interfaz al sistema para conectarse con el mundo exterior. La biblioteca de componentes de Altera brinda un periférico para la implementación de puertos de Entrada/Salida, denominado **PIO (Parallel I/O)**.



Para insertar el puerto de Entrada/Salida, se debe expandir la categoría **Peripherals** de la biblioteca de componentes (Figura 5-7), expandir la subcategoría **Microcontroller Peripherals** y seleccionar el componente **PIO (Parallel I/O)**. Al hacer clic en el botón **Add ...**, se abrirá la pantalla de configuración del temporizador de intervalos (Figura 5-14).

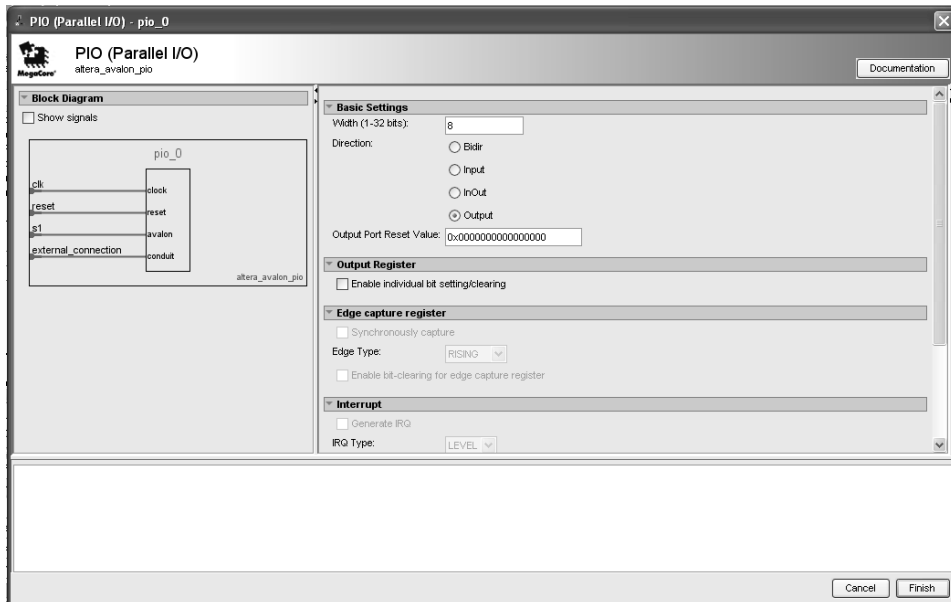


Figura 5-14: Pantalla de configuración del puerto de Entrada/Salida.

La pantalla de configuración del **PIO Parallel IO** presenta los siguientes parámetros:

- **Configuración básica (Basic Settings):** permite definir las características y cantidad de puertos de entrada y de salida:
 - **Width:** especifica la cantidad de entradas o salidas que contendrá el periférico. Es un número entero entre 1 y 32.

- **Direction:** especifica si los puertos son: (1) del tipo bidireccionales (**Bidir**) en los cuales se lee y se escribe el valor del mismo puerto del dispositivo, (2) del tipo entrada (**Input**) en los cuales solo se puede leer su valor, (3) del tipo salida (**Output**) en los cuales se puede escribir el valor o (4) del tipo entrada/salida (**Inout**) los cuales posee puertos de entrada y puertos de salida independientes.
 - **Output Port Reset Value:** permite especificar el valor que tendrán los puertos definidos como salida en el momento en que se produzca un **reset** del sistema.
- **Registro de salida (Output Register):** esta opción permite especificar si los puertos de salida pueden ser direccionados por bit. Esto significa que si se desea poner a uno o a cero una salida, no es necesario conocer el valor de las restantes salidas del puerto para realizarlo. Si esta opción no es seleccionada, el hardware no implementa los registros necesarios para tal función de direccionamiento.
- **Registro de entrada (Edge capture register):** esta opción permite especificar la captura de flancos ascendentes (**Rising**), descendentes (**Falling**) o ambos (**Any**). Esta opción no está disponible para puertos de salida.
 - **Enable bit-clearing for edge capture register:** si esta opción está seleccionada, entonces cada entrada que detecta flanco puede ser reinicializada mediante un direccionamiento por bit.
- **Interrupción (Interrupt):** si esta opción está seleccionada, entonces el periférico genera una señal de interrupción al procesador ante un evento en las señales de entrada. Si la opción es seleccionada, entonces se crea un registro de máscara para enmascarar las señales de entrada que no se desean que produzcan interrupción al procesador.
- **Cableado para pruebas (Testbench wiring):** esta opción permite especificar si se podrán modificar el valor de las entradas del periférico, cuando se simule o si se les fija un determinado valor.

Para el ejercicio propuesto, puede configurarse el componente “Parallel I/O” con los siguientes parámetros (Figura 5-14):

- **Basic Settings**
 - **Width (1-32 bits):** 8
 - **Direction:** InOut
 - **Output port reset value:** 0x0000000000000000
- **Output Register**
 - **Enable individual bit setting/clearing:** No
- **Edge capture register**
 - **Synchronously capture:** No

- Interrupt
 - Generate IRQ: No
- Testbench wiring
 - Hardwire PIO inputs in testbench: Si
 - Drive inputs to: 0x0000000000000000

Al finalizar la configuración, se debe seleccionar el botón **Finish** para que el módulo se inserte en el diseño del sistema.

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ
☒		clk_reset nios2_qsys_0 clk reset_n data_master instruction_master jtag_debug_module_re... jtag_debug_module custom_instruction_m...	Reset Output Nios II Processor Clock Input Reset Input Avalon Memory Mapped Master Avalon Memory Mapped Master Reset Output Avalon Memory Mapped Slave Custom Instruction Master	Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export Double-click to export	clk_0 [clk] [clk] [clk] [clk] [clk] [clk]	IRQ 0	IRQ 31	
☒		memoria_ram clk1 s1 reset1	On-Chip Memory (RAM or ROM) Clock Input Avalon Memory Mapped Slave Reset Input	Double-click to export Double-click to export Double-click to export	clk_0 [clk1] [clk1]	0x0	0x7fff	
☒		timer_0 clk reset s1	Interval Timer Clock Input Reset Input Avalon Memory Mapped Slave	Double-click to export Double-click to export Double-click to export	clk_0 [clk] [clk]	0x001f		
☒		pio_0 clk reset s1 external_connection	PIO (Parallel I/O) Clock Input Reset Input Avalon Memory Mapped Slave Conduit	Double-click to export Double-click to export Double-click to export Double-click to export	clk_0 [clk] [clk] [clk]	0x0	0x000f	

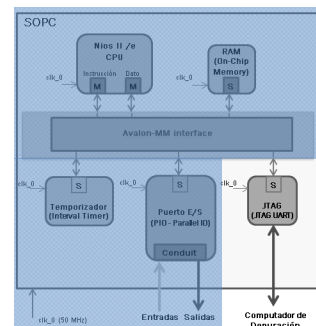
Figura 5-15: Detalle de conexión del temporizador al puente de adaptación de señales de reloj.

De manera similar a la que se realizó con el temporizador, debe conectarse el puerto esclavo **s1** del periférico **PIO Parallel IO** al puerto maestro **data_master** del procesador. También debe especificarse la señal de reloj y de reset del periférico **PIO parallel IO** como se muestra en la Figura 5-15.

El diseñador del sistema puede insertar tantos periféricos **PIO (Parallel I/O)** como crea conveniente para interconexión del sistema con la aplicación que debe implementar.

Selección y configuración de un puerto de comunicaciones JTAG (JTAG UART)

El componente **JTAG UART** permite comunicar, a través de un enlace serie JTAG, al sistema implementado en un dispositivo FPGA, basado en el procesador NIOS II y una computadora personal para la programación y depuración del diseño. Altera brinda un sistema operativo básico, denominado HAL (Capa de abstracción de hardware, *Hardware Abstraction Layer*, en inglés) que permite el acceso a esta interfaz



mediante la ejecución de rutinas de la biblioteca estándar de ANSI C (**stdio.h**). La inclusión de este componente en el ejemplo permitirá una comunicación flexible y eficiente para la prueba y depuración del diseño propuesto.

Para insertar el componente **JTAG UART**, se debe expandir la categoría **Interface Protocols** de la biblioteca de componentes (Figura 5-7), expandir la subcategoría **Serial** y seleccionar el componente **JTAG UART**. Al hacer clic en el botón **Add ...**, se abrirá la pantalla de configuración del temporizador de intervalos (Figura 5-16).

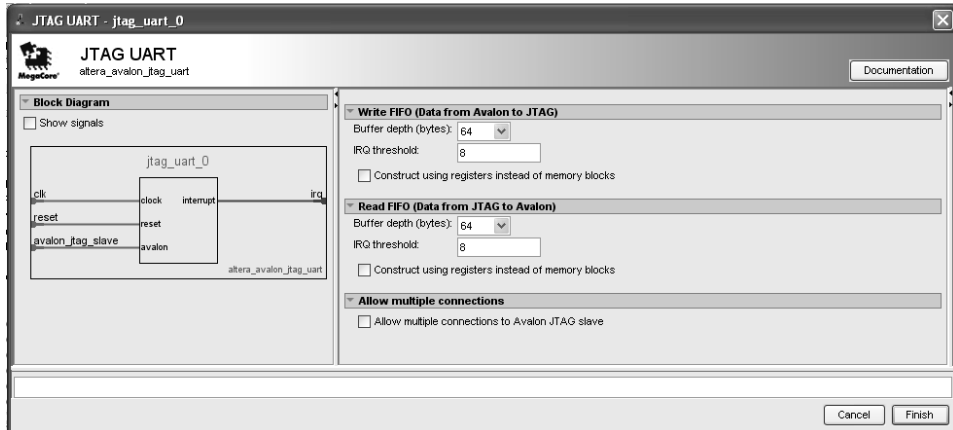


Figura 5-16: Pantalla de configuración del puerto de comunicación JTAG UART.

La pantalla de configuración del componente **JTAG UART** presenta los siguientes parámetros:

- **Configuración de la cola de escritura (Write FIFO (Data from Avalon to JTAG)):** configura la transferencia de datos desde la interfaz Avalon a la computadora personal. Las opciones disponibles son:
 - **Buffer depth (bytes):** permite definir la profundidad de la cola de almacenamiento entre 8 y 32.768 bytes en potencias de 2, aunque valores mayores que 64 son raramente utilizados.
 - **IRQ Threshold:** este parámetro determina la cantidad de bytes por debajo del cual el componente genera una interrupción al procesador para que este ejecute la rutina encargada de escribir más datos a la cola. Si es muy bajo, puede ser que la cola se vacíe antes de que la rutina recargue la cola. Si es muy elevado, entonces el componente estará interrumpiendo frecuentemente al procesador, disminuyendo su eficiencia para ejecutar las restantes tareas del sistema. Un valor típico es 8 bytes.
 - **Construct using registers instead of memory blocks:** esta opción permite seleccionar si la cola de escritura será implementada mediante registros en el dispositivo FPGA (si está seleccionada) o si se implementará mediante bloques de memoria (si no está seleccionada).

- **Configuración de la cola de lectura (*Read FIFO (Data from JTAG to Avalon)*):** configura la transferencia de datos desde la computadora personal a la interfaz Avalon. Las opciones disponibles son:
 - **Buffer depth (bytes):** permite definir la profundidad de la cola de almacenamiento entre 8 y 32.768 bytes en potencias de 2, aunque valores mayores que 64 son raramente utilizados.
 - **IRQ Threshold:** este parámetro determina la cantidad de bytes por encima del cual el componente genera una interrupción al procesador para que este ejecute la rutina encargada de leer datos a la cola. Si es muy alto, puede ser que la cola se llene antes que la rutina lea la cola. Si es muy bajo, entonces el componente estará interrumpiendo frecuentemente al procesador, disminuyendo su eficiencia para ejecutar las restantes tareas del sistema. Un valor típico es 8 bytes.
 - **Construct using registers instead of memory blocks:** esta opción permite seleccionar si la cola de escritura será implementada mediante registros en el dispositivo FPGA (si está seleccionada) o si se implementará mediante bloques de memoria (si no está seleccionada).
- **Allow multiple connections:** esta opción permite conectar varios dispositivos a la misma interfaz JTAG UART. Debe considerarse la integridad de la comunicación cuando se utiliza esta opción.

Para el ejercicio propuesto, puede configurarse el puerto de comunicaciones JTAG UART con los siguientes parámetros (Figura 5-16):

- *Write FIFO (Data from Avalon to JTAG)*
 - *Buffer depth (bytes):* 64
 - *IRQ threshold:* 8
 - *Construct using registers instead of memory blocks:* No
- *Read FIFO (Data from JTAG to Avalon)*
 - *Buffer depth (bytes):* 64
 - *IRQ threshold:* 8
 - *Construct using registers instead of memory blocks:* No
- *Allow multiple connections*
 - *Allow multiple connections to Avalon JTAG slave:* No

Al finalizar la configuración, se debe seleccionar el botón **Finish** para que el módulo se inserte en el diseño del sistema.



Figura 5-17: Detalle de conexión del temporizador al puente de adaptación de señales de reloj.

De manera similar a la que se realizó con el temporizador y el puerto de Entrada/Salida, debe conectarse el puerto esclavo **avalon_jtag_slave** del periférico **JTAG UART** al puerto maestro **data_master** del procesador. También debe especificarse la señal de reloj, de reset y de interrupción del periférico **JTAG UART** como se muestra en la Figura 5-17.

CONFIGURACIÓN DEL SOPC

Una vez que se incluyeron los diferentes componentes que integrarán el SOPC, es necesario asignar las direcciones e interrupciones para cada uno de ellos. Este proceso puede realizarse manualmente mediante la modificación de los campos que cada componente tiene al efecto en la ventana del sistema o puede seleccionarse el comando para que Qsys realice la asignación automáticamente. La opción manual puede ser conveniente cuando se desea asignar determinadas direcciones para no modificar el software que hubiera sido creado para una determinada configuración. Sin embargo, en la mayoría de los casos, es conveniente que Qsys realice la asignación automática para evitar cualquier inconsistencia en la configuración.

Posteriormente a la asignación de direcciones de memoria e interrupciones, será necesario asignar las direcciones de reset y excepción del procesador Nios II conforme a la asignación de direcciones de las memorias del sistema realizada anteriormente.

Se podrá observar que durante el transcurso de la configuración del sistema, se irán eliminando los mensajes de advertencia y error de la ventana de mensajes de Qsys.

Asignación automática de direcciones de memoria

Todos los componentes del sistema deben ser asignados a diferentes direcciones de memoria para una correcta configuración. Cada componente del sistema demande diferente cantidad de direcciones de memoria de acuerdo a la cantidad de registros que este requiera en el mapa de memoria. Es necesario establecer la dirección de comienzo de cada componente para determinar el rango de direcciones que el componente utilizará.

Qsys permite asignar automáticamente las direcciones de los componentes del sistema mediante la ejecución del comando **System > Assign Base Addresses** de la barra de menú (Figura 5-18).

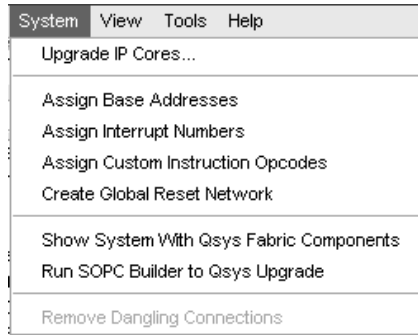


Figura 5-18: Menú para la asignación automática de direcciones e interrupciones.

Al ejecutar el comando de asignación de direcciones, se modificarán las direcciones **base** y de finalización (**end**) de cada uno de los componentes, definiéndolas en forma consistente.

Asignación automática de interrupciones

En forma similar a la asignación de las direcciones de memoria de cada uno de los componentes, pueden existir componentes del sistema que requieran adicionalmente la asignación de una señal de interrupción al procesador. En el sistema propuesto, los componentes *Interval Timer* y *JTAG UART* requieren una línea de interrupción del procesador. Para la asignación automática de las interrupciones del sistema, se debe seleccionar el comando **System > Assign Interrupt Numbers** de la barra de menú (Figura 5-18).

Cuando ambas asignaciones hayan sido realizadas, el sistema quedará configurado como muestra la Figura 5-19.

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ
☒		clk_reset	Reset Output	Double-click to				
☒		nios2_qsys_0	Nios II Processor	Double-click to	clk_0			
		clk	Clock Input	Double-click to	[clk]			
		reset_n	Reset Input	Double-click to	[clk]			
		data_master	Avalon Memory Mapped Master	Double-click to	[clk]			
		instruction_master	Avalon Memory Mapped Master	Double-click to	[clk]			
		jtag_debug_module_re...	Reset Output	Double-click to	[clk]			
		jtag_debug_module	Avalon Memory Mapped Slave	Double-click to	[clk]	# 0x0001_0800	0x0001_0fff	
		custom_instruction_m...	Custom Instruction Master	Double-click to	[clk]			
☒		memoria_ram	On-Chip Memory (RAM or ROM)	Double-click to	clk_0			
		clk1	Clock Input	Double-click to	[clk1]	# 0x0000_8000	0x0000_ffff	
		s1	Avalon Memory Mapped Slave	Double-click to	[clk1]			
		reset1	Reset Input	Double-click to	[clk1]			
☒		timer_0	Interval Timer	Double-click to	clk_0			
		clk	Clock Input	Double-click to	[clk]			
		reset	Reset Input	Double-click to	[clk]	# 0x0001_1000	0x0001_101f	
		s1	Avalon Memory Mapped Slave	Double-click to	[clk]			
☒		p1o_0	PIO (Parallel I/O)	Double-click to	clk_0			
		clk	Clock Input	Double-click to	[clk]			
		reset	Reset Input	Double-click to	[clk]	# 0x0001_1020	0x0001_102f	
		s1	Avalon Memory Mapped Slave	Double-click to	[clk]			
		external_connection	Conduit	Double-click to				
☒		jtag_uart_0	JTAG UART	Double-click to	clk_0			
		clk	Clock Input	Double-click to	[clk]			
		reset	Reset Input	Double-click to	[clk]	# 0x0001_1030	0x0001_1037	
		avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to	[clk]			

Figura 5-19: Sistema con la asignación de direcciones de memoria e interrupciones.

Una vez que las direcciones de memoria y las interrupciones del sistema han sido configuradas, solo resta configurar las direcciones de los vectores que requiere el procesador Nios II, de acuerdo a la dirección que le fue asignada a la memoria del sistema.

Configuración de los vectores de *reset* y *exception* del procesador Nios II del sistema

Todo procesador comienza la ejecución del software de aplicación desde una dirección de memoria predefinida denominada, generalmente, **dirección de reset**. En esta dirección, debe comenzar el programa de aplicación puesto que de lo contrario el procesador ejecutaría un contenido de memoria desconocido, perdiéndose el control de la ejecución. De la misma manera, es necesario establecer la dirección de memoria en la cual el procesador comienza a ejecutar el código cuando se produce una excepción. Dicha dirección de memoria se denomina generalmente **dirección de excepción**.

Para configurar ambas direcciones, es necesario acceder nuevamente a la pantalla de configuración del procesador Nios II mediante: (1) la realización de un doble clic sobre el procesador en la ventana del sistema en el Qsys, o (2) la selección del procesador y presionando el botón **Edit...** Cualquiera de las opciones abrirá la pantalla de configuración del procesador Nios II (Figura 5-8).

The screenshot shows a configuration window with two sections: "Reset Vector" and "Exception Vector".

Reset Vector	
Reset vector memory:	memoria_ram.s1
Reset vector offset:	0x00000000
Reset vector:	0x00008000

Exception Vector	
Exception vector memory:	memoria_ram.s1
Exception vector offset:	0x00000020
Exception vector:	0x00008020

Figura 5-20: Selección memoria de vectores y desplazamiento (Offset).

La Figura 5-20 muestra en detalle la lista de selección de las diferentes opciones válidas para asignar la dirección de memoria de cada vector. En el ejemplo propuesto, al incluir el sistema solo una memoria, la selección se reduce a la memoria RAM denominada por defecto “**memoria_ram.s1**”. El campo **Offset** determina el desplazamiento a partir del comienzo de la memoria que se asignará a la dirección del vector. Es conveniente dejar los desplazamientos que por defecto asigna Qsys para lograr una asignación más eficiente en la utilización de memoria.

Para configurar la asignación de los vectores de *reset* y de *exception* seleccionar la opción “**memoria_ram.s1**” de cada una de las listas.

Luego de realizar la asignación, seleccionar el botón **Finish** para salir de la pantalla de configuración del procesador Nios II.

Configuración de las entradas y salidas del SOPC

Qsys permite diseñar un SOPC mediante la incorporación y configuración de los diferentes componentes que lo integran. El SOPC generado será posteriormente instanciado como un módulo en Quartus II para luego ser compilado e implementado en un dispositivo FPGA.

Sin embargo, es necesario definir las entradas y salidas que poseerá el SOPC y que estarán disponibles para su conexión en Quartus II. Las señales del SOPC que estarán disponibles en Quartus II se definen mediante la columna “Export” de la solapa **System Contents** de Qsys. Dicha columna permite definir el nombre con que se podrá acceder desde Quartus II a cada señal de cada componente del SOPC con solo ingresarlo en la casilla de texto correspondiente.

En el ejemplo propuesto, es necesario acceder al puerto de entrada/salida del componente **PIO (Parallel I/O)**, por lo que se asignará el nombre “**pio_o_connection**” en la columna “Export” a la conexión “external_connection” (Figura 5-21).

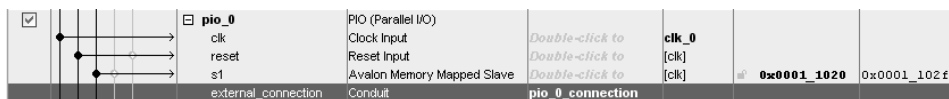


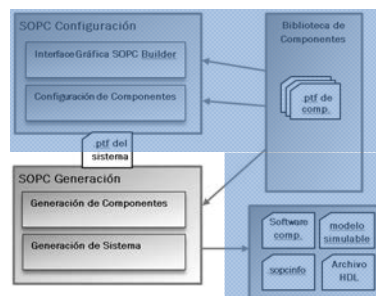
Figura 5-21: Asignación de nombre al puerto de entrada/salida del componente PIO (Parallel I/O) para exportar del SOPC.

En este punto, deben haber desaparecido todos los errores de validación de la ventana de mensajes, dejando solo mensajes de advertencia recordando la posibilidad de que existan señales adicionales de los componentes incluidos en el SOPC que se desean exportar. Se habilita entonces la posibilidad de generar el sistema

GENERACIÓN DEL SISTEMA EN QSYS

La etapa de configuración del SOPC permite ingresar el sistema en Qsys, configurar cada uno de sus componentes y asignar los parámetros del sistema como direcciones de memoria e interrupciones.

Una vez que se han seleccionado y configurado todos los componentes del sistema, es necesario generarlo. En el proceso de generación, Qsys crea, entre otros archivos, el archivo con extensión **.ptf**, el archivo con extensión **.sopcinfo** y el archivo de descripción de hardware ("módulo del sistema") que requiere el Quartus II para su síntesis al hardware del dispositivo FPGA.



De no existir ningún error de diseño, la generación del sistema en Qsys permitirá definir en Quartus II las conexiones de las entradas y salidas del sistema a los pines físicos del dispositivo FPGA, para una correcta interconexión.

La generación del sistema se realiza mediante la simple selección del botón **Generar** en la parte inferior de la solapa **Generation**. Sin embargo, primero es conveniente revisar las opciones brindadas por esta solapa.

Solapa del generador del sistema (Generation)

La generación del sistema (*Generation*) se accede mediante la solapa ubicada en la parte superior de la pantalla de Qsys (Figura 5-22). Esta pantalla permite observar todo el proceso de generación del sistema en la ventana principal y seleccionar la opción para crear los archivos de simulación del hardware.

Simulation

The simulation model contains generated HDL files for the simulator, and may include simulation-only features.

Create simulation model: Verilog

Testbench System

The testbench system is a new Qsys system that instantiates the original system, adding bus functional models to drive the top-level interfaces. Once generated, the bus functional models can interact with the system in the simulator.

Create testbench Qsys system: Simple, BFMs for clocks and resets

Create testbench simulation model: Verilog

Synthesis

Synthesis files are used to compile the system in a Quartus II project.

Create HDL design files for synthesis: Verilog

☒ Create block symbol file (.bsf)

Output Directory

Path: C:\n2sopc\sopcsys

Simulation: C:\n2sopc\sopcsys\simulation\

Testbench: C:\n2sopc\sopcsys\testbench\

Synthesis: C:\n2sopc\sopcsys\synthesis\

Generate

Figura 5-22: Solapa Generation de Qsys.

La solapa **Generation** posee tres secciones diferenciadas:

- **Simulation:** permite definir los archivos que se crearán para la simulación del SOPC. Qsys permite generar automáticamente archivos de prueba *testbench* para la simulación del comportamiento del SOPC y el archivo de simulación del SOPC en forma separada. Para esto, presenta diferentes opciones:
 - **Create simulation model:** esta opción permite seleccionar el lenguaje de descripción de hardware en el cual será creado el modelo de simulación del SOPC.
- **Testbench System:** permite especificar la manera en que se desea crear el escenario de simulación.
 - **Create testbench Qsys system:** esta opción permite la generación de *testbench* automática para la simulación del SOPC diseñado.
 - **Create testbench simulation model:** permite seleccionar el lenguaje en el que se describirá el *testbench* generado.
- **Synthesis:** permite determinar las opciones del archivo “modulo de sistema” que se generará del SOPC diseñado. Las opciones disponibles son:
 - **Create HDL design files for synthesis:** crea el archivo de “modulo de sistema” requerido por Quartus II para la compilación del SOPC a hardware.
 - **Create block symbol file (.bsf):** genera un archivo de símbolo para poder instanciar en Quartus II el componente en un diagrama de bloques de símbolos.

- **Output Directory:** permite definir el directorio en donde se crearán los archivos generados. El campo **Path** permite definir el directorio raíz y los demás campos definirán los directorios correspondientes a la simulación y síntesis según hayan sido configurados.

Para la realización del proyecto propuesto, se seleccionarán las siguientes opciones:

- **Simulation:**
 - *Create simulation model:* Verilog.
 - *Create testbench Qsys system:* Simple, BFM's for clocks and reset.
 - *Create testbench simulation model:* Verilog.
- **Synthesis:**
 - *Create HDL design files for synthesis:* Verilog.
 - *Create block symbol file (.bsf):* Si.
- **Output Directory:**
 - Path: "c:\n2sopc\sopcqsys"

Seleccionar el botón **Generate** para comenzar el proceso de generación.

En caso de que el proyecto de Qsys no haya sido guardado con anterioridad, una pantalla de confirmación para guardarlo aparecerá (Figura 5-23): seleccionar el botón **Save** y asignar el nombre "sopcqsys" al archivo que se guardará en el directorio "c:\n2sopc". Este proceso determina que la especificación del SOPC sea guardada en el archivo "sopcqsys.sopcinfo".



Figura 5-23: Confirmación para guardar el proyecto Qsys.

En forma sencilla para el diseñador, pero requiriendo tiempo para su finalización, el programa generador realiza las siguientes tareas:

- Lee el archivo de descripción del sistema (.ptf o .spocinfo de acuerdo a la versión de Qsys o SOPC Builder).
- Crea los archivos de software (*drivers*, bibliotecas y utilidades) para cada componente del sistema que provee soporte de software en su definición de biblioteca.

- c. Ejecuta el programa generador de cada componente. De esta manera, ocurre un proceso jerárquico donde el generador principal ejecuta el generador de cada componente que es el que crea la descripción de hardware pertinente.
- d. Genera el archivo HDL a nivel de sistema, el cual puede ser tanto VHDL o Verilog. Este archivo contiene:
 - Una instancia de cada componente del sistema.
 - La lógica de *bus* para implementar la interconexión de los módulos la cual incluye: (1) Multiplexores del *bus* de datos, (2) Árbitros de recursos compartidos, (3) Lógica de reseteo y lógica condicional, lógica de prioridad de interrupciones, (4) Ajuste dinámico de *buses*, para permitir la adaptación entre maestros y esclavos con buses de datos de diferente ancho y (5) Todas las interconexiones pasivas entre los puertos maestros y esclavos.
 - Un archivo de pruebas (*testbench*) que: (1) Instancia el modulo del sistema, (2) Genera las entradas de reloj y reset con los comportamientos por defecto y (3) Instancia y conecta los modelos de simulación de componentes externos (por ejemplo, modelos de memoria)
- e. Crea un símbolo para el “módulo de sistema” (archivo .bsf).
- f. Crea un directorio de proyecto de ModelSim que incluye:
 - Archivos de datos de simulación para todos los componentes de memoria que especifiquen contenido.
 - Un archivo **stop_sim.do** que contiene un conjunto inicial de formas de ondas de la interfaz de *bus*.
 - Un proyecto de ModelSim para el sistema (archivo .mpf).
- g. Escribe un *script* Tcl de Quartus II para compilación. Este *script* Tcl coloca todos los archivos necesarios para la compilación en Quartus II y es generado en el directorio del proyecto.
- h. Escribe un *script* de generación para el intérprete de comandos. Mediante este *script*, el sistema puede ser regenerado sin la utilización de la GUI del Quartus II.

Si el sistema fue correctamente generado, entonces se mostrará el mensaje *Info: System generation was successful* en la ventana de reporte. En esta etapa del proceso, todos los archivos de hardware del sistema y los de especificación del soc fueron creados en la carpeta del proyecto (“c:\n2sopc\” en el ejemplo propuesto), incluyendo el módulo del sistema que puede ser instanciado en Quartus II para su compilación a hardware.

Para finalizar el diseño en Qsys y retornar a la plataforma Quartus II, seleccionar el comando **File > Exit** de la barra de menús. Con este comando, se cierra Qsys y se retorna al Quartus II luego de seleccionar **Save** en la pantalla de confirmación que aparece antes de que se cierre Qsys.

INSTANCIACIÓN DEL SOPC EN QUARTUS II

Una vez generado el “módulo del sistema”, el SOPC está disponible en el proyecto de Quartus II para ser instanciado como componente en el archivo de mayor jerarquía y conformar el hardware que luego se implementará en el dispositivo FPGA.

Retornando al navegador del proyecto, abrir el archivo de mayor jerarquía realizando un doble clic en el archivo “n2sopc.bfd” de la solapa **Files** (Figura 5-24). Esta acción abrirá el archivo esquemático en la ventana principal del Quartus II para su edición.

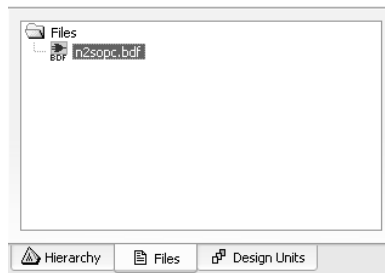


Figura 5-24: Selección de archivo de mayor jerarquía del proyecto.

Con el archivo “n2sopc.bfd” activo en la ventana principal, seleccionar el botón **Symbol Tool** o el comando **Insert > Symbol...** del menú que se despliega al hacer clic en el botón derecho del *mouse* sobre el archivo esquemático de la ventana principal. Cualquiera de los dos métodos abrirá el explorador de símbolos de Quartus II (Figura 5-25) y permitirá seleccionar componentes de la biblioteca de Altera, como el SOPC generado con Qsys.

En el explorador de símbolos de Quartus II, se debe expandir la categoría **Project**, seleccionar el componente “sopcqsys”, como lo muestra la Figura 5-25. Luego se debe presionar el botón **OK** para que el símbolo sea seleccionado para inserción.

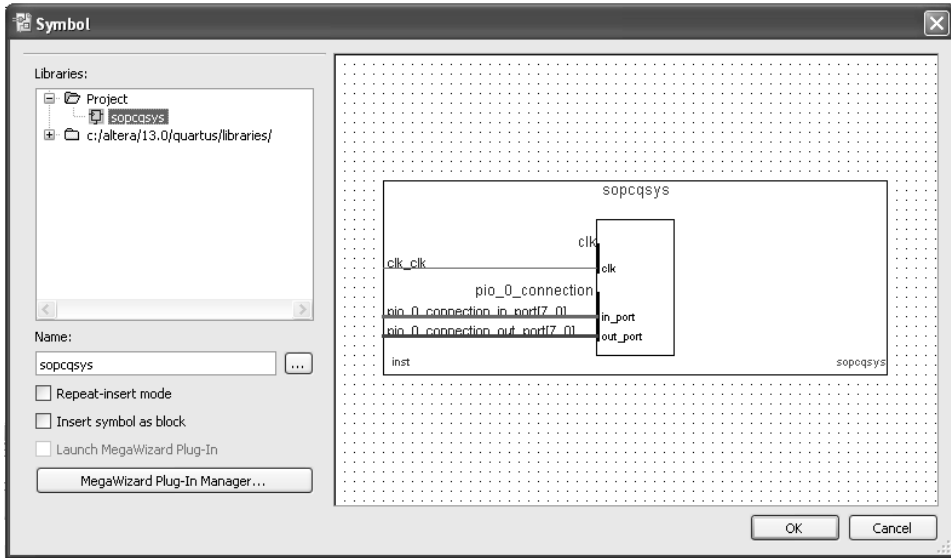


Figura 5-25: Explorador de símbolos con el SOPC seleccionado.

Con el símbolo seleccionado sobre el archivo esquemático, hacer clic para posicionarlo sobre el esquemático y luego presionar la tecla **Esc** para dejar de seleccionarlo. El símbolo del SOPC se instanciará en el archivo de mayor jerarquía del proyecto permitiendo la conexión de los pines de entrada y de salida (Figura 5-26). Queda entonces, para finalizar el diseño del hardware, la interconexión de la lógica externa y la asignación de pines de entrada y salida.

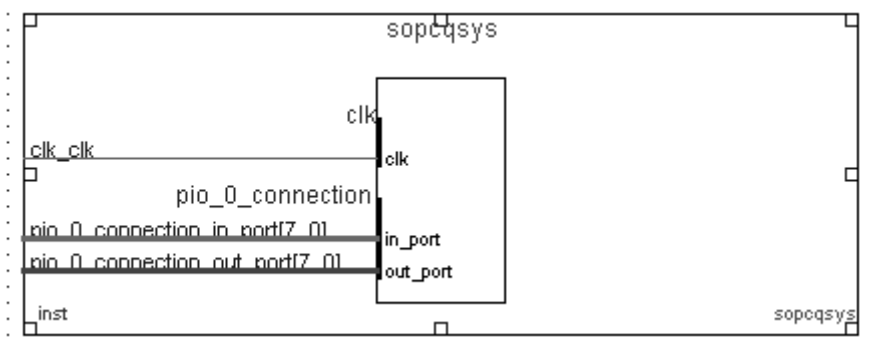


Figura 5-26: Símbolo del SOPC creado.

AGREGADO DE LÓGICA ADICIONAL Y ASIGNACIÓN DE PINES

En muchas aplicaciones, puede ser posible implementar hardware externo al SOPC para realizar alguna función o tarea. En particular, siempre se deberá asignar lógica de entrada y de salida para poder conectar el hardware al mundo exterior. En el SOPC propuesto, deben agregarse los puertos de entrada y salida al sistema y luego asignar dichos puertos a los pines del dispositivo FPGA para su implementación física.

AGREGADO DE LÓGICA ADICIONAL

En el Capítulo 2, se detalló que la estructura *module* permite la definición de la lista de puertos (*port list*) que tendrá un determinado hardware digital que se describe mediante Verilog HDL. En una descripción mediante esquema, debe realizarse la misma función pero a través de los símbolos correspondientes a los diferentes tipos de puertos de entrada, salida y bidireccional.

Para acceder a los símbolos de cada uno de los diferentes tipos de puerto, se debe desplegar la lista del ícono “” ubicado en la barra de herramientas de la ventana principal de Quartus II (Figura 5-27).



Figura 5-27: Lista de símbolos de puertos.

Seleccionar el símbolo de puerto de entrada “”(Input) y posicionarlo en el archivo esquemático realizando un clic en la posición que se desea realizar cada una de las instancias. Al finalizar la última instancia, presionar la tecla **Esc** para finalizar la selección.

Similarmente, seleccionar el símbolo de puerto de salida “” (Output) y instanciarlo en todas las ubicaciones deseadas del archivo esquemático. La Figura 5-28 muestra el archivo esquemático del proyecto al haber instanciado todos los puertos de entrada y salida.

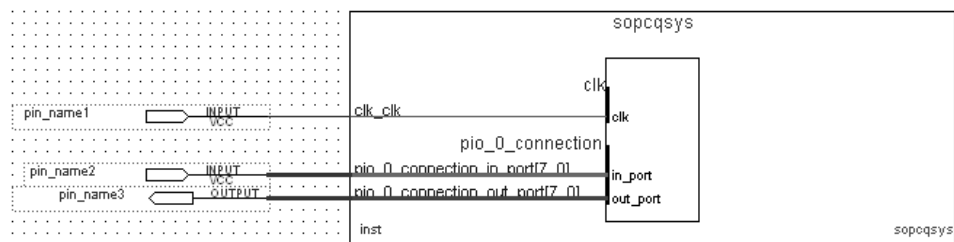


Figura 5-28: Esquema con puertos de Entrada y Salida Instanciados.

El cambio de orientación de los símbolos puede realizarse mediante la selección del comando **Flip Horizontal** o **Flip Vertical** del menú sensitivo mostrado en la Figura 5-29.

Asignación de nombres a los puertos

Al instanciar cada puerto de entrada o salida, Quartus II asigna por defecto un nombre que va numerando secuencialmente posterior a “pin_name”. Para una correcta especificación del diseño, es necesario asignar un nombre a cada uno de los puertos para que las interconexiones se realicen en forma correcta. Para asignar un nombre a cada puerto, se debe seleccionar haciendo clic sobre él y eligiendo el comando **Properties** del menú que se despliega al hacer clic en el botón derecho del mouse (Figura 5-29).

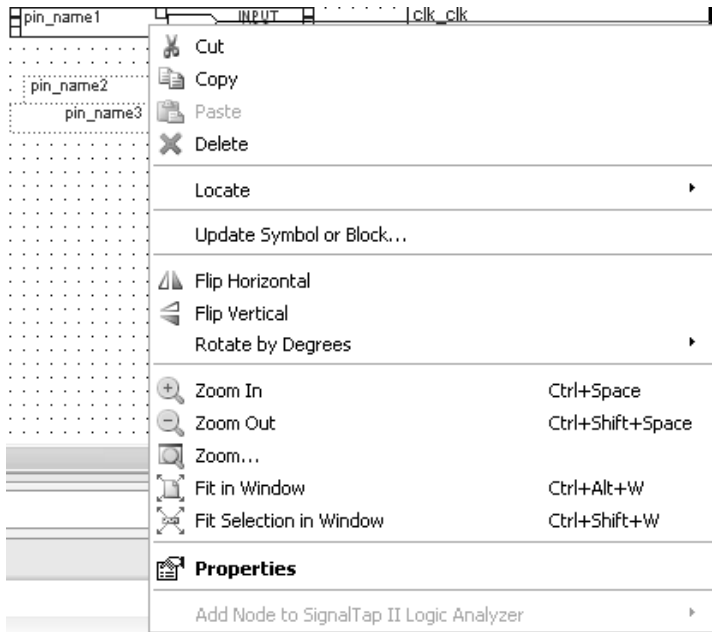


Figura 5-29: Acceso a las propiedades del puerto.

En la pantalla de configuración del puerto, se debe completar el campo **pin name(s)** con el nombre que se desea asignar al puerto. Para una identificación más fácil, es conveniente asignarle un nombre que pueda ser asociado con el puerto del símbolo al que se conectará. En el sistema propuesto, se sugiere la asignación de nombres de la Tabla 5-1.

TABLA 5-1: NOMBRES DE LOS PUERTOS DE ENTRADA Y SALIDA DEL SISTEMA

Nombre del puerto	Puerto del SOPC al que irá conectado	Tipo de puerto	Tipo de conexión
clk	clk	entrada (Input)	Nodo ()
entrada[7..0]	in_port_to_the_pio_0[7..0]	entrada (Input)	Bus (“  ”)
salida[7..0]	out_port_to_the_pio_0[7..0]	salida (Output)	Bus (“  ”)

Conexión de la lógica externa

Las interconexiones entre los diferentes símbolos del archivo esquemático se realizan mediante el graficado de líneas que la representan. Para seleccionar la herramienta de dibujo de interconexiones, se debe hacer clic sobre uno de los símbolos     según el tipo de conexión que se desea realizar:

-  o  para la realización de conexiones de nodo simples en gráficos ortogonales o diagonales, respectivamente.
-  o  para la realización de conexiones de buses en gráficos ortogonales o diagonales, respectivamente.
-  o  para la realización de conjuntos de señales (**Conduit**) en gráficos ortogonales o diagonales, respectivamente.

Para realizar las interconexiones, se debe seleccionar el tipo de conexión deseado y presionando el botón izquierdo del *mouse* en el punto de origen y, sin soltar el botón, desplazarse hasta el punto destino en el cual debe liberarse. Al finalizar todas las conexiones, deseleccionar el tipo de conexión presionando la tecla **Esc** o seleccionando la herramienta de puntero “  ”.

Realizar todas las conexiones de acuerdo a lo especificado en la Tabla 5-1. Una vez realizadas las conexiones, el archivo quedará configurado según la Figura 5-30. Para preservar todos los cambios realizados, es conveniente guardar todos los archivos del proyecto mediante el comando **File > Save All** de la barra de menú o seleccionando el ícono “

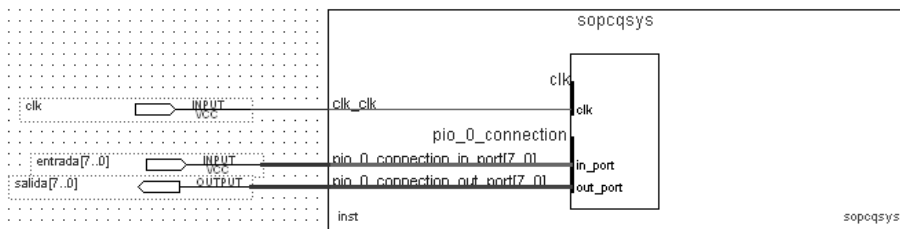


Figura 5-30: Sistema con puertos conectados.

ASIGNACIÓN DE PINES

La asignación de pines permite que las señales de los puertos estén disponibles en los pines del dispositivo FPGA. Esta asignación puede ser realizada automáticamente por Quartus II que aplicará heurísticas para determinar la asignación más conveniente tanto en recursos utilizados como tiempo de propagación del hardware. Este tipo de asignación automática fuerza a que el diseño de la placa de circuito impreso sea adecuada a tal asignación para una correcta conexión con los componentes externos al dispositivo FPGA.

Generalmente, los diseños son implementados en placas de desarrollo (*kits*) en las que todos los componentes ya están interconectados entre sí en una arquitectura definida por el fabricante. En estos casos, la asignación de los pines debe realizarse en forma manual para una correcta conexión del hardware implementado con los componentes externos al dispositivo FPGA. Por ejemplo, la interfaz de comunicación JTAG UART implementada en el SOPC propuesto debe ser asignado a los pines del dispositivo FPGA que se encuentran conectados al puerto de comunicaciones con la computadora personal.

Los dispositivos FPGA modernos presentan una elevada cantidad de pines disponibles para entradas y salidas (de 100 a 700 pines). Por lo general, el proceso de asignación de pines puede resultar tedioso y conducente a la introducción de errores de asignación. Es por este motivo que muchas placas de desarrollo se ofrecen con archivos de comandos (*scripts*) para la asignación de pines. Estos archivos (con extensión *.tcl*) contienen los comandos necesarios para asignar un **nombre** a cada uno de los pines, teniendo en cuenta la señal a la que está conectado dicho pin del dispositivo FPGA. Posteriormente, deben respetarse dichos **nombres** en la definición de la lista de puertos del módulo para que su relación sea inmediata. Por ejemplo, la asignación del pin “**PIN_E1**” a la señal de reloj de entrada se realiza mediante el comando de archivo *.tcl*:

```
set_location_assignment PIN_E1 -to CLK
```

Para ejecutar un archivo de comandos, se debe seleccionar **Tools > Tcl Scripts...** de la barra de menú de Quartus II y escoger el archivo *.tcl* correspondiente. La utilización de archivos de comandos para la asignación de pines facilita la reutilización de una placa de desarrollo entre diferentes proyectos al reducir significativamente el tiempo que demanda esta etapa en diseños con gran cantidad de puertos.

Otra manera de asignación de pines es la establecida en la sección “Asignación de pines (*Pin Planner*)” del Capítulo 3 (página 53) que puede emplearse en el ejemplo propuesto debido a su reducida utilización de pines.

CONCLUSIONES

En este capítulo, se describieron todas las etapas del flujo de diseño de un sistema-en-chip-programable (SOPC). Se utilizaron las herramientas de desarrollo que brinda Altera y se diseñó un sistema basado en el procesador Nios II hasta su síntesis para configuración a un dispositivo FPGA. En el siguiente capítulo, se describirán las etapas y herramientas de

desarrollo disponibles para el diseño y compilación del software que se ejecutará sobre el hardware diseñado.

CAPÍTULO 6

GENERACIÓN DEL SOFTWARE

GENERACIÓN DE SOFTWARE PARA NIOS II

Qsys y Quartus II generan los archivos de configuración necesarios para implementar el hardware del sistema en el dispositivo FPGA seleccionado. Sin embargo, si el sistema incluye uno o más procesadores Nios II, entonces es necesario establecer el software que se ejecutará para implementar la funcionalidad del sistema. La realización del software se realiza mediante el desarrollo de un código que, luego de compilado, es ejecutado por el o los procesadores durante el tiempo de funcionamiento. Existen diversos lenguajes de programación, siendo el lenguaje C ((17)), el más utilizado para el desarrollo de sistemas embebidos.

La programación de SOPC de Altera pueden realizarse mediante la utilización de diferentes plataformas o alternativas. Entre las más utilizadas, se encuentran:

- La plataforma **Nios Eclipse** incluida con *SOPC Builder*. Esta plataforma permite el desarrollo de proyectos complejos en un ambiente gráfico y con posibilidades de depuración. La plataforma Nios Eclipse se instala configurada para soporte de librerías (en especial, newlib) que facilita el desarrollo de aplicaciones. Altera no recomienda esta alternativa para proyectos nuevos.
- Las utilidades de compilación incluidas en la consola **Nios II SDK**. Esta consola posee una terminal de comandos con diversas utilidades para configuración y depuración de aplicaciones. La utilidad **Nios-build** permite la compilación de un código C en ejecutable por el procesador Nios II. La consola también posee utilidades para la transferencia de las configuraciones y aplicaciones a la placa de desarrollo.
- Las utilidades de compilación basadas en **GNU-gcc**. El compilador gcc ha sido desarrollado con la filosofía de código libre y es el utilizado para la

compilación del sistema operativo Linux y sus diferentes versiones. Este compilador ha sido adecuado (*ported*, en inglés) a diversos procesadores, entre ellos el Nios II.

La selección de algunas de estas alternativas será acorde con el soporte que brinda para una determinada aplicación, la compatibilidad con versiones anteriores del software y la flexibilidad de programación.

Todas las herramientas de desarrollo de sistemas embebidos modernos se basan en la necesidad de brindar al diseñador un soporte de software que le permita abstraerse lo máximo posible de la arquitectura de hardware y, al mismo tiempo, este soporte requiera la menor cantidad de recursos del sistema (tanto en tiempo de ejecución como en memoria) para su implementación. Es por esta necesidad que se proponen alternativas para la implementación de **capas de abstracción de hardware** (HAL, *Hardware Abstraction Layer*, en inglés) y **sistemas operativos** (OS, *Operating System*, en inglés).

CAPA DE ABSTRACCIÓN DE HARDWARE (HAL) Y SISTEMAS OPERATIVOS (OS)

En muchas ocasiones, se considera que la realización del software de una aplicación se resuelve mediante la simple programación de un código. De este modo, la programación de un determinado hardware se reduciría a la realización de un código que se ejecutara en el procesador y que accediera a los diferentes recursos del sistema. La Figura 6-1 muestra la estructura de un sistema en el cual el software es desarrollado para el hardware específico. En este tipo de metodología de desarrollo, el software debe contemplar todas las especificaciones del hardware (direcciones de memoria, tipo de periféricos, interrupciones, etc.) y la modificación del hardware podría requerir la modificación de gran parte del software de aplicación.

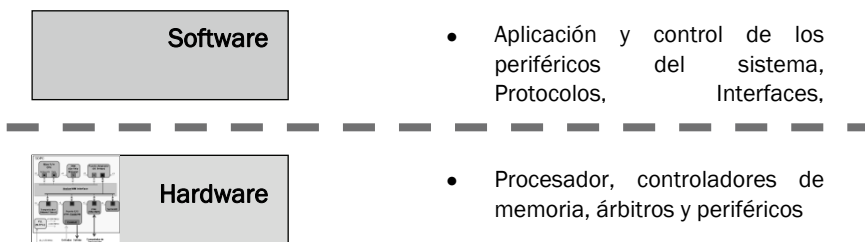


Figura 6-1: Sistema con software de aplicación dedicado.

Es evidente que la estructura de sistema de la Figura 6-1, si bien puede optimizar los recursos de memoria o de procesamiento requeridos, es poco flexible y de difícil mantenimiento. Este tipo de programación no es adecuada para el diseño de SOPC en los cuales se ha demostrado que el hardware puede ser realizado con gran flexibilidad.

Cuando los sistemas se tornan más complejos, las aplicaciones requieren la realización de funciones más sofisticadas que requieren el manejo y control de dispositivos periféricos específicos. Es de esta manera que es conveniente que el diseñador cuente con un soporte en el sistema embebido que le permita abstraerse de las particularidades que el hardware necesita para su funcionamiento y se concentre en la realización de la aplicación propiamente dicha. Es conveniente que el diseñador tenga un soporte para el desarrollo de la aplicación que lo abstraiga de la implementación del control de los periféricos, los protocolos y las interfaces con el hardware. Generalmente, este tipo de soporte se brinda al diseñador a través de una capa de abstracción de hardware (HAL) o un sistema operativo (OS).

CAPA DE ABSTRACCIÓN DE HARDWARE (HAL)

La realización de una aplicación de software según la estructura de la Figura 6-1 requiere un amplio conocimiento de los diferentes periféricos y de la arquitectura del sistema para su programación.

Cuando los sistemas a desarrollar se vuelven más complejos o realizados sobre plataformas de hardware que pueden modificarse, es conveniente la utilización de una “capa de abstracción de hardware” (HAL, *Hardware Abstraction Layer*, en inglés). Una HAL brinda al diseñador una interfaz estándar con el hardware de manera que este puede realizar su software con el solo conocimiento de los recursos con los que cuenta el sistema. La Figura 6-2 muestra la estructura de un sistema utilizando una HAL como interfaz al software de la aplicación.

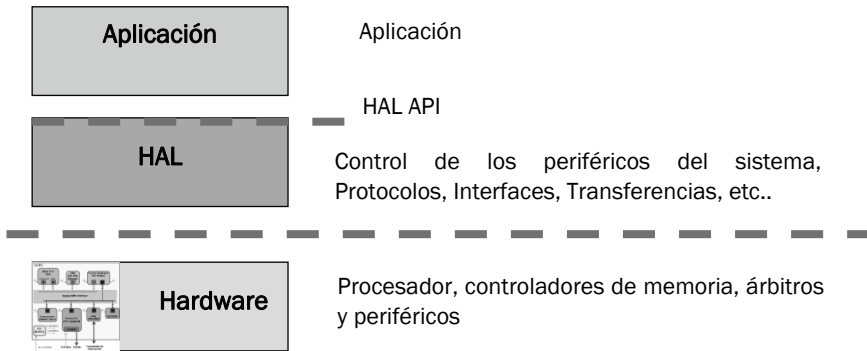


Figura 6-2: Sistema con software de aplicación y HAL.

La utilización de una HAL para el diseño de la aplicación presenta beneficios muy importantes entre los que se encuentran:

- **Sencillez:** facilita la reutilización de código entre diferentes diseños, permite la integración automática de los controladores de los periféricos del SOPC y permite la inicialización automática del procesador, los periféricos e interrupciones.

- **Confiabilidad:** permite la configuración de los diferentes periféricos mediante estructuras bien definidas y mejora la consistencia del software mediante el acceso a los periféricos mediante una interfaz común.
- **Estandarización:** permite la realización del código mediante la utilización de una interfaz estándar (ej.: POSIX).

Como contraparte a estos beneficios, se debe tener en cuenta que la implementación de la HAL requiere recursos de memoria y tiempo del procesamiento del procesador para su funcionamiento. Sin embargo, estos requerimientos pueden considerarse mínimos ante los beneficios que presenta su implementación en un SOPC.

SISTEMA OPERATIVO (OS)

Un sistema operativo (OS, *Operating System*, en inglés), desde el punto de vista de su estructura, es una colección de rutinas de software que son ejecutadas por el procesador. Estas rutinas implementan el control que hace funcionar al sistema embebido y proporcionan un entorno para la ejecución de las diferentes tareas que lo conforman. Por supuesto que la ejecución del sistema operativo requiere tiempo de ejecución del procesador y memoria del sistema que no estarán disponibles para las tareas realizadas por el diseñador. Esta sobrecarga del sistema operativo es una desventaja que tendrá que analizarse frente a la flexibilidad, portabilidad y facilidad de desarrollo que produce su utilización.

Una característica importante de muchos sistemas operativos, es que soportan multitarea. Esto consiste en que el sistema operativo permite compartir el tiempo de ejecución del procesador entre diferentes tareas o procesos, cada uno de los cuales implementa una funcionalidad específica de la aplicación. Por ejemplo, podría existir un proceso que implemente un servidor Web, al mismo tiempo que otro proceso escribe en pantalla información para el usuario. El soporte de un sistema operativo permite, al programador, implementar diferentes tareas que serán ejecutadas compartiendo entre sí la utilización del procesador.

Un sistema operativo resguarda al programador de particularidades del hardware y le presenta una interfaz simple basada en recursos que pueden leerse o escribirse como si se trataran, en forma genérica, de archivos. Esta característica es muy importante en un sistema operativo puesto que permite al diseñador programar su aplicación abstrayéndose relativamente del hardware de la plataforma de desarrollo. De esta manera, una aplicación desarrollada para un determinado hardware, podría **migrarse** sin mayores inconvenientes a otro hardware si ambas plataformas soportan el mismo sistema operativo. Generalmente, el sistema operativo se implementa sobre una “capa de abstracción de hardware” con el objetivo de adaptarlo fácilmente a las diferentes arquitecturas de hardware (Figura 6-3).

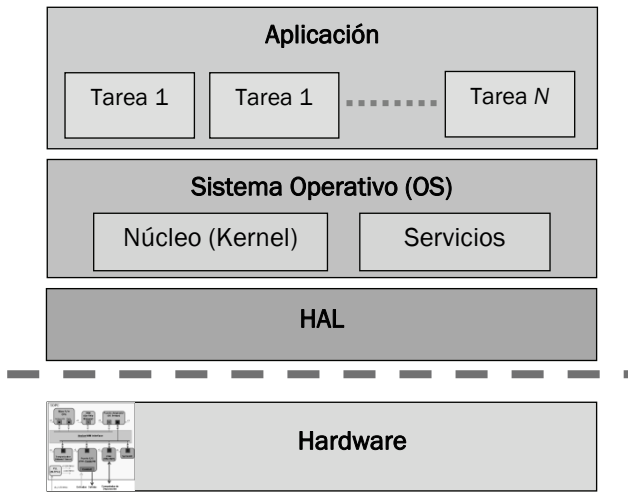


Figura 6-3: Estructura de software-hardware basada en sistema operativo.

Una de las principales características de un sistema operativo es que pueden implementar manejo de memoria y mecanismos de protección de memoria. Para eso, el sistema operativo es ejecutado en un modo protegido que permite preservar la integridad del sistema aun cuando alguna de las tareas tenga un comportamiento indeseado. De esta manera, el malfuncionamiento de una tarea no impedirá que las restantes del sistema y las funciones del sistema operativo colapsen en una falla general del sistema. La elección de las diferentes funcionalidades que se ejecutan en modo protegido permite diferenciar entre distintas arquitecturas de sistemas operativos.

Un sistema operativo puede implementarse de diferentes maneras que brindarán diversas características en su funcionamiento y su facilidad de utilización y modificación.

Estructura monolítica

En un sistema monolítico, podría decirse que no existe una estructura formal. El sistema operativo se concibe como un conjunto de procedimientos, cada uno de los cuales puede llamar a los demás cada vez que lo requiera. Estos procedimientos implementan los diferentes **servicios** del sistema operativo y son referidos generalmente como **llamadas al sistema**.

Cada uno de los procedimientos definidos en el sistema operativo tiene una interfaz muy bien delimitada con respecto a los parámetros de entrada y los resultados que cada uno de ellos requiere. Por lo general, los parámetros de entrada a un procedimiento están estructurados para ser realizados a través de registros del procesador o la pila del sistema.

En caso de que el procesador lo permita, la ejecución de los procedimientos del sistema operativo (denominados “llamadas al sistema”) se realiza en un nivel de privilegio mayor al de las tareas de la aplicación. Esto permite que, ante fallas de funcionamiento de una de las

tareas de la aplicación, los servicios del sistema operativo continúen brindando servicio al resto de las tareas de la aplicación.

Un sistema operativo de este tipo concentra todas las funcionalidades (planificación, sistema de archivos, redes, controladores de dispositivos, manejo de memoria, etc.) en un “gran código”. Un error en una rutina del sistema operativo podría propagarse a todo el núcleo. Cualquier cambio que se desee realizar al núcleo requiere la recompilación total del sistema operativo.

Entre los sistemas operativos monolíticos, podemos citar:

- Tipo Unix: Linux, BSD, Solaris
- Tipo DOS: MS-DOS, DR-DOS
- Tipo Windows: Windows 95, 98, Me
- Tipo MAC: Mac OS hasta Mac OS 8.6

Estructura micronúcleo

En un sistema operativo con estructura de micronúcleo, el núcleo está constituido por conjunto mínimo de llamadas al sistema que implementan los servicios básicos que necesita el sistema operativo como son: manejo de espacio de direcciones, comunicación entre procesos y planificación básica. Todos los demás servicios (gestión de memoria, sistemas de archivos, accesos a dispositivos de entrada/salida, etc.), que en un sistema monolítico forman parte del núcleo, se ejecutan en un sistema operativo micronúcleo como procesos servidores ejecutados en el espacio de usuario.

En un principio, este tipo de estructura pretendía ser una solución a la creciente complejidad de los sistemas operativos. La principal ventaja en su utilización es la reducción de la complejidad, la descentralización de las fallas (un fallo en una parte del sistema no lo haría colapsar por completo) y la facilidad de crear y depurar controladores de dispositivos. Por otro lado, sus principales desventajas son la complejidad en la sincronización entre todos los módulos servidores que componen el micronúcleo y la menor eficiencia que posee.

Entre algunos de los sistemas que poseen estructura micronúcleo, se pueden citar:

- Tipo Unix: AIX de IBM, GNU Hurd, QNX
- Tipo específico: AmigaOS, Amoeba, Minix, ChorusOS

Estructura híbrida

Los núcleos híbridos son fundamentalmente micronúcleos que incorporan servicios adicionales “no esenciales” para mejorar la eficiencia al ejecutarse más rápido que si estuviera en espacio de usuario.

Una de las características de estos sistemas operativos, y que los diferencia de los monolíticos, es que pueden incorporar y ampliar los servicios del sistema sin que requiera un reinicio en la mayoría de los casos. De esta manera, el sistema operativo puede continuar en

funcionamiento mientras que se realizan las operaciones de mantenimiento, configuración o instalación de servicios adicionales.

Un ejemplo de estos tipos de sistemas operativos lo podemos encontrar en Microsoft Windows NT (2000, XP, 2003, 2008, Vista, 7) y Mac OS X.

SISTEMAS OPERATIVOS PARA SOPC

Los sistemas embebidos implementados en dispositivos FPGA soportan la ejecución de diversos sistemas operativos. Estos sistemas operativos son desarrollados por empresas dedicadas a los mismos o por comunidades de diseñadores. Cada sistema operativo requiere para su implementación recursos de hardware que deben considerarse en el momento de diseñar el hardware del sistema. Adicionalmente, los sistemas operativos pueden requerir de herramientas especiales para su compilación o el desarrollo y depuración de aplicaciones.

La Tabla 6-1 lista algunos de los sistemas operativos utilizados para el desarrollo de sistemas embebidos en dispositivos FPGA.

TABLA 6-1: SISTEMAS OPERATIVOS UTILIZADOS PARA SISTEMAS EMBEBIDOS EN DISPOSITIVOS FPGA

Desarrollador	Sistema operativo
Micrium	µC/OS-II
Código libre	eCos
Green Hills Software	Integrity
LynuxWorks	LynuxOS
QNX	Neutrino
Mentor Graphics	Nucleus
eSol	PrKernel (µITRON4.0)
ETAS	RTA PPC
expresslogic	ThreadX
Wind River Systems	VxWorks
LynuxWorks	BlueCat Linux
Código libre	Embedded Linux
Diversos desarrolladores	Linux
Wasabi Systems	NetBSD
Mispo	NORTi/ulTRON
Enea	OSE
Código libre	µCLinux

La ejecución de un sistema operativo en un SOPC requiere determinar los requerimientos necesarios para su soporte por la arquitectura del hardware y la metodología de compilación e implementación. Esta información debe obtenerse de la documentación brindada por los desarrolladores. En este libro, se detallará el desarrollo de SOPC utilizando la HAL que brinda Altera.

ESTRUCTURA DE LA HAL DE ALTERA

Altera brinda una estructura de software basada en HAL que puede ser opcionalmente utilizada en los diseños realizados con Qsys. En esta estructura de software, Altera define dos proyectos bien diferenciados:

- **Proyecto del aplicación:** posee todos los archivos de código realizados por el diseñador para implementar su aplicación.
- **Paquete de soporte de la placa** (BSP, *Board Support Package*, en inglés): el cual implementa la HAL y es desarrollado como un proyecto diferente al de aplicación.

Esta estructura de software permite una gran flexibilidad en el diseño de aplicaciones para sistemas embebidos con hardware configurable. La Figura 6-4 muestra la estructura de software basada en la HAL brindada por Altera.

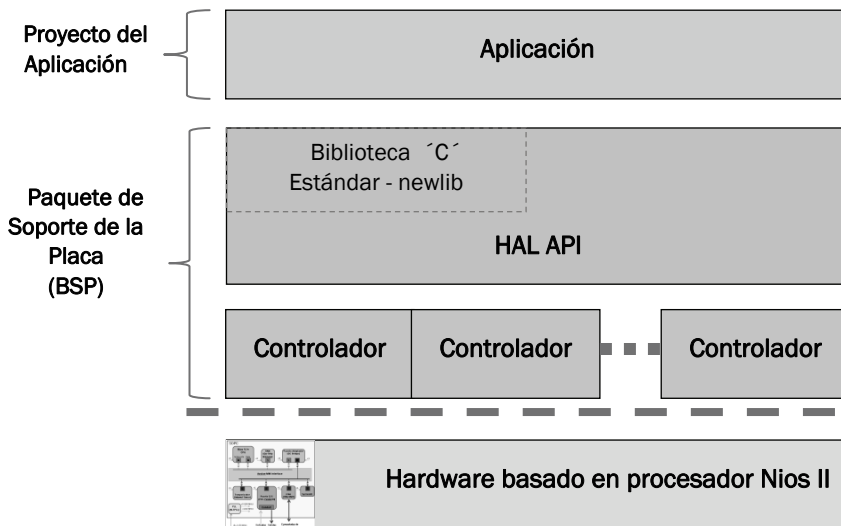


Figura 6-4: Estructura de software basada en la HAL de Altera.

La HAL de Altera brinda una **interfaz API** (“interfaz de programa de aplicación”, *Application Program Interface*, en inglés) con el software de aplicación y una estructura adecuada para la incorporación de **controladores de dispositivos** (*device drivers*, en inglés) conforme a los recursos y periféricos con que cuenta el hardware. De esta manera, modificaciones realizadas

a la configuración o a la arquitectura del hardware, no requerirán una modificación mayor del código de la aplicación, sino una adecuación de los controladores de dispositivos de la HAL. Debe tenerse en cuenta que la mayoría de las modificaciones de la HAL la realiza en forma automática la plataforma de desarrollo **Nios SBT** brindada por Altera.

Los principales servicios que brinda la HAL de Altera son (18):

- **Biblioteca ANSI C estándar 'newlib':** newlib es una implementación de la biblioteca estándar de ANSI C orientada para sistemas embebidos (ANSI C es una estandarización del lenguaje de programación C originalmente propuesto en *The C Programming Language* de B. Kernighan y D. Ritchie (17)). La HAL de Altera integra a la biblioteca newlib en su código durante el tiempo de corrida. La biblioteca newlib es de código libre y no requiere ningún tipo de licencia para su utilización o comercialización. Entre las funciones más conocidas que implementa la biblioteca newlib se encuentran: `printf()`, `fopen()`, `fwrite()`, etcétera.
- **Controladores de dispositivos (device drivers, en inglés):** permiten el acceso a los dispositivos del hardware del sistema. La HAL de Altera provee un modelo genérico basado en **clases** de periféricos más frecuentemente utilizados en sistemas embebidos como son: temporizadores, puertos de comunicaciones de Ethernet, controladores de acceso directo a memoria (DMA, *Direct Memory Access*, en inglés), puertos de comunicación serie, etcétera.
- **Interfaz HAL API:** permite una interfaz estándar y consistente a los servicios de la HAL como: acceso a los dispositivos, manejo de interrupciones y facilidades para la generación de señales del sistema.
- **Inicialización del sistema:** realiza las tareas de inicialización del procesador y del ambiente del sistema, antes de la ejecución del `main()` de la aplicación.
- **Inicialización de los dispositivos:** realiza la instanciación y las tareas de inicialización de los dispositivos del hardware del sistema, antes de la ejecución del `main()` de la aplicación.

Las facilidades que brinda una HAL otorgan al diseñador la posibilidad de abstraerse de las particularidades de los dispositivos y de la arquitectura del hardware al realizar el código de su aplicación. Esta característica permite obtener aplicaciones portables entre diferentes arquitecturas y plazos de programación y depuración más reducidos, consistentes con el desarrollo de sistemas embebidos.

Modelo genérico de los controladores de dispositivos de la HAL de Altera

La definición de la estructura de la HAL requiere agrupar los diferentes controladores de dispositivos en **clases** para unificar los mecanismos de acceso a los mismos y permitir la abstracción de programación deseada. Cada clase de dispositivo utilizará controladores con las mismas características de acceso y programación por parte de la aplicación. De esta manera, el diseñador programa el código de la aplicación utilizando las mismas funciones de la API de la HAL, sin necesidad de conocer del hardware del sistema más que la clase de cada uno de los dispositivos que lo componen.

La HAL de Altera define un **modelo genérico** para cada una de las siguientes **clases de dispositivos** (18):

- **Dispositivos de modo-carácter (*Character-mode devices*):** incluye todos los dispositivos de hardware que pueden enviar y recibir caracteres de manera serie, tal como una UART. Entre los dispositivos de esta clase que ofrece Altera se encuentran: UART, JTAG UART y el controlador de *display* LCD 16207.
- **Dispositivos de temporizado (*Timer devices*):** incluye los dispositivos que pueden contar flancos de una señal de reloj y generar requerimientos de interrupción periódicos. El *Interval Timer* es el dispositivo de esta clase ofrecido por Altera.
- **Subsistemas de archivos (*File subsystems*):** ofrece un mecanismo para el acceso de archivos almacenados en dispositivos físicos de memoria.
- **Dispositivos de ethernet (*Ethernet devices*):** incluye dispositivos que proveen acceso a conexiones de Ethernet para un determinado **protocolo de red** (“**networking stack**”) como por ejemplo el protocolo TCP/IP.
- **Dispositivos de acceso directo de Memoria (DMA) (*DMA devices*):** incluye periféricos que permiten la transferencia de un volumen de datos desde un origen a un destino. Los orígenes y destinos de información pueden ser memoria u otros dispositivos del sistema, como por ejemplo una conexión de Ethernet.
- **Dispositivos de memoria flash (*Flash memory devices*):** incluye dispositivos de memoria no-volátil que utilizan protocolos de programación especiales para el almacenamiento de datos.

La mayoría de los dispositivos de hardware ofrecidos por Altera o por otra empresa pueden ser asociados a una de estas clases y, por consiguiente, ofrecidos con controladores adecuados para su compatibilidad. Sin embargo, pueden existir algunos dispositivos de hardware, como por ejemplo el **PIO-Parallel I/O** utilizado en el Capítulo 5, que requieren características especiales para su programación y configuración que no permiten asociarlos a las clases definidas. Para este tipo de dispositivos, la HAL de Altera provee una función `ioctl()` que permite un acceso a los dispositivos de entrada y salida particulares. Las opciones de configuración y programación a través de la función `ioctl()` deben estar bien documentadas en la descripción de cada dispositivo de hardware. Como alternativa, los dispositivos que no pueden ser asociados a ninguna clase de los modelos genéricos provistos por la HAL, pueden proveer los encabezados de las funciones (*headers* en archivos **.h**) de acceso para una configuración y programación más simple de sus características.

La HAL de Altera no brinda soporte para la unidad de protección de memoria (MPU) ni para la unidad de manejo de memoria (MMU).

DESARROLLO DE SOFTWARE EMBEBIDO PARA EL PROCESADOR NIOS II DE ALTERA

El desarrollo de software para sistemas embebidos requiere de herramientas adecuadas que consideren las características del hardware en el cual se ejecutará. En las secciones anteriores, se describieron las estructuras de hardware que pueden implementarse para lograr flexibilidad en la realización de las aplicaciones embebidas. Los fabricantes de dispositivos FPGA ofrecen plataformas para el desarrollo de software embebido que facilitan la realización del flujo de diseño, integrándose también con las herramientas de diseño del hardware.

Altera agrupa todas las herramientas para el desarrollo de software embebido orientado su procesador Nios II en la plataforma que denomina **Nios II Embedded Design Suite (Nios II EDS)**. Cuando se instala la plataforma **Nios II EDS**, en la computadora personal, se crean todos los archivos y ejecutables necesarios para el desarrollo de software embebido para el procesador Nios II y se integran con la plataforma Quartus II y Qsys.

AMBIENTE DE DESARROLLO DE SOFTWARE PARA EL PROCESADOR NIOS II

La plataforma Nios II EDS provee un ambiente de desarrollo de software para sistemas basados en el procesador Nios II. Con la plataforma Nios II EDS instalada en una computadora personal, un dispositivo FPGA y un cable de programación JTAG de Altera es suficiente para el desarrollo e implementación de software embebido para sistemas mono o multi-procesador basados en el procesador Nios II.

La plataforma Nios II EDS está totalmente integrada con Qsys lo que permite automáticamente crear el proyecto del paquete de soporte de la placa (BSP), lo cual evita que el usuario realice la creación en forma manual. La creación automática del proyecto BSP permite al diseñador del software independizarse de los detalles constructivos del hardware en donde se ejecutará su aplicación.

La plataforma Nios II EDS incluye herramientas propietarias y herramientas de código libre (tales como el GNU C/C++) para la creación de proyectos ((18)).

FLUJO DE DISEÑO CON NIOS II EDS

El flujo de diseño del software para Nios II puede variar de acuerdo a las diferentes opciones que brinda la plataforma Nios II EDS, siendo estas:

- **Nios II Software Build Tools (SBT):** la cual provee dos interfaces para el usuario:
 - La línea de comandos de Nios II SBT: permite la realización del flujo de diseño mediante la ejecución de comandos Tcl y utilidades de línea de comandos. Esta opción es muy adecuada para la integración de diferentes herramientas de diseño y para realizar proyectos documentados y de fácil mantenimiento.

- La interfaz Eclipse de Nios II SBT: es una interfaz gráfica (GUI, *Graphic User Interface*, en inglés) basada en Eclipse y contiene un conjunto de complementos para la edición, programación, configuración, ejecución y depuración de proyectos de software para el procesador Nios II.
- **Nios II *Integrated Development Environment* (IDE):** es un ambiente de desarrollo utilizado en versiones anteriores y que Altera recomienda solo cuando se trabaja en proyectos previamente creados con este ambiente.

Si bien ambas opciones pueden compartir los códigos de las aplicaciones, las bibliotecas y los controladores, los flujos de diseños no son compatibles y los proyectos no pueden intercambiarse debido a cómo los *makefile* son desarrollados. Nios II IDE brinda una herramienta la cual permite convertir proyectos a Nios II SBT. Para proyectos nuevos, Altera recomienda realizarlos utilizando Nios II SBT ya que Nios II IDE no será soportada por Qsys que reemplaza a *SOPC Builder*. La Figura 6-5 muestra las dos alternativas de diseño que ofrece Nios II EDS y la relación con *SOPC Builder* y Qsys. En lo que sigue, se considerará la alternativa Nios II SBT para la creación de proyectos nuevos.

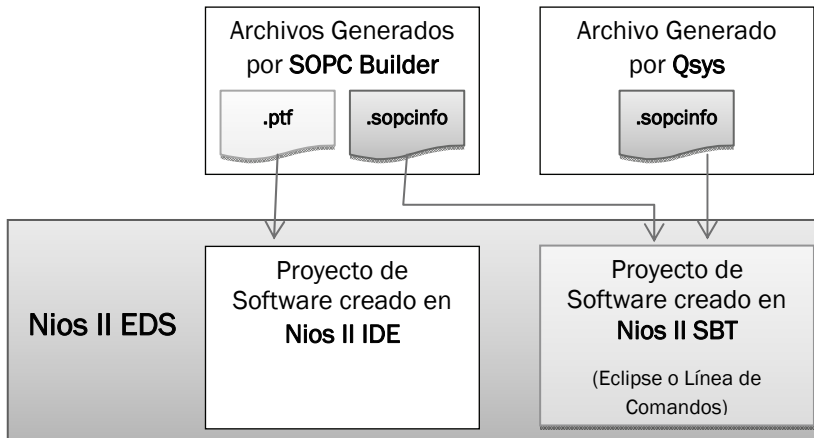


Figura 6-5: Alternativas de diseño en Nios II EDS ((18)).

MAKEFILE Y NIOS II SBT

El archivo *makefile* es el componente fundamental de un proyecto de software de Nios II cuando es creado con Nios II SBT. El archivo *makefile* describe todos los componentes que integran el proyecto de software y la manera en la que deben ser compilados y enlazados. La plataforma Nios II SBT crea automáticamente el archivo *makefile*, pero permite su modificación para controlar la creación y compilación de los archivos del proyecto.

Existen tres diferentes tipos de proyectos de Nios II:

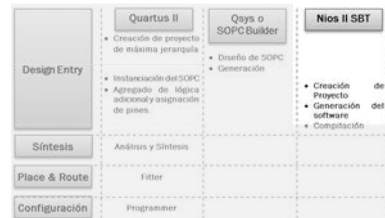
- **Proyecto de aplicación:** consiste en un conjunto de archivos de código más un archivo *makefile* necesarios para la generación de un archivo ejecutable *.elf*.

Uno de los archivos del proyecto contiene la función “`main()`” y llamadas a funciones incluidas en las bibliotecas y en el proyecto BSP.

- **Proyecto de biblioteca de usuario:** consiste en una colección de archivos de código que pueden ser compilados para crear un archivo de biblioteca **.a**. Las bibliotecas frecuentemente contienen código que generan funciones reutilizables y que pueden ser compartidas entre varios proyectos. Los archivos de código de bibliotecas no contienen la función “`main()`”.
- **Proyecto BSP:** consiste en el proyecto que genera la capa de abstracción de hardware (HAL). Opcionalmente, puede dar soporte del sistema operativo de tiempo real $\mu C/OS-II$ (RTOS). Generalmente, el proyecto BSP es generado automáticamente por el Nios II SBT.

CREACIÓN DE UN PROYECTO DE SOFTWARE EN NIOS II SBT

Es una costumbre muy arraigada que, para el inicio en la programación de sistemas embebidos, se comience por la compilación de ejemplos o códigos ya comprobados, para luego modificarlos según las propias necesidades. Altera ofrece diversos ejemplos con sus placas de desarrollo abarcando desde el original “Hello world” a ejemplos con sistemas operativos de tiempo real (RTOS) y servidor Web. Estos ejemplos se encuentran documentados y listos para ser compilados y ejecutados.



La manera más natural de comenzar con la programación de sistemas embebidos basados en el procesador Nios II es mediante la utilización de la plataforma Eclipse de Nios II SBT. Esta plataforma debe estar instalada en la misma computadora personal con la que se realizó el diseño del proyecto de hardware con Qsys (Capítulo 5). El Anexo 2 indica el procedimiento para la instalación de Nios II SBT.

EJECUCIÓN DE ECLIPSE PARA NIOS II SBT Y LA INICIALIZACIÓN DEL ESPACIO DE TRABAJO

La plataforma Eclipse permite el desarrollo de software para el procesador Nios II. La compilación del código desarrollado resultará en un archivo ejecutable con extensión **.elf**, el cual podrá ser descargado en memoria del sistema para su ejecución por el procesador Nios II. Existen diferentes alternativas para comenzar la ejecución de Eclipse:

- Seleccionando el comando del menú de Windows **Start** (o **Inicio** en versión español) > **All Programs** (o **Todos los programas** en la versión en español) > **Altera** > **Nios II EDS XXX** > **Nios II XXX Software Built Tools for Eclipse**.
- Seleccionando de la barra de menú de Quartus II el comando **Tools** > **Nios II Software Built Tools for Eclipse**.

- Seleccionando de la barra de menú de Qsys el comando **Nios II > Nios II Software Built Tools for Eclipse**.
- Seleccionando el botón **Nios II Software Built Tools for Eclipse** de la solapa **System Generation** del Qsys.

El software Eclipse es un ambiente que integra diferentes herramientas de desarrollo en una interfaz gráfica amigable para el usuario (GUI). Para el correcto funcionamiento y coordinación entre estas diferentes herramientas, Eclipse solicita que se establezca el espacio de trabajo (*Workbench*, en inglés) para el proyecto. La determinación del espacio de trabajo establecerá la carpeta en la cual se almacenarán todos los archivos requeridos por Eclipse para el manejo del estado del ambiente.

Estableciendo el espacio de trabajo del proyecto (Workbench)

Al comienzo de la ejecución, Eclipse solicita que se especifique un directorio en el cual se almacenarán todos los archivos utilizados en la sesión actual. Es conveniente definir la carpeta del espacio de trabajo como una subcarpeta en la carpeta del proyecto de hardware para evitar inconvenientes cuando se trabaja con varios proyectos diferentes. En el ejemplo propuesto, se establecerá dentro de la carpeta “c:\n2sopc” y se creará la subcarpeta “eclipse_workspace” como espacio de trabajo (Figura 6-6). Luego de especificar la carpeta en el campo **Workbench**, seleccionar el botón **OK**.

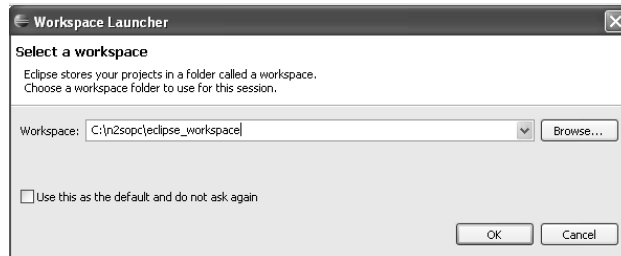


Figura 6-6: Definición del espacio de trabajo en Eclipse.

Al finalizar el proceso de inicialización, se despliega la pantalla principal del software Eclipse. Cada espacio de trabajo contiene una o más perspectivas. Cada una de ellas provee un conjunto de características adecuadas para la edición y vista de diferentes recursos del proyecto. Las diferentes perspectivas pueden ser seleccionadas mediante la solapa superior derecha (Figura 6-7).

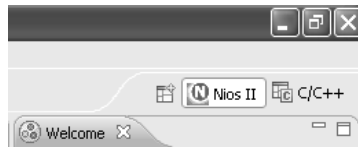


Figura 6-7: Definición del espacio de trabajo en Eclipse.

Por defecto, Eclipse inicia su ejecución en la perspectiva “Nios II”.

CREACIÓN DEL PROYECTO EN NIOS II SBT

La primera acción para realizar el software de aplicación es la creación de un proyecto de software que agrupará todos los archivos requeridos para su edición y compilación.

La creación del proyecto es conveniente realizarla mediante la guía de creación que ofrece Eclipse. Para realizar una aplicación completa que utilice la HAL de Altera es necesario incluir, además del proyecto de aplicación, el proyecto BSP asociado al hardware diseñado mediante Qsys. Para la creación de un proyecto mediante la guía de creación que incluya el soporte de la HAL de Altera, debe seleccionarse de la barra de menú de Eclipse el comando **File > New > Nios II Application and BSP from Template**. Este comando abrirá la primera pantalla de la guía de creación del proyecto de software (Figura 6-8).

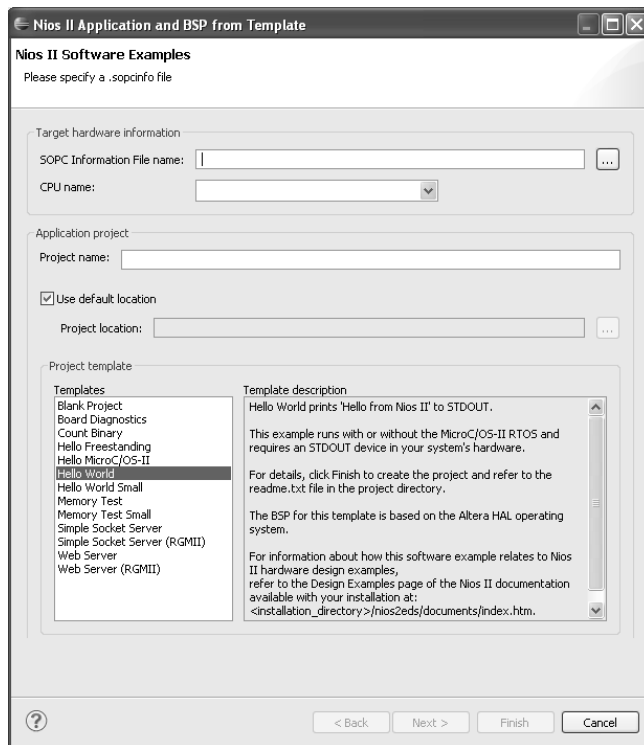


Figura 6-8: Pantalla de creación del proyecto de Eclipse.

La pantalla de creación del proyecto solicita diferentes campos para su configuración:

- **Target hardware information:** esta sección de la pantalla de creación de proyecto requiere la información de la ubicación del proyecto de hardware para obtener la información referente a la arquitectura del sistema generado por Qsys (Figura 6-5). Los campos requeridos son:

- **SOPC Information File name:** debe especificarse la carpeta y el nombre del archivo `“.sopcinfo”` generado por *SOPC Builder* o *Qsys*. Para su especificación, puede utilizarse el botón de navegación “”.
- **CPU name:** debe seleccionarse el nombre del procesador para el cual se compilará el proyecto que se está creando. En el caso de los sistemas con un solo procesador, se deberá seleccionar la única opción disponible de la lista.
- **Application Project:** esta sección permite definir el nombre que se le dará al proyecto que se está creando. Este campo definirá el nombre de la carpeta en donde se almacenarán todos los archivos correspondientes al proyecto.
- **Project name:** requiere el nombre que se le dará al proyecto que se está creando. Este campo definirá el nombre de la carpeta que contendrá todos los archivos del proyecto.
- **Use default location:** permite seleccionar la ubicación de la carpeta del proyecto. Si se selecciona, la carpeta será creada en el espacio de trabajo definido en Eclipse. Si no se selecciona, permite definir la ubicación de la carpeta en el sistema de archivos de la computadora personal.
- **Project location:** permite especificar la ubicación de la carpeta del proyecto en caso de que no se haya seleccionado la opción anterior.
- **Project template:** esta sección permite seleccionar diferentes ejemplos de proyectos ya creados. La utilización de estas aplicaciones ayuda a obtener una estructura de software que permite ser modificada según los requerimientos del diseño que se desea implementar. Las aplicaciones abarcan desde el tradicional “Hello World” a un servidor Web.
- **Templates:** permite seleccionar entre diferentes ejemplos de una lista de ejemplos disponibles.
- **Template description:** brinda información relativa al ejemplo seleccionado y los requerimientos de hardware necesarios para su implementación.

Para el ejemplo propuesto, se pueden realizar las siguientes especificaciones:

- **Target hardware information:**
 - SOPC Information File name: `“c:\n2sopc\sopcqsys.sopcinfo”`
 - CPU name: `nios2_qsys_0`.
- **Application Project:**
 - *Project name:* `hello_eclipse`
 - *Use default location:* seleccionado.
 - *Project location:*
- **Project template:**
 - *Templates:* “Hello World”.

Luego de especificar los campos de la pantalla, se selecciona el botón **Next >**. Dicha acción habilita la segunda pantalla de creación de proyecto en la cual se pueden especificar las características del proyecto BSP asociado al proyecto de aplicación (Figura 6-9). En el ejemplo propuesto, se dejarán todas las opciones por defecto y se seleccionará el botón **Finish**. Esta acción especificará que el proyecto BSP que contendrá la HAL de Altera, se creará con el nombre “hello_eclipse_bsp”.

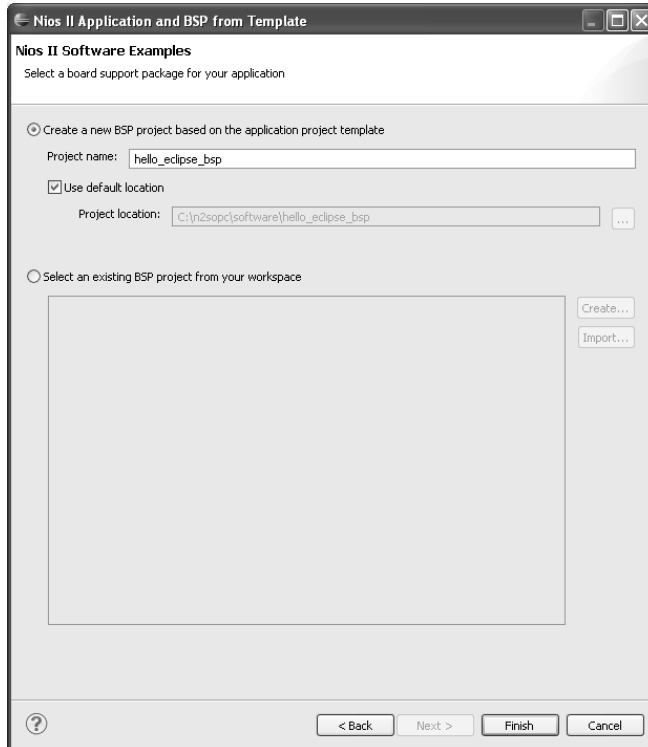


Figura 6-9: Pantalla de especificación del proyecto BSP.

Luego de un periodo de procesamiento, la herramienta Eclipse creará una carpeta “software” en el directorio del proyecto de hardware realizado con Quartus II (“c:\n2sopc\”) y dentro de dicha carpeta, dos subcarpetas:

- “hello_eclipse” para el proyecto de aplicación el cual contendrá todos los archivos de la aplicación propiamente dicha, y
- “hello_eclipse_bsp” para el proyecto BSP que construirá la HAL de Altera para la plataforma de hardware construida con *SOPC Builder* o *Qsys* y especificada en la pantalla de creación del proyecto (Figura 6-8).

Ambos proyectos están bien diferenciados en la ventana del **Project Explorer** de Eclipse (Figura 6-10).

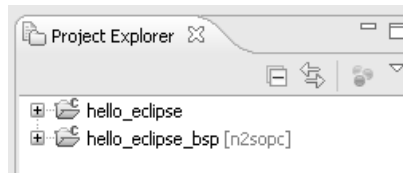


Figura 6-10: Proyectos en el *Project Explorer* de Eclipse.

PROGRAMACIÓN DEL CÓDIGO DE LA APLICACIÓN

La utilización de la HAL de Altera permite que el diseño de la aplicación se restrinja generalmente a la programación en el proyecto de aplicación, relegando toda la interfaz con el hardware al proyecto BSP. El código de la aplicación estará totalmente en el proyecto de aplicación.

En el sistema propuesto, el proyecto de aplicación contendrá el archivo **hello_word.c** con el siguiente código:

```
#include <stdio.h>
int main()
{
    printf("Hello from Nios II!\n");
    return 0;
}
```

La resolución de la función `printf` estará resuelta mediante la inclusión del archivo **stdio.h** del proyecto BSP. De esta manera, el proyecto de aplicación se abstrae de las particularidades propias de la arquitectura del hardware, permitiendo que cualquier modificación en esta última no implique modificaciones en el código de la aplicación (si requerirá una nueva generación del proyecto BSP y una nueva compilación de todo el software).

Existen casos en los que se requiere un acceso directo al hardware. En estos casos, es conveniente la utilización de las macros de cada componente del SOPC y de las definiciones disponibles en el archivo **system.h** del proyecto BSP. En el ejemplo propuesto, el acceso a los puertos del componente PIO Parallel I/O puede realizarse mediante la macro **IOWR_ALTERA_AVALON_PIO_DATA** (definida en el archivo **altera_avalon_pio_regs.h**) a la dirección `PIO_0_BASE` (es el nombre del componente "PIO_0" con referencia a la dirección base de memoria "_BASE" en la cual se encuentra el registro de entrada y salida), mediante la instrucción:

```
IOWR_ALTERA_AVALON_PIO_DATA(PIO_0_BASE, valor_salida);
```

en donde *valor_salida* es la variable que contiene la información de las señales de salida que se desean poner en '0' o en '1' de acuerdo a los bits que la componen.

Cada componente posee un modelo de programación del software (*Software Programming Model*, en inglés) que determina la manera en que se accede a sus registros de estado y configuración. Más información sobre las macros definidas para los componentes o la estructura de su modelo de programación, se puede obtener en (19) .

CONFIGURACIÓN DEL PROYECTO BSP: EDITOR DE BSP (BSP EDITOR)

La configuración del proyecto BSP permite especificar los recursos de hardware que serán utilizados por el proyecto de aplicación. En dicha configuración, se puede establecer, entre otros parámetros, los puertos estándar del sistema y las posibles memorias para alojar código y datos.

La plataforma Eclipse del Nios II EDS provee el editor de BSP (*BSP Editor*) que permite configurar parámetros simples y complejos del proyecto BSP. El *BSP Editor* brinda la posibilidad de configuración mediante archivos de comandos “Tcl” y a través de una interfaz gráfica amigable para el usuario.

Para acceder al *BSP Editor*, se debe realizar un clic con el **botón derecho** del *mouse* sobre el proyecto BSP en la ventana del **Project Explorer** y seleccionar el comando **Nios II > BSP Editor...**(Figura 6-11).

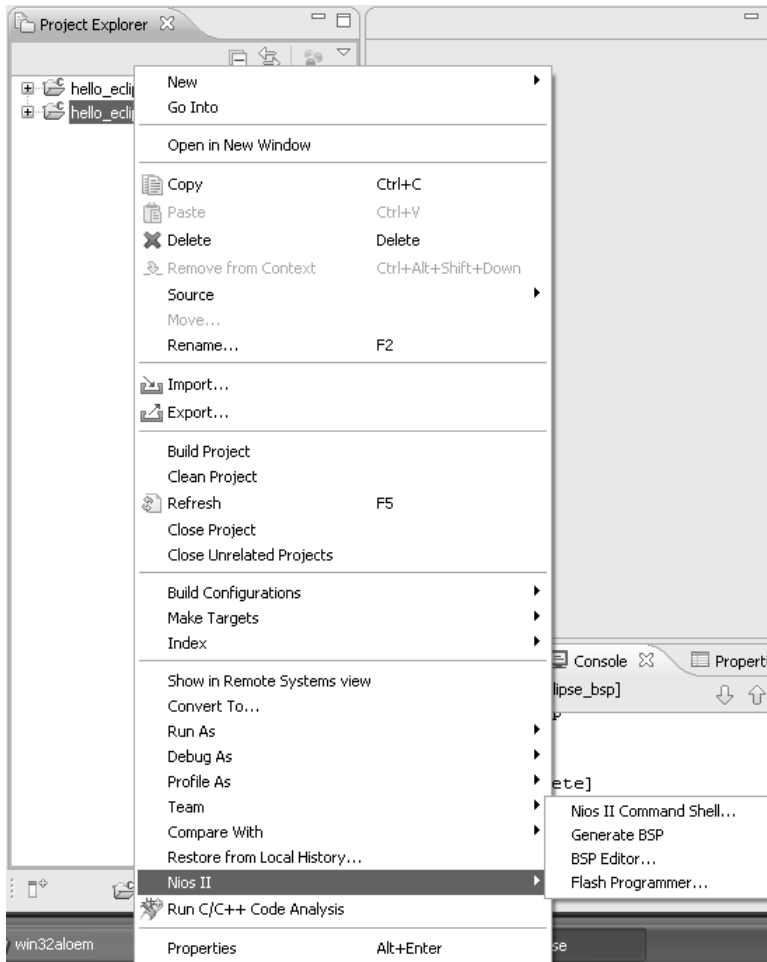


Figura 6-11: Acceso al BSP Editor.

La pantalla del editor de BSP presenta diferentes solapas para la configuración de diferentes características del proyecto BSP (Figura 6-12). Cada una de las solapas permite especificar ajustes o parámetros del proyecto ((18)).

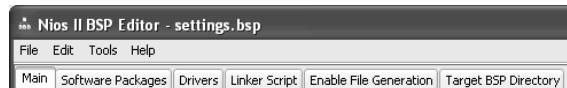


Figura 6-12: Solapas de la pantalla del BSP Editor.

SOLAPA MAIN

La solapa **Main** permite definir ajustes y parámetros del proyecto BSP y de soporte para sistemas operativos. En la parte superior de la ventana, se muestran el directorio del archivo de especificación del hardware **.sopcinfo**, el nombre del procesador, el sistema operativo y su versión, y el directorio del proyecto BSP. Esta información coincide con la configuración que se realizó durante la creación del proyecto.

No puede modificarse el sistema operativo de un proyecto BSP ya creado. En caso de necesitar soporte de otro tipo de sistema operativo, debe crearse nuevamente el proyecto BSP.

Los diferentes ajustes (*Settings*, en inglés) que permite realizar al proyecto el *BSP Editor* en la solapa **Main** están ordenados en una estructura del tipo árbol en la parte central izquierda de la ventana. Los ajustes están agrupados en dos categorías principales: **Common** y **Advanced**. Cada categoría permite ajustes agrupados en grupos funcionales que dependen del sistema operativo seleccionado para el proyecto BSP.

El detalle de los diferentes ajustes se realizarán para la selección de la HAL de Altera como capa de abstracción de hardware del proyecto BSP.

Categoría Common

La categoría **Common** contiene una subcategoría “*hal*” que permite configurar las siguientes propiedades:

- **enable_gprof**: si se habilita esta opción, la aplicación se enlazará (*link*) con la biblioteca *GPROF*. Esta configuración permite realizar un perfil del código (*Code Profiling*, en inglés) para analizarlo durante el tiempo de corrida ((20)).
- **enable_reduced_device_drivers**: si se habilita esta opción, algunos controladores de dispositivos son instalados con prestaciones reducidas y otros son eliminados. Es proporcionada para cuando se desea reducir el tamaño de memoria requerida para el sistema, pero debe considerarse su selección para no ocasionar comportamientos inadecuados.
- **enable_sim_optimize**: si se habilita esta opción, el código es compilado para mejorar el tiempo de simulación. Sin embargo, no debe ser habilitada cuando la compilación es realizada para implementarse en el hardware real.
- **enable_small_c_library**: si se habilita, se utiliza una versión reducida de la biblioteca *newlib*. Debe considerarse su uso para no ocasionar comportamientos inadecuados.
- **stderr**: permite especificar el dispositivo que se empleará como consola estándar de error. La opción permite seleccionar de una lista de dispositivos que pueden ser de modo-carácter (página 130).

- **stdin**: permite especificar el dispositivo que se utilizará como consola estándar de entrada. La opción permite seleccionar de una lista de dispositivos que pueden ser de modo-carácter (página 130).
- **stdout**: permite especificar el dispositivo que se utilizará como consola estándar de salida. La opción permite seleccionar de una lista de dispositivos que pueden ser de modo-carácter (página 130).
- **sys_clk_timer**: permite especificar el dispositivo temporizador del sistema. Este temporizador proveerá una interrupción periódica (*tick*) que generalmente es requerido para el funcionamiento del sistema operativo.
- **timestamp_timer**: permite especificar el dispositivo temporizador para medición de tiempo de alta resolución. Este tipo de característica es necesaria para la realización de perfiles de código (*code profiling*) y requieren un componente **high_resolution time** contenido en el hardware del sistema.
- **Linker**: es un grupo de opciones que permiten configurar la utilización de pilas (*stacks*) separadas para excepciones e interrupciones. Por lo general, estas opciones deben estar deseleccionadas.
- **make**: es un grupo de opciones que especifican la optimización de compilación del proyecto BSP. En caso de que se desee depurar (*debug*) el código, se debe compilar sin optimización para preservar los símbolos en el ejecutable **.elf**. Los ajustes de optimización modifican el archivo **Makefile**.

Categoría Advanced

La categoría **Advanced** contiene una subcategoría “*hal*” que permite configurar propiedades avanzadas de la HAL de Altera. Generalmente, estas opciones deben dejarse en los valores por defecto que le son asignados. Como caso especial, puede necesitarse la modificación de las herramientas de compilación, para lo que debe especificarse los valores correspondientes en los ítems del grupo **make**.

SOLAPA SOFTWARE PACKAGES

La solapa **Software Packages** permite insertar y eliminar paquetes de software brindados por Altera y configurar sus propiedades. En la parte superior de la solapa, están listados los posibles paquetes de software disponibles para la versión de HAL o sistema operativo elegido.

La selección en la creación del proyecto BSP de HAL o sistema operativo determina los paquetes de software disponibles. Cuando se desea incluir un paquete de software específico, se debe seleccionar la opción **Enable** correspondiente.

En la HAL que ofrece Altera, existen tres paquetes de software que pueden ser seleccionados (no se detalla el paquete `altera_quad_seven_seg`):

- **Paquete de sistema de archivos basados en el host (*host-based file system*, **altera_hostfs**)**: este paquete de software permite manipular archivos que se encuentran en la computadora personal a través de la conexión JTAG. Las API

que ofrece este paquete de software son transparentes para el acceso de dato en los archivos de la computadora personal y no requieren ningún componente especial en el hardware del sistema. Esta opción es adecuada para la depuración de aplicaciones o para cuando se desea registrar datos en la computadora personal obtenidos por el procesador Nios II ((18)).

- **Paquete de lectura de sistema de archivos .zip (*Zip Read-Only File System, altera_ro_zipfs*):** este paquete de software permite el acceso a un sistema de archivos de solo lectura almacenado en la memoria *flash* del sistema. El sistema de archivos está basado en un archivo sin comprimir **.zip** que puede ser accedido en forma transparente a través de las funciones de entrada y salidas estándar de ANSI C (tales como `fopen()` y `fread()`).

SOLAPA DRIVERS

La solapa **Drivers** permite configurar los diferentes controladores de los dispositivos considerados en el hardware del sistema y la posibilidad de incluirlos en el proyecto BSP.

En la parte superior de la pantalla, se detalla cada uno de los componentes que fueron incluidos en el proyecto de hardware en Qsys y el nombre del controlador, su versión y su clase. Todos los dispositivos que brinda Altera para ser incluidos en proyectos de hardware en Qsys contienen el controlador correspondiente para su inclusión en el proyecto BSP de la HAL de Altera.

Es importante considerar la disponibilidad de controladores cuando se obtienen componentes de hardware de diferentes fabricantes y su compatibilidad con la HAL de Altera para que puedan ser integrados al diseño sin inconvenientes.

Existen algunos dispositivos a los que se le pueden configurar diferentes características en la parte central de la solapa. Al seleccionar el dispositivo se habilitan las diferentes categorías con las propiedades que pueden ser configuradas. La mayoría de los controladores permite la selección de una versión reducida del mismo (*small_driver*) que implementa el controlador con funcionalidades restringidas pero que requiere una menor cantidad de recursos del sistema.

SOLAPA LINKER

El comportamiento de un sistema basado en procesador depende de la ejecución del código almacenado en memoria del sistema. Es importante determinar las diferentes direcciones de memoria en las que se almacenará el código para un correcto funcionamiento de todo el sistema.

El *linker* es una herramienta utilizada durante el proceso de compilación del código que producirá una imagen de memoria del programa que será almacenado en memoria para su ejecución por el procesador. Es necesario especificar correctamente las direcciones del

sistema en las que existen dispositivos de memoria y la asignación de tales memorias a las diferentes partes del código compilado.

La solapa **Linker** permite asignar la memoria del sistema para la compilación de las diferentes secciones del programa. Por otro lado, cada dispositivo de memoria del sistema puede estructurarse en regiones para una adecuada asignación de cada una de las secciones del código a regiones de memoria. La Figura 6-13 muestra la relación entre las secciones del código y las regiones de memoria que pueden especificarse en la solapa **Linker**.

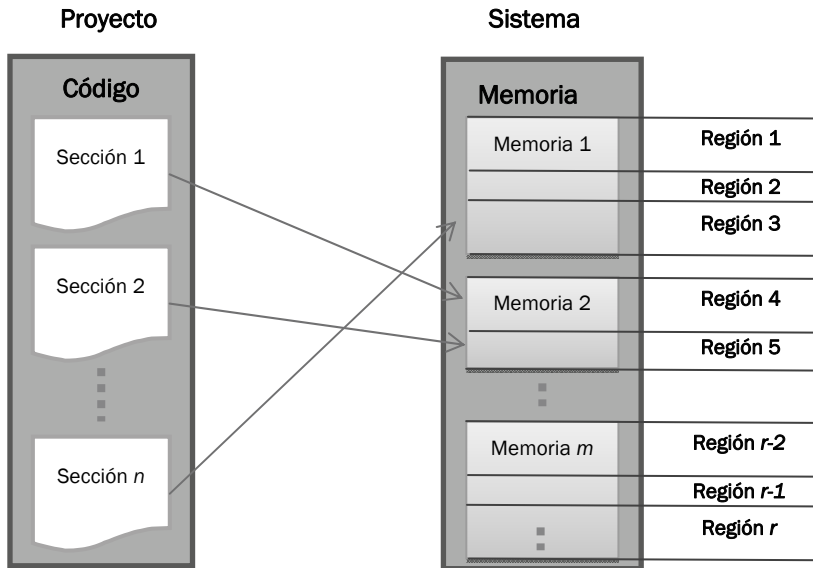


Figura 6-13: Definición de sección, memoria y región para Linker.

La solapa tiene dos secciones principales:

- **Linker Section Mappings:** esta sección muestra una tabla que permite asignar las diferentes secciones del código del proyecto a las regiones de memoria. La columna “Memory Device Name” permite seleccionar el dispositivo de memoria de acuerdo a los dispositivos que se incorporaron en el diseño del hardware e identificado con el mismo nombre que se le asignó en Qsys. La definición de nuevas secciones de código pueden realizarse mediante el botón **Add...**
- **Linker Memory Regions:** esta sección permite definir o modificar las regiones en las que se estructura la memoria del sistema. Existen dos botones cuyas funciones merecen mencionarse:
 - **Memory Usage:** permite determinar el porcentaje de utilización de cada una de las regiones de memoria.

- **Memory Map:** permite visualizar la organización de memoria y dispositivos de entrada y salida del sistema.

SOLAPA ENABLE FILE GENERATION

La generación del sistema involucra el procesamiento de los diferentes archivos de código para producir el programa ejecutable. La solapa **Enable File Generation** permite modificar qué archivos son considerados para la generación del programa. En esta solapa, se listan todos los módulos del programa y permite deshabilitar la generación del código correspondiente a diferentes archivos Qsys, es conveniente habilitar al menos la primera vez que se genera el código del proyecto BSP para que la generación sea consistente con la nueva arquitectura del hardware.

SOLAPA TARGET BSP DIRECTORY

La solapa **Target BSP Directory** lista toda la estructura de software que se creará cuando se genera el proyecto BSP.

Todas las modificaciones realizadas con el **BSP Editor** serán realizadas solo si se selecciona el botón **Generate**.

Siempre que se realice una modificación del SOPC en Qsys y se genere un nuevo archivo **.sopcinfo**, se deberá generar el proyecto BSP mediante el comando del menú sensitivo **Nios II > Generate BSP** (Figura 6-11).

En el ejemplo propuesto, no es necesario realizar ninguna modificación a las asignaciones por defecto del proyecto BSP. Para finalizar la configuración del proyecto BSP, se debe seleccionar el botón **Exit**.

PROPIEDADES DEL PROYECTO BSP

El editor de BSP (**BSP Editor**) permite configurar las diferentes características que contendrá la HAL de Altera o el sistema operativo considerado en el proyecto BSP. Por otro lado, el proyecto será un proyecto de software compilado en Eclipse. Para configurar las opciones de construcción del proyecto, se utiliza la pantalla de propiedades que se accede mediante el comando **Properties** del menú desplegado cuando se hace clic con el botón derecho del mouse sobre el nombre del proyecto BSP ("hello_eclipse_bsp" en el ejemplo propuesto).

La Figura 6-14 muestra la pantalla de propiedades del proyecto BSP con el árbol de opciones totalmente expandido. La selección de las propiedades **Nios II BSP Properties** permite

especificar algunas de las opciones disponibles también mediante el editor de BSP (BSP Editor).

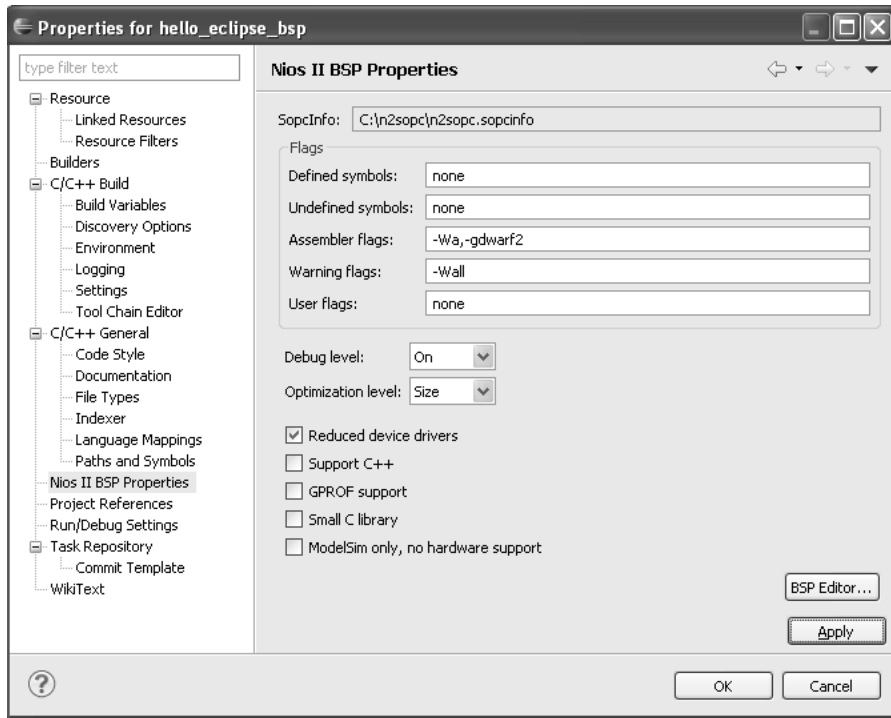


Figura 6-14: Pantalla de propiedades del proyecto BSP.

Para el ejemplo propuesto, especificaremos el campo **Optimization level** del ítem **Nios II BSP Properties** en **Size**. Esta selección configurará la propiedad **bsp_cflags_optimization** (subcategoría **make** de la solapa **Main**, página 142) en “-Os” para indicar optimización del tamaño del código generado. También se seleccionará la opción **Reduced Device Drivers** para realizar un código más compacto. Debido a que el proyecto de aplicación no requerirá soporte para el lenguaje C++, se deseleccionará la opción **Support C++**. Seleccionar **Apply**.

CONFIGURACIÓN DEL PROYECTO DE APLICACIÓN

Tal como se ha configurado las propiedades de compilación del proyecto BSP, se puede configurar las propiedades del proyecto de aplicación. Para acceder a la pantalla de propiedades, se debe hacer clic con el botón derecho del *mouse* sobre el nombre del proyecto e aplicación (“hello_eclipse” en el ejemplo propuesto) y seleccionar el comando **Properties** (Figura 6-15).

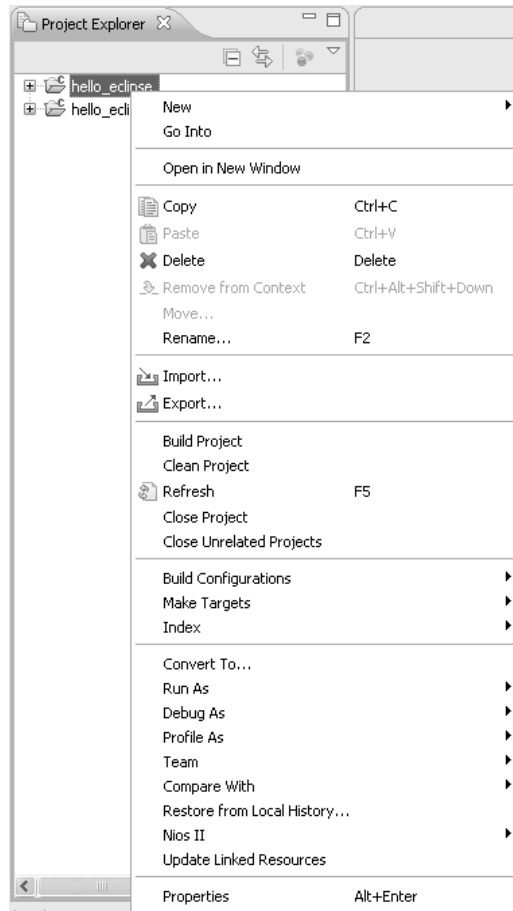


Figura 6-15: Acceso a propiedades del proyecto de aplicación.

La pantalla de propiedades del proyecto de configuración es similar a la del proyecto BSP con diferentes ítems. La Figura 6-16 muestra la ventana de configuración del ítem Nios II *Application Properties*.

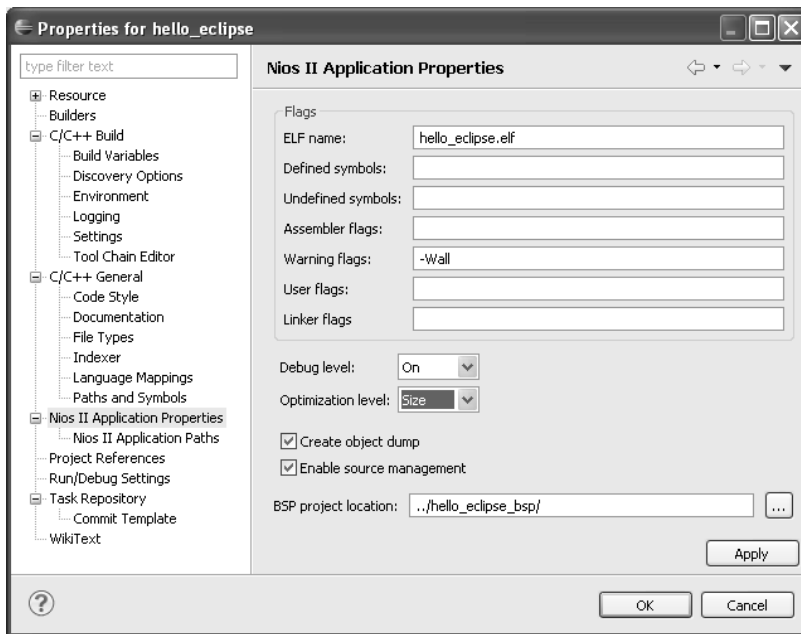


Figura 6-16: Pantalla de propiedades del proyecto de aplicación.

Los diferentes ítems permiten seleccionar las diferentes ventanas para la configuración de los parámetros de generación que Eclipse utilizará cuando compile el código del proyecto de aplicación.

Para el ejemplo propuesto, se seleccionará **Size** como parámetro de optimización en el campo **Optimization Level**. Seleccionar **Apply**.

CONSTRUCCIÓN DE LOS PROYECTOS

Una vez configurados todos los parámetros de los proyectos, se está en condiciones de pasar a la etapa de construcción de cada uno de los proyectos. Para la construcción del proyecto, Eclipse procesa los códigos de los proyectos de acuerdo a la configuración realizada y produce un archivo ejecutable y enlazado (**.elf**) que puede ser transferido a la memoria del sistema para su ejecución.

La construcción del proyecto involucra el procesamiento de los diferentes archivos que lo integran. Este procesamiento está determinado por la ejecución de un archivo que contiene los diferentes pasos de construcción y que se denomina *Makefile*.

Para la construcción de un proyecto, se debe hacer clic con el botón derecho del *mouse* sobre el nombre del proyecto y seleccionar el comando **Build Project** (Figura 6-15 para proyecto de aplicación).

Para el ejemplo propuesto, primero se ejecutará el comando **Build Project** para el proyecto BSP y posteriormente para el proyecto de aplicación.

La consola de Eclipse indicará las diferentes etapas de construcción que se estarán procesando e indicará con un mensaje **[BSP build complete]** y **[hello_eclipse build complete]** cuando finalice exitosamente la construcción del proyecto BSP y el proyecto de aplicación respectivamente.

El archivo ejecutable se incluirá en la carpeta del proyecto de aplicación ("c:\n2sopc\software\hello_eclipse" para el ejercicio propuesto), con el nombre del proyecto de aplicación y extensión ".elf" ("hello_eclipse.elf" para el ejercicio propuesto). Este archivo contiene todo el software necesario para ejecutarse en el hardware diseñado mediante Qsys.

En los próximos capítulos, se detallará la metodología para simulación e implementación del hardware y software diseñado.

CONCLUSIONES

En el presente capítulo, se describieron las diferentes etapas para la realización de software embebido para implementar un SOPC. Se detallaron las características más importantes de la capa de abstracción de hardware (HAL) y la flexibilidad de diseño que permite su utilización. Las herramientas de diseño de software para sistemas embebidos presentan importantes características de configuración orientadas a soportar la gran dinámica que presenta el diseño de arquitecturas de hardware basadas en dispositivos FPGA.

CAPÍTULO 7

IMPLEMENTACIÓN DEL SOPC

INTRODUCCIÓN

La implementación de un SOPC abarca la correcta configuración del hardware como así también del software. En el Capítulo 2, se describen las arquitecturas de los dispositivos FPGA cuyas configuraciones están realizadas acorde a la información que se almacena en los registros desplazamientos de las LUT. Debido a la característica no-volátil de estos registros, la configuración del dispositivo FPGA debe realizarse cada vez que este se energiza.

Los dos mecanismos más frecuentemente utilizados para realizar la configuración de un dispositivo FPGA son:

- **Transferencia desde una memoria *flash* conectada al dispositivo FPGA:** esta alternativa es utilizada cuando se desea que el sistema funcione de manera independiente a la de una computadora personal de configuración. La mayoría de las placas de desarrollo basadas en dispositivos FPGA cuentan con memorias *flash* de las cuales el dispositivo FPGA autónomamente transfiere la información para su configuración.
- **Transferencia desde una computadora personal con un software de configuración:** esta alternativa es adecuada cuando el proyecto está en etapa de desarrollo y pruebas de funcionamiento. La configuración del dispositivo FPGA es transferida desde la computadora personal mediante la adecuada utilización del software de programación.

Es importante remarcar que ambas alternativas configuran los recursos con que cuenta el dispositivo FPGA para la implementación final del hardware diseñado para tal dispositivo. En el caso del diseño de un SOPC, el cual incluye también un componente de memoria implementado con recursos internos del dispositivo FPGA, entonces se podría inicializar dicho componente para que la configuración del dispositivo abarcara el hardware y software del SOPC. Si el software del SOPC es almacenado en una memoria externa al dispositivo FPGA, entonces será necesaria la utilización de una memoria no volátil para su almacenamiento que sea direccionada por el procesador en el momento de arranque del sistema.

IMPLEMENTACIÓN DEL HARDWARE Y SOFTWARE

Los SOPC requieren algún mecanismo para correcta configuración del hardware y el software. En algunos casos, la arquitectura del SOPC permite implementar en memoria interna del dispositivo FPGA todo el software lo que permite realizar la configuración del hardware y del software simultáneamente. En otros casos, el software debe almacenarse en memoria externa volátil para ser ejecutada por el procesador y, por consiguiente, debe inicializarse dicha memoria en el momento de arranque del sistema. Tal inicialización de memoria con el software del sistema, generalmente denominado *boot-loading*, abarca: (1) el acceso a un componente de almacenamiento no-volátil con la información a cargar en la memoria del sistema, (2) el procesamiento de dicha información en caso de que esté comprimida o compactada, (3) la copia de la información a la memoria correspondiente del SOPC para su ejecución y (4) la transferencia del control de ejecución del procesador a la dirección de memoria de comienzo del software cargado.

EJECUCIÓN DEL SOFTWARE DESDE MEMORIA INTERNA DEL DISPOSITIVO FPGA

Todo sistema basado en procesador debe contener una memoria para almacenar el software del sistema. La memoria es utilizada entonces para almacenar los datos y las instrucciones de la aplicación que el sistema ejecuta para lograr la funcionalidad deseada. Es muy importante entonces que dicha memoria contenga el código correcto al momento que el procesador comienza a buscar la primera instrucción a ejecutar. En sistemas basados en procesadores tradicionales (*hard-processors*), por lo general, se utilizan memorias del tipo ROM debido a su característica “no-volátil” la cual permite que el procesador encuentre un código válido al buscar la primera instrucción a ejecutar. En contraparte, una memoria del tipo ROM no es adecuada para almacenar datos producidos durante el funcionamiento del sistema debido a que el proceso de grabación requiere consideraciones especiales eléctricas y temporales.

Por otra parte, un SOPC implementado en un dispositivo FPGA puede utilizar la memoria interna del dispositivo para almacenar el código y datos de la aplicación a ejecutar. Debido a que los recursos de memoria interna del dispositivo FPGA son del tipo RAM, estos deben ser inicializados antes de que el procesador busque la primera instrucción a ejecutar. La inicialización de la memoria interna puede realizarse en el momento en que se configura el dispositivo FPGA. De esta manera, el archivo de programación del dispositivo FPGA contendrá la configuración del hardware y del software que se realizará en el mismo proceso. Es muy importante entonces, que en el momento de generación del archivo de configuración, se incluyan las inicializaciones de la memoria interna con el código y datos que deben contener en el momento de arranque del sistema.

Altera ofrece alternativas para la inicialización de las memorias internas del dispositivo FPGA con el código y datos generados al compilar la aplicación.

CREACIÓN DE LOS ARCHIVOS DE INICIALIZACIÓN DE MEMORIA INTERNA

El proceso de construcción de un proyecto de aplicación involucra la creación de los archivos necesarios para la inicialización de la memoria del sistema. Existen diversos formatos de archivos de memoria, siendo el más divulgado el de “formato hexadecimal de Intel” (archivos con extensión “.hex”). La plataforma Nios II SBT puede generar archivos “.hex” y de otros formatos, como un paso intermedio para la generación del archivo de configuración del dispositivo FPGA (.sof o .pof). Los archivos de inicialización de memoria deben ser incorporados al proyecto de Quartus II para que sean considerados al momento de la síntesis total del sistema. El proceso de generación de los archivos de inicialización está realizado mediante archivos de comandos (**Makefile**) en Eclipse.

Inclusión automática de los archivos de inicialización de memoria al proyecto Quartus II

Altera ofrece una alternativa que automatiza la generación del archivo de inicialización del sistema y su inclusión en el proyecto de Quartus en caso de que sea implementada con recursos de memoria interna del dispositivo FPGA. Sin embargo, esta alternativa **no es aconsejada** debido a que puede haber inconvenientes originados en los diferentes directorios de proyectos involucrados.

Para proceder con esta alternativa, se debe realizar un clic con botón derecho del **mouse** sobre el proyecto de aplicación de Eclipse, seleccionar el comando **Make targets > Build**, seleccionar **mem_init_install** y hacer clic en **Build** (Figura 7-1).

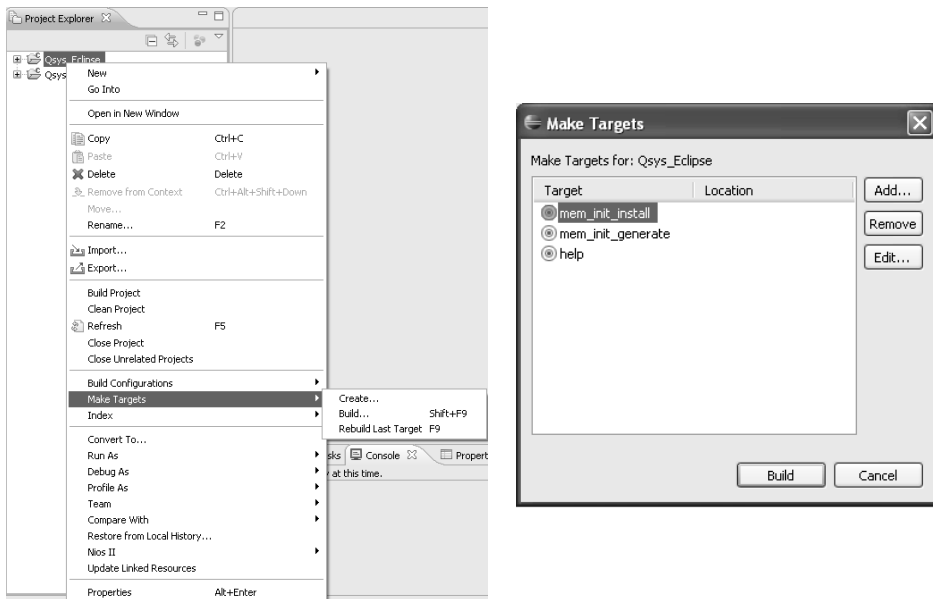


Figura 7-1: Menú inclusión automática de archivo de inicialización de memoria interna.

Este archivo **makefile mem_init_generate** genera el archivo de inicialización de cada memoria del SOPC y lo incorpora al proyecto de Quartus II. Sin embargo, pueden existir inconvenientes en la realización del proceso, por lo que se muestra un mensaje que sugiere la realización manual.

Inclusión manual de los archivos de inicialización de memoria al proyecto Quartus II

Para realizar la incorporación manual de los archivos de inicialización, deben realizarse los siguientes pasos:

- 1) Seleccionar el comando **Make targets > Build** del menú que se despliega haciendo clic con el botón derecho del *mouse* sobre el proyecto de aplicación.
- 2) Seleccionar la opción **mem_init_generate** y hacer clic en el botón **Build** (Figura 7-1).

Esta opción ejecuta el archivo **makefile** y genera los archivos **.hex** por cada memoria del sistema y un archivo **.qip** para incorporarlo al proyecto Quartus II antes de compilar todo el sistema.

En el ejemplo propuesto, los archivos de inicialización del componente “memoria_ram” se crearán en el directorio “c:\n2sopc\software\hello_eclipse\mem_init\mem_init.qip”. Este archivo deberá ser incluido en el proyecto de Quartus II mediante el siguiente proceso:

- 1) Abrir el archivo “c:\n2sopc\software\hello_eclipse\mem_init\mem_init.qip” a través del comando **File > Open...** de la barra de menú de Quartus II.
- 2) Incorporar el archivo abierto al proyecto mediante la ejecución del comando **Project > Add Current File to Project**.

Luego de la incorporación del archivo de inicialización de memoria, se debe recompilar todo el proyecto en Quartus II para que los cambios sean considerados en la creación del archivo de configuración (**.sof** o **.pof**). La configuración del hardware y del software se realiza entonces mediante la programación del dispositivo FPGA como se detalló en “Programador del dispositivo”, página 54.

NOTA: la compilación del proyecto final en Quartus II puede realizarse para la **simulación** o para la **síntesis**. Dependiendo del objetivo de la compilación se deben seleccionar el archivo de descripción del SOPC correspondiente generado por Qsys. Para **simulación**, se encuentra en la carpeta “../software/simulation”, mientras que para **síntesis** se encuentra en la carpeta “../software/synthesis”. Los archivos de descripción de los componentes del SOPC se encuentran en una subcarpeta “/submodules” en cada una de las carpetas respectivas. Estas deben ser incluidas en las librerías de compilación en la pantalla que se accede por **Tools -> Options** en la categoría **Libraries**, seleccionando las carpetas mediante el botón ... e incorporándolas mediante el botón **Add** (Figura 7-2).

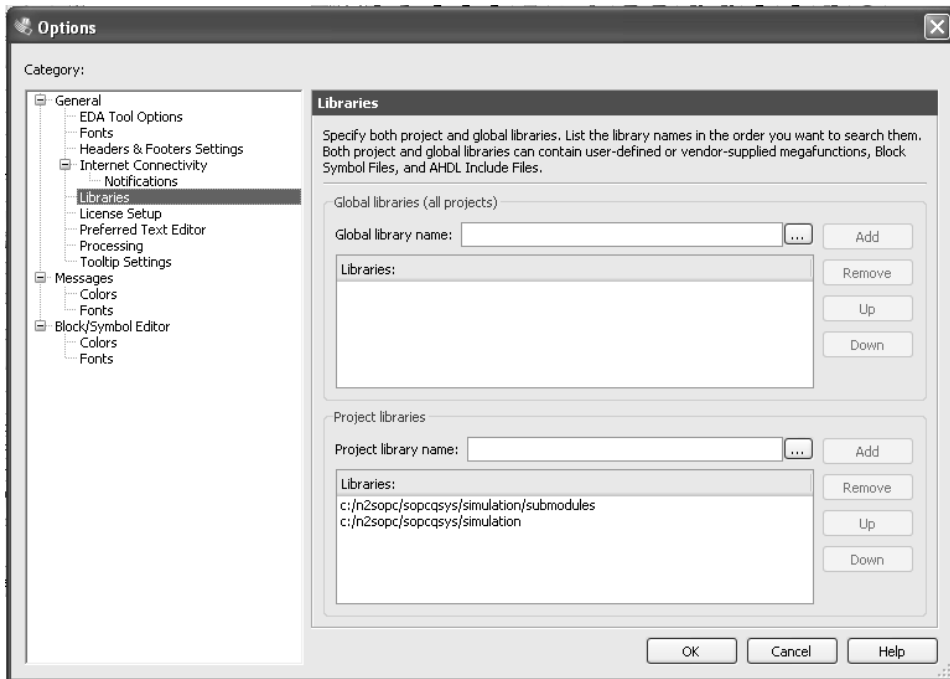


Figura 7-2: Inclusión de carpetas de simulación o síntesis del SOPC en la compilación del proyecto Quartus.

VENTAJAS Y DESVENTAJAS DE LA UTILIZACIÓN DE MEMORIA INTERNA DEL DISPOSITIVO FPGA

La memoria interna del dispositivo FPGA es el tipo más simple de utilización en el desarrollo de SOPC (21). En este tipo de configuración, la memoria es implementada propiamente en la FPGA y, consecuentemente, no requiere conexiones a dispositivos externos en la placa del circuito impreso. La implementación de este tipo de memoria se simplifica a agregar un componente de *On-Chip Memory* desde la biblioteca de componente en el diseño del SOPC en Qsys.

Como ventaja principal en la utilización de este tipo de memoria es su alta eficiencia, reduciendo la latencia de acceso a un solo ciclo de reloj. También puede configurarse para acceso *dual-port* para incrementar el ancho de banda al realizar lecturas y escrituras en forma simultánea. Otra ventaja adicional es que su inicialización se restringe a la inicialización del dispositivo FPGA, no requiriendo ningún proceso adicional.

La desventaja que presenta este tipo de memoria es su baja capacidad de almacenamiento, forzando al diseñador a utilizar memorias externas cuando el software alcanza un tamaño apreciable.

EJECUCIÓN DEL SOFTWARE DESDE LA MEMORIA *FLASH* DE CONFIGURACIÓN DEL DISPOSITIVO FPGA

La limitada capacidad de almacenamiento de los recursos de memoria interna de los dispositivos FPGA generalmente obliga al diseñador a almacenar los datos e instrucciones de la aplicación en dispositivos de memoria externos.

Existen dos categorías en las que se pueden agrupar las memorias de acuerdo a la preservación de su contenido cuando no se encuentran energizadas:

- **Memorias no-volátiles:** preservan los datos almacenados aun cuando la memoria no se encuentra energizada. Es necesario que el sistema contenga al menos un componente de este tipo para almacenar el código que el sistema debe ejecutar en el arranque. La mayoría de los dispositivos FPGA tienen asociada, en la misma placa, una memoria del tipo *flash* en la cual se almacena la configuración del hardware del dispositivo FPGA, y que permite también almacenar software para ser ejecutado por el SOPC.
- **Memorias volátiles:** no preservan los datos cuando son desenergizadas. Se utilizan para almacenar datos en tiempo de funcionamiento o cargar el software para ser ejecutado por el procesador. Existen diversas tecnologías que permiten diferentes capacidades de almacenamiento y tiempo de acceso a los datos, tanto para lectura como para escritura.

Generalmente, el tiempo de acceso a los datos almacenados en una memoria no-volátil es mayor que los correspondientes a memorias volátiles. Es por esta razón que es conveniente copiar el software a ejecutar desde la memoria no-volátil en la cual se almacena para preservarlo mientras el sistema está desenergizado, a una memoria volátil para mejorar la velocidad de ejecución del mismo. Esta transferencia entre la memoria no-volátil a volátil puede realizarse por una simple rutina de copia (ejecutada en el arranque del sistema) o mediante una rutina de *boot-loading* que puede comprender algún tipo de procesamiento de los datos almacenados en la memoria no-volátil antes de ser transferidos a la memoria volátil (por ejemplo, descompresión cuando se desea reducir el tamaño de los datos almacenados en la memoria no-volátil)

APLICACIÓN *BOOT LOADING*

La mayoría de los dispositivos FPGA tienen asociada, en la misma placa del circuito impreso, una memoria no-volátil (por lo general, del tipo *flash*) en la cual se almacena la configuración de hardware para programar el dispositivo FPGA en el momento en que se energiza el circuito. Es frecuente que la capacidad de dicha memoria no-volátil sea mayor que la requerida para almacenar la configuración del hardware del dispositivo y puede ser utilizada entonces para almacenar también el software del SOPC. Sin embargo, debido a que la capacidad libre que deja la configuración de hardware es acotada, es conveniente almacenar una versión comprimida del software a la que se le denomina **imagen de software**. Esta alternativa requiere que el primer software ejecutado por el procesador, denominado *boot loading*,

acceda a la imagen de software almacenada en la memoria *flash*, lo descomprima y lo almacene en la memoria volátil del SOPC para su ejecución por el procesador. Todo este proceso requiere una adecuada implementación para su funcionamiento. Altera ofrece una configuración por defecto en el proyecto BSP para el proceso de *boot loading* (22).

Configuración por defecto de boot loading en la HAL de Altera

Altera incluye un archivo de comandos (*script*), denominado **nios2-bsp**, que determina si el SOPC requiere la implementación de una aplicación de *boot loading* ((21)). Por defecto, **nios2-bsp** aplica las siguientes reglas para determinar si el proyecto requiere la aplicación de *boot loading* y sus características:

- **Proceso de *boot loading*:** **nios2-bsp** asume que debe implementarse esta aplicación si alguna de las siguientes condiciones se cumplen:
 - La dirección de inicio (*reset*) del procesador Nios II no se encuentra en la sección `.text` (Solapa **Linker** del *BSP Editor*, página 143), o
 - La dirección de inicio (*reset*) del procesador Nios II está definida en la memoria *flash* del sistema (Solapa **Linker** del *BSP Editor*, página 143).
- **Proceso de copia de las secciones por defecto:** **nios2-bsp** habilita la transferencia de todas las demás secciones diferentes a la de código, a memoria volátil si la dirección de inicio (*reset*) está en la sección `.text` (Solapa **Linker** del *BSP Editor*, página 143).

La configuración por defecto generalmente es adecuada para la mayoría de las implementaciones de SOPC. En caso de requerir una configuración diferente, puede modificarse las opciones de la aplicación *boot loading* mediante el *BSP Editor*.

Configuración de opciones de la aplicación boot loading

Las diferentes opciones de configuración de la aplicación *boot loading* se pueden acceder mediante la solapa **Main**, categoría **Advanced**, subcategoría “hal”, grupo *linker* del *BSP Editor* (página 141). Las propiedades disponibles para sobrescribir la configuración por defecto son:

- **hal.linker.allow_code_at_reset:** si la opción está seleccionada, entonces la HAL inserta la macro `ALT_ALLOW_CODE_AT_RESET` en la dirección de inicio del procesador (*reset*). Esta opción debe ser deshabilitada solo si se utiliza una aplicación *boot loading* externa (por ejemplo, el software **Das U-Boot** detallado en (23)).
- **hal.linker.enable_alt_load:** si esta opción está seleccionada, entonces permite ejecutar la función `alt_load()` para copiar de la memoria *flash* a la memoria volátil diferentes secciones. Esta opción es generalmente deshabilitada cuando se utiliza una aplicación *boot loading* externa.
- **hal.linker.enable_alt_load_copy_rwddata:** si esta opción está seleccionada, se copia la sección `.rwddata` de la memoria *flash* a la memoria volátil.
- **hal.linker.enable_alt_load_copy_exceptions:** si esta opción está seleccionada, se copia la sección `.exceptions` de la memoria *flash* a la memoria volátil.

- **hal.linker.enable_alt_load_copy_rodata**: si esta opción está seleccionada, se copia la sección `.rodata` de la memoria *flash* a la memoria volátil.

Como puede observarse, existen diferentes alternativas para seleccionar las secciones de memoria que se desean copiar desde la memoria *flash* a la memoria volátil. Estas elecciones dependerán de la naturaleza de la información contenida en cada una de las secciones y de la necesidad de copiarlas en memoria volátil.

INCLUSIÓN DE MEMORIA *FLASH* EN EL SOPC

En el Capítulo 3 (página 54), se presenta la utilidad para programar el dispositivo FPGA o la memoria *flash* del sistema con la configuración de hardware. En dicho capítulo, la utilidad de la memoria *flash* del sistema finalizaba cuando la configuración del hardware se transfería al dispositivo FPGA. Sin embargo, cuando se almacena en dicha memoria el software del SOPC, es necesario que el procesador implementado en el dispositivo FPGA pueda acceder a su contenido. Es por esta razón que es necesario insertar un componente de memoria *flash* equivalente al que cuenta la placa del sistema en el diseño del SOPC en Qsys. De esta manera, la aplicación *boot loading* podrá acceder a la “imagen de software” y copiarla en memoria volátil para su ejecución.

SELECCIÓN Y CONFIGURACIÓN DE MEMORIA *FLASH* EN SOPC

La inclusión de un componente de memoria *flash* de configuración de hardware para dispositivo FPGA se realiza en forma similar a la de los restantes componentes del SOPC como se detalló en la sección “Configuración de los componentes” del Capítulo 5 (página 91).

Para incluir la memoria *flash* de configuración del dispositivo FPGA en el SOPC propuesto, se debe abrir el proyecto “sopcqsys” en Qsys y mostrar toda la arquitectura del sistema en la solapa **System Contents**. Luego, en la biblioteca de componentes (Figura 5-7), expandir la categoría **Memories and Memory Controllers**, expandir la subcategoría **External Memory Interfaces**, expandir el grupo **Memory Interfaces**, expandir el subgrupo **Flash** y seleccionar el componente **EPCS/EPCQx1 Serial Flash Controller**. Al hacer clic en el botón **Add ...**, se abrirá la pantalla de configuración del controlador de la memoria *flash* del sistema (Figura 7-3). No existen mayores configuraciones que realizar, por lo que se puede incorporar el componente al sistema seleccionando el botón **Finish**.

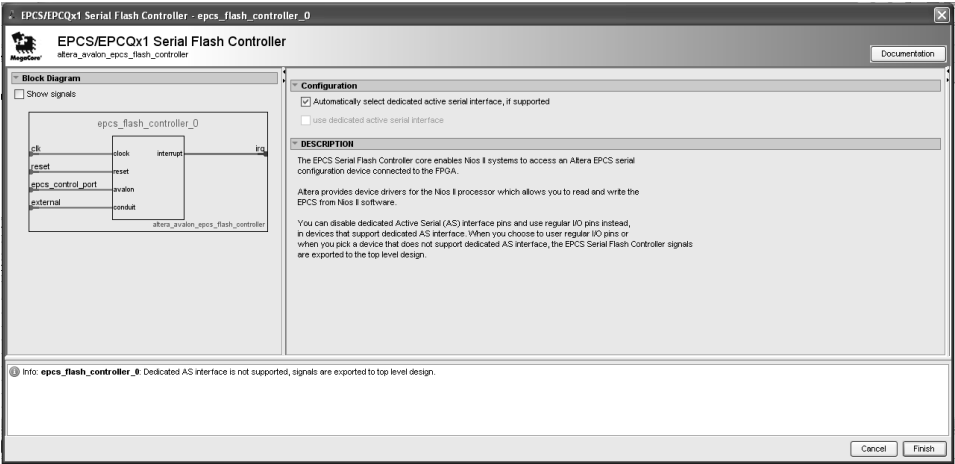


Figura 7-3: Pantalla de configuración de memoria *flash* del sistema.

Una vez incorporado el componente, se deben realizar las conexiones referentes a las señales de reloj, *reset*, interrupción y puerto Avalon. Debe considerarse si el puerto Avalon se conecta solo al puerto de datos del procesador Nios II, al de instrucciones o a ambos. En la Figura 7-4, se observa la conexión de la memoria *flash* a ambos puertos del procesador Nios II, conjuntamente con las restantes señales del sistema. La conexión al puerto de instrucciones permitirá utilizar una dirección de la memoria *flash* como vector de *reset* o de excepción.

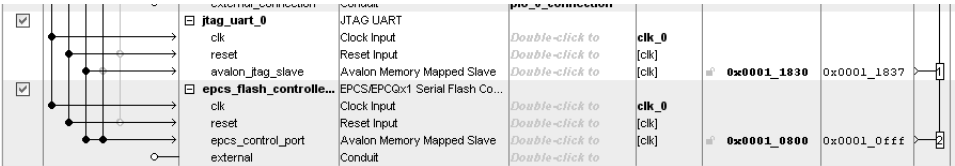


Figura 7-4: Conexión en el SOPC del controlador memoria *flash*.

Al incorporar un nuevo componente al SOPC, es conveniente una reasignación de las direcciones e interrupciones como se detalló en la sección “Configuración del SOPC” (página 106).

Puede observarse en este punto, que la especificación de la dirección de inicio (*reset*) del procesador Nios II (sección “Configuración de los vectores de *reset* y *exception* del procesador Nios II del sistema”, página 108) permite la selección entre las dos memorias incorporadas al SOPC: (1) la memoria RAM interna del dispositivo FPGA y (2) la memoria *flash*.

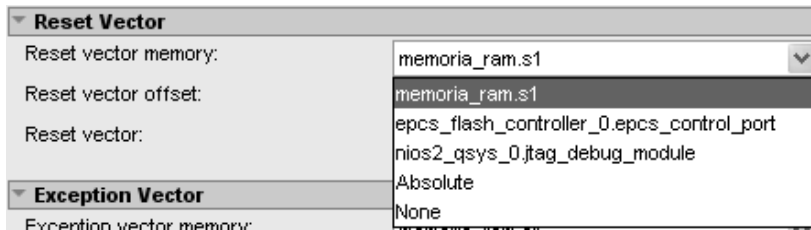


Figura 7-5: Opciones de especificación vector de reset del procesador Nios II con memoria flash incorporada al SOPC.

La selección de la memoria *flash* en el campo **Reset vector memory** indicará la incorporación de la aplicación *boot loading* de la HAL de Altera para ejecutar el software desde dicha memoria.

La modificación del campo **Reset vector memory** requerirá una nueva generación del software en Eclipse y de los archivos de memoria del sistema (Capítulo 6) y compilación de todo el proyecto de Quartus II (Capítulo 5).

Una vez generados todos los archivos del proyecto, es necesario configurar el contenido de la memoria *flash* para su programación conteniendo todos los archivos requeridos para el funcionamiento del hardware y del software del sistema.

PROGRAMACIÓN DE LA FLASH DEL SISTEMA

En primer lugar, debe programarse correctamente la memoria *flash* del sistema que puede entonces contener:

- La configuración de hardware para el dispositivo FPGA (realizada por Quartus II al compilar el diseño de hardware), incluida en el archivo **.sof** generado por Quartus II,
- Los datos y el código de programa para ser ejecutado por el procesador del SOPC que deben transferirse a la memoria datos e instrucciones respectivamente (obtenidos mediante la compilación del proyecto de aplicación y el proyecto BSP en Eclipse), incluida en el archivo **.elf** generado por Nios II SBT para Eclipse.
- El sistema de archivos que puede, opcionalmente, implementarse para almacenar información general del sistema y accederse en tiempo de corrida.

Altera provee una utilidad en el Nios II SBT para Eclipse que permite especificar todo el contenido que se desee almacenar en la memoria *flash* de configuración del hardware del sistema ((18)).

La utilidad gráfica para la programación de la memoria *flash* que ofrece Nios II SBT para Eclipse se denomina *Flash Programmer* y se accede mediante el comando **Nios II > Flash Programmer** de la barra de menús de Eclipse. Dicho comando abrirá la pantalla de la aplicación (Figura 7-6).

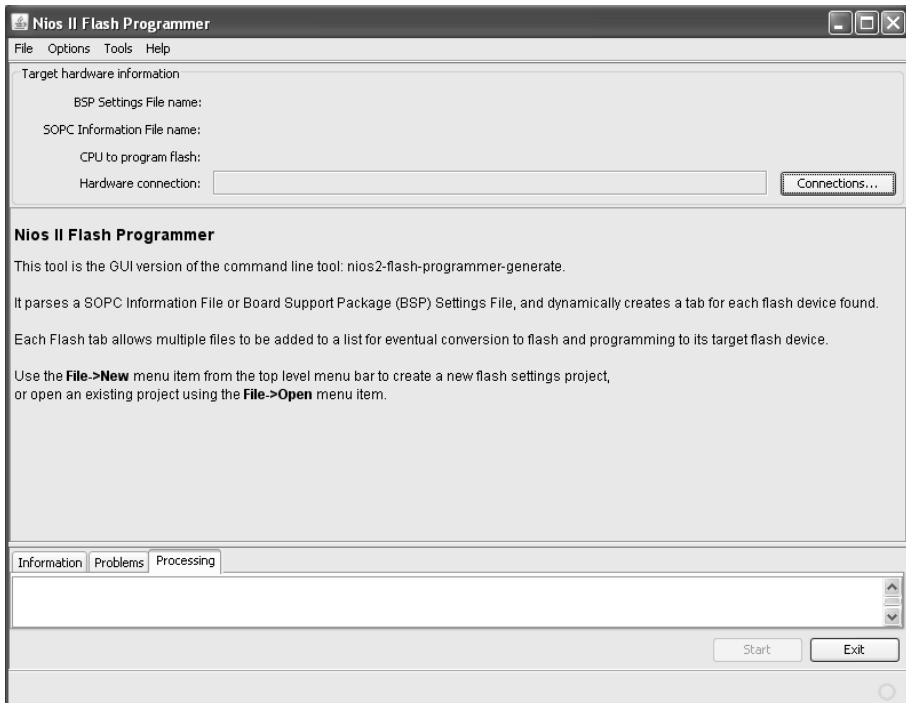


Figura 7-6: Pantalla del Flash Programmer.

CREACIÓN DEL ARCHIVO DE CONFIGURACIÓN DE *FLASH PROGRAMMER*

Toda la configuración de la programación de una memoria *flash* es almacenada por “Flash Programmer” en un archivo con extensión **.flash-settings**. Dicha información incluye todos los archivos a ser incluidos en la programación de la memoria *flash*, un archivo **.sopcinfo** describiendo la arquitectura del sistema y las direcciones de programación.

Para la creación de un nuevo archivo de ejecución se debe ejecutar el comando **File > New** de la barra de menú del *Flash Programmer*. Este comando abrirá la pantalla **New Flash Programmer Settings File** (Figura 7-7) la cual permite especificar la información del sistema desde un archivo BSP o desde el archivo **.sopcinfo** del sistema.

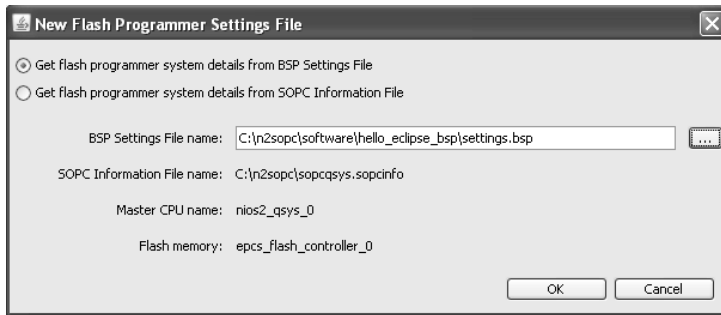


Figura 7-7: Pantalla creación de archivo de configuración de Flash Programmer.

En el ejemplo propuesto, puede seleccionarse cualquiera de los dos tipos de archivos (**.bsp** o **.sopcinfo**), de los cuales *Flash Programmer* extraerá toda la información referente a la arquitectura del sistema cuando se selecciona el botón **OK**, habilitando la pantalla de configuración de *Flash Programmer* (Figura 7-8). Es importante definir la conexión a hardware a través del botón **Connections...** para que el programador adquiriera la información de la memoria *flash* que se programará.

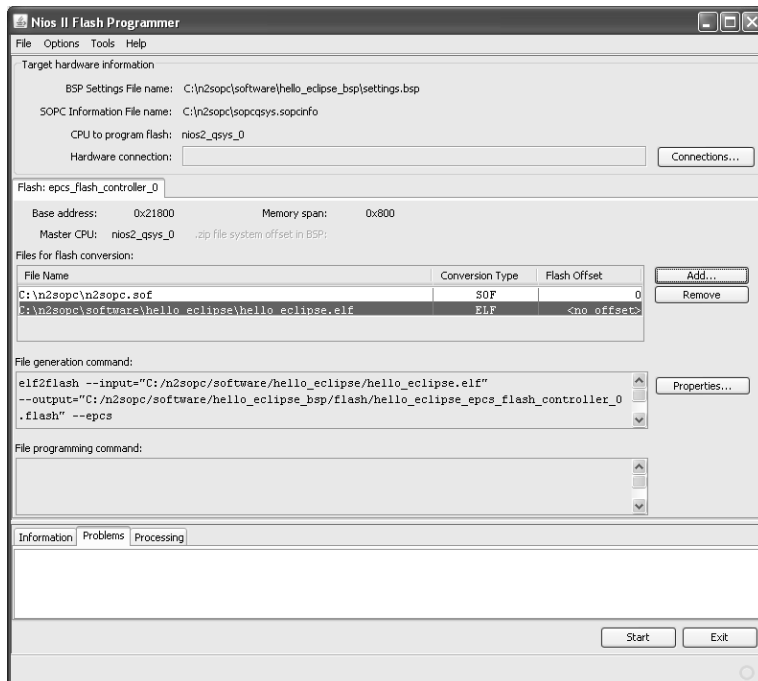


Figura 7-8: Pantalla de configuración de Flash Programmer.

ESPECIFICACIÓN DE LA CONFIGURACIÓN DE *FLASH PROGRAMMER*

La pantalla de *Flash Programmer* presenta dos áreas principales:

- **El área de comando:** es el área en la cual se especifican los archivos a almacenar en la memoria *flash* y los parámetros de configuración. Principalmente, hay que incorporar el archivo **.sof** para la configuración del hardware del dispositivo FPGA y el archivo **.elf** correspondiente a la imagen de software para ser ejecutado por el procesador Nios II.
- **El área de consola:** esta área posee tres solapas que muestran el resultado del proceso de configuración y programación:
 - **Solapa *Information*:** detalla los cambios realizados en los ajustes del programa.
 - **Solapa *Problems*:** muestra los errores y advertencias en el proceso de configuración y de generación. Es importante reconocer los errores para poder llegar a una generación correcta.
 - **Solapa *Processing*:** muestra el avance de las acciones que el programador lleva a cabo.

Cuando toda la configuración se realiza y no existe advertencias o errores en la solapa “Problems”, el proceso de generación del archivo de programación de la memoria *flash* puede comenzarse seleccionando el botón **Start**.

Opciones de *Flash Programmer*

El programador de *flash* permite seleccionar algunas opciones para controlar algunos aspectos de su comportamiento. Se puede acceder a estas opciones mediante la selección de menú **Options** de la barra de menús:

- ***Staging Directories*:** permite sobrescribir los directorios por defecto en los que se almacenarán el archivo de comando (*script*) que *Flash Programmer* utiliza para generar el archivo de configuración y el archivo a transferir a la memoria *flash* “**.flash-settings**”.
- ***Generate Files*:** si se deshabilita esta opción, entonces *Flash Programmer* permite programar la memoria *flash* con un archivo anteriormente creado.
- ***Program Files*:** si esta opción se deshabilita, *Flash Programmer* permite generar el archivo de configuración pero no programa ninguna memoria *flash*. Esta opción es adecuada cuando se desea generar el archivo de programación sin conectar ninguna memoria al computador personal.
- ***Erase Flash Before Programming*:** si esta opción se habilita, entonces la memoria *flash* se borra antes de ser programada con el archivo creado por *Flash Programmer*.

- **Run From Reset After Programming:** si esta opción está habilitada, el procesador Nios II se inicia (*reset*) luego de que la memoria *flash* es programada.

La mayoría de las placas de desarrollo (*kit*) son comercializadas con una configuración por defecto en la memoria *flash* y debe tenerse en cuenta las consideraciones que brinda el fabricante, para no sobrescribir involuntariamente dicha configuración si posteriormente se desea retornar al comportamiento original de fábrica de la placa. Muchas placas de desarrollo permiten almacenar más de una configuración en la memoria *flash*, permitiendo mantener siempre la configuración por defecto y escribir la configuración del usuario en las direcciones de memoria para tal fin. De todas maneras, es conveniente preservar el archivo de configuración original de la placa (generalmente, brindado por el fabricante en un CD/DVD-ROM) como alternativa para restituir el comportamiento de fábrica de la placa.

EJECUCIÓN DEL SOFTWARE DESDE MEMORIA EXTERNA DE ALMACENAMIENTO MASIVO

Existen aplicaciones que requieren memorias de una capacidad mayor a las que pueden ofrecer la memoria interna del dispositivo FPGA o su memoria de configuración *flash* para almacenar la imagen de software. Actualmente, existen en el mercado alternativas de memorias SD (*Secure Digital*, en inglés) (Figura 7-9) que poseen una gran capacidad de almacenamiento y una simple conexión al sistema. Diversas placas de desarrollo basadas en dispositivos FPGA (*kits*) soportan la conexión de este tipo de memorias.



Figura 7-9: Memorias SD y micro SD.

CARACTERÍSTICAS DE LAS MEMORIAS SD

La memoria SD fue diseñada para satisfacer los requerimientos de seguridad, capacidad y desempeño de sistemas digitales de audio y diferentes dispositivos electrónicos. La comunicación está basada en una interfaz de 9 pines, con las siguientes funciones:

- **Clock:** es una señal que sincroniza todas las transferencias realizadas al dispositivo.
- **Comando:** selecciona una operación a realizar sobre la memoria. El comando se transfiere serialmente sobre la línea “SD_CMD” bidireccional.

- **Datos:** consta de 4 líneas de señal bidireccional, denominada “SD_DAT”, que transfieren el dato. Las transferencias son realizadas en bloques y cada bloque transmite también un CRC (Código de Redundancia Cíclica).
- **Alimentación:** posee tres líneas dedicada a los diferentes modos de alimentación de memoria.

Actualmente, existen diferentes tipos de memoria SD con diversas características de capacidad de almacenamiento y transferencia. Sin embargo, la evolución de las memorias SD ha preservado su compatibilidad, permitiendo que dispositivos electrónicos modernos puedan acceder a la información almacenada en memorias SD de versiones anteriores.

Modo de funcionamiento de las memorias SD

El modo de funcionamiento de las memorias SD está basado en una serie de comandos que regulan la transferencia de datos desde y hacia la memoria. Estos comandos conforman un protocolo cuya implementación puede requerir autorización de la organización que posee sus derechos de comercialización (SD Card Association (24)). Sin embargo, la mayoría las memorias SD permiten un **modo SPI** que constituye un subconjunto de los comandos del protocolo original. El modo SPI es de libre implementación, pero su eficiencia de transferencia es mucho menor que la obtenida mediante la transferencia estándar de memorias SD.

Altera ofrece alternativas para implementar controladores de memoria SD en SOPC, pero requieren licencias o son restringidos para uso académico. La alternativa de modo SPI puede implementarse con componentes disponibles en el paquete Nios II EDS.

Selección y configuración de memoria SD en SOPC

La incorporación de periféricos en el SOPC para el acceso a memorias SD puede realizarse mediante los componentes brindados por el fabricante de la placa de desarrollo que posee soporte para este tipo de memorias. El funcionamiento en modo SPI puede ser implementado mediante la interfaz serie **SPI (3 Wire Serial)** provista por Nios II SBT, a la que se le deberá agregar el controlador correspondiente para poder acceder desde el proyecto de aplicación al contenido de la memoria. Existen diversos de estos controladores implementados por desarrolladores independientes que pueden ser utilizados.

CONCLUSIONES

En este capítulo, se describieron las posibles alternativas para la implementación de SOPC en dispositivos FPGA. Debe considerarse que el proceso requiere la configuración del hardware del dispositivo FPGA y la configuración del software del SOPC para un correcto funcionamiento. De acuerdo a la capacidad de almacenamiento necesaria y a la eficiencia en la ejecución de las aplicaciones, pueden determinarse diferentes estructuras de memoria para las áreas de instrucciones y datos. También, se detallaron los diferentes mecanismos y consideraciones para cada una de las alternativas existentes.

CAPÍTULO 8

VERIFICACIÓN DEL FUNCIONAMIENTO DE SISTEMAS EMBEBIDOS EN DISPOSITIVOS FPGA

INTRODUCCIÓN

La verificación del correcto funcionamiento es una etapa muy importante en el desarrollo de un SOPC. La complejidad de los sistemas embebidos implementados en dispositivos FPGA requiere la utilización de diferentes alternativas para verificar un diseño e implementación correctos. Técnicas tradicionales basadas en equipamiento e instrumental de medición pueden no resultar adecuadas cuando se implementan SOPC en dispositivos FPGA.

Las técnicas que se detallan en este capítulo abarcan:

- **Simulación:** permite verificar el comportamiento del diseño ante escenarios que en la implementación final pueden ser difíciles de producir y que mediante la simulación en un ambiente virtual pueden ser precisamente generados.
- **Depuración en placa de desarrollo:** permite verificar el funcionamiento en la implementación real con el soporte de herramientas de depuración (*debugging*, en inglés).
- **Medición y monitoreo de comportamiento:** permite comprobar diferentes señales y contenidos de memoria durante el funcionamiento del sistema. Estas herramientas son adecuadas para analizar el comportamiento del sistema ante determinados eventos que se producen durante su funcionamiento.

Todas estas técnicas pueden ser utilizadas en diferentes etapas del diseño de acuerdo a las características de verificación que se deseen realizar del sistema.

SIMULACIÓN DE SOPC

En el Capítulo 3 (sección “Simulación de diseños digitales con lenguajes de descripción de hardware: *testbench*”), se describió el proceso de simulación de sistemas digitales. Se detallaron los diferentes niveles de abstracción que pueden ser utilizados para producir simulaciones más precisas o procesos de simulación más rápidos.

Los SOPC implementados en dispositivos FPGA contienen generalmente al menos un procesador y diversos componentes periféricos, los cuales producen circuitos complejos y requieren gran cantidad de recursos digitales. Por consiguiente, una simulación a *Gate Level* podría requerir un tiempo de simulación excesivamente extenso, inadecuado para la mayoría de los procesos de diseño. Sin embargo, si se desea realizar una simulación a *Gate Level* es suficiente con ejecutar el mismo procedimiento que el llevado a cabo cuando se simula un circuito digital sintetizado desde una descripción de hardware tradicional (página 56).

Por otra parte, Nios II SBT permite la simulación del diseño de SOPC realizado mediante Qsys. El nivel de abstracción que permite en dicha simulación es RTL (nivel de transferencia de registros) con generación automática de *testbench* genéricos o mediante la especificación de un *testbench* particular realizado por el diseñador. Un detalle general de los pasos para realizar una simulación de un SOPC basado en el procesador Nios II de Altera se encuentra en la **Application Note 351** ((25)).

Los archivos utilizados para simular el SOPC no son los mismos que los utilizados para la generación del **módulo del sistema**, por lo que el comportamiento observado en la simulación puede ser restringido con respecto al que se obtendrá en la implementación final.

GENERACIÓN DE UN MODELO DE SIMULACIÓN EN QSYS

En (25), se detallan todos los pasos necesarios para la generación de un modelo de simulación para Modelsim del SOPC creado en Qsys. La especificación de la generación de los modelos de simulación del SOPC se realizan en la solapa **Generation** del Qsys (sección “Solapa del generador del sistema (Generation)”, Capítulo 5).

La generación del modelo de generación de un SOPC involucra la interconexión de los modelos de simulación de cada uno de los componentes incluidos en el SOPC. Es por esta razón, que cada componente de biblioteca debe poseer un modelo de simulación y, puede ser conveniente, que cuente con versiones para diferentes lenguajes de descripción de hardware (Verilog HDL y VHDL). El proceso de generación del modelo de simulación de Qsys interconectará los modelos de simulación en el lenguaje especificado por el diseñador en la sección *Simulation* de la solapa **Generation** (Figura 5-22).

La verificación del comportamiento de un SOPC debe incluir la producción de las señales del ambiente externo al sistema con las que este interactúa. Este ambiente es especificado por un *testbench* que puede ser generado en forma automática por Qsys para la mayoría de los SOPC que no requieren consideraciones particulares. La generación de modelos de simulación

es configurable en la solapa **Generation** de Qsys, en la sección *Simulation*, mediante las siguientes opciones:

- **Create simulation model:** esta opción permite seleccionar el lenguaje de descripción de hardware en el cual será creado el modelo de simulación del SOPC a nivel RTL. El modelo puede ser simulado en ModelSim-Altera.
- **Create testbench Qsys system:** esta opción permite la generación de *testbench* automática para la simulación del SOPC diseñado. Posee dos opciones:
 - **Standard, BFM for standard Avalon interfaces:** genera un modelo de simulación del SOPC de jerarquía menor instanciado en un *testbench* con una interfaz **Bus Functional Models (BFM)** que permite generar vectores de verificación a los diferentes componentes del SOPC. Con este tipo de interfaz, se pueden realizar pruebas del software generando diferentes estímulos al sistema. Los modelos BFM no están soportados para VHDL.
 - **Simple, BFM for clocks and resets:** produce un *testbench* solo con señales de reloj y reset al SOPC.
- **Create testbench simulation model:** permite seleccionar el lenguaje en el que se describirá el *testbench* generado.

La versión **ModelSim-Altera** soporta solo modelos generados por la interconexión de componentes descritos en el mismo lenguaje de descripción de hardware (Verilog HDL o VHDL). Si los componentes de biblioteca no poseen versiones de modelos de simulación en el mismo lenguaje, entonces la simulación no podrá ser llevada a cabo y será necesario adquirir otra versión de ModelSim para su simulación o modelar los componentes en un lenguaje común.

La modificación o visualización del *testbench* generado automáticamente por Qsys puede realizarse accediendo al archivo `<nombre_proyecto_Qsys>_tb.qsys` en la subcarpeta `<qsys_system>\testbench`. En el ejemplo propuesto, el archivo será `sopcqsys_tb.qsys` en la carpeta `"c:\n2sopc\sopcqsys\testbench\"`.

Los *testbench* generados automáticamente Qsys pueden modificarse para adaptarse a las particularidades del SOPC o simulación que se desea realizar. Para mayor detalle sobre la estructura y manejo de los BFM, se debe remitir a (26).

Luego de generar los archivos de simulación del SOPC en Qsys, se debe: (1) realizar el software en Nios II SBT para Eclipse, (2) crear los proyectos BSP y de aplicación y luego se podrá (3) simular todo el SOPC (hardware y software) en ModelSim mediante el acceso de la opción **Run As > 4. Nios II ModelSim** (Figura 8-1).

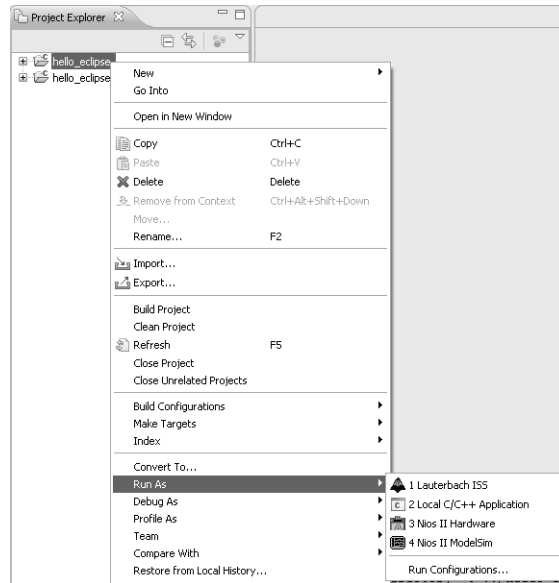


Figura 8-1: Menú para la simulación del SOPC en ModelSim.

La selección del comando **4 Nios II ModelSim** creará archivos de comandos para la inicialización de ModelSim, abrirá la aplicación ModelSim y ejecutará el archivo de comandos para la compilación automática del proyecto SOPC desarrollado en Qsys (hardware) y Nios II SBT (software). La Figura 8-2 muestra la configuración de ModelSim luego de ejecutar el archivo de comando de inicialización para simular el SOPC desarrollado.

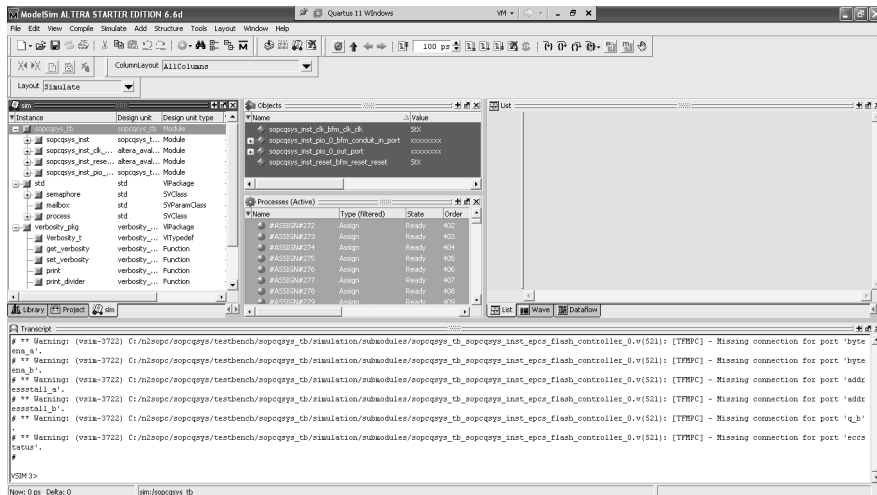


Figura 8-2: SOPC en ModelSim.

La ventana **Sim** muestra la estructura del proyecto a simular (Figura 8-3). Esta ventana detalla los módulos, clases y funciones incluidas en la simulación. Puede observarse el módulo **sopccsys_tb** que es la descripción del *testbench* a simular y que contiene las instancias de los módulos:

- **sopccsys_inst**: módulo simulable del SOPC generado en Qsys.
- **sopccsys_inst_clk_bfm**: módulo de simulación de la señal de reloj producida para el SOPC.
- **sopccsys_inst_pio_bfm**: módulo de simulación de los puertos de entrada/salida producidos por el componente “PIO (Parallel I/O)” incluido en el SOPC.
- **sopccsys_inst_reset_bfm**: módulo de simulación de la señal de *reset* producida para el SOPC.

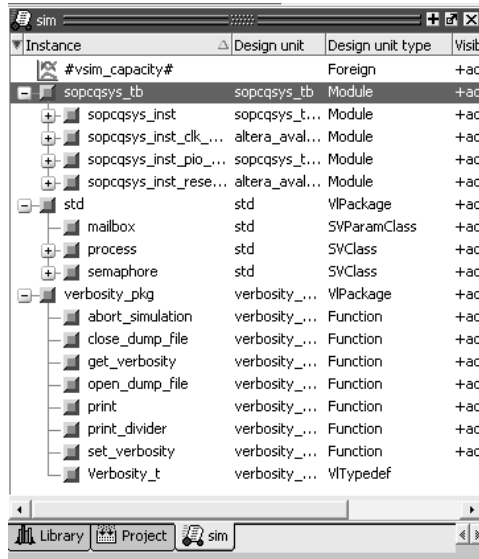


Figura 8-3: Ventana *Sim* de ModelSim.

La ventana **Sim** también muestra los paquetes (*packages*) y funciones necesarias para simular los diferentes comportamientos del software y de las funciones de la HAL.

Los módulos, paquetes y funciones incluidas en la simulación son modelos de simulación y no son los que se compilan para la implementación final, por lo que pueden existir diferencias de comportamientos entre la simulación y el sistema implementado.

La ventana **Objects** muestra los componentes que integran el módulo seleccionado en la ventana **Sim**. Cuando se selecciona el módulo **sopcqsys_tb** en la ventana **Sim**, entonces se muestran los componentes que lo integran (Figura 8-4).



Figura 8-4: Ventana Objects de ModelSim.

Seleccionando el objeto **sopcqsys_inst_pio_0_bfm_**"Conduit"**_in_port** y ejecutando el comando **Add > To Wave > Selected Signals** de la barra de menú, se abrirá la ventana **Wave** de ondas (Figura 8-5) y se podrá visualizar su comportamiento cuando se simule el sistema. Puede ocurrir que la incorporación de gran cantidad de señales en la ventana **Wave** provoque que el proceso de simulación requiera mayor tiempo de procesamiento, capacidad de disco rígido y memoria de la computadora personal.

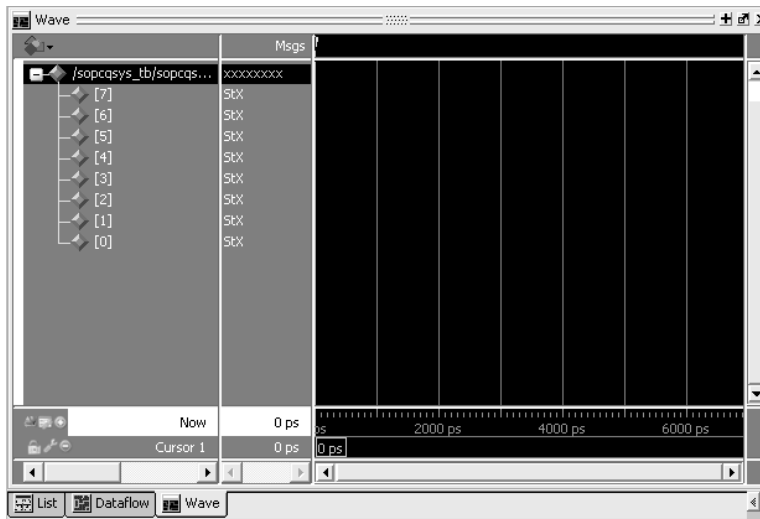


Figura 8-5: Ventana "Wave" de ModelSim

La simulación del sistema puede realizarse especificando en la línea de comando el tiempo de simulación que se desea realizar, como por ejemplo:

```
VSIM > run 2 ms
```


El comando comenzará la simulación de todo el sistema considerando los modelos de simulación de cada uno de los componentes que los integran. Estos modelos cuentan con procedimientos de inicialización que generalmente informan esta condición mediante mensajes a la consola y que pueden observarse cuando la simulación comienza. En el ejemplo propuesto, puede observarse que la consola **Transcript** es utilizada por el componente JTAG UART para mostrar los mensajes de salidas, en este caso aparecerá el mensaje **Hello from Nios II** producido por la instrucción:

```
printf("Hello from Nios II!\n");
```

En este caso, puede observarse que el modelo de simulación del componente JTAG UART permite una interacción con el ambiente de ModelSim, mientras que el modelo para síntesis difiere totalmente al no requerir esta característica. Puede ocurrir que los modelos de simulación describan características que en la realidad han dejado de ser soportadas en versiones más actuales del componente. Por ejemplo, la interacción de “Entrada / Salida” del componente JTAG UART no es soportada en las versiones más modernas del componente siendo la interacción de salida (*Output*) la única que funciona correctamente.

La simulación de componentes de biblioteca puede tener funcionalidades reducidas de acuerdo a la versión del mismo. Esto no significa que el componente carezca de dichas funcionalidades cuando es sintetizado.

La simulación de SOPC mediante ModelSim requiere un tiempo de procesamiento que justifica su utilización solo en casos especiales en los que se desean verificar el funcionamiento del hardware durante escenarios particulares.

DEPURACIÓN EN PLACA DE DESARROLLO DE SOPC

El ambiente de desarrollo para Nios II ofrece diferentes alternativas para la depuración de proyectos SOPC. Es importante recordar que la depuración de un desarrollo requiere un profundo conocimiento de las herramientas a utilizar y del sistema embebido a depurar. Es por este motivo que puede ser necesaria la utilización de varias de estas herramientas de acuerdo al tipo de depuración que se desea realizar.

NIOS II SOFTWARE BUILD TOOLS FOR ECLIPSE

La plataforma de desarrollo basada en Eclipse presenta diferentes herramientas para la depuración del software, tanto en compilación como en funcionamiento.

Componente Nios II System ID

La primera consideración que debe tenerse en cuenta es que la versión de software que se compila coincida con la arquitectura de hardware sobre la que se desea ejecutar. Es muy frecuente que durante el proceso de depuración de un sistema embebido en dispositivos FPGA se modifique el hardware y que, posteriormente, no se adecúe la generación de software. Para evitar este tipo de inconvenientes, Altera ofrece el componente **Nios II System ID** que permite identificar el hardware con una etiqueta única y que debe coincidir con la que se genera el software (específicamente, el .sopcinfo con que se forma el software).

El componente System ID permite prevenir al diseñador del intento de ejecutar el software en un hardware incorrecto. Para esto, debe incluirse en el SOPC el componente **System ID Peripheral** mediante la selección del componente en la biblioteca de componentes en la categoría **Peripherals**, grupo **Debug and Performance**.

Este componente asigna una “estampilla de tiempo” (*Timestamp*, en inglés) en el momento de generación del SOPC. Esta estampilla es leída por el programador de Quartus II y debe coincidir con la utilizada para compilar el software del sistema. Si ambas estampillas no coinciden, entonces significa que la aplicación fue compilada considerando un archivo .sopcinfo que no es el que se utilizó para configurar el hardware del dispositivo FPGA.

Generación del archivo OBJdump

En ciertas circunstancias, puede ser conveniente tener información sobre la manera en que el código de la aplicación es compilado. Un archivo .objdump contiene la información sobre las direcciones de memoria, estructura, direcciones, funciones e instrucciones de lenguaje ensamblador relacionadas con las instrucciones originales del código C.

La generación del archivo .objdump se realiza en la herramienta **Nios II SBT for Eclipse** seleccionando la opción **Properties** del menú que se despliega al hacer clic en el botón derecho del mouse sobre el proyecto de aplicación (Figura 8-6).

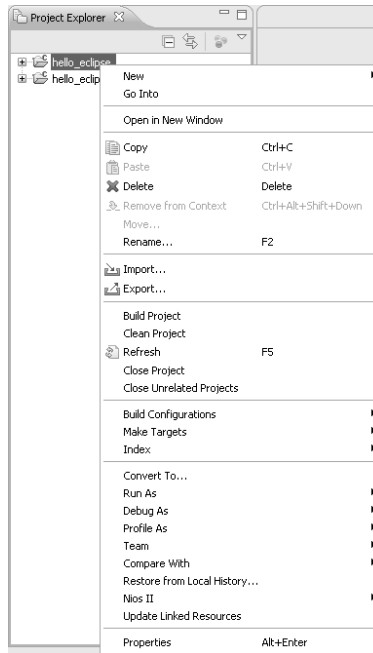


Figura 8-6: Selección de *Properties* del proyecto de aplicación.

Luego de la selección, se abre la ventana de propiedades en la cual se debe seleccionar la categoría **Nios II Application Properties**, marcar la opción **Create object dump** (como muestra la Figura 8-7) y hacer clic en el botón **OK**.

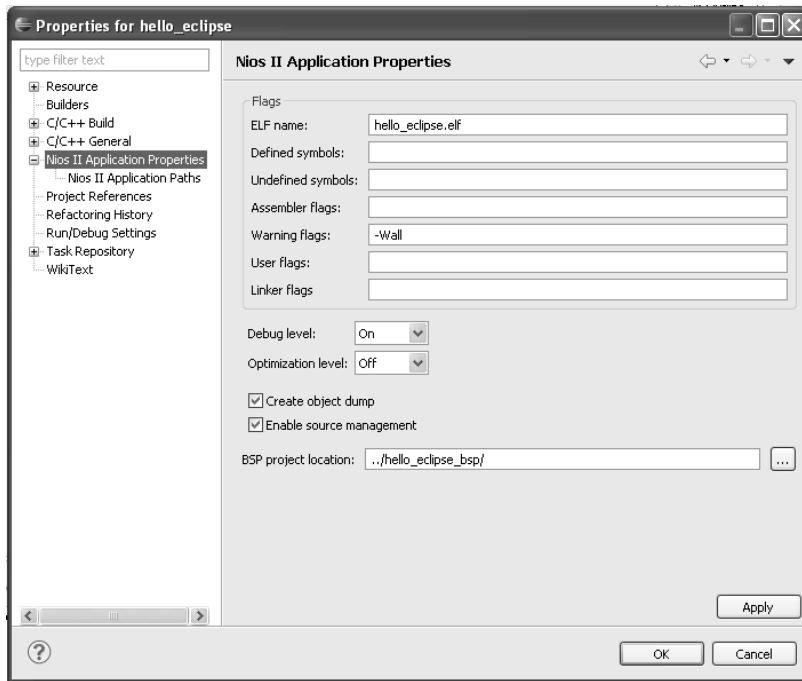


Figura 8-7: Selección de Create object dump.

Luego de la próxima compilación del código, se generará el archivo **.objdump** en el mismo directorio en el cual se genera el archivo **.elf** (en el ejemplo propuesto, “c:\n2sopc\software\hello_eclipse”). El archivo **.objdump** generado contendrá, entre mucha otra, la siguiente información:

```
#include <stdio.h>
int main()
{
    101f0:    deffff04    addi    sp,sp,-8
    101f4:    dfc00115    stw     ra,4(sp)
    101f8:    df000015    stw     fp,0(sp)
    101fc:    d839883a    mov     fp,sp
    printf("Hello from Nios II!\n");
    10200:    01000074    movhi   r4,1
    10204:    2107a704    addi    r4,r4,7836
    10208:    00102b40    call    102b4 <puts>
    return 0;
    1020c:    0005883a    mov     r2,zero
}
```

Consola de Nios II y funciones de la biblioteca stdio

Nios II SBT para Eclipse provee una consola para visualizar mensajes producidos por la función `printf` o la función `alt_log_printf`. Ambas funciones pueden ser utilizadas en diferentes partes del código para indicar que la ejecución ha alcanzado dichos puntos de ejecución y enviar a la consola de Nios II SBT para Eclipse el estado de diferentes variables de la aplicación. Es importante remarcar que la inserción de estas funciones modifica el código original y por consiguiente su comportamiento puede ser afectado.

Desbordamiento de la pila

El desbordamiento de la pila es un problema frecuente en sistemas embebidos en los cuales los recursos de memoria y, por consiguiente, los asignados a la pila del sistema, son reducidos. Nios II SBT para Eclipse permite habilitar que la HAL incorpore una función de excepción para el manejo de desbordamiento de pila. Dicha excepción permitirá analizar las causas del desbordamiento de la pila y, según corresponda, modificar el código de la aplicación o asignar más memoria para la implementación de la pila del sistema. La habilitación de esta característica se realiza en el **BSP Editor** en la solapa **Main**, categoría **Settings**, subcategoría **Advance**, grupo **hal**, seleccionando la opción **enable_runtime_stack_checking**.

La selección de esta opción puede incrementar el tiempo de ejecución de la aplicación y requiere memoria adicional del sistema para su implementación.

Puntos de quiebre (Breakpoints) y ejecución paso-a-paso

Nios II SBT para Eclipse permite la ejecución paso-a-paso del código y la inserción de puntos de quiebre (*breakpoints*, en inglés) en el código. Esto permite un seguimiento de la evolución de la aplicación durante el tiempo de corrida. Nios II posibilita la inclusión de puntos de quiebre por hardware mediante la unidad funcional de depuración JTAG que contiene (Figura 4-1). Para la configuración de dicha unidad funcional, debe accederse en Qsys a la ventana de configuración del procesador Nios en la solapa **JTAG Debug Module** y seleccionar un nivel mayor o igual a 2 para habilitar el soporte de puntos de quiebre por hardware. El nivel 1 solo posee soporte de puntos de quiebre por software en los cuales el código del punto de quiebre es insertado en el código de la aplicación que se ejecuta.

Es importante no optimizar la compilación del código cuando se desean utilizar puntos de quiebre o ejecución paso-a-paso para que el código ejecutable coincida con el código original.

La imposibilidad de optimizar la compilación cuando se desea utilizar puntos de quiebre o ejecución paso-a-paso, puede provocar que la aplicación depurada difiera de la finalmente implementada si se compila con optimización para mejorar su eficiencia.

IN-SYSTEM MEMORY CONTENT EDITOR

Muchas veces es conveniente acceder al contenido de la memoria del sistema para analizar el comportamiento de la aplicación. De esta forma, en lugar de controlar la manera en que se ejecuta el código, se puede analizar los resultados que este provoca. Altera ofrece la herramienta **In-System Memory Content Editor** que permite visualizar y modificar el contenido de la memoria del sistema a través de la interfaz JTAG del dispositivo FPGA.

Mientras que la posibilidad de leer el contenido de la memoria permite comprobar el correcto funcionamiento del código, la escritura de dicho contenido permite generar diferentes condiciones de ejecución de la aplicación para su evaluación. Por ejemplo, se puede modificar el contenido de una determinada dirección de memoria para comprobar el correcto funcionamiento de una función de detección de errores de paridad. En (9), se detalla la utilización de la herramienta **In-System Memory Content Editor** para la depuración de aplicaciones.

SIGNALTAP II LOGIC ANALYZER

El desarrollo de hardware digital requiere de instrumental adecuado para visualizar la correcta ocurrencia de las señales del sistema. Un instrumental muy utilizado para este propósito es el “analizador lógico” el cual permite visualizar diferentes señales del sistema y determinar si la relación entre ellas es la establecida por la especificación. Los analizadores lógicos brindan diferentes herramientas para facilitar la captura de eventos de acuerdo a las condiciones de funcionamiento que se desean verificar en tiempo real. Sin embargo, los analizadores lógicos tradicionales requieren el acceso a los nodos físicos de las señales para poder conectar sus puntas de medición. Esta posibilidad no está disponible en los dispositivos FPGA debido a que todas las señales se encuentran dentro del circuito integrado que imposibilitan su acceso.

Los fabricantes de dispositivos FPGA brindan herramientas de verificación que permiten adicionar al hardware realizado por el diseñador, componentes para la captura de señales para su posterior análisis. Estos componentes de captura de señales son implementados mediante recursos internos de los dispositivos FPGA, conjuntamente con el hardware que se desea verificar. De esta manera, se tiene acceso a las señales internas del diseño y son estos componentes los encargados de enviar al exterior del dispositivo la información recolectada para su análisis.

La herramienta de análisis lógico que ofrece Altera se denomina *SignalTap II Logic Analyzer*. Esta herramienta permite la verificación del diseño en tiempo real a la velocidad de funcionamiento del sistema mientras se realiza el análisis de las señales en Quartus II. La utilización de *SignalTap II Logic Analyzer* requiere su configuración antes de su implementación en el dispositivo FPGA. Dicha configuración se realiza mediante el software *SignalTap II Logic Analyzer Editor* con el cual se crea un archivo **SignalTap II File (.stp)** el cual contiene toda la información de configuración del componente *SignalTap II Logic Analyzer*. La configuración de un componente de este tipo permite la definición de condiciones de disparo (*trigger*) para los eventos que se desean analizar.

La Figura 8-8 muestra la estructura de verificación cuando se utiliza *SignalTap II Logic Analyzer*. Esta estructura inserta el componente para ser implementado conjuntamente a la lógica del diseño a verificar. El componente *SignalTap* detecta los diferentes eventos y registra en memoria interna del dispositivo FPGA las diferentes señales que fueron configuradas. Posteriormente, dichas señales son transferidas a la computadora personal mediante la interfaz JTAG para ser visualizadas y analizadas mediante el software *SignalTap* *Logic Analyzer* en Quartus II.

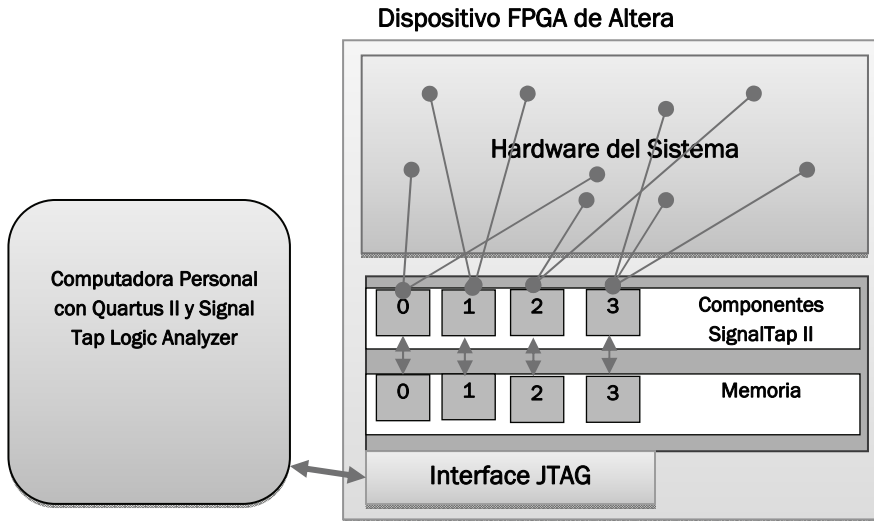


Figura 8-8: Estructura de verificación mediante *SignalTap II Logic Analyzer*.

SignalTap II Logic Analyzer es una poderosa herramienta que permite un análisis interno del funcionamiento de la lógica del SOPC con características solo encontradas en instrumental de muy alta gama.

CONCLUSIONES

La depuración de SOPC requiere de diferentes herramientas y mecanismos acordes a la complejidad de diseño del hardware y del software. En este capítulo, se abarcaron diferentes alternativas que pueden ser utilizadas para la depuración de sistemas embebidos en dispositivos de lógica programable. La selección de los mecanismos y herramientas dependerá de la arquitectura del sistema embebido y de la naturaleza de las fallas que se desean depurar, lo que requiere un profundo conocimiento del hardware y el software del sistema.

CAPÍTULO 9

EL LENGUAJE DE COMANDOS TCL

INTRODUCCIÓN

Actualmente, el diseño digital moderno requiere de diversas configuraciones para lograr implementaciones eficientes y adecuadas. Por ejemplo, los componentes de biblioteca deben ser adaptados a las características propias del sistema embebido en el que se desea utilizar. Por lo general, estas adaptaciones involucran la especificación de diferentes parámetros cuyos nombres y códigos requieren un conocimiento muy profundo de los componentes y su implementación. Los archivos de comandos Tcl permiten especificar secuencias de configuración de componentes para facilitar diferentes procesos de configuración o diseño. En este capítulo, se describen los principales conceptos del lenguaje Tcl.

EL LENGUAJE DE COMANDOS TCL

El lenguaje de comandos Tcl fue desarrollado por John Ousterhout de la Universidad de California en Berkeley en la década del ochenta. El deseo original en su propuesta era controlar diferentes herramientas de software, de diferentes fabricantes, utilizadas en el proceso de diseño de circuitos integrados. Es de esta manera que el nombre del lenguaje Tcl tiene su origen en *Tool Command Language* que en español puede traducirse como “Lenguaje de comando de herramienta”.

Tcl es un lenguaje interpretado en lugar de compilado. Esto permite que los comandos puedan ser introducidos por el diseñador en un interpretador para su ejecución y visualizar los resultados mientras los comandos son ejecutados. Estos son ejecutados inmediatamente puesto que no requieren compilación. El lenguaje Tcl ha sido ampliamente adoptado por las principales compañías dedicadas al diseño electrónico y en consecuencia existen diversas aplicaciones libres y comerciales que lo soportan. **TK toolkit** es una extensión muy utilizada que permite el desarrollo de interfaces gráficas de usuario (GUI) independientes de la plataforma.

SINTAXIS BÁSICA DEL LENGUAJE TCL

El lenguaje Tcl es un lenguaje basado en cadenas de caracteres (*string*, en inglés). Esto significa que cualquier expresión automáticamente tratará a los caracteres numéricos como enteros o números en punto flotante según corresponda.

El lenguaje posee un número reducido de estructuras y reglas y la sintaxis básica es un comando seguido por los argumentos separados por espacios y finaliza con un fin de línea o con un punto y coma (“;”).

```
comando arg1 arg2 arg3 .. ;
```

INTÉRPRETE TCL

Por lo general, el intérprete Tcl viene incluido con los sistemas Unix, Linux y Mac OS X. La versión para Microsoft Windows puede obtenerse en su sitio oficial “www.tcl.tk”. Quartus II generalmente actualiza a las versiones más recientes de Tcl/Tk que incluye la aplicación `tclsh`. La mayoría de las aplicaciones de Quartus II (`quartus_sh`, `quartus_fit`, etc.) permiten evaluar un archivo de comandos Tcl si se utiliza la opción `-t` cuando es invocado.

Para comprender más fácilmente el lenguaje Tcl, utilizaremos la consola provista por Altera con Quartus II que puede invocarse en Microsoft Windows mediante el comando aplicado (en **Inicio > Buscar Aplicaciones y archivos** en Windows 7 o **Inicio > Ejecutar** en Windows XP):

```
quartus_sh -s
```

Este comando abrirá la consola Tcl en una ventana de Windows (Figura 9-1).

```

C:\altera\Sub11.0\quartus\bin\quartus_sh.exe
Info: *****
Info: Running Quartus II Shell
Info: Version 11.0 Build 157 04/27/2011 SJ Full Version
Info: Copyright (C) 1991-2011 Altera Corporation. All rights reserved.
Info: Your use of Altera Corporation's design tools, logic functions
Info: and other software and tools, and its AMPP partner logic
Info: functions, and any output files from any of the foregoing
Info: (including device programming or simulation files), and any
Info: associated documentation or information are expressly subject
Info: to the terms and conditions of the Altera Program License
Info: Subscription Agreement, Altera MegaCore Function License
Info: Agreement, or other applicable license agreement, including,
Info: without limitation, that your use is for the sole purpose of
Info: programming logic devices manufactured by Altera and sold by
Info: Altera or its authorized distributors. Please refer to the
Info: applicable agreement for further details.
Info: Processing started: Wed Sep 19 10:50:07 2012
Info: *****
Info: The Quartus II Shell supports all TCL commands in addition
Info: to Quartus II Tcl commands. All unrecognized commands are
Info: assumed to be external and are run using Tcl's "exec"
Info: command.
Info: - Type "exit" to exit.
Info: - Type "help" to view a list of Quartus II Tcl packages.
Info: - Type "help <package name>" to view a list of Tcl commands
Info: available for the specified Quartus II Tcl package.
Info: - Type "help -tcl" to get an overview on Quartus II Tcl usages.
Info: *****
tcl>

```

Figura 9-1: Pantalla de la consola de comandos Tcl provista por Quartus II.

Como ejemplo, se puede introducir el comando `puts` para implementar el ejemplo tradicional de “Hello World”.

```
tcl> puts "Hello World"
```

El comando `puts` escribe la cadena de caracteres ingresada como argumento en la salida estándar (en este caso, la consola de comandos). El argumento se declara entre comillas para que sea considerado como un solo argumento y que debe ser considerado como la cadena de caracteres a escribir en la consola.

CREACIÓN DE ARCHIVOS DE COMANDOS TCL

Los archivos de comando Tcl son archivos de texto, comúnmente con extensión `.tcl`, que contienen comandos Tcl y que no requieren encabezamientos o símbolos especiales. La creación de un archivo Tcl se restringe entonces a la creación de un archivo de texto que contenga todos los comandos Tcl que se desea que el intérprete ejecute cuando el archivo es ejecutado. De esta manera, la ejecución de un archivo de comandos Tcl producirá el mismo resultado que la ejecución individual de los mismos en la consola de comandos.

EJECUCIÓN DE ARCHIVOS DE COMANDOS TCL

Existen dos alternativas para la ejecución de archivos de comandos Tcl:

- Especificando el archivo de comandos, como argumento con la cláusula **-t o -script**, cuando se invoca la consola de comandos. Por ejemplo:

```
c:>quartus_sh -t <nombre_del_archivo>.tcl
```

- Ejecutando el comando **source** en la consola de comandos, especificando el nombre del archivo como argumento. Por ejemplo:

```
tcl> source <nombre_del_archivo>.tcl
```

CREACIÓN Y SUSTITUCIÓN DE VARIABLES EN TCL

El lenguaje Tcl está basado principalmente en cadenas de texto (*strings*). De esta manera, cuando una determinada instrucción requiere un argumento numérico, entonces considera la cadena de texto respectiva como un número.

Creación de variables

Para crear una variable y asignarle un valor se utiliza el comando **set**. Por ejemplo, el comando

```
tcl> set pi 3.14
```

crea una variable denominada **pi** y le asigna el valor 3,14.

Las variables pueden no declararse antes de su uso y su nombre puede contener cualquier longitud de caracteres. El nombre de las variables es sensible a la utilización de minúsculas y mayúsculas (*case sensitive*).

Sustitución \$

Cada vez que se desea que el intérprete sustituya el nombre de una variable por su valor, se debe preceder el nombre de la variable con el símbolo **\$**. Por ejemplo,

```
tcl> puts $pi
```

producirá la salida **3,14**, mientras que la ejecución del comando

```
tcl> puts pi
```

producirá la salida **pi**.

Operaciones aritméticas con variables: comando `expr`

Para la realización de operaciones aritméticas con variables, debe utilizarse el comando `expr`. De esta forma, la cadena de caracteres que figura como el argumento del comando `expr` es evaluada como una expresión aritmética. Por ejemplo,

```
tcl> expr 2*$pi
```

retornará el valor 6,28 dado que la variable `pi` había sido asignada con el valor 3,14 anteriormente.

Sustitución anidada []

Un tipo de sustitución soportada por Tcl es la sustitución de comandos anidados mediante la utilización de corchetes ("`[ ]`"). Este uso permite asignar a una variable un comando que luego podrá ser valorado al realizar una evaluación de la variable. No existe un límite para el anidamiento de comandos, siendo los comandos de mayor anidamiento los primeros en ser evaluados y los de menor anidamiento los siguientes. Por ejemplo, el comando

```
tcl> set aux [expr 2*$pi]
```

creará una variable con el nombre **aux** a la que le asignará el valor 6,28 resultante de la evaluación aritmética del comando `expr 2*$pi`.

Sustitución backslash

Un tercer tipo de sustitución en Tcl es la *backslash* que permite sustituir por los caracteres especiales como, por ejemplo, `$` y `[`. El comando

```
tcl> set b \$
```

asignará a la variable `b` el valor `'$'`.

El carácter `\` también es utilizado para continuar con el texto en la línea siguiente (*newline*).

Evaluación de cadenas de caracteres: comando `eval`

Como se puede observar, los valores asignados a variables pueden ser "cadenas de caracteres" si están expresados entre comillas y [comandos] si están expresados entre corchetes. Para evaluar una cadena de caracteres como un comando, debe utilizarse el comando **eval**. Por ejemplo,

```
tcl> set aux "expr 2*$pi"
```

asignará a la variable **aux** la cadena de caracteres `expr 2*$pi`. Si se ejecuta

```
tcl> eval $aux
```

entonces se obtendrá el resultado 6,28 que resulta de duplicar el valor que se le había asignado a la variable pi (3,14).

LISTAS

Las listas son agrupaciones de cualquier tipo de elemento de Tcl, incluyendo otras listas. Los elementos que conforman la lista deben escribirse entre llaves (“{ }”) separados por espacios. Por ejemplo,

```
tcl> set beatles {paul ringo george john}
```

asigna a la variable **beatles** la lista de cuatro ítems: paul, ringo, george y john.

Existen varias operaciones que pueden realizarse sobre las listas.

llength

La operación **llength** retorna la cantidad de ítems que contiene la lista. Por ejemplo,

```
tcl> llength $beatles
```

retornará el valor 4 y

```
tcl> set integrantes_beatles [llength $beatles]
```

creará la variable **integrantes_beatles** y le asignará el valor 4.

lindex

La operación **lindex** retorna el ítem que ocupa una posición determinada en la cadena. El índice inicial de la lista es 0. Por ejemplo,

```
tcl> lindex $beatles 2
```

retornará el valor **george**.

lsearch

La operación **lsearch** devuelve la posición del primer ítem que cumple con el patrón de búsqueda o **-1** en caso de que no exista ítem que cumpla con el patrón de búsqueda. Por ejemplo,

```
tcl> lsearch $beatles j*
```

devolverá 3 dado que el ítem **john** cumple con el patrón de búsqueda **j***.

Los caracteres globales son:

- ***** para indicar cualquier carácter y número de caracteres,
- **?** para indicar un carácter
- **[caracteres]** para indicar que puede ser cualquiera de los caracteres contenidos entre los corchetes.

lappend

La operación **lappend** añade un ítem al final de la lista. Por ejemplo,

```
tcl> lappend beatles pete  
tcl> puts $beatles
```

producirá la salida **paul ringo george john pete**.

foreach

El comando **foreach** permite iterar sobre cada ítem de la lista. Por ejemplo,

```
tcl> foreach i $beatles {puts $i}
```

retornará:

```
paul  
ringo  
george  
john  
pete
```

ESTRUCTURAS DE CONTROL EN TCL

Los comandos para el control de flujo son similares a los del lenguaje C. Dichos comandos pueden agruparse en las siguientes categorías:

- **Comandos condicionales:** agrupa los comandos **if/then/else** y **switch**.
- **Lazos (loops, en inglés):** agrupa los comandos **for**, **foreach** y **while**.
- **Control de lazos:** agrupa los comandos **break** y **continue**.

A continuación, se detallarán las principales estructuras de estos comandos.

Comando if/then/else/elseif

El comando **if/then/else** es el comando condicional básico del lenguaje Tcl. Su sintaxis es de la forma:

```
if {condición} {comandos si condición verdadera }
```

o

```
if {condición} {comandos condición verdadera} else {  
comandos condición falsa  
}
```

o

```
if {condición} {comandos condición verdadera} elseif {condición 2  
}{  
comandos condición falsa condición 2 verdadera  
} else {  
comandos condición falsa condición 2 falsa  
}
```

Nota: la apertura de la llave que contiene los comandos debe estar en la misma línea que el condicional o la palabra **else** o **elseif**. Caso contrario debe utilizarse la barra \ para indicar una nueva línea.

Ejemplo:

```
if {$aux > 0} {  
puts "aux es positivo"  
} elseif {$aux == 0} {  
puts "aux es cero"  
} else {  
puts "aux es negativo"  
}
```

Comando switch

EL comando **switch** evalúa una de varias alternativas dependiendo del valor definido. Su sintaxis es de la siguiente forma:

```
switch opciones string {patrón {  
  cuerpo} patrón {  
  cuerpo} patrón {  
  cuerpo}}
```

Las opciones disponibles abarcan:

- **exact**: es la opción por defecto y especifica que la concordancia debe ser exacta.
- **glob**: permite la utilización de consignas para determinar los rangos de comparación.
- **regexp**: permite la especificación de caracteres comodines en la comparación.
- **nocase**: especifica que la comparación no tenga en cuenta si los caracteres están en mayúsculas o minúsculas.
- **matchvar** *varName*: esta opción permite la comparación con el nombre de una variable.
- **indexvar** *varName*: permite la especificación de una variable en la cual una lista de índices referirá a la comparación de *substrings*.

El campo de las opciones debe finalizar con los caracteres “—”, luego de los cuales el intérprete tratará como *string* al próximo argumento del comando.

Ejemplo:

```
switch -exact -- $valor {  
  1 { puts "el valor es 1"}  
  2 { puts "el valor es 2"}  
  default { puts "el valor no es ni 1 ni 2"}  
}
```

Comando for

El comando **for** permite definir una estructura de lazo. Su sintaxis es de la siguiente forma:

```
for {valor inicial} {condición} {comando post-ejecución del lazo}  
{  
  Cuerpo del lazo  
}
```

Ejemplo:

Este ejemplo mostrará los números de 0 a 9.


```
for {set x 0} {$x<10} {incr x} {
    puts "x is $x"
}
```

Comando foreach

El comando **foreach** permite iterar el lazo asignando a una variable cada uno de los valores de una lista. La sintaxis es de la forma:

```
foreach variable lista {
    Cuerpo del lazo
}
```

Ejemplo:

El siguiente ejemplo muestra los nombres en una lista que tienen la letra j:

```
set nombres { "María José" "Juan Carlos" Esteban Hernán Erika
Germana Joaquín}
foreach nombre $nombres {
    if { [string match {[jJ]*} $nombre] } {
        puts "$nombre posee la letra j."
    }
}
```

Comando while

El comando **while** permite ejecutar el lazo mientras la condición del comando es verdadera. La sintaxis del comando tiene la siguiente forma:

```
while { condición } {
    Cuerpo del lazo
}
```

Ejemplo:

```
set valor 0
while { $valor <= 10 } {
    puts $valor
    incr valor
}
```

Este lazo imprimirá los valores de 0 a 10 hasta que la condición deje de ser verdadera.

Comando break y continue

El comando **break** finaliza la ejecución del lazo inmediatamente y el intérprete continúa con el siguiente comando fuera del lazo. El comando **continue** fuerza a la ejecución de la iteración siguiente del lazo si la condición se cumple sin finalizar la iteración actual.

PROCEDIMIENTOS

Los procedimientos encapsulan una secuencia de comandos Tcl. Los procedimientos permiten el pasaje de parámetros o la definición de valores por defecto. La estructura del procedimiento es de la siguiente forma:

```
proc nombre { parámetros } {  
    Cuerpo del Procedimiento  
}
```

Ejemplo:

```
proc sumador { a { b 1 } } {  
    set suma [ expr { $a + $b } ]  
    return $suma  
}
```

Este procedimiento realiza la suma entre dos parámetros **a** y **b**, teniendo una asignación por defecto de 1 al parámetro **b**.

El comando

```
puts [sumador 7]
```

imprimirá el valor 8, mientras que el comando

```
puts [sumador 7 6]
```

imprimirá el valor 13.

Alcance de las variables en procedimientos (Scope)

Los procedimientos permiten una definición “global” y una definición “local” en el alcance de las variables. Las variables definidas en el procedimiento poseen un alcance local y pueden accederse a ellas solamente dentro del procedimiento.

Para un alcance global de las variables, el lenguaje Tcl posibilita dos opciones:

- comando **global** *variable*: permite acceder dentro de un procedimiento a una variable global.
- comando **upvar** *oldvar newvar*: permite que la variable global *oldvar* sea accesible dentro del procedimiento mediante una variable de nombre *newvar*.

COMANDOS DE ENTRADA/SALIDA

El lenguaje Tcl incluye una variedad de comandos para el acceso a archivos de entrada y salida. Los comandos de entrada y salida a archivos es un subconjunto de los comandos generales de entrada y salida los cuales incluyen conexiones de red (*network sockets*) y canales de entrada y salida en general.

Generalmente, los comandos de acceso a archivos son los más requeridos en la mayoría de las aplicaciones. El acceso a archivos puede dividirse en tres pasos básicos:

- **open**: permite abrir el archivo asignándole un identificador para su posterior utilización en los accesos de lectura o escritura del archivo.
- **gets** o **puts**: permite acceder al archivo para leer la información que contiene o para escribir información en él. El archivo debe haber sido abierto previamente con el comando **open**.
- **close**: permite cerrar el acceso al archivo una vez que las operaciones de lectura y escritura hayan sido realizadas.

Comando open

El comando **open** abre un archivo del disco y devuelve el identificador (*handle*) que se debe utilizar en futuras transferencias a dicho archivo. La estructura del comando es de la siguiente forma:

```
open nombre_archivo <tipo_de_acceso> <permisos>
```

El comando devuelve el **identificador** del canal para el acceso al archivo.

Se permiten los siguiente *tipo_de_acceso*:

- **r**: acceso de solo lectura (el archivo debe existir).
- **r+**: acceso de lectura y escritura (el archivo debe existir).

- **w**: acceso de solo escritura (si el archivo existe lo sobrescribe).
- **w+**: acceso de lectura y escritura (si el archivo existe lo sobrescribe).
- **a**: acceso de solo escritura, añadiendo al final del archivo los nuevos datos (el archivo debe existir).
- **a+**: acceso de lectura y escritura (el archivo debe existir y los nuevos datos son añadidos al final del mismo).

Los permisos son definidos según los permisos de UNIX.

Ejemplo:

El siguiente comando abre el archivo **resultados.txt** para escritura y asigna a la variable **archivo_ID** el identificador para futuros accesos al archivo.

```
set archivo_ID [ open "resultados.txt" w+ ]
```

El archivo **resultados.txt** es creado si no existe o sobrescrito en caso de existir.

Comando close

El comando **close** cierra el canal establecido para manejar el archivo. Su sintaxis es de la siguiente forma:

```
set archivo_ID [ open "resultados.txt" w+ ]
```

Nota: siempre es conveniente cerrar correctamente el archivo para asegurarse que todas las transferencias de escritura se realicen.

Comando puts

El comando **puts** escribe a un archivo que se accede a través del identificador obtenido previamente mediante la ejecución del comando **open**. La estructura del comando es de la siguiente forma:

```
puts <canal> string
```

donde <canal> es opcional e indica el identificador del archivo al que se desea escribir el *string*. Si <canal> es omitido, entonces el *string* es escrito en la consola estándar de salida (**stdout**).

Ejemplo:

El siguiente comando escribe en el archivo **resultados.txt** abierto previamente con el comando **open** y asignado a la variable **archivo_ID** el identificador del archivo, el *string* “Hola Mundo”:

```
puts $archivo_ID "Hola Mundo"
```

Puede ocurrir que la escritura efectiva se realice al cerrar el acceso al archivo.

Comando gets

El lenguaje Tcl soporta diferentes alternativas de acceso a archivos para lectura. Una alternativa es la lectura de una línea del archivo la cual retornará todos los caracteres del archivo hasta que la línea finalice. La segunda opción es transferir un determinado número de bytes a una variable. La forma de estas dos alternativas es:

```
gets canal
```

retorna una línea completa leída del archivo que se accede mediante el identificador *canal*,

```
gets $archivo_ID línea
```

transfiere la línea del archivo identificado por *canal* a la variable **línea** y retorna el número de bytes transferidos. Si se llegó al final del archivo, el comando retorna -1.

Ejemplo

```
set archivo_ID [ open "resultados.txt" w+ ]
puts $archivo_ID "Hola Mundo"
close $archivo_ID

set archivo_ID [ open "resultados.txt" r+ ]
gets $archivo_ID línea
close $$archivo_ID
```

transfiere a la variable *línea* “Hola Mundo” previamente escrito en el archivo e imprime la cantidad de bytes de la línea (10). Debe cerrarse el archivo para que las transferencias se realicen y abrirlo sin sobrescritura.

Comentario

El lenguaje Tcl permite introducir comentarios para mejorar la documentación y el mantenimiento del código. Las líneas de comentario deben comenzar con el símbolo ‘#’. Los

comentarios pueden ser introducidos al comienzo de una línea o luego del símbolo ';' como se muestra en el siguiente ejemplo:

```
# Este comentario que comienza en la línea  
set i 2; # Este comentario está en la misma línea que el comando
```

EJECUCIÓN DE ARCHIVOS TCL POR LÍNEA DE COMANDO

El lenguaje Tcl permite la creación de archivos de comandos que pueden ser ejecutados desde la consola o invocados desde otros archivos. En estos casos, es conveniente transferir argumentos mediante la línea de comandos. Este tipo de lenguaje predefine las siguientes variables para tal fin:

- **argc**: contiene el número de argumentos de la línea de comandos.
- **argv**: contiene la lista de los argumentos, siendo su primer elemento el nombre del comando ejecutado (que coincidirá con el nombre del archivo correspondiente).

Estas variables pueden ser procesadas para determinar los diferentes argumentos con los que se invocó el archivo de comandos.

CAPÍTULO 10

LENGUAJE DE COMANDOS TCL EN HERRAMIENTAS DE DISEÑO: QUARTUS II

LENGUAJE DE COMANDOS TCL EN HERRAMIENTAS DE DISEÑO

Como se ha descrito en capítulos anteriores, el flujo de diseño requiere la ejecución de diversas aplicaciones que son accesibles de menú en las diferentes interfaces gráficas de las distintas herramientas. Esto implica que la realización de un cambio en el diseño de la aplicación involucra una nueva ejecución de las diferentes aplicaciones comprendidas en el flujo de diseño. Esta metodología puede resultar inadecuada cuando se desea automatizar el proceso de diseño o independizarse de las actualizaciones que las interfaces gráficas presentan para sus diferentes versiones.

La utilización del lenguaje de comandos Tcl en herramientas de diseño permite:

- **Automatizar el proceso de diseño:** mediante la eliminación de la selección manual de los comandos desde la interfaz gráfica de las herramientas de diseño. Además, permite automatizar la interrelación entre las diferentes herramientas de diseño.
- **Realizar reportes personalizados:** que permitan extraer la información que el diseñador desea analizar y prescindir de toda la información restante que brindan las herramientas. Esta característica permite ejecutar mejores procesos de documentación de los proyectos de diseño.
- **Reproducir los procesos de diseño:** independizándose de las diferencias entre las interfaces gráficas de las distintas versiones de las herramientas de diseño. Esta característica permite un mantenimiento más eficiente y actualización de los proyectos.

La mayoría de las herramientas de diseño (generalmente, referenciadas como EDA) presenta soporte para la especificación de archivos de comandos Tcl. En las próximas secciones, se describirán las alternativas que ofrece Altera en sus herramientas de diseño para el soporte del lenguaje de comandos Tcl. Un completo detalle del soporte de lenguaje de comandos Tcl en Quartus II puede encontrarse en (27).

SOPORTE DE ARCHIVOS DE COMANDOS TCL EN QUARTUS II

Altera ofrece un amplio soporte para la generación de archivos de comandos Tcl que involucran todo el proceso de diseño de hardware. Dicho soporte está implementado mediante la definición de diferentes **interfaces de programación de la aplicación** (API, *Application Programming Interface*, en inglés). La implementación de las diferentes funciones de estas interfaces está agrupada en paquetes (*packages*, en inglés) que deben ser precargados para que el intérprete de comandos Tcl pueda ejecutarlos correctamente. Las API de Quartus II soportan cientos de comandos agrupados en más de 20 paquetes, posibilitando de esta manera una gran potencialidad en la creación de archivos de comandos para la automatización del flujo de diseño.

PAQUETES TCL (*TCL PACKAGES*)

Altera agrupa los diferentes comandos Tcl soportados por Quartus II en paquetes de acuerdo a sus funciones. Por defecto, solo un número mínimo de paquetes es cargado automáticamente cuando Quartus II es ejecutado. Esta metodología permite mantener en bajos niveles la utilización de memoria de la computadora personal. Los restantes paquetes deben ser cargados en el sistema con anterioridad a ejecutar las funciones que definen. Para determinar los paquetes que son o están cargados por una determinada aplicación, puede ejecutarse el siguiente comando:

```
<nombre_de_la_aplicación> --tcl_eval help
```

Por ejemplo, para determinar los paquetes que carga la aplicación `quartus_map`, se puede ejecutar el comando:

```
quartus_map --tcl_eval help
```

La Tabla 10.1 describe los paquetes de comandos Tcl definidos para Quartus II.

TABLA 10.1: PAQUETES DE COMANDOS TCL DE QUARTUS II

Nombre del paquete	Descripción
::quartus::backannotate	Comandos para la asignación del análisis de la síntesis realizada.
::quartus::chip_planner	Comandos para la identificación y modificación de las asignaciones de recursos y ruteo realizadas con la aplicación <i>Chip Editor</i> de Quartus II.
::quartus::database_manager	<i>Manage version-compatible database files.</i>
::quartus::device	Comandos para la obtención de información de los dispositivos desde la base de datos de las diferentes familias.
::quartus::flow	Comandos para la compilación de proyectos y la ejecución de flujos de diseño.
::quartus::incremental compilation	Comandos para la manipulación de las particiones del diseño y las regiones fijas (<i>LogicLock regions</i>), y la configuración relacionada con la compilación incremental.
::quartus::insystem_memory_edit	Comandos para la lectura y edición de los contenidos de memoria en los dispositivos de Altera.
::quartus::insystem_source_probe	Comandos para la interacción con las herramientas de prueba implementadas en los dispositivos de Altera.
::quartus::jtag	Comandos para el control de la cadena de dispositivos JTAG.
::quartus::logic_analyzer_interface	Comandos para consulta y modificación de la interfaz del componente <i>Logic Analyzer</i> de Altera.
::quartus::misc	Comandos para la realización de diferentes tareas generales.
::quartus::project	Comandos para la creación de proyectos y la realización de asignaciones.
::quartus::rapid_recompile	Comandos para la manipulación de las características <i>Rapid Recompile</i> de Quartus II.
::quartus::report	Comandos para la creación y acceso a reportes personalizados.
::quartus::rtl	Comandos para el análisis de los archivos <i>netlist</i> del diseño.
::quartus::sdc	Comandos para la especificación de restricciones y excepciones a la herramienta <i>TimeQuest Timing Analyzer</i> .

::quartus::sdc_ext	Comandos específicos para la configuración de parámetros SDC (<i>Synopsys Design Constraints</i> : formato de archivo que contiene características de diseño como temporizado, consumo y área) de dispositivos de Altera.
::quartus::simulator	Comandos para la configuración y ejecución de la simulación.
::quartus::sta	Comandos para la obtención de información avanzada de Quartus II <i>TimeQuest Timing Analyzer</i> .
::quartus::stp	Comandos para la ejecución de la herramienta SignalTap II <i>Logic Analyzer</i> .

La carga de un paquete específico se realiza mediante el comando `load_package` con la siguiente sintaxis:

```
load_package [-version <número_de_versión>] <nombre_del_paquete>
```

Si el número de versión no es especificado, entonces el intérprete cargará la versión más actualizada disponible.

Nota: no todos los paquetes pueden ser cargados en todas las aplicaciones, sino que algunos de ellos pueden cargarse en aplicaciones específicas.

EJECUTABLES DE LÍNEA DE COMANDOS DE QUARTUS II

Todos los procedimientos y configuraciones realizados mediante la interfaz gráfica de Quartus II pueden ser efectuados a través de la ejecución de los ejecutables de línea de comandos que ofrece Quartus II y los comandos Tcl de los diversos paquetes. Estos ejecutables de línea de comando responden a las mismas aplicaciones que son invocadas mediante la interfaz gráfica de Quartus II. De la misma manera que los comandos Tcl, la ejecución por línea de comando de las aplicaciones permite procedimientos más documentados y de fácil mantenimiento. Todas las aplicaciones cargan por defecto algunos de los paquetes de comandos Tcl de Quartus II detallados en la sección anterior.

La Tabla 10.2 describe los paquetes de comandos Tcl definidos para Quartus II. Todas las aplicaciones poseen el argumento `-h` que permite la impresión en pantalla de la ayuda de dicha aplicación.

TABLA 10.2: EJECUTABLES DE LÍNEA DE COMANDOS DE QUARTUS II

Ejecutable	Descripción
quartus_asm	La aplicación Quartus II Assembler genera los archivos de configuración para los dispositivos CPLD o FPGA. Esta aplicación puede generar archivos con formato: • .pof (<i>Programmer Object File</i>) para la configuración de dispositivos CPLD o memorias de configuración E2PROM de FPGA. • .sof (<i>SRAM Object File</i>) para la configuración de dispositivos FPGA directamente desde Quartus II. • .hexout (formato Intel Hexadecimal), .tff (<i>Tabular Text File</i>), .rbf (<i>Raw Binary File</i>) para la configuración de los dispositivos mediante herramientas de otros fabricantes. Los archivos con formato .sof y .pof pueden ser procesados mediante la aplicación Quartus II Programmer para la configuración de los dispositivos. La aplicación Quartus II Fitter debe haber sido ejecutada satisfactoriamente antes de la ejecución de esta aplicación.
quartus_cdb	La aplicación Quartus II Compiler Database Interface es la interfaz que permite la compatibilidad entre bases de datos de diferentes versiones y la compilación incremental soportando la asignación de recursos <i>logiclock</i>. Se debe realizar una etapa satisfactoria de <i>Analysis & Synthesis</i> del diseño antes de ejecutar esta aplicación.
quartus_cpf	La aplicación Quartus II Convert Programming Files permite convertir el formato de archivos de configuración entre diversos formatos posibles.
quartus_drc	La aplicación Quartus II Design Assistant verifica el diseño mediante la aplicación de diferentes reglas de diseño. Esta aplicación es adecuada para comprobar la confiabilidad de un diseño cuando se desea migrar a dispositivos <i>Gate Array</i> como los <i>Hardcopy</i> de Altera.
quartus_eda	La aplicación Quartus II EDA Netlist Writer genera diferentes formatos de archivos <i>netlist</i> compatibles herramientas de desarrollo de otras compañías.
quartus_fit	La aplicación Quartus II Fitter realiza el proceso de <i>place and route</i> para ubicar en los recursos del dispositivo CPLD o FPGA el diseño realizado. La aplicación Quartus II Analysis & Synthesis debe ser ejecutada satisfactoriamente antes de ejecutar esta aplicación.
quartus_jbcc	La aplicación Quartus II JAM Compiler convierte archivos en formato JAM/STAPL (JAM) a formato JAM/STAPL <i>Byte Code files</i> (JBC).
quartus_jli	El Quartus II JBI Player ejecuta archivos de comandos JAM/STAPL (JBC).
quartus_map	La aplicación Quartus II Analysis & Synthesis construye una única base de datos del proyecto que integra todos los archivos de diseño del proyecto. Esta aplicación soporta archivos de diseño en Verilog HDL, VHDL y el lenguaje específico de Altera AHDL.

quartus_pgm	La aplicación Quartus II Programmer permite programar los dispositivos de Altera. Esta aplicación soporta únicamente formatos de archivos propios de Altera como son: Programmer Object Files (.pof), SRAM Object Files (.sof), Jam File (.jam) o Jam Byte-Code File (.jbc).
quartus_pow	La aplicación Quartus II PowerPlay Power Analyzer estima el consume de potencia dinámico y estático del diseño. La aplicación Quartus Fitter debe haberse ejecutado satisfactoriamente antes de esta aplicación.
quartus_sh	La aplicación Quartus II Shell es un intérprete de comandos Tcl de Quartus II. Permite la ejecución de comandos en línea de comandos o de archivos de comandos.
quartus_si	La aplicación Quartus II ssn Analyzer estima las perturbaciones que produce el ruido de conmutación al voltaje y temporizado. Antes de ejecutar esta aplicación, debe haberse ejecutado satisfactoriamente la aplicación Quartus II Fitter .
quartus_sim	La aplicación Quartus II Simulator permite la realización de simulaciones funcional y temporal. Esta aplicación posee soporte para la interpretación de comandos Tcl. La ejecución de una simulación temporal debe ser precedida por la ejecución satisfactoria de la aplicación Quartus II Timing Analyzer .
quartus_sta	La aplicación TimeQuest timing analyzer computa el retardo de un determinado diseño para un dispositivo específico y lo incorpora al archivo <i>netlist</i> para uso futuro. Esta aplicación debe ser ejecutada luego de la ejecución satisfactoria de Quartus II Fitter .
quartus_stp	La aplicación Quartus II SignalTap II Logic Analyzer permite la captura de señales internas del dispositivo para su posterior análisis. Las señales capturadas son visualizadas mediante formas de onda y almacenadas en un archivo <i>.st</i> .
quartus_tan	La aplicación Quartus II timing analyzer computa el retardo de un determinado diseño para un dispositivo específico y lo incorpora al archivo <i>netlist</i> para uso posterior.

La ejecución de las diversas aplicaciones por línea de comandos permite desarrollar todo el proceso de diseño de hardware en Quartus II de manera eficiente, documentable y de fácil mantenimiento.

UTILIDAD PARA AYUDA DE PAQUETES Y COMANDOS TCL EN QUARTUS II

Existe un gran número de comandos Tcl y ejecutables que pueden ser utilizados para la realización de los flujos de diseño en Quartus II. Si bien la mayoría de los comandos poseen una opción en su línea de ejecución para que imprima la ayuda y alternativas que brinda, muchas veces es conveniente obtener dicha ayuda en una manera estructurada que involucre todos los comandos y ejecutables disponibles. Para tal fin, Altera ofrece la utilidad **Qhelp** que brinda información sobre todos los ejecutables, paquetes Tcl y comandos con sus descripciones y opciones disponibles. Para invocar esta utilidad debe ejecutarse el siguiente comando en Microsoft Windows (en **Inicio > Buscar Aplicaciones y archivos** en Windows 7 o **Inicio > Ejecutar** en Windows XP):

```
quartus_sh -qhhelp
```

Este comando ejecutará la aplicación **Quartus II Command-Line and Tcl API Help** como muestra la Figura 10-1.

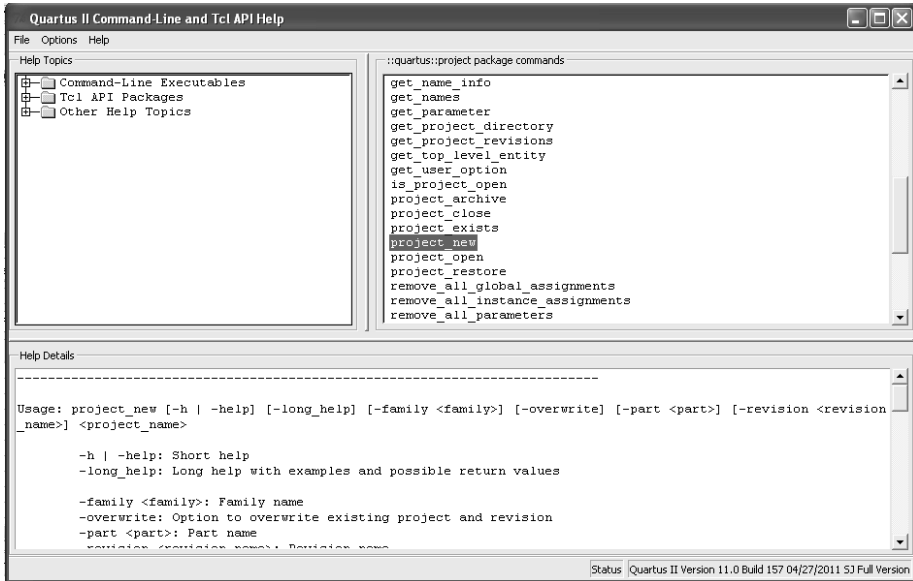


Figura 10-1: Utilidad Qhelp de Quartus II.

La opción Qhelp brinda de manera gráfica información detallada y estructurada de todos los paquetes, comandos y opciones brindados por Altera para la realización de comandos Tcl. A continuación, se detalla la utilización de estos comandos para la creación de proyectos y ejecución del flujo de diseño de hardware.

CREACIÓN Y MANIPULACIÓN DE PROYECTOS EN QUARTUS II CON COMANDOS TCL

Uno de los paquetes de comandos Tcl más frecuentemente utilizados es el paquete `::quartus::project` el cual permite manipular proyectos de Quartus II y realizar diferentes asignaciones para la configuración del flujo de diseño del proyecto. A continuación, se describen los principales comandos que permite este paquete y se utilizan para crear y manipular proyectos en Quartus II.

PAQUETE ::QUARTUS::PROJECT

El paquete `::quartus::project` contiene la definición de diferentes comandos Tcl para trabajar con proyectos de Quartus II y sus revisiones. La siguiente tabla lista los comandos incluidos en la versión 6.0 del paquete `::quartus::project`:

Comando	Descripción
<code>assignment_group</code>	Agrega, elimina, obtiene los miembros o las excepciones de un grupo de asignaciones.
<code>create_revision</code>	Crea una nueva revisión del proyecto.
<code>delete_revision</code>	Elimina una revisión específica del proyecto actual.
<code>execute_assignment_batch</code>	Ejecuta los comandos Tcl pasados por una lista Tcl en forma secuencial.
<code>export_assignments</code>	Exporta las asignaciones realizadas en el proyecto abierto al archivo de configuración (.qsf).
<code>get_all_assignment_names</code>	Retorna una lista filtrada de todos los nombres de las asignaciones posibles en el proyecto.
<code>get_all_assignments</code>	Retorna el conjunto de todas las asignaciones actuales del proyecto.
<code>get_all_user_option_names</code>	Retorna el conjunto de todos los nombres de las opciones de usuario.
<code>get_assignment_info</code>	Retorna la información de identificación basada en una opción específica.
<code>get_assignment_name_info</code>	Retorna la información para el nombre de la asignación especificada.
<code>get_current_revision</code>	Retorna el nombre de la revisión actual para un proyecto especificado.
<code>get_global_assignment</code>	Retorna el valor de una asignación global especificada.
<code>get_instance_assignment</code>	Retorna el valor de la asignación de una instancia.
<code>get_location_assignment</code>	Retorna el valor de una asignación de locación.
<code>get_name_info</code>	Muestra el tipo de información especificado para un determinado identificación de nombre de nodo.
<code>get_names</code>	Retorna una lista de identificadores de nodos que coinciden con un determinado criterio de búsqueda.
<code>get_parameter</code>	Retorna el valor de un parámetro.
<code>get_project_directory</code>	Retorna el directorio del proyecto actual.
<code>get_project_revisions</code>	Retorna una lista con las revisiones de un proyecto específico.
<code>get_top_level_entity</code>	Retorna el nombre de identificación de la entidad de mayor jerarquía.
<code>get_user_option</code>	Retorna el valor de la opción de usuario especificada para un determinado nombre de opción.

is_project_open	Verifica si el proyecto especificado está actualmente abierto. Retorna '1' si el proyecto está abierto y '0' en caso contrario.
project_archive	Archiva un proyecto abierto y todos sus archivos relacionados en un archivo .qar . Este tipo de archivo puede ser restituido mediante el comando project_restore .
project_close	Cierra un proyecto abierto.
project_exists	Verifica si un determinado proyecto existe, Retorna '1' si el proyecto existe y '0' en caso contrario.
project_new	Crea y abre un nuevo proyecto con un nombre determinado.
project_open	Abre un determinado proyecto.
project_restore	Restablece un proyecto archivado en un archivo .qar mediante el comando project_archive .
remove_all_global_assignments	Elimina todas las asignaciones globales que satisfacen un determinado criterio.
remove_all_instance_assignments	Elimina todos los valores de las instancias que satisfacen un determinado criterio.
remove_all_parameters	Elimina todos los parámetros que satisfacen un determinado criterio.
resolve_file_path	Retorna el directorio completo de un determinado archivo. Si el archivo no existe, retorna el nombre original del archivo.
revision_exists	Verifica si una revisión existe para un determinado proyecto o para el proyecto actual.
set_current_revision	Asigna un determinado nombre a la revisión actual.
set_global_assignment	Asigna o elimina asignaciones globales.
set_instance_assignment	Asigna o elimina asignaciones de instancias.
set_io_assignment	Asigna o elimina asignaciones de entrada salida.
set_location_assignment	Asigna o elimina asignaciones de locación.
set_parameter	Asigna o elimina asignaciones de parámetros.
set_power_file_assignment	Asigna o elimina la asignación de un archivo de energía de entrada.
set_user_option	Asigna el valor a una opción de usuario especificada por un determinado nombre de opción. La opción de usuario es escrita al archivo quartus2.ini.
test_assignment_trait	Verifica si una determinada asignación tiene una característica específica.

Existen, entre estos comandos del paquete `:quartus:project`, comandos que por su frecuencia de utilización son detallados a continuación.

Comando `project_exist`

El comando `project_exist` verifica si un determinado proyecto existe. El comando retornará el valor '1' si el proyecto existe y "0" en caso contrario. Su sintaxis es:

```
project_exists [-h | -help] [-long_help] <nombre_del_proyecto>
```

La siguiente tabla describe la función de cada uno de los argumentos del comando:

Argumento	Descripción	
-h -help	Imprime una breve ayuda sobre el comando.	Opcional
-long_help	Imprime una ayuda detallada y con ejemplos sobre el comando.	Opcional
<nombre_del_proyecto>	Nombre del proyecto que se desea verificar su existencia.	Obligatorio

Comando `project_new`

El comando `project_new` crea y abre un proyecto nuevo con un determinado nombre de proyecto. Su sintaxis es:

```
project_new [-h | -help] [-long_help] [-family <family>] [-  
  overwrite] [-part <part>] [-revision <revision_name>]  
  <nombre_del_proyecto>
```

La siguiente tabla describe la función de cada uno de los argumentos del comando:

Argumento	Descripción	
-h -help	Imprime una breve ayuda sobre el comando.	Opcional
-long_help	Imprime una ayuda detallada y con ejemplos sobre el comando.	Opcional
-family <family>	Especifica la familia del dispositivo considerado en el proyecto.	Opcional
-overwrite	Especifica que el proyecto y revisión será sobrescrito en caso de existir.	Opcional
-part <part>	Especifica el modelo de dispositivo considerado en el proyecto.	Opcional

-revision <revision_name>	Especifica el nombre de la revisión con que se creará el proyecto.	Opcional
<nombre_del_proyecto>	Nombre que se le asigna al proyecto creado.	Obligatorio

La ejecución del comando devolverá un valor '0' si el comando se realizó correctamente o '1' si alguno de los siguientes errores ocurrieron:

Error	Solución
Can't create project: <nombre_del_proyecto>	El nombre del proyecto no es un nombre correcto. Especificar un nombre de proyecto válido.
Can't create revision: <revision_name>	El nombre de la revisión no es un nombre correcto. Especificar un nombre de revisión válido.
Can't create settings files for project: <nombre_del_proyecto>	No pueden crearse los archivos de configuración del proyecto. Asegurarse que los archivos .psf, .csf y .ssf tengan permiso de escritura.
Can't remove Quartus II Settings File: <nombre_del_proyecto>	No se puede eliminar el archivo de configuración del proyecto <nombre_del_proyecto>. Asegurarse que los archivos del proyecto tengan permiso de escritura.
Can't run Tcl command while a process is in progress: <nombre_del_proyecto>.	Para ejecutar un comando, deben detenerse los procesos de simulación y compilación, o esperar a que estos finalicen.
Project already exists: <nombre_del_proyecto>	El proyecto ya existe y no se especificó la opción <code>-overwrite</code> . Debe asegurarse que el proyecto no existe o utilizar la opción <code>-overwrite</code> .

Comando `project_open`

El comando `project_open` abre un proyecto ya existente. Su sintaxis es:

```
project_open [-h | -help] [-long_help] [-current_revision] [-force]
             [-revision <revision_name>] <project_name>
```

La siguiente tabla describe la función de cada uno de los argumentos del comando:

Argumento	Descripción	
-h -help	Imprime una breve ayuda sobre el comando.	Opcional

-long_help	Imprime una ayuda detallada y con ejemplos sobre el comando.	Opcional
-current_revision	Especifica que se abra la revisión actual del proyecto.	Opcional
-force	Fuerza a que la base de datos del proyecto se reconstruya si su versión no coincide con la versión de Quartus II con que se desea abrir el proyecto. Si no se especifica la opción <code>-force</code> y las versiones de la base de datos y de Quartus II son incompatibles, entonces el comando no se completa y retorna error.	Opcional
<nombre_del_proyecto>	Nombre que se le asigna al proyecto creado.	Obligatorio

Si la opción `-revision` no es especificada, entonces el `<nombre_del_proyecto>` es utilizado como nombre de la revisión. El comando retorna '0' si se completó correctamente y '1' si ocurrió un error durante su ejecución y no pudo ser completado.

Comando `project_close`

El comando `project_close` cierra un proyecto abierto. Todas las asignaciones realizadas mientras el proyecto estuvo abierto son transferidas al archivo de configuración del proyecto (.qsf) a menos que se utilice la opción `dont_export_assignments`. Su sintaxis es:

```
project_close [-h | -help] [-long_help] [-dont_export_assignments]
```

La siguiente tabla describe la función de cada uno de los argumentos del comando:

Argumento	Descripción	
-h -help	Imprime una breve ayuda sobre el comando.	Opcional
-long_help	Imprime una ayuda detallada y con ejemplos sobre el comando.	Opcional
-dont_export_assignments	No exporta las asignaciones realizadas mientras el proyecto estuvo abierto al archivo del proyecto.	Opcional

El comando retorna '0' si se ejecutó correctamente o '1' si existió un error durante su ejecución y no se completó correctamente.

Comando `create_revision`

El comando `create_revision` crea una nueva versión especificada. Si la revisión no está incluida en el proyecto actual una nueva revisión es creada con la configuración por defecto. Su sintaxis es:

```
create_revision [-h | -help] [-long_help] [-based_on
    <revision_name>] [-copy_results] [-set_current] <revision_name>
```

La siguiente tabla describe la función de cada uno de los argumentos del comando:

Argumento	Descripción	
-h -help	Imprime una breve ayuda sobre el comando.	Opcional
-long_help	Imprime una ayuda detallada y con ejemplos sobre el comando.	Opcional
-based_on <revision_name>	Permite obtener la configuración de la revisión creada de la revisión especificada en <revision_name>.	Opcional
-copy_results	Copia los resultados de la revisión especificada en la opción -based_on.	Opcional
-set_current	Especifica que la nueva revisión creada sea la actual.	Opcional
<revision_name>	Especifica el nombre dado a la revisión creada.	Obligatorio

El comando retorna '0' si se ejecutó correctamente o '1' si existió un error durante su ejecución y no se completó correctamente.

Comando `export_assignments`

El comando `export_assignments` exporta las asignaciones realizadas en el proyecto abierto al archivo de configuración (**.qsf**). Es importante notar que todas las asignaciones creadas o modificadas mientras el proyecto está abierto, no son almacenadas en el archivo de configuración hasta que se realice una exportación de las asignaciones mediante el comando `export_assignments` o `project_close`. Su sintaxis es:

```
export_assignments [-h|-help] [-long_help] [-reorganize]
```

La siguiente tabla describe la función de cada uno de los argumentos del comando:

Argumento	Descripción	
-h -help	Imprime una breve ayuda sobre el comando.	Opcional
-long_help	Imprime una ayuda detallada y con ejemplos sobre el comando.	Opcional
-reorganize	Permite reorganizar el archivo de configuración .qsf.	Opcional

El comando retorna '0' si se ejecutó correctamente o '1' si existió un error durante su ejecución y no se completó correctamente.

Nota: es necesario exportar todas las asignaciones realizadas antes de ejecutar alguna aplicación de Quartus II para que dichas asignaciones sean tenidas en cuenta.

CREACIÓN DE PROYECTOS EN QUARTUS II MEDIANTE COMANDOS TCL

La utilización del lenguaje de comandos Tcl en el flujo de diseño de hardware permite registrar en un archivo todos los pasos necesarios para la generación de un proyecto y utilizarlo en un futuro para restaurar la configuración a un estado conocido. Quartus II permite la generación de un archivo de comandos Tcl cuya ejecución permite configurar el flujo de diseño al estado en que se realizó su generación. Para acceder a esta opción, se debe ejecutar el comando **Project > Generate Tcl File for Project...** de la barra de menú (Figura 10-2).

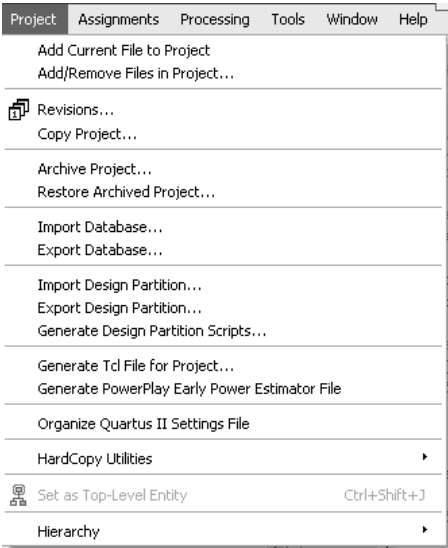


Figura 10-2: Comando para la generación de archivo de comandos Tcl del proyecto.

El archivo generado es un buen punto de comienzo para el aprendizaje de la generación de proyectos mediante la utilización del lenguaje de comandos Tcl. Sin embargo, en lo que sigue, se realizará un archivo de comandos para que realice el mismo procedimiento descrito en el Capítulo 5, preservando la estructura del archivo de comandos Tcl generado por Quartus II.

Ejemplo: creación de proyecto en Quartus II mediante comandos Tcl

En este ejemplo, se realizará un procedimiento similar al realizado en el Capítulo 5 para la generación y configuración del flujo de diseño de un SOPC (Página 88). El código expuesto conformará un archivo de texto que se denominará **tcl_n2sopc.tcl**.

El primer comando del archivo debe cargar el paquete `::quartus::project` para tener definidos todos los comandos y ejecutables requeridos para la creación y manejo de proyectos:

```
# Carga el paquete ::quartus::project
# para la creación y manejo de proyectos
load_package ::quartus::project
```

La creación de un nuevo proyecto debe verificar si dicho proyecto no existe con anterioridad. Si el proyecto existe, entonces el proyecto es abierto, caso contrario el proyecto es creado. La configuración necesaria para la creación del proyecto será similar a la realizada en la sección “Realización del flujo de diseño de un SOPC” del Capítulo 5 (página 88) con la siguiente información:

- Directorio del proyecto: **c:\tcl\n2sopc** (especificación realizada en la pantalla de la Figura 3-6 del flujo de diseño de Quartus II).
- Nombre del proyecto: **n2sopc** (especificación realizada en la pantalla de la Figura 3-6 del flujo de diseño de Quartus II).
- Familia de dispositivo: **Cyclone IV E**. Configurada mediante la asignación global **FAMILY** (especificación realizada en la pantalla de la Figura 3-8 del flujo de diseño de Quartus II).
- Dispositivo: **EP4CE22F17C7**. Configurado mediante la asignación global **DEVICE** (especificación realizada en la pantalla de la Figura 3-8 del flujo de diseño de Quartus II).
- Entidad de máxima jerarquía: **n2sopc**. Configurada mediante la asignación global **TOP_LEVEL_ENTITY** a la entidad **n2sopc** definida en el archivo **n2sopc.bdf** (especificación realizada en la pantalla de la Figura 3-6 del flujo de diseño de Quartus II).
- Aplicación de simulación: **ModelSim-Altera (Verilog)**. Configurada mediante la asignación global **EDA_SIMULATION_TOOL** (especificación realizada en la pantalla de la Figura 3-9 del flujo de diseño de Quartus II).

- Formato de archivo de salida para simulación: **VERILOG HDL**. Configurada mediante la asignación global **EDA_OUTPUT_DATA_FORMAT** (especificación realizada en la pantalla de la Figura 3-9 del flujo de diseño de Quartus II).
- Ejecución automática de la herramienta de simulación: **ON**. Configurada mediante la asignación global **EDA_RUN_TOOL_AUTOMATICALLY** (especificación realizada en la pantalla de la Figura 3-9 del flujo de diseño de Quartus II).

El archivo de comandos Tcl comenzará con la verificación de la existencia del directorio del proyecto. En caso de no existir, el directorio es creado. Posteriormente, se cambia el directorio de trabajo al directorio del proyecto.

```
# Verifica si el directorio del proyecto está creado
# El formato del directorio debe ser tipo UNIX
if {[eval "file exist /tcl/n2sopc"]} {
    file mkdir /tcl/n2sopc
} else {
    puts "El directorio eciste, no es creado"
}
# Se cambia el directorio de trabajo al directorio del proyecto
cd /tcl/n2sopc
```

En forma similar, se comprueba si el proyecto existe. Si existe, entonces se comprueba si el proyecto está actualmente abierto (el comando `is_project_open` retorna si existe un proyecto abierto actualmente y la variable `quartus(project)` retorna el valor del proyecto actual que es comparado con el proyecto que se desea abrir). Si el proyecto existe pero no es actual, entonces el proyecto es abierto (mediante el comando `project_open`). Si el proyecto no existe, el proyecto es creado (mediante el comando `project_new`). El comando **project_new** permite definir la familia y el dispositivo utilizado para el proyecto, pero estas asignaciones serán realizadas posteriormente para una mejor estructura del código del archivo.

```
# Verifica si existe un proyecto actualmente abierto en Quartus II
if {[is_project_open]} {
    # Si hay proyecto abierto, verifica si
    # concuerda con el que se desea crear
    if {[string compare $quartus(project) "n2sopc"]} {
        puts "El proyecto n2sopc está actualmente abierto"
    }
} else {
    # Verifica si el proyecto existe
    if {[project_exists n2sopc]} {
        # Si existe entonces es abierto con revisión "n2sopc"
```

```
        project_open -revision n2sopc n2sopc
    } else {
        # Si no existe, es creado con revisión "n2sopc"
        project_new -revision n2sopc n2sopc
    }
}
```

La familia y el dispositivo se asignan mediante el comando `set_global_assignment`.

```
# Asignación de la Familia de FPGA del proyecto
set_global_assignment -name FAMILY "Cyclone IV E"
# Asignación del dispositivo FPGA del proyecto
set_global_assignment -name DEVICE EP4CE22F17C7
```

La definición de la entidad de máxima jerarquía requiere la incorporación del archivo de diseño que la describa.

```
# Incorpora el archivo n2sopc.bdf al proyecto
set_global_assignment -name BDF_FILE n2sopc.bdf
# Define la entidad "n2sopc" como la de mayor jerarquía
set_global_assignment -name TOP_LEVEL_ENTITY n2sopc
```

NOTA: el archivo "n2sopc.bdf" debe ser copiado en el directorio "c:\tcl\n2sopc" para que pueda ser accedido correctamente.

La asignación de la herramienta de simulación y su configuración puede realizarse de la siguiente manera:

```
# Asignación de la aplicación de simulación
set_global_assignment -name EDA_SIMULATION_TOOL "ModelSim-Altera (Verilog)"
# Asignación del format de archive de salida para simulación
set_global_assignment -name EDA_OUTPUT_DATA_FORMAT "VERILOG HDL"
# Asignación de ejecución no automática del simulador
set_global_assignment -name EDA_RUN_TOOL_AUTOMATICALLY OFF
```

Finalmente, una vez creado el proyecto y realizado todas las asignaciones propuesta, el proyecto puede ser cerrado para almacenar toda la configuración realizada.

```
# El proyecto es cerrado
project_close
```

Asignación de pines

En los dispositivos FPGA modernos, la asignación de las entradas y salidas puede resultar dificultosa mediante la utilización de la interfaz gráfica debido a la gran cantidad de pines que pueden tener tales dispositivos. Además, pueden introducirse errores en el proceso de asignación de pines cuando se realiza manualmente.

Generalmente, en proyectos de mediana a gran envergadura es conveniente realizar la asignación de pines del dispositivo FPGA mediante archivos de comandos Tcl de configuración. La asignación de los pines se realiza mediante una asignación de locación que asocia la identificación del pin con el nombre de la señal que se desea en dicho pin. De esta manera, si se desea asignar la señal `clk` al pin G1 del dispositivo FPGA se puede ejecutar el siguiente comando mientras el proyecto está abierto:

```
# Asigna el pin G1 del dispositivo FPGA
# a la señal "clk" del proyecto
set_location_assignment -to clk Pin_G1
```

La asignación de los restantes pines se realiza en forma similar.

En la próxima sección, se introduce el paquete `::quartus::flow` que define comandos para la compilación de proyectos de Quartus II para continuar con el flujo de diseño mediante el lenguaje de comandos Tcl.

COMPILACIÓN DE PROYECTOS EN QUARTUS II CON COMANDOS TCL

El paquete `::quartus::project` define diversos comandos para la compilación de proyectos en Quartus II. Estos comandos permiten realizar el flujo de diseño de Quartus II mediante comandos Tcl. A continuación, se describen los principales comandos que permite este paquete y se utilizan para realizar el flujo de diseño de un proyecto en Quartus II.

PAQUETE `::QUARTUS::FLOW`

El paquete `::quartus::flow` contiene la definición de diferentes comandos Tcl para compilar proyectos de Quartus II. La siguiente tabla lista los comandos incluidos en la versión 1.1 del paquete `::quartus::flow`.

Comando	Descripción
delete_netlist	Elimina una determinada red de componentes (<i>netlist</i>) del proyecto actualmente abierto.
execute_flow	Ejecuta uno o varios flujos de diseño de Quartus II.
execute_hc	Ejecuta un determinado flujo para dispositivo HardCopy.
execute_module	Ejecuta una determinada aplicación mediante un comando y exportando todas las asignaciones del proyecto.
netlist_exists	Verifica si una determinada red de componentes existe.

A continuación, se detallan los más frecuentemente utilizados.

Comando `execute_flow`

El comando `execute_flow` ejecuta uno de los diferentes flujos de diseño disponibles en Quartus II. Su sintaxis es:

```
execute_flow [-h | -help] [-long_help] [-analysis_and_elaboration]
             [-check_ios] [-check_netlist] [-compile] [-compile_and_simulate]
             [-create_companion_revision] [-dont_export_assignments] [-
             early_timing_estimate] [-eco] [-eda_synthesis] [-
             export_database] [-generate_functional_sim_netlist] [-
             import_database] [-incremental_compilation_export] [-
             incremental_compilation_import] [-signalprobe] [-vqm_writer]
```

La siguiente tabla describe la función de cada uno de los argumentos del comando:

Argumento	Descripción	
-h -help	Imprime una breve ayuda sobre el comando.	Opcional
-long_help	Imprime una ayuda detallada y con ejemplos sobre el comando.	Opcional
-analysis_and_elaboration	Ejecuta el flujo de Análisis y Elaboración (<i>Analysis & Elaboration</i>) de Quartus II.	Opcional
-check_ios	Ejecuta el análisis de asignación de Entradas/Salidas (<i>I/O assignment analysis</i>).	Opcional
-check_netlist	Ejecuta el chequeo y guarda de la red de componentes (<i>Check and Save Netlist</i>).	Opcional
-compile	Ejecuta una compilación completa (<i>Full Compilation</i>).	Opcional

-compile_and_simulate	Ejecuta una compilación completa seguida de una simulación.	Opcional
-create_companion_revision	Crea una revisión para HardCopy.	Opcional
-dont_export_assignments	Evita la exportación de asignaciones que por defecto realiza este comando.	Opcional
-early_timing_estimate	Ejecuta la estimación temporal temprana (<i>Early Timing Estimate</i>).	Opcional
-eco	Ejecuta la opción de compilación ECO (<i>Fitter ECO compilation</i>).	Opcional
-eda_synthesis	Ejecuta una síntesis EDA (<i>EDA Synthesis</i>).	Opcional
-export_database	Exporta una versión compatible de la base de datos del proyecto.	Opcional
-generate_functional_sim_netlist	Genera una red de componentes para simulación funcional.	Opcional
-import_database	Importa una base de datos compatible con la versión del proyecto.	Opcional
-incremental_compilation_export	Exporta la partición de un diseño a un archivo .qxp .	Opcional
-incremental_compilation_import	Importa la partición de un diseño desde un archivo .qxp al proyecto actual.	Opcional
-signalprobe	Ejecuta una compilación "SignalProbe".	Opcional
-vqm_writer	Ejecuta <i>VQM Writer</i> .	Opcional

El comando retorna '0' si se ejecutó correctamente o '1' si existió un error durante su ejecución y no se completó correctamente.

COMPILACIÓN DE PROYECTOS EN QUARTUS II MEDIANTE COMANDOS TCL

El paquete `::quartus::flow` permite la realización de todas las opciones de compilación disponibles en la interfaz gráfica de Quartus II mediante sus íconos de acceso directo y los comandos de la barra de menú. La utilización del lenguaje de comandos Tcl para la realización de estas operaciones permite obtener las ventajas de describir estos flujos de diseño en archivos de comandos.

Ejemplo: compilación de proyecto en Quartus II mediante comandos Tcl

En este ejemplo, se realizará un procedimiento similar al realizado en el Capítulo 5 para la compilación del proyecto creado en la sección anterior mediante los comandos contenidos en

el archivo **tcl_n2sopc.tcl**. El código expuesto podrá anexarse a dicho archivo para que automáticamente compile el proyecto creado.

El primer comando debe cargar el paquete `::quartus::project` definir el comando `execute_flow`:

```
# Carga el paquete ::quartus::flow
# para la compilación de proyectos
load_package ::quartus::project
```

La realización de una compilación completa se realiza posteriormente a abrir el proyecto:

```
# El proyecto es abierto con revisión "n2sopc"
project_open -revision n2sopc n2sopc

# Ejecuta una compilación completa
# execute_flow -compile

# Cierra el proyecto
project_close
```

NOTA: para que la compilación se ejecute correctamente, el directorio del proyecto debe contener todos los archivos de diseño del proyecto. Para la ejecución de este comando, pueden copiarse los archivos creados en los Capítulos 5 y 6.

CONCLUSIONES

La utilización de archivos de comandos Tcl para la realización del flujo de diseño de un proyecto mejora su mantenimiento su documentación y automatización. De esta manera, la especificación de los diferentes pasos de creación de una determinada aplicación en archivos de comandos Tcl reduce significativamente los tiempos de desarrollo de los proyectos y los errores introducidos al modificar las condiciones en las que se debe realizar la compilación del proyecto.

CAPÍTULO 11

UTILIDADES Y ARCHIVOS DE COMANDOS PARA LA GENERACIÓN DE SOPC

INTRODUCCIÓN

La generación de sistemas embebidos basados en dispositivos FPGA requiere la configuración de diversos parámetros acorde a las características que se desean lograr. Todos los fabricantes de dispositivos FPGA que soportan el diseño de sistemas embebidos, ofrecen interfaces gráficas para la configuración de estas características de generación. Estas interfaces gráficas son muy eficientes en las etapas de diseño puesto que permiten un acceso fácil a la configuración de los diferentes parámetros de la aplicación y la ejecución de diferentes utilidades sin un conocimiento detallado de su funcionamiento. Sin embargo, cuando se desea automatizar un determinado proceso de diseño o realizar procedimientos en forma específica, la ejecución de utilidades y configuración de parámetros puede resultar inconveniente o propensa a la introducción de errores.

En este capítulo, se describen las utilidades que brinda la plataforma Nios II y que pueden ser ejecutadas desde la línea de comando o mediante la ejecución de archivos de comandos. La posibilidad de ejecución por líneas de comandos de estas utilidades permite la automatización de diferentes procedimientos de diseño y la fácil interacción con herramientas de diseño de otros fabricantes.

UTILIDADES EN QSYS Y NIOS II SBT

En los Capítulos 5 y 6, se desarrollaron todas las etapas para el diseño de un SOPC y la generación de su software asociado utilizando las plataformas de desarrollo brindada por Altera. Estas etapas fueran realizadas mediante interfaces gráficas (GUI) que permiten una rápida y simple configuración de la aplicación. Sin embargo, estas interfaces gráficas permiten la modificación de valores en archivos de configuración del proyecto de diseño o la ejecución de utilidades mediante el despliegue de menús de la plataforma o la selección de

íconos de acceso directo. En realidad, todas las etapas de diseño descritas en los Capítulos 5 y 6 se realizan mediante la ejecución de estas utilidades que se acceden a través de la interfaz gráfica que ofrece Eclipse para Nios II SBT. Por lo tanto, todas las operaciones que se realizan mediante Eclipse para Nios SBT pueden ser realizadas ejecutando las utilidades en línea de comando.

Por ejemplo, puede ocurrir que se desee tener una configuración del proyecto de aplicación o el proyecto BSP para realizar el desarrollo diferente de la que finalmente se implementará. La configuración de la optimización de la compilación, las consolas de entrada o salida pueden ser convenientes configurarlas de determinada manera para realizar el desarrollo y modificarlas para su implementación final. El cambio entre una configuración y otra puede resultar complejo si se desean modificar varios parámetros entre las diferentes pruebas. En estos casos, es muy conveniente contar con archivos de comandos que ejecuten las utilidades para realizar las modificaciones necesarias en la configuración de la herramienta y generación automática de la aplicación.

UTILIDADES Y ARCHIVOS DE COMANDOS EN QSYS

Muchas de las funciones disponibles en la interfaz gráfica pueden ser realizadas mediante la línea de comandos con utilidades que brinda Qsys. Estas utilidades pueden ejecutarse desde cualquier ventana terminal o consola del sistema operativo desde el directorio:

```
<directorio_de_instalación_de_quartus>\quartus\sopc_builder\bin\
```

Las utilidades disponibles como interfaz de línea de comandos de Qsys son:

- `qsys-script`: permite crear y manipular SOPC desarrollados en Qsys mediante archivos de comandos Tcl.
- `ip-generate`: permite configurar parámetros del SOPC creado en Qsys y generar los archivos de salida para sistemas Qsys o componentes Qsys.
- `ip-make-simscript`: permite crear archivos de simulación y especificar opciones que no están disponibles en la interfaz gráfica de Qsys.

CREACIÓN Y MANIPULACIÓN DE SOPC CON QSYS-SCRIPT

La utilidad `qsys-script` permite crear y manipular los SOPC en Qsys con archivos de comandos Tcl. Al igual que se realiza en la interfaz gráfica de Qsys, debe especificarse la versión que se utiliza. Esta versión determinará también la versión de los componentes que integrarán el SOPC por defecto. La versión debe especificarse mediante la opción **—package-versión** de la utilidad `qsys-script` o estar especificada en el archivo de comandos Tcl cuando se carga el paquete `qsys` (mediante el comando Tcl `package require - exact qsys <version>`).

La utilidad **qsys-script** es copiada, durante la instalación de Quartus II en la carpeta “`carpeta_de_instalacion>\quartus\sopc_builder\bin`”.

La sintaxis del comando es:

```
qsys-script [--cmd=<value>] [--help] [--jvm-max-heap-size=<value>]
  [--package-version=<value>] [--search-path=<value>] [--
  script=<file>] [--system-file=<file>]
```

La utilidad `qsys-script` posee los siguientes argumentos opcionales:

- **--cmd=<value>**: Opcional. <value> es un *string* que contiene comandos Tcl para crear o manipular sistemas Qsys. Si ambos argumentos `-cmd` y `-script` son especificados, entonces los comandos del argumento `-cmd` son ejecutados antes de los especificados con el argumento `-script`.
- **--help**: Opcional. Muestra la ayuda de la utilidad `qsys-script`.
- **--jvm-max-heap-size=<value>**: Opcional. Este parámetro determina el máximo monto de memoria que puede ser asignado a la utilidad `qsys-script` para su ejecución. El parámetro <value> es especificado como <size> <unit> donde <unit> puede ser ‘m’ o ‘M’ para *megabytes* o ‘g’ o ‘G’ para *gigabytes*.
- **--package-version=<value>**: Obligatorio. Especifica la versión del paquete de comandos Tcl que se utilizará. Si no se especifica esta opción, debe especificarse la versión del paquete Tcl en el archivos de comandos Tcl especificados con el argumento `-script` con el comando `package require -exact qsys <version>`.
- **--search-path=<value>**: Opcional. Establece los directorios en donde se buscarán las definiciones de los componentes utilizados en el SOPC creado o manipulado por el comando. Múltiples directorios deben ser separados por comas.
- **--script=<file>**: Opcional. El parámetro <file> contiene un archive de comandos Tcl que se ejecutará para crear o manipular el SOPC utilizando la utilidad **qsys-script** de Qsys.
- **--system-file=<file>**: Opcional. Especifica el archivo **.qsys** del sistema. Si el archivo existe, es cargado antes de ejecutar los comandos Tcl.

La utilidad brinda soporte para la ejecución de comandos Tcl que permiten crear y manipula SOPC realizados en Qsys. Un detalle de todos los comandos es realizado en el Anexo 4.

Un SOPC realizado en Qsys posee las siguientes definiciones:

- **Módulo (*module*)**: los módulos son los bloques funcionales que constituyen un SOPC.
- **Componentes (*component*)**: los componentes son los diferentes elementos que pueden ser elegidos para formar un SOPC. Qsys brinda una biblioteca de

componentes que pueden ser seleccionados para conformar los diferentes módulos del SOPC.

- **Instancia (*instance*):** una instancia es la parametrización de un determinado componente para conformar un determinado módulos del SOPC. La instancia recibe un nombre único con el que será referenciado en el SOPC.
- **Conexión (*connection*):** los módulos de un SOPC se comunican entre sí mediante conexiones definidas por una determinada interfaz de conexión (“Especificación Avalon” en el caso de Altera).
- **Interfaz (*interface*):** las conexiones entre dos módulos pueden ser definidas mediante diferentes interfaz de acuerdo a las características de transferencia entre los módulos. Las diferentes interfaces de la especificación Avalon son listadas en la sección “Bus Avalon” en página 77.
- **Puerto (*port*):** los puertos de una determinada interfaz están conformados por las señales de dicha interfaz. Existen algunos casos en los que una interfaz puede modificar sus puertos de acuerdo a la parametrización que se realiza. Por ejemplo: si una interfaz se configura como solo escritura, entonces las señales de lectura de la interfaz pueden no ser efectuadas, modificando así la implementación de los puertos de dicha interfaz.

Cada componente, módulo, interfaz y puerto posee sus propiedades que pueden ser configuradas y parametrizadas. Estas características permiten mucha flexibilidad ya que el hardware de los diferentes componentes puede ser diseñado en forma genérica y adecuado a las diferentes aplicaciones mediante diferentes parámetros que determinarán cómo serán implementados en el SOPC particular.

Ejemplo de arquitectura de SOPC en Qsys

Para ilustrar las definiciones anteriores, se describe, a continuación, el SOPC implementado en el Capítulo 5 y que se muestra en la Figura 4-2 y consta de los siguientes 5 módulos:

- Módulo **nios2_qsys_0**: es una instancia del componente **Nios II Processor** cuya parametrización es detallada en la configuración realizada en la sección “Selección y configuración del procesador Nios II” (página 92). Este módulo tiene dos interfaces *Avalon Memory Mapped Master* con los nombres **data_master** e **instruction master** que son los puertos maestros de sus respectivas conexiones con otros módulos del SOPC.
- Módulo **memoria_ram**: es una instancia del componente **On-Chip Memory (RAM or ROM)** cuya parametrización es detallada en la configuración realizada en la sección “Selección y configuración de la memoria RAM interna del dispositivo FPGA” (página 94). El módulo tiene un puerto de interfaz **Avalon Memory Mapped Slave** con nombre **s1** que se conecta a las conexiones de los puertos maestros del procesador nios2_qsys_0.
- Módulo **timer_0**: es una instancia del componente **Interval Timer** cuya parametrización es detallada en la configuración realizada en la sección “Selección y configuración del temporizador del sistema (Interval Timer)” (página 97). El módulo posee un puerto de interfaz **Avalon Memory Mapped Slave** con nombre **s1** que se conecta a la conexión del puerto *data master* del procesador nios2_qsys_0.

- Módulo **pio_0**: es una instancia del componente **PIO (Parallel I/O)** cuya parametrización es detallada en la configuración realizada en la sección “Selección y configuración de un puerto de entrada/salida (PIO Parallel I/O)” (página 101). El módulo posee un puerto de interfaz **Avalon Memory Mapped Slave** con nombre **s1** que se conecta a la conexión del puerto *data master* del procesador `nios2_qsys_0` y un puerto *Conduit Endpoint* con nombre **pio_0** con conexión externa al SOPC.
- Módulo **jtag_uart_0**: es una instancia del componente **JTAG UART** cuya parametrización es detallada en la configuración realizada en la sección “Selección y configuración de un puerto de comunicaciones JTAG (JTAG UART)” (página 103). El módulo posee un puerto de interfaz **Avalon Memory Mapped Slave** con nombre **s1** que se conecta a la conexión del puerto *data master* del procesador `nios2_qsys_0`.

Las conexiones de cada módulo pueden ser modificadas de acuerdo a la configuración de las intersecciones mostradas en la columna **Connections** en la interfaz gráfica de Qsys o mediante los comandos Tcl correspondiente mediante la utilidad `qsys-script`.

Un reporte detallado de la arquitectura del SOPC es realizado por Qsys en el archivo `<nombre_del_proyecto>.html` que en el proyecto del Capítulo 5 se encuentra en el archivo “c:\n2sopc\sopcsys.html”.

GENERACIÓN DEL SOPC EN QSYS CON IP-GENERATE

La utilidad `ip-generate` permite configurar diferentes parámetros del SOPC diseñado y generar todos los archivos que produce Qsys. Esta utilidad permite la configuración de todas las opciones que brinda la interfaz gráfica de Qsys y realizar la generación del SOPC en forma similar a la que se produce cuando se selecciona el botón **Generate** en Qsys.

La sintaxis del comando es:

```
ip-generate [--project-directory = <directory>] [--help] [--output-
directory=<directory>] [--file-set = <QUARTUS_SYNTH /
SIM_VERILOG / SIM_VHDL>] [--report-file= <type><filename>] [--
standard-reports] [--search-path =<value>] [--component-
file=<file>] [--component-name=<value>] [--component-
parameter=<value>] [--system-info=<parameter>=<value>]
```

La utilidad `ip-generate` posee los siguientes argumentos opcionales:

- **--project-directory=<directory>**: Opcional. Determina el directorio relativo al directorio del proyecto en el cual los componentes serán buscados. El directorio actual es “.”, para excluir cualquier directorio “”.
- **--output-directory=<directory>**: Opcional. Establece el directorio en donde se escribirán los archivos de salida de la generación. Si el directorio no existe, entonces es creado. En caso de omitirse, se establece el directorio actual como directorio de salida.

- **--file-set=<QUARTUS_SYNTH | SIM_VERILOG | SIM_VHDL>:** Opcional. Especifica el tipo de salida generada. QUARTUS_SYNTH produce una salida HDL que puede ser compilada por Quartus II para ser implementada en un dispositivo FPGA. SIM_VERILOG y SIM_VHDL producen modelos de simulación en los respectivos lenguajes de descripción de hardware.
- **--report-file=<type><filename>:** Opcional. Especifica si el reporte generado será parcial o total y el nombre del archivo (por ejemplo: html:reporte.html). Para generar varios tipos de reportes, se debe especificar esta opción múltiples veces. Los diferentes tipos pueden ser:
 - **bsf:** archivo *Block Symbol File* (.bsf)
 - **html:** *Hypertext Markup Language File* (.html)
 - **qip:** *Quartus II IP File* (.qip)
 - **sip:** *Quartus II Simulation IP File* (.sip)
 - **sopcinfor:** *SOPC Information File* (.sopcinfor)
 - **spd:** *Simulation Package Descriptor File* (.spd)
- **--standard-reports:** Opcional. Produce los archivos de reporte estándar: **.sopcinfor** y **.qip**.
- **--search-path=<value>:** Opcional. Provee el PATH que será utilizado en la generación del SOPC. Pueden agregarse varios directorios separados por comas. Para incorporar el PATH del sistema, debe incluirse el símbolo "\$".
- **--component-file=<file>:** Opcional. Especifica el nombre de un archivo del cual extraer un determinado componente (_hw.tcl) o sistema (.qsys).
- **--component-name=<value>:** Opcional. Especifica el nombre del componente que es instanciado en el archive a extraerlo.
- **--component-parameter=<value>:** Opcional. Asigna un determinado valor a un parámetro del componente. Si se desean múltiples asignaciones, se debe utilizar esta opción múltiples veces.
- **--system-info=<parameter>=<value>:** Opcional. Asigna un determinado valor al algún parámetro del sistema (por ejemplo, familia FPGA a ser implementado). Para múltiples asignaciones, se debe utilizar esta opción varias veces.

La utilidad `ip-generate` permite generar todos los archivos que produce Qsys para Quartus II y para la generación del software.

La generación del hardware de un SOPC es un proceso dinámico en el que cada componente es implementado de acuerdo a la especificación del diseñador y conforme a la arquitectura general del sistema. Este proceso dinámico requiere flexibilidad en la configuración del SOPC para una generación exitosa. Es de esta manera que cada componente de biblioteca tiene un archivo de comandos Tcl, denominado "<nombre_del_componente>_hw.tcl". Este archivo posee comandos que son ejecutados por la utilidad `ip-generate` cada vez que se

instancia en el sistema el componente, para que pueda ser configurado en la arquitectura general del sistema conforme a la especificación del diseñador.

GENERACIÓN DE LOS ARCHIVOS DE SIMULACIÓN DEL SOPC CON IP-MAKE-SIMSCRIPT

La utilidad `ip-make-simscript` permite generar los modelos de simulación del SOPC diseñado.

La sintaxis del comando es:

```
ip-make-simscript [--project-directory = <directory>] [--help] [--  
output-directory=<directory>] [--file-set = <QUARTUS_SYNTH /  
SIM_VERILOG / SIM_VHDL>] [--report-file= <type><filename>] [--  
standard-reports] [--search-path =<value>] [--component-  
file=<file>] [--component-name=<value>] [--component-  
parameter=<value>] [--system-info=<parameter>=<value>]
```

La utilidad `ip-make-simscript` posee los siguientes argumentos opcionales:

- **--spd=<file>**: Obligatorio. El archivo **.spc** describe la lista de archivos a ser compilados y los modelos jerárquicos de memoria.
- **--output-directory=<directory>**: Opcional. Especifica el directorio en el que se alojarán los archivos producidos por la utilidad. Si no es especificado, los archivos son alojados en el directorio en el que se ejecuta la utilidad `ip-make-simscript`.
- **--compile-to-work**: Opcional. Compila todos los diseños a la biblioteca por defecto `“./work”`.
- **--use-relative-paths**: Opcional. Especifica que utilice directorios relativos siempre que no sean especificados en forma absoluta.

UTILIDADES Y ARCHIVOS DE COMANDOS EN NIOS II SBT

En el Capítulo 6, se describen las principales alternativas para la implementación del software del sistema embebido para dispositivos FPGA y el flujo de diseño para su generación. Altera brinda la plataforma **Nios II SBT** para el desarrollo del software, cuya interfaz gráfica está basada en la herramienta de programación **Eclipse para Nios II SBT**. **Eclipse para Nios II SBT** brinda una interfaz gráfica amigable para la configuración de la generación del software que se ejecutará en el sistema embebido.

Por otro lado, **Nios II SBT** ofrece una interfaz de línea de comandos para la ejecución de diferentes utilidades para la configuración y generación de software. Todas las

configuraciones y procesos realizados en **Eclipse para Nios II SBT** pueden ser realizados mediante las utilidades de la línea de comando. Por esto, es muchas veces conveniente utilizar la herramienta **Eclipse para Nios II SBT** para la creación y verificación de las aplicaciones y posteriormente la línea de comandos para la automatización y mantenimiento del proyecto.

ARCHIVOS MAKEFILE EN NIOS II SBT

La configuración y generación de software requiere la realización, en un determinado orden, de diferentes acciones. Nios II SBT utiliza archivos **makefile** para la especificación de dichas acciones tendientes a la configuración y generación de los diferentes proyectos de hardware. Nios II SBT posibilita la generación de tres diferentes tipos de proyectos y, por consiguiente, especifica una definición de archivo **makefile** para cada uno de ellos:

- **Proyecto de aplicación:** consiste en un proyecto que implementa una determinada funcionalidad de acuerdo a la aplicación a la que está destina el sistema. Nios II SBT genera un archivo privado denominado **Makefile** el cual es utilizado para la generación del archivo **.elf** que contiene la compilación de la aplicación.
- **Proyecto de librería:** implementa una colección de funciones que pueden ser reutilizadas en diferentes aplicaciones. Nios II SBT genera un archivo privado denominado **Makefile** y un archivo público denominado **public.mk** los cuales son utilizados para la generación del archivo **.a** que contiene la compilación de la librería.
- **Proyecto BSP:** implementa diferentes librerías para el acceso al hardware de un sistema basado en el procesador Nios II. En su forma básica, implementa una HAL, pero opcionalmente puede incluir un sistema operativo y paquetes de software para el soporte de protocolos de comunicaciones, entre otros. Nios II SBT genera un archivo privado denominado **Makefile**. La generación de un proyecto BSP produce una librería **.a**, un archivo de encabezados (*headers*) (**system.h**) y un archivo de enlace **linker.x**.

Cada tipo de proyecto y, en consecuencia, cada tipo de archivo **makefile**, tiene asociado en Nios II SBT diferentes utilidades para su creación y modificación. Los archivos **makefile** pueden ser cerrados o modificados mediante diferentes procesadores de texto, pero sin embargo es conveniente utilizar las herramientas que brinda Nios II SBT para preservar las estructuras de los mismos y, en consecuencia, su documentación y mantenimiento.

ARCHIVOS DE COMANDOS Y UTILIDADES DE LÍNEA DE COMANDOS DE NIOS II SBT

Las siguientes son las diferentes utilidades y archivos de comandos (*script*) de línea de comandos que brinda Nios II SBT para la configuración y generación de software.

- **nios2-app-generate-makefile:** (utilidad) genera un archivo **makefile** (denominado “Makefile”) para un proyecto de aplicación. La utilidad requiere un directorio en donde se encuentre un proyecto BSP generado para determinar la arquitectura del hardware del sistema embebido.
- **nios2-lib-generate-makefile:** (utilidad) genera un archivo **makefile** (denominado **Makefile**) para un proyecto de librería.
- **nios2-bsp-create-settings:** (utilidad) crea un archivo de configuración de un proyecto BSP con los valores por defecto.
- **nios2-bsp-generate-files:** (utilidad) genera todos los archivos del proyecto BSP en la carpeta BSP del proyecto.
- **nios2-bsp-query-settings:** (utilidad) provee información de un archivo **.bsp**. El comando no modifica ninguna información del archivo.
- **nios2-bsp-update-settings:** (utilidad) permite modificar la configuración de un archivo **.bsp** existente.
- **nios2-bsp-editor:** (utilidad) habilita una interfaz gráfica para la creación y edición de diseños BSP.
- **nios2-app-update-makefile:** (utilidad) actualiza el archivo **makefile** de un proyecto de aplicación para agregar o eliminar archivos de código al proyecto.
- **nios2-lib-update-makefile:** (utilidad) actualiza el archivo **makefile** de un proyecto de librería para agregar o eliminar archivos de código al proyecto.
- **nios2-swexample-create:** (utilidad) crea una plantilla de ejemplo para un determinado SOPC.
- **nios2-elf-insert:** (utilidad) permite insertar información específica al archivo **.elf** del diseño.
- **nios2-elf-query:** (utilidad) permite consultar información específica de un archivo **.elf** del diseño.
- **nios2-convert-ide2sbt:** (utilidad) convierte un proyecto en Nios II IDE a Nios II SBT.
- **nios2-flash-programmer-generate:** (utilidad) permite convertir diferentes formatos de archivos al formato S de Motorola y programar dispositivos *flash*.
- **nios2-c2h-generate-makefile:** (utilidad) crea un archivo **makefile**, denominado **c2h.mk** que especifica todas las opciones de todos los aceleradores especificados para una aplicación.
- **nios2-bsp:** (utilidad) permite crear o modificar la configuración de un archivo BSP ejecutando la utilidad **nios2-bsp-create-settings** o **nios2-bsp-generate-files**. Nios II SBT soporta dos tipos de BSP: **hal** y **ucosii**.
- **nios2-bsp-console:** (utilidad) abre una consola interactiva para la ejecución de comandos Tcl o archivos de comandos Tcl.

- **create-this-bsp:** (archivo de comandos) es un archivo de comandos de alto nivel que genera el proyecto BSP para un determinado diseño de hardware.
- **create-this-app:** (archivo de comandos) es un archivo de comandos de alto nivel que crea un proyecto de aplicación para un diseño de hardware específico.

A continuación, se describen los principales comandos Tcl que pueden utilizarse para configurar un proyecto BSP.

COMANDOS TCL PARA LA CONFIGURACIÓN DEL PROYECTO BSP

En el Capítulo 6, se describieron algunas de las diferentes opciones de configuración que pueden especificarse para el proyecto BSP. Como se detalló, el proyecto BSP implementará una HAL o un sistema operativo que actuará como interfaz entre la aplicación y el hardware del sistema. La generación del proyecto BSP implementará esta interfaz de acuerdo a los recursos de hardware que posee el sistema (extraídos del archivo **.sopcinfo** generado por Qsys) y realizará la definición de dispositivos para poder acceder a través de la aplicación. La configuración del proyecto BSP permite determinar diferentes maneras en la implementación de estas interfaces (mediante la selección de diferentes controladores de dispositivos) y la especificación de los diferentes recursos accedidos por el software de aplicación (temporizadores, consolas de entrada y salida, etc.).

La creación y manipulación de archivos de configuración **.bsp** para proyectos BSP se realiza mediante las utilidades **nios2-bsp-create-settings**, **nios2-bsp-update-settings**, y **nios2-bsp-query-settings**. Un detalle de los diferentes comandos Tcl soportados por estas utilidades para la configuración de proyectos BSP es dado en el Anexo 5.

GENERACIÓN DEL PROYECTO BSP Y COMANDOS TCL “CALLBACKS”

La generación de un proyecto BSP debe tener en cuenta, además de su configuración, las particularidades propias de cada uno de los módulos incluidos en el SOPC. A su vez, las instancias de los componentes que conforman cada uno de los módulos pueden ser parametrizados de acuerdo a la configuración realizada en el proyecto BSP.

De esta manera, cada componente instanciado en el SOPC debe tener la posibilidad de parametrización conforme a la configuración que el diseñador realizó para el SOPC. Es por esta razón, que el controlador de cada componente (*device driver*) posee un archivo de comandos que es ejecutado cada vez que el proyecto BSP encuentra una instancia de dicho componente. La parametrización de la instancia del componente se efectúa entonces mediante la ejecución de un archivo de comandos Tcl asociado al componente y que se denomina **<nombre_del_componente>_sw.tcl**.

La ejecución de los comandos del archivo **<nombre_del_componente>_sw.tcl** durante la generación del proyecto BSP posibilita la parametrización de la instancia del componente a la configuración del hardware o a la configuración del proyecto BSP.

Los comandos Tcl contenidos en el archivo `<nombre_del_componente>_sw.tcl` se denominan funciones Tcl *callback* y las modificaciones que realizan en la configuración del proyecto BSP durante su generación son insertadas en el archivo `settings.bsp` del proyecto.

Las funciones Tcl *callback* pueden ser ejecutadas en cada una de las siguientes etapas de la generación del proyecto BSP:

- Inicialización del proyecto BSP
- Generación del proyecto BSP
- Validación del proyecto BSP

Por otro lado, las acciones que pueden ser realizadas por una función *callback* pueden tener alcance en la **clase** del componente o en su **instancia**.

Una de las principales funciones *callback* definidas en el archivo `<nombre_del_componente>_sw.tcl` del componente es la inserción de los archivos fuentes necesarios para la generación de los controladores para dicho componente el proyecto BSP. Un detalle de las funciones Tcl *callback* soportados por Nios II SBT se encuentra en (28).

HERRAMIENTAS DE LÍNEA DE COMANDO GNU PARA NIOS II (CONSOLA “BASH”)

La compilación de proyectos en Nios II SBT está basada en las herramientas GNU del compilador gcc. Estas herramientas son de código abierto y han sido adaptadas (“ported”, en inglés) al procesador Nios II de Altera, creando versiones específicas de cada una de ellas para dicho procesador.

Luego de la configuración del proyecto BSP y del proyecto de aplicación, se puede compilar todo el software mediante la ejecución del comando **make**, el cual ejecuta los diferentes comandos y configuraciones de los archivos **Makefile**.

Un archivo **Makefile** puede contener diferentes procedimientos de acuerdo a los procesos que se desea realizar. Cada uno de estos procedimientos dentro del archivo **Makefile** puede ser ejecutado mediante la especificación del correspondiente **target** cuando se invoca el comando **make**. Por ejemplo, `make clean` elimina todos los archivos objetos generados en compilaciones anteriores para que la próxima compilación genere todos los archivos objetos nuevamente. En Windows, el directorio que contiene la utilidad `make` es `<directorio_altera>\quartus\bin\cygwin\bin\`.

Sin embargo, todas las herramientas de línea de comando GNU están basadas en una consola de comandos nativa en UNIX denominado *bash* (*Bourne again shell*). Esta consola de comandos interpreta cada uno de los comandos introducidos por el usuario y los ejecuta. Nios II SBT brinda esta consola de comandos en la aplicación:

- En Windows, si se desea ejecutar la consola en forma interactiva:

```
<directorio_altera>\nios2eds\Nios II Command Shell.bat
```

- En Windows, si se desea ejecutar un comando:

```
"<directorio_altera>\nios2eds\Nios II Command Shell.bat" <comando>
```

— En Windows, si se desea ejecutar un archivo de comandos:

```
"<directorio_altera>\nios2eds\Nios II Command Shell.bat"  
<archivo_comandos.sh>
```

— En Linux, si se desea ejecutar la consola en forma interactiva:

```
<directorio_altera>\nios2eds\nios2_command_shell.sh
```

— En Linux, si se desea ejecutar un comando:

```
<directorio_altera>\nios2eds\nios2_command_shell.sh <comando>
```

— En Linux, si se desea ejecutar un archivo de comandos:

```
<directorio_altera>\nios2eds\nios2_command_shell.sh  
<archivo_comandos.sh>
```

UTILIDADES GNU PARA NIOS II DE ALTERA

Altera ha adoptado para su proceso de generación y depuración de software las utilidades de código abierto GNU basadas en el compilador GCC y la biblioteca newlib. Todos estas utilidades pueden ser ejecutadas desde la consola de comandos **bash** brindada por Altera, ya sea en forma iterativa o mediante la interpretación de un archivo **Makefile**. Un detalle de todas las utilidades GNU disponibles para Nios II de Altera son brindadas en el Capítulo 4 de (21). A continuación, se detallarán las utilidades más frecuentemente utilizadas por el diseñador.

nios2-elf-gcc y nios2-elf-g++

Las utilidades nios2-elf-gcc y nios2-elf-g++ son los compiladores GNU de C y C++ para el procesador Nios II de Altera, respectivamente. Su sintaxis es de la forma siguiente:

```
nios2-elf-gcc <opciones> -o <archivos_objetos> <archivos_C>  
nios2-elf-g++ <opciones> -o <archivos_objetos> <archivos_C>
```

El compilador brinda como salida un archivo ejecutable en formato **.elf**.

nios2-elf-objdump

La utilidad nios2-elf-objdump muestra la información de la estructura del archivo objeto, generalmente un archivo del formato **.elf**. Su sintaxis es de la forma siguiente:

```
nios2-elf-objdump -D -S -x <proyecto>.elf <proyecto>.elf.objdump
```

CREACIÓN DE ARCHIVOS DE INICIALIZACIÓN DE MEMORIA

En la sección “Creación de los archivos de inicialización de memoria interna” en la página 153, se detalló el procedimiento para la generación de los archivos de inicialización de memoria del SOPC. Estos archivos deben ser incorporados al proyecto de Quartus II para que efectivamente sean considerados para la inicialización del contenido de las memorias correspondientes cuando se configura el hardware del SOPC.

Para crear tales archivos de inicialización de memoria desde la línea de comandos, se puede invocar el mismo archivo de comandos desde la consola de comandos de Nios II mediante:

```
make mem_ini_generate
```

Este comando crea y genera los archivos de inicialización y simulación para todos los dispositivos de memoria del SOPC. También genera el archivo de IP para Quartus II “**.qip**”. El archivo **.qip** contiene la información para que Quartus II encuentre los archivos de inicialización correspondientes y debe ser incluido en el proyecto de Quartus II.

CONCLUSIONES

En este capítulo, se describieron las diferentes utilidades de línea de comando que ofrece Altera para la realización del flujo de diseño. Estas utilidades permiten una mayor automatización de los procedimientos y configuraciones involucradas en el diseño lo que posibilita una mayor documentación y mantenimiento de los proyectos. De esta manera, todas las etapas de creación, configuración y generación de un SOPC pueden almacenarse en archivos de comandos en lugar de requerir la ejecución de diferentes comandos en una interfaz gráfica. Además, posibilita la integración con otras herramientas de diseño de diferentes fabricantes.

REFERENCIAS

1. **Heath, Steve.** *Embedded System Design (2nd Edition)*. Oxford: Newnes, 2003. pág. 430. 0-7506-5546-1.
2. *Managing the Development of Large Software Systems.* **Royce, Winston W.** : IEEE CS Press, 1970. Proc. Westcon. Vol. c, págs. 328-339.
3. *Operational Prototyping: A new development approach.* **Davis, Alan M.** Colorado Springs: IEEE Software, September de 1992, págs. 70-78.
4. **Wakerly, John F.** *Digital Design: Principles and Practices Package (4th Edition)*. s.l.: Prentice Hall , 2005. pág. 928. 0-13-173349-4.
5. **Altera Corporation.** *MAX7000 Programmable Logic Device Family*. San Jose: s.n., 2005.
6. **Altera Coporation.** *Cyclone III Device Handbook*. San Jose: Altera, 2012. Device Handbook. CIII5V1-4.2.
7. **IEEE Computer Society.** *IEEE Standard 1364-2005 for Verilog Hardware Description Language*. New York: IEEE Computer Society, 2006. pág. 590.
8. *A Signed Binary Multiplication Technique.* **Booth, Andrew.** 1951, Journal Mechanic and Applied Mathematic, págs. 236-240.
9. **Altera Corporation.** *Quartus II Handbook Version 13.0.* [ed.] Altera. San Jose, CA, USA: Altera, November de 2013. II5V1-13.0.0.
10. **Altera Corporation.** *Altera Quartus II Software v13.0 — Subscription Edition vs. Web Edition.* [http://www.altera.com/literature/po/ss_quartussevswe.pdf] San Jose, USA: SS-1006-4.0, March de 2013.
11. **IEEE.** *1666-2011 - IEEE Standard for Standard SystemC Language Reference Manual.* IEEE. New York: IEEE Computer Society, 2012. pág. 638, Standard. 978-0-7381-6801-2.
12. **Lattice Semiconductor Corporation.** *LatticeMico32 Processor Reference Manual*. Hillsboro, OR, USA: s.n., December de 2011.

13. **Altera Corporation.** *Nios II Processor Reference: Handbook*. San Jose: Altera, 2011. pág. 284.
14. **Altera Corporation.** *Nios II C2H Compiler - User Guide*. San Jose: Altera, 2009. pág. 138.
15. **Altera Corporation.** *Avalon Interface Specifications*. [ed.] Altera. San Jose, CA, USA: Altera, May de 2013. MNL-AVABUSREF-2.1.
16. **Altera Corporation.** *SOPC Builder User Guide*. Altera. San Jose, CA, USA: s.n., December de 2012.
17. **Kernigham, Brian W. y Rotchie, Dennis M.** *The C Programming Language*. s.l.: Prentice Hall, 1988. 0-13-110362-8.
18. **Altera Corporation.** *Nios II Software Developer's Handbook*. San José, CA, USA: s.n., May de 2011.
19. **Altera Corporation.** *Embedded Peripherals IP: User Guide*. San Jose: Altera Corporation, 2011. UG-01085-11.0.
20. **Altera Corporation.** *Profiling Nios II Systems*. San Jose: Altera, 2011. pág. 20, Application Note 391. AN-391-3.0.
21. **Altera Corporation.** *Embedded Design Handbook*. San Jose: Altera Corporation, 2011. ED_HANDBOOK-2.9.
22. **Altera Corporation.** *Alternative Nios II Boot Methods*. San Jose: Altera Corporation, 2011. Application Note. AN-458-2.1.
23. **Source, Open.** <http://www.denx.de/wiki/U-Boot/>. [En línea] denx software engineering.
24. **Association, SD Card.** <https://www.sdcard.org/home/>. [En línea]
25. **Altera Corporation.** *Simulating Nios II Embedded Processor Designs*. San Jose: Altera, 2011. págs. 1-5, Application Note. AN-351-1.3.
26. **Altera Corporation.** *Avalon Verification IP Suite: User Guide*. San Jose: Altera, 2013. págs. 1-196. UG-01073-3.2.
27. **Altera Corporation.** *Quartus II Scripting Reference Manual: For Command-Line Operation & Tool Command Language (Tcl) Scripting*. San Jose: Altera, 2013. MNL-Q2101904-9.1.1.
28. **Altera Corporation.** *Nios II Software Build Tools Reference*. San Jose: Altera, 2011. pág. 128. NI52016-11.0.0.
29. **Red Hat Incorporated, y otros.** *newlib*. [En línea] <http://sourceware.org/newlib/>.

ANEXO 1

INSTALACIÓN DE LINUX CENTOS-5

El sistema operativo CentOS-5 es una de las versiones de Linux recomendada por Altera para la instalación de su software de desarrollo Quartus II en versión de 32 bits. En este anexo, se describirá los pasos necesarios para la instalación de CentOS-5 con todas las dependencias necesarias para la instalación del software de desarrollo de Altera en sus versiones libres (Quartus II Web Edition versión 13, Nios II EDS y ModelSim-Altera Starter). La versión de instalación CentOS-5 puede obtenerse de su sitio oficial: www.centos.org.

Los requerimientos computacionales sugeridos para ejecutar el sistema operativo y los paquetes de desarrollo de Altera son:

- Procesador Pentium Dual Core o superior
- Memoria RAM de 1 GB o superior
- Monitor con resolución de 1024x768 o superior
- Disco rígido de 40 GB o superior
- Mouse y teclado
- DVD-ROM player

Pasos para la instalación

- Colocar el DVD-ROM con la versión 5 de CentOS en el DVD-ROM player. Iniciar la computadora personal y seleccionar la opción de arranque de DVD-ROM. La versión de CentOS iniciará automáticamente.

La pantalla de presentación permite seleccionar el modo en que se realizará la instalación, Presionar la tecla <Intro> para seleccionar el modo gráfico de instalación.

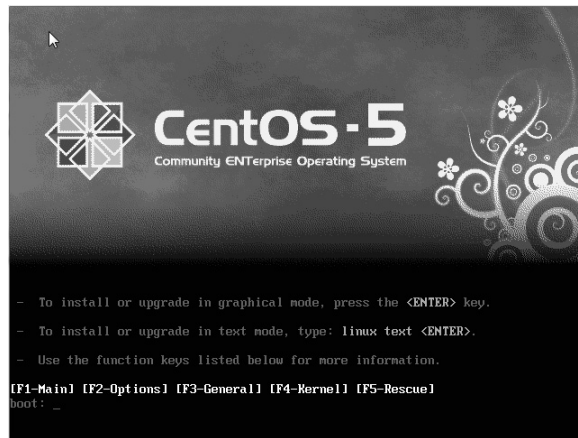


Figura A1-1

Antes de cambiar a modo gráfico, el proceso de instalación continúa en modo texto por algunas pantallas. Las opciones que se presentan en las pantallas en modo texto se pueden modificar mediante la tecla de tabulación (<TAB>) y la selección se realiza mediante la barra espaciadora.

En la siguiente pantalla, se puede elegir comprobar si la distribución con la que se cuenta está corrupta o en condiciones de continuar el procesor. Para agilizar el proceso de instalación, se saltea esta comprobación, seleccionando la opción **Skip**.



Figura A1-2

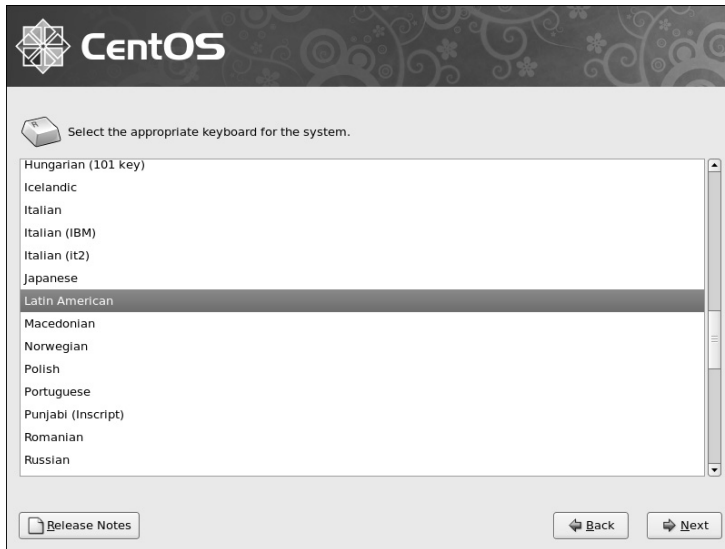
Luego de la selección, la instalación se realizará con soporte de *mouse* en modo gráfico. En la pantalla de bienvenida, se debe hacer clic en **Next**.

**Figura A1-3**

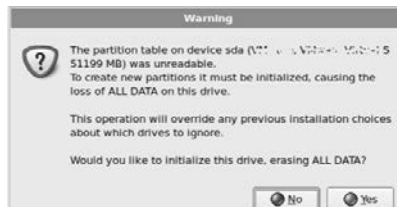
En el próximo paso, se debe seleccionar el lenguaje que se desea utilizar durante la instalación. Para compatibilidad con el software de Altera, se realizará la instalación seleccionando **English (English)** como idioma deseado. Luego hacer clic en **Next**.

**Figura A1-4**

En el paso siguiente, seleccionar la distribución de teclado apropiada de acuerdo al teclado con que se cuente y hacer clic en **Next**.

**Figura A1-5**

Posteriormente, la instalación verificará los discos rígidos en donde podrá instalarse. En caso de que no se encuentre ningún disco rígido formateado, la instalación consultará si se desea crear una partición, advirtiéndole que todos los datos serán eliminados. Seleccionar **Yes** para crear una partición nueva, eliminando cualquier información previa en el disco rígido.



Seleccionar **Remove Linux partitions on selected drives and create default layout** y hacer clic en **Next**.

**Figura A1-6**

Aparecerá una ventana de verificación para confirmar que todos los datos del disco rígido serán eliminados mediante selección. Seleccionar **Yes** para confirmar.



Deberá configurarse la conexión a Internet. La información suministrada en esta pantalla depende de la configuración de la red a la que está conectada la computadora personal. En este ejemplo, dejaremos por defecto la opción de adquisición automática de los parámetros de configuración (la red debe contar con un servidor de configuración automática para que esta opción funcione correctamente). Hacer clic en **Next**.



The image shows the CentOS Network Configuration window. At the top is the CentOS logo. Below it, the 'Network Devices' section contains a table with columns: 'Active on Boot', 'Device', 'IPv4/Netmask', and 'IPv6/Prefix'. The first row shows 'eth0' with 'DHCP' and 'Auto' settings. An 'Edit' button is to the right. Below the table is a 'Hostname' section with the instruction 'Set the hostname:' and two radio buttons: 'automatically via DHCP' (selected) and 'manually' (with a text field containing 'localhost.localdomain' and a hint '(e.g., host.domain.com)'). A 'Miscellaneous Settings' section follows with fields for 'Gateway:', 'Primary DNS:', and 'Secondary DNS:'. At the bottom are buttons for 'Release Notes', 'Back', and 'Next'.

Figura A1-7

Posteriormente, se debe seleccionar la zona horaria de la región que corresponda y hacer clic en **Next**.



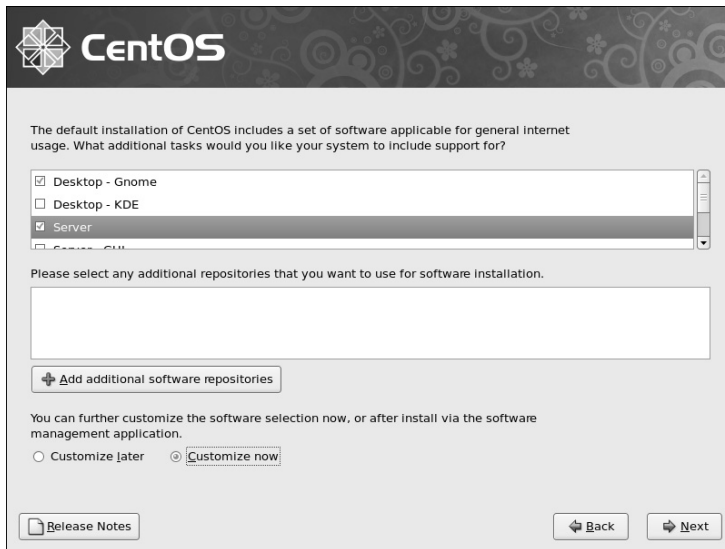
The image shows the CentOS Time Zone Selection window. It features the CentOS logo at the top. The main instruction is 'Please click into the map to choose a region:'. Below this is a world map. Under the map is a dropdown menu showing 'America/Mexico_City' and the text 'Central Time - most locations'. A checkbox labeled 'System clock uses UTC' is checked. At the bottom are buttons for 'Release Notes', 'Back', and 'Next'.

Figura A1-8

En la próxima ventana, debe definirse la clave (*password*) del usuario administrador (*root*). En este ejemplo, se utilizará “altera” como clave que debe ser ingresada en ambos cuadros de texto. Hacer clic en **Next**.

**Figura A1-9**

Luego de un proceso de configuración automática, la instalación permite seleccionar paquetes de aplicaciones para diferentes configuraciones de Linux. En nuestro ejemplo, seleccionaremos “Desktop – Gnome” y “Server” y marcaremos la selección **Customize now** para realizar la tarea de configuración en próximas pantallas. Hacer clic en **Next**.

**Figura A1-10**

En la pantalla de selección de paquetes, seleccionamos **Development** para seleccionar **Development Libraries**, **Development Tools** y **X Software Development**.



Figura A1-11

En la categoría de **Base System**, seleccionar **Administration Tools** y **System Tools**, siempre dejando las opciones por defecto sin modificar. Hacer clic en **Next**.

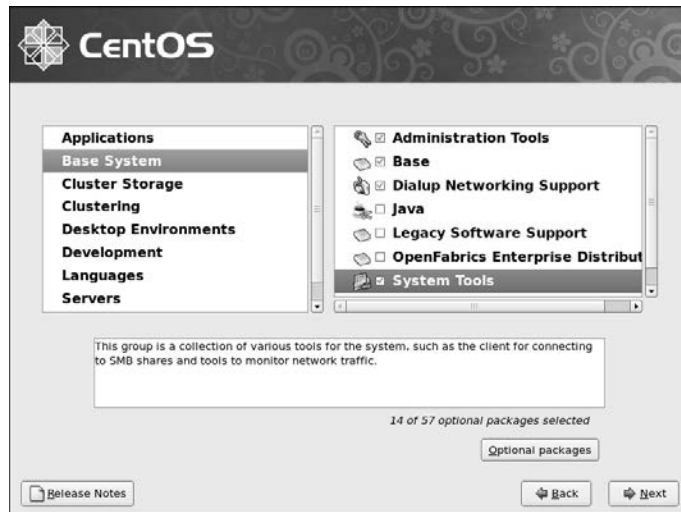


Figura A1-12

Luego de un pequeño proceso de configuración, el sistema nos indica que está listo para la instalación. Hacer clic en **Next**.

**Figura A1-13**

El proceso entrará ahora en la etapa de instalación en el cual dará formato al sistema de archivos, transferirá la información necesaria y ejecutará los procesos para la instalación del sistema. Este proceso puede tardar varios minutos dependiendo de las características de la computadora personal utilizada. Al finalizar la instalación, el proceso indica que debe reiniciar la computadora personal. Retirar el DVD-ROM de instalación del DVD-ROM player y hacer clic en **Reboot**.

**Figura A1-14**

El sistema se reiniciará y comenzará la ejecución del Linux instalado en la computadora. Una pantalla de bienvenida nos indicará que restan algunas configuraciones necesarias para que el nuevo sistema funcione correctamente. Hacer clic en **Forward**.

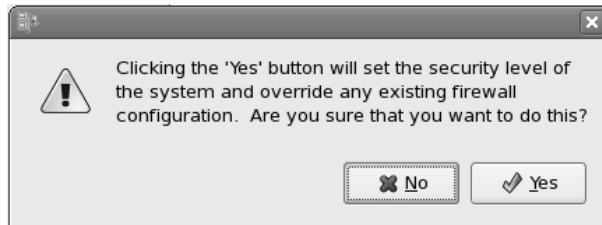


Figura A1-15

La primera pantalla de configuración es la del Firewall. En esta pantalla, se deben habilitar los servicios que consideraremos confiables y que el Firewall no bloqueará. Seleccionaremos **Samba** (para compartir archivos con red de Microsoft) y **Telnet** (para consola remota). Dejaremos seleccionado el servicio **SSH** marcado por defecto. Hacer clic en **Forward**.

**Figura A1-16**

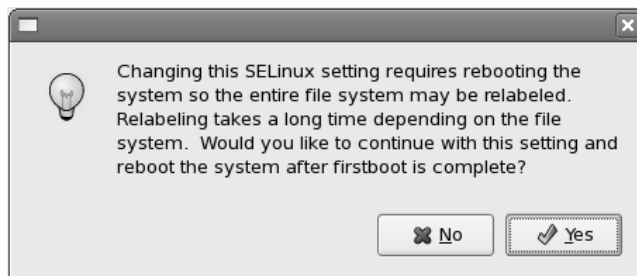
Una pantalla de verificación nos pedirá confirmar que deseamos desbloquear en el Firewall los servicios seleccionados en la pantalla anterior. Hacer clic en **Yes**.

**Figura A1-17**

La configuración de la instalación continuará con la pantalla de SELinux. Esta característica deberá ser deshabilitada para evitar inconvenientes en la instalación de aplicaciones. Seleccionar la opción **Disabled** y hacer clic en **Forward**.

**Figura A1-18**

Confirmar con **Yes** en la pantalla de verificación.

**Figura A1-19**

La próxima pantalla permite configurar un sistema de seguimiento de fallas “Kdump” del sistema operativo. Dejaremos esta opción por defecto. Hacer clic en **Forward**.



Figura A1-20

La siguiente pantalla permite configurar la fecha y hora del sistema. Configurar correctamente los valores y hacer clic un **Forward**.

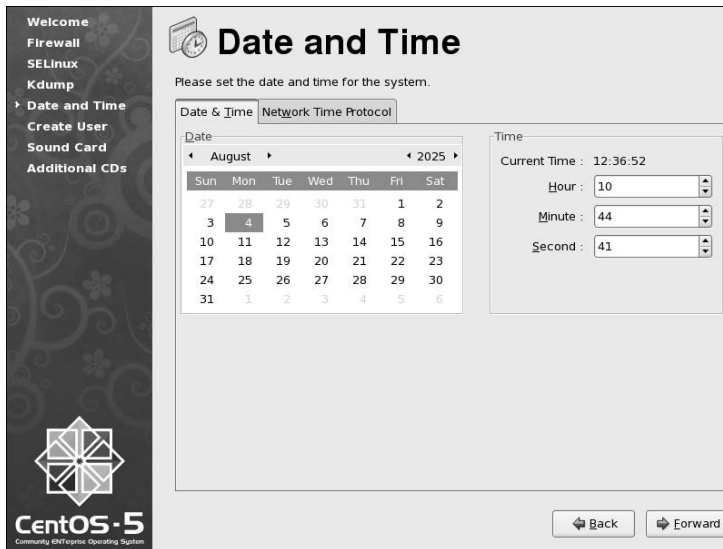


Figura A1-21

La configuración del sistema continúa con la creación de usuario. Para este ejemplo, crear un usuario (*username*) “altera” con clave (*password*) “altera”. Hacer clic en **Forward**.



CentOS-5
Community Enterprise Operating System

Welcome
Firewall
SELinux
Kdump
Date and Time
➤ **Create User**
Sound Card
Additional CDs

Create User

It is recommended that you create a 'username' for regular (non-administrative) use of your system. To create a system 'username,' please provide the information requested below.

Username:

Full Name:

Password:

Confirm Password:

If you need to use network authentication, such as Kerberos or NIS, please click the Use Network Login button.

[Use Network Login...](#)

[Back](#) [Forward](#)

Figura A1-22

La pantalla siguiente permite configurar la placa de sonido que posee la computadora. Seleccionar las opciones adecuadas de acuerdo a la placa disponible o dejar las selecciones por defecto. Hacer clic en **Forward**.



CentOS-5
Community Enterprise Operating System

Welcome
Firewall
SELinux
Kdump
Date and Time
Create User
➤ **Sound Card**
Additional CDs

Sound Card

An audio device has been detected in your computer.

Click the "Play" button to hear a sample sound. You should hear a series of three sounds. The first sound will be in the right channel, the second sound will be in the left channel, and the third sound will be in the center.

The following audio device was detected.

Selected card

Vendor: Ensoniq
Model: ES1371 [AudioPCI-97]
Module: snd-ens1371

Sound test

... Stopped ... ☐ Repeat

Volume settings

Device settings

PCM device

[Back](#) [Forward](#)

Figura A1-23

Posteriormente, la instalación permitirá seleccionar la posibilidad de incluir DVD-ROM de instalación con aplicaciones adicionales. En este ejemplo, no será necesario. Hacer clic en **Finish**.



Figura A1-24

Hacer clic en **OK** en la ventana de confirmación en la que informa que el sistema deberá ser reiniciado.

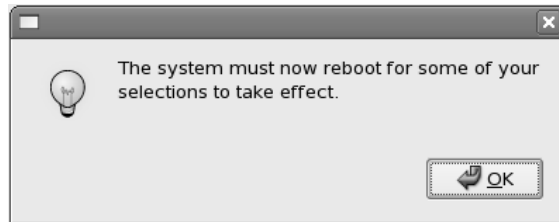
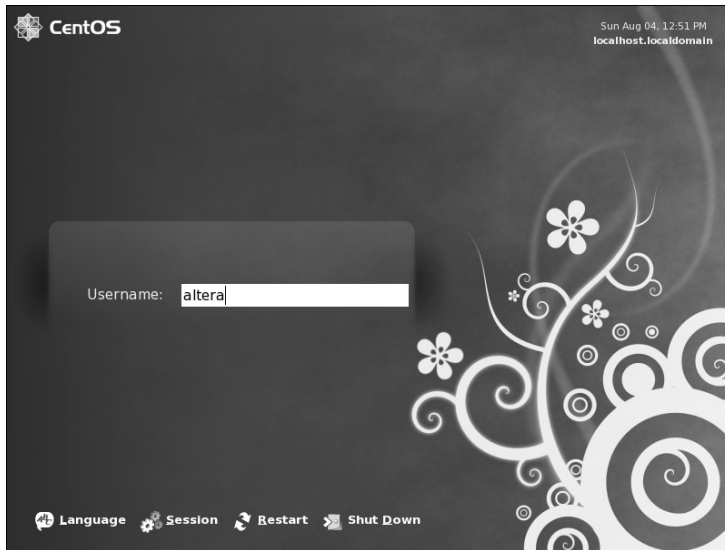
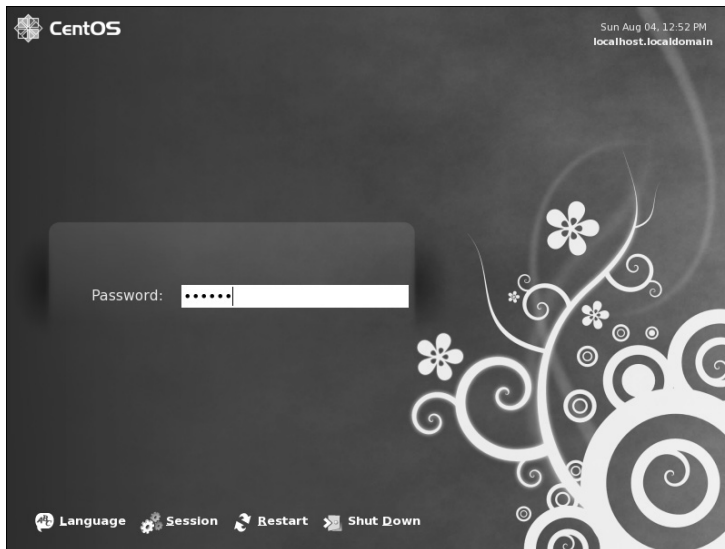


Figura A1-25

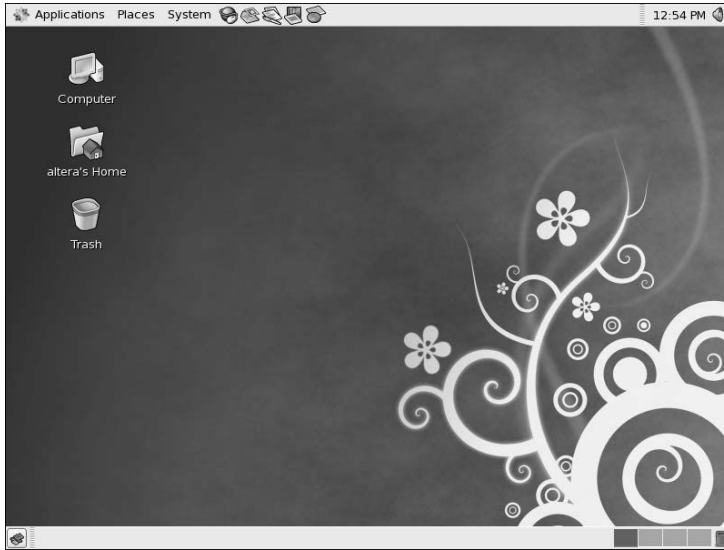
En este punto, toda la configuración de instalación se ha realizado y el sistema permite el acceso a través de la pantalla de inicio. Ingresar en el campo “Username” (nombre de usuario) “altera” y presionar la tecla <Intro> (<Enter>).

**Figura A1-26**

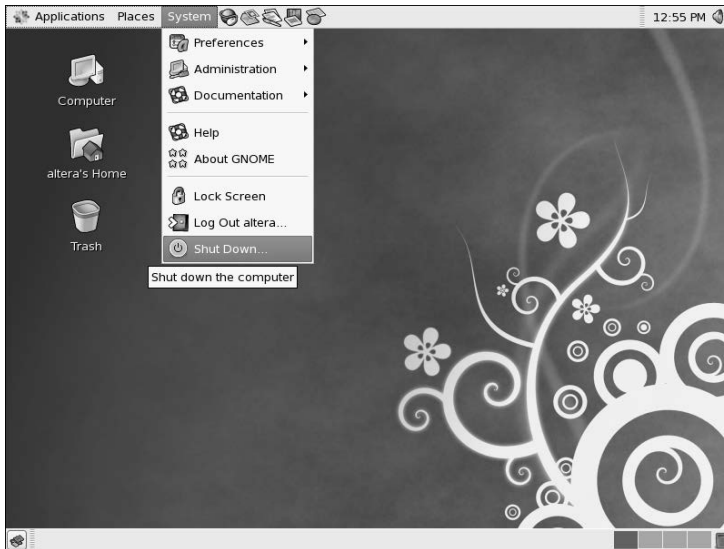
El sistema requerirá el ingreso de la clave (*password*). Ingresar “altera” en el campo correspondiente y presionar la tecla <Intro> (<Enter>).

**Figura A1-27**

El sistema mostrará el escritorio del usuario “altera”, indicando que la instalación ha sido realizada correctamente.

**Figura A1-28**

El sistema ya a sido configurado con Linux CentOS-5 y está listo para instalar las aplicaciones que brinda Altera para desarrollar aplicaciones. Cuando se desea apagar el sistema, debe hacerse mediante la selección de **Shutdown** del menú **System**.

**Figura A1-29**

El sistema solicitara una confirmación antes de proceder con el apagado. Hacer clic en **Shutdown**.

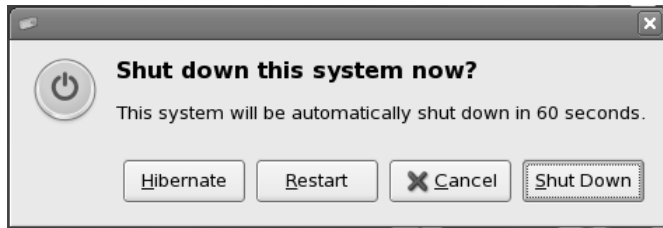


Figura A1-30

Luego de realizar todo el proceso de finalización de servicios, el sistema se apagará. Cuando se encienda, el sistema requerirá otra nuevamente el usuario y clave.

ANEXO 2

INSTALACIÓN DE QUARTUS II WEB EDITION VERSIÓN 13

En este anexo, se describe la instalación del software de desarrollo Quartus II Web Edition versión 13, que es libremente disponible del sitio web de Altera (www.altera.com).

PASOS PARA LA INSTALACIÓN

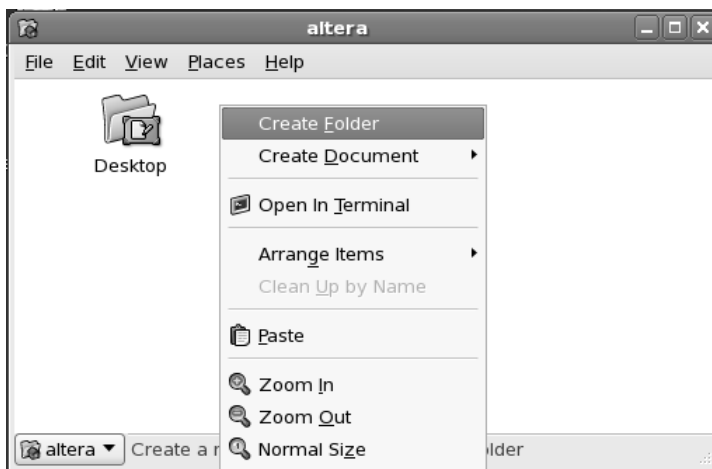
Ingresar con el usuario “altera” al sistema operativo y esperar a que aparezca el escritorio del usuario.

Abrir el explorador de archivos mediante el comando **Home Folder** del menú **Places**.

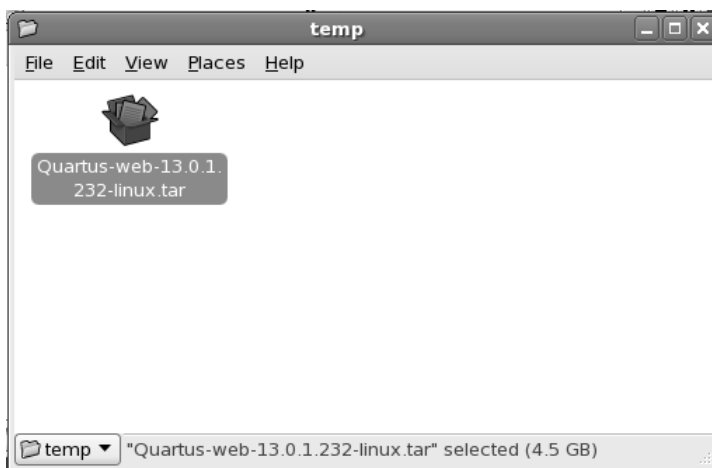


Figura A2-1

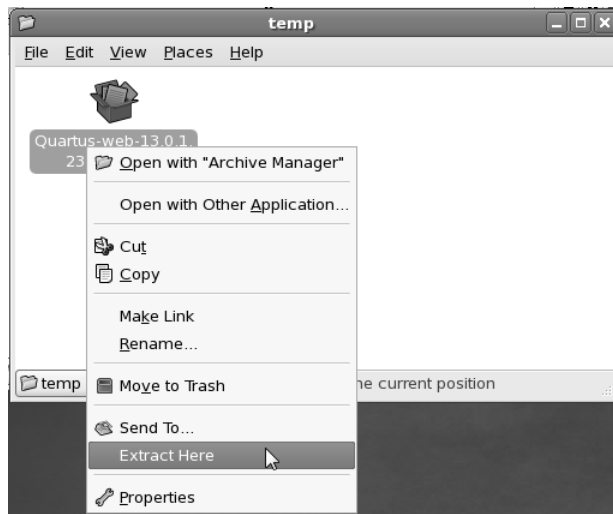
En el explorado de archivos, crear un directorio “altera” mediante un clic con el botón derecho del *mouse* y seleccionado el comando **Create Folder**.

**Figura A2-2**

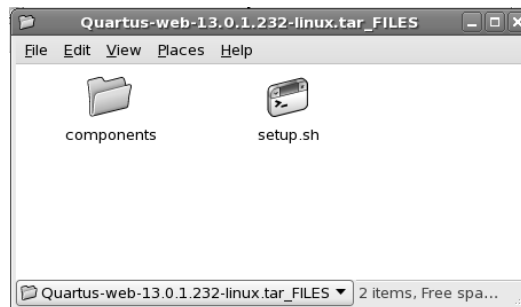
Ingresa a la carpeta “altera” recién creada y mediante el mismo procedimiento cree una carpeta con nombre “temp” e ingrese a ella. Transfiera a la carpeta “temp” el archivo descargado de la página de Altera.

**Figura A2-3**

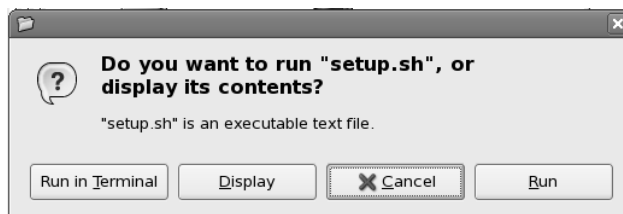
En la ventana de exploración de archivos, seleccionar el archivo de Quartus II y con el botón derecho del mouse seleccionar el comando **Extract Here** del menú desplegable.

**Figura A2-4**

El proceso de extracción requerirá un tiempo para completarse, pero no requerirá atención del usuario. Una vez finaliza la extracción de archivos, abrir la carpeta creada durante la extracción.

**Figura A2-5**

Hacer doble clic sobre el archivo **“setup.sh”** que abrirá la pantalla de opción sobre su ejecución. Seleccionar el botón **Run**.

**Figura A2-6**

El programa de instalación comenzará a ejecutarse, mostrando el mismo proceso que el realizado cuando se instala en un sistema operativo Microsoft Windows.

La primera pantalla es de bienvenida al proceso de instalación a la cual solo debe seleccionarse el botón **Next >**.

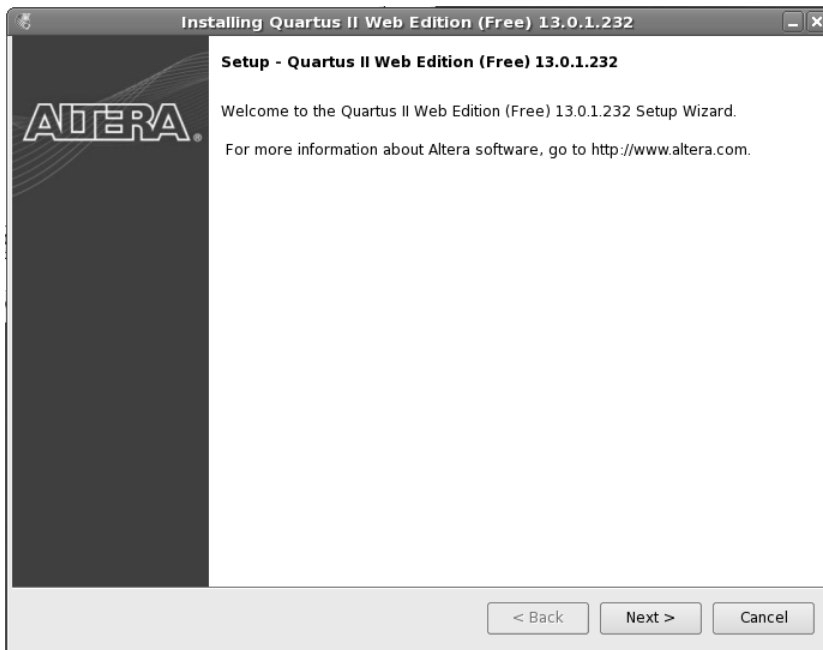
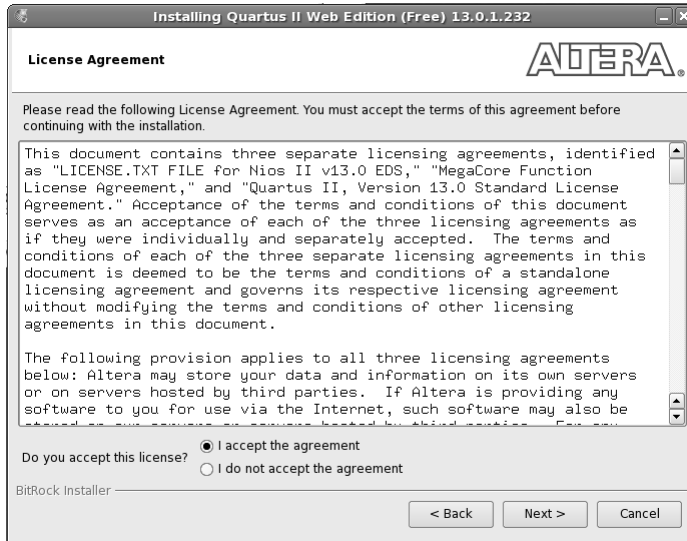


Figura A2-7

La siguiente pantalla muestra el acuerdo de licencia bajo el cual puede utilizarse el software provisto por Altera. En caso de estar de acuerdo con las cláusulas, debe seleccionarse la opción **I accept the agreement** y presionar el botón **Next >**.

**Figura A2-8**

El proceso de instalación permitirá definir el directorio de instalación de Quartus II, siendo aconsejable conservar el sugerido. Seleccionar el botón **Next >**.

**Figura A2-9**

En la pantalla siguiente, el proceso permitirá seleccionar las diferentes familias de dispositivos que se desean instalar. En caso de no contar con suficiente espacio disponible en el disco rígido del sistema, se pueden de-seleccionar las familias que no se utilizarán.

Caso contrario pueden mantenerse todas las familias de dispositivos. Seleccionar el botón **Next>**.

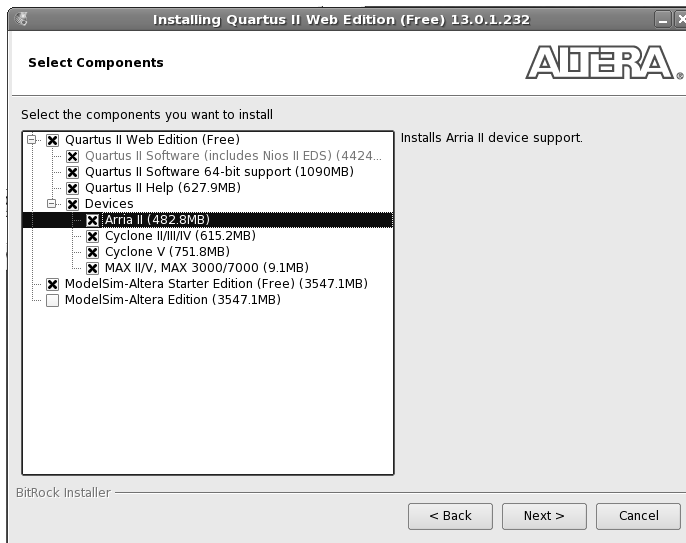


Figura A2-10

Aparecerá una pantalla de información de las selecciones realizadas. Seleccionar el botón **Next >**.

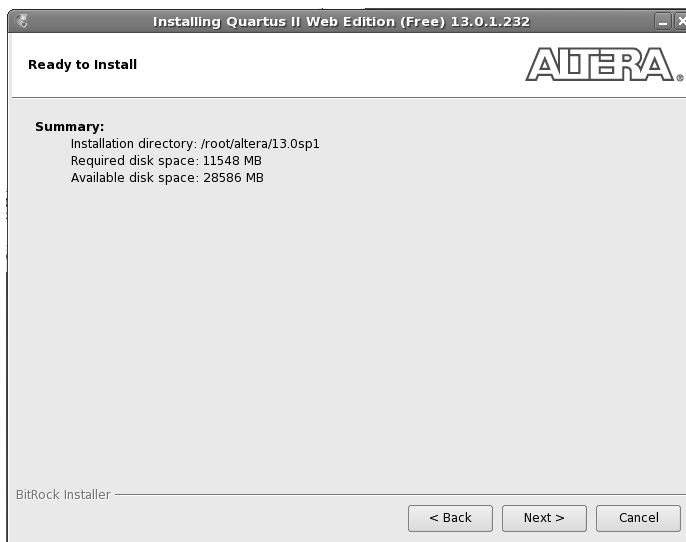


Figura A2-11

Al finalizar el proceso de instalación, se mostrará una pantalla en que permite seleccionar si se desea crear un acceso directo en el escritorio, ejecutar Quartus II y llenar el registro de información para Altera. Para simplificar la instalación, des-seleccionar la opción **Provide your feedback** y seleccionar el botón **Finish**.

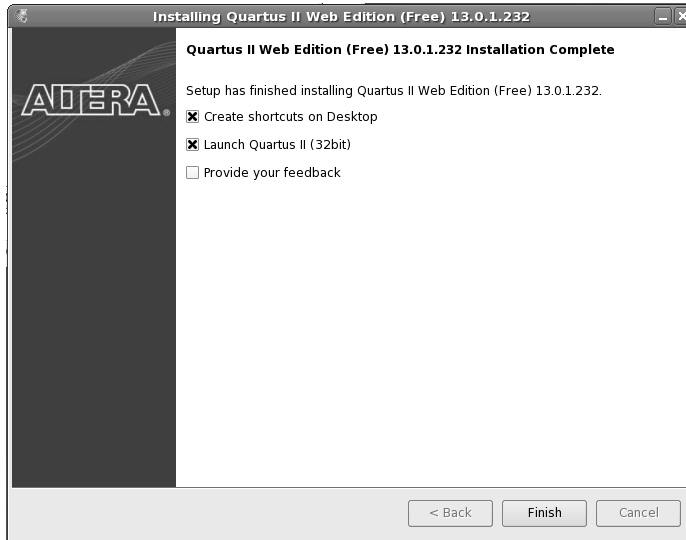


Figura A2-12

Quartus II se instaló en el directorio especificado. Para su ejecución, debe abrirse una pantalla de “Terminal”. Quartus II requiere el “Bourne Shell” para su ejecución. Por tal motivo, su ejecución requiere los siguientes comandos:

```
cd /<carpeta_de_instalación>/quartus/bin  
sh quartus
```

Otra alternativa es la utilización del acceso directo que la instalación deja en el escritorio del sistema. Una vez realizada la instalación puede eliminarse el archivo de instalación y el directorio “temp” para liberar espacio en el disco rígido.

La primera vez que se ejecuta Quartus II Web Edition, seleccionar la opción **Run the Quartus II software** y hacer clic en el botón **OK**.

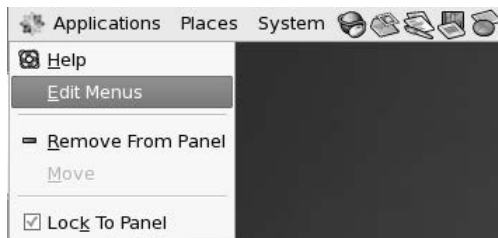
**Figura A2-13**

La ejecución de Quartus II comenzará y podrá utilizarse para todos los ejemplos desarrollados en este libro.

CONFIGURACIÓN DEL MENÚ DE APLICACIONES EN CENTOS

A diferencia que la instalación en Microsoft Windows, en Linux no se configura el menú del sistema con cada una de las aplicaciones que se pueden ejecutar. Solo puede ejecutarse la aplicación Quartus II mediante el acceso directo creado en el escritorio del sistema.

Para incorporar un menú de acceso a las aplicaciones instaladas, es conveniente la utilización de utilidad **Edit Menus** a la que se puede acceder haciendo clic con el botón derecho sobre el acceso a **Applications** en la esquina superior-izquierda del escritorio:

**Figura A2-14**

La ejecución de este comando abrirá la aplicación **Menu Layout** que permite crear comandos en el menú de aplicaciones del sistema. Para la creación de una carpeta exclusiva para Quartus, seleccionar mediante un clic sobre el separador (“—”) y seleccionar el botón **New Separator**.

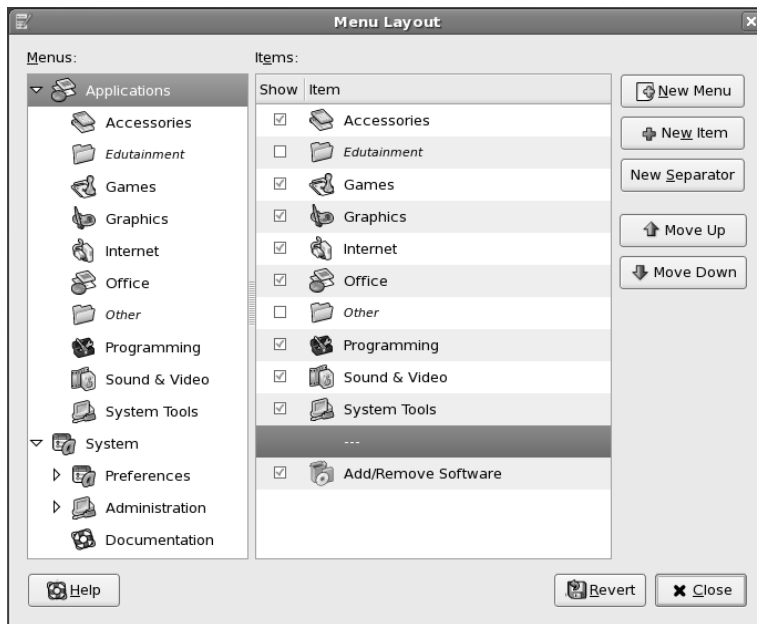


Figura A2-15

Crear, seleccionado el botón **New Menu**, una carpeta y asignarle el nombre “Altera”, especificándolo en el cuadro de texto **Name**. Seleccionar el botón **OK**.



Figura A2-16

Mediante los botones **Move Up** y **Move Down** ubicar la carpeta entre los dos separadores o en la ubicación que se desee. Habilitar el checkbox **Show** para que la carpeta sea visible en el menú.

En la carpeta “Altera” realizar el mismo procedimiento para la creación de tres menús:

- ModelSim-Altera Starter Edition
- Nios II EDS
- Quartus II Web Edition

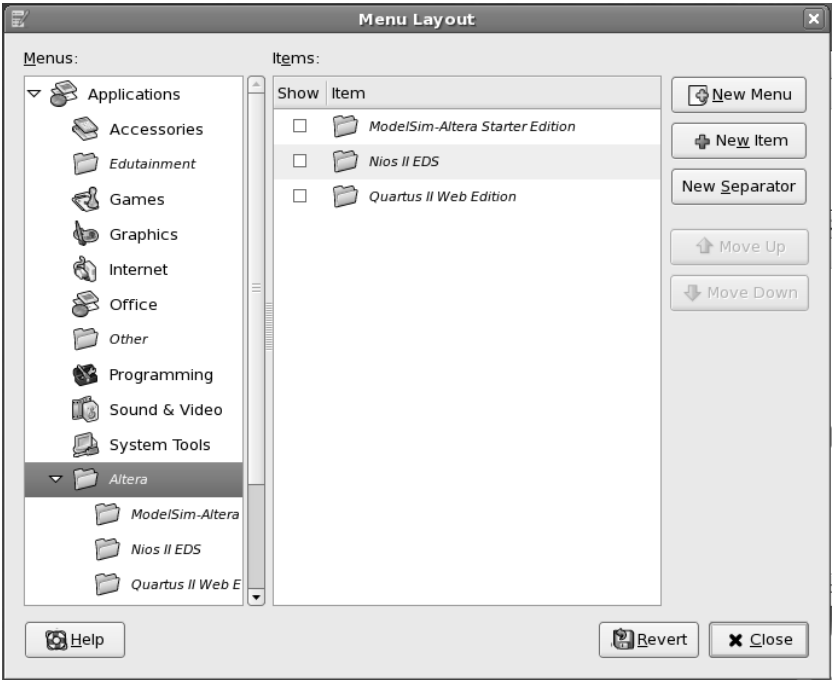


Figura A2-17

En la carpeta “ModelSim-Altera Starter Edition”, mediante la selección del botón **New Item**, crear el siguiente acceso directo (habilitando la celda **Run command in a terminal**):

Nombre	Comando
ModelSim-Altera (Quartus II)	sh <directorio_instalación>/modelsim_ase/bin/vsim

En la carpeta Nios II EDS, crear el siguiente acceso directo (sin habilitar la celda **Run command in a terminal**):

Nombre	Comando
Nios II Software Build Tools for Eclipse	<directorio_instalación>/nios2eds/bin/eclipse- nios2/eclipse

En la carpeta Quartus II, crear los siguientes accesos directos (habilitando la celda **Run command in a terminal**):

Nombre	Comando
Quartus II	sh <directorio_instalación>/quartus/bin/quartus
Quartus II Designer Space Explorer	sh <directorio_instalación>/quartus/bin/quartus_sh --dse
Quartus II MegaWizard Plug-in Manager	sh <directorio_instalación>/quartus/bin/qmegawizq

Mediante esta configuración se podrán acceder a las aplicaciones de la plataforma de Altera en forma similar a la realizada en un ambiente de Microsoft Windows.

ANEXO 3

FLUJO DE DISEÑO CON UTILIDADES DE LÍNEA DE COMANDOS

INTRODUCCIÓN

En diversos capítulos del libro, se describen las diferentes utilidades disponibles para la realización de las etapas del flujo de diseño de un SOPC. En este anexo, se describen las diferentes utilidades involucradas en dicho proceso:

- La Figura A3-1 muestra el flujo de comandos requeridos para el diseño de un proyecto en Quartus II.
- La Figura A3-2 muestra el flujo de diseño en Qsys para la obtención del archivo de descripción del SOPC **.sopcinfo** que será utilizado para la generación de la HAL en el proyecto BSP. También el proceso genera los archivos de descripción de hardware utilizados por Quartus para la síntesis del diseño al dispositivo FPGA.
- La Figura A3-3 muestra el flujo de diseño del proyecto BSP. El flujo toma como entrada el archivo **.sopcinfo** generado por Qsys y produce un archivo de librería **.a** y **system.h** que serán incluidos en la compilación del proyecto de aplicación para la implementación de la HAL.
- La Figura A3-4 muestra el proceso para la generación del software que se ejecutará en el SOPC. El archivo **.elf** contiene el ejecutable binario con diferentes opciones para la depuración según las opciones de optimización con las que se haya realizado la compilación. El proceso genera también los archivos de inicialización de memoria del SOPC que deben ser incluidos en el proyecto de Quartus II para la síntesis del diseño al dispositivo FPGA.

FLUJO DE DISEÑO DEL PROYECTO EN QUARTUS II MEDIANTE LÍNEA DE COMANDOS

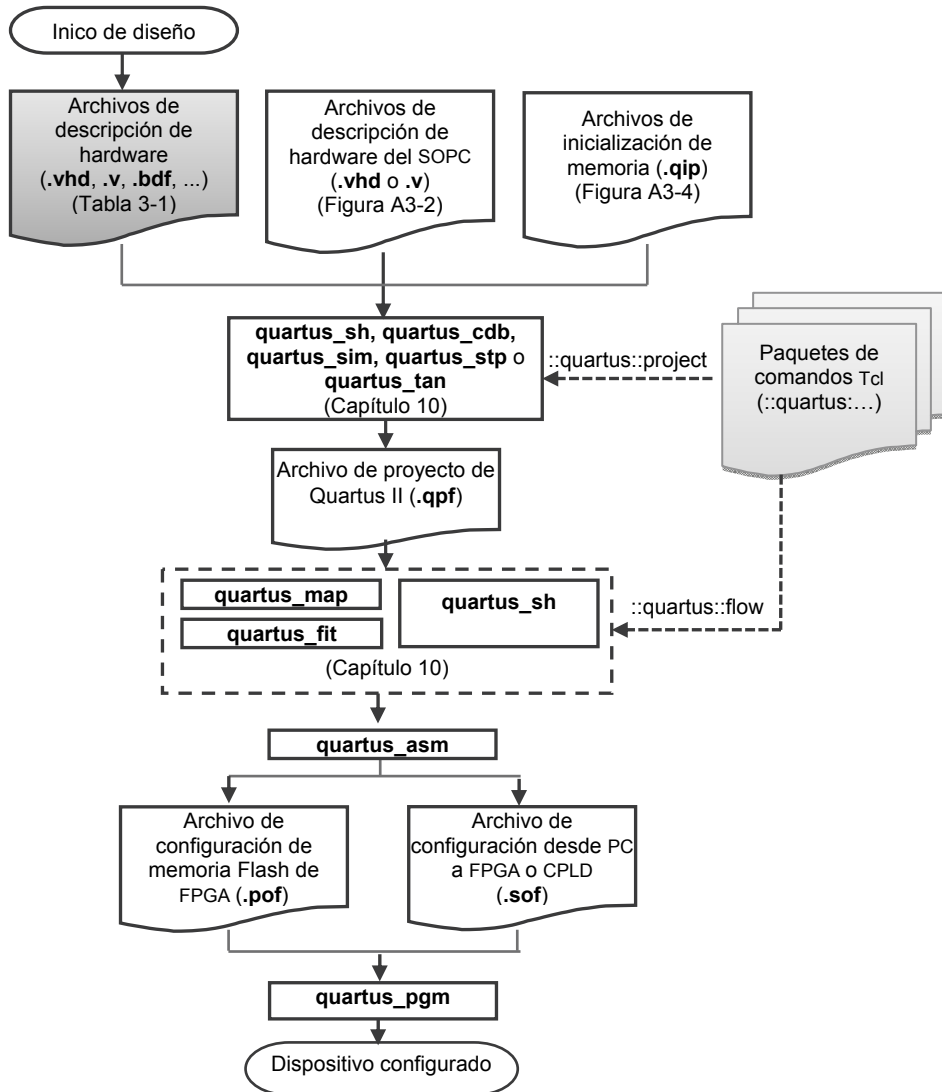


Figura A3-1: Flujo de diseño de proyecto de Quartus II mediante línea de comandos.

El diseño de un proyecto en Quartus II requiere los archivos de descripción de hardware de todo el sistema, los archivos de descripción de hardware generados por *SOPC Builder* o Qsys y los archivos de inicialización de memoria generados por la compilación de la aplicación en Eclipse para Nios II. El control del flujo de diseño puede realizarse mediante comandos Tcl y las diversas utilidades brindadas por Altera. El resultado final es el archivo para la configuración directa del dispositivo FPGA a través de la computadora personal (.sof) y para la grabación de la memoria *flash* de configuración en placa del dispositivo FPGA (.pof).

FLUJO DE DISEÑO DE PROYECTO SOPC EN QSYS MEDIANTE LÍNEA DE COMANDOS

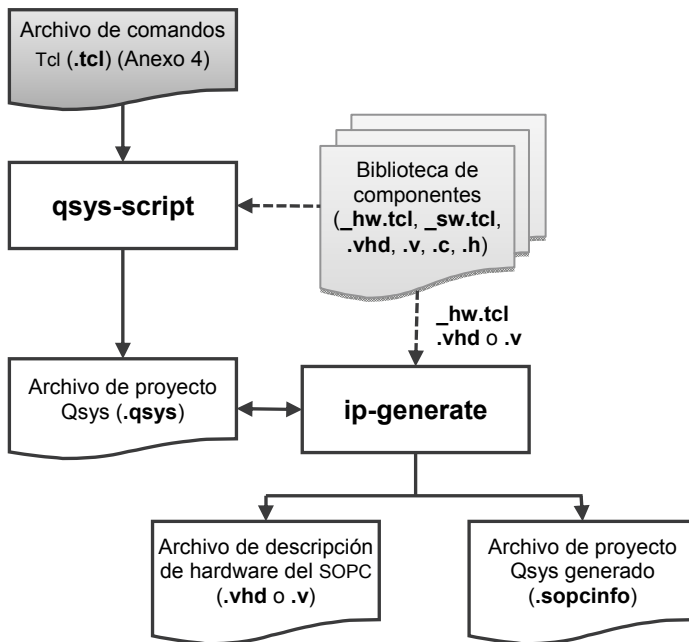


Figura A3-2: Flujo de Diseño de Proyecto de SOPC en Qsys.

Un SOPC en Qsys puede realizarse mediante comandos Tcl y la aplicación “qsys-script” de Altera. El flujo de diseño de un proyecto en Qsys permite obtener el archivo de descripción de hardware del SOPC (utilizado por Quartus II para la síntesis del hardware final, Figura A3-1) y el archivo **.sopcinfo** del proyecto realizado (utilizado para la especificación del proyecto BSP, Figura A3-3).

FLUJO DE DISEÑO DE PROYECTO BSP MEDIANTE LÍNEA DE COMANDOS

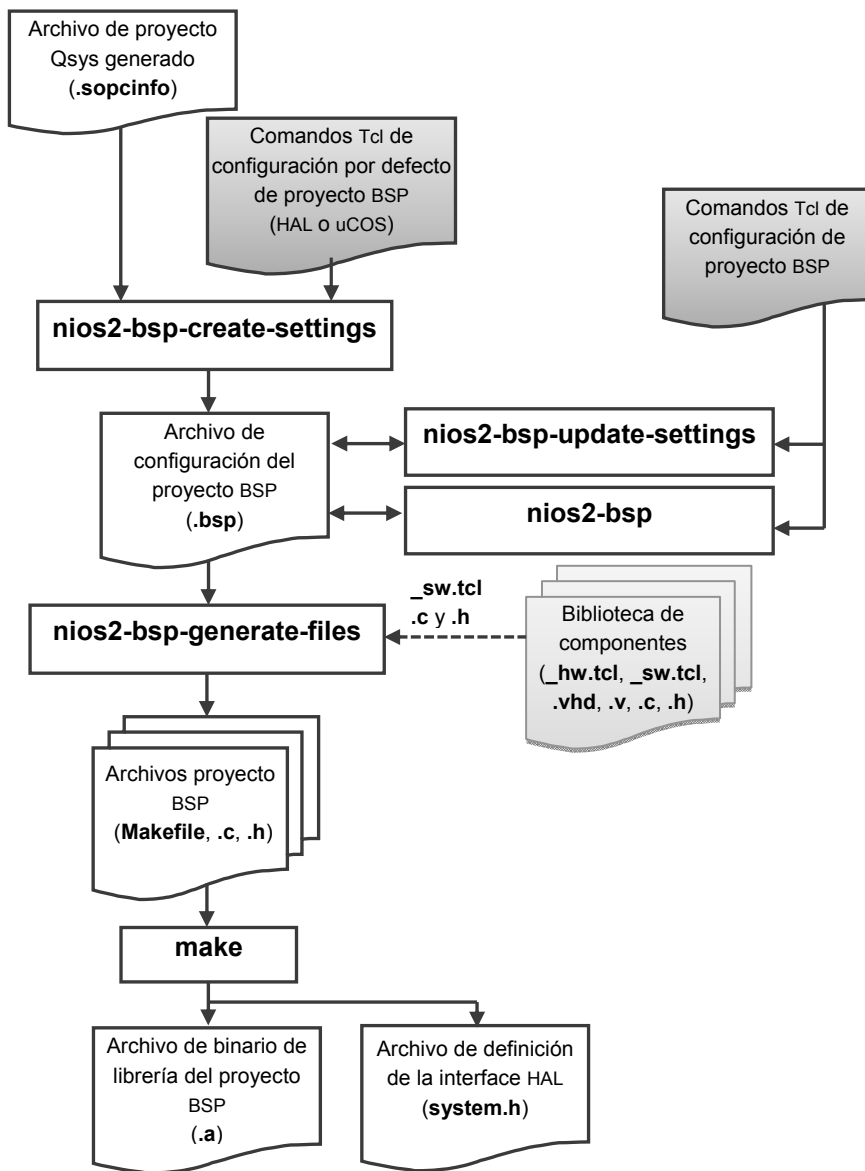


Figura A3-3: Flujo de diseño de proyecto BSP en línea de comandos.

El flujo de diseño del proyecto BSP permite obtener la biblioteca de software que contiene todos los controladores de dispositivos especificados en el hardware del SOPC (utilizados en la compilación final de la aplicación, Figura A3-4). El archivo **system.h** contiene todos los encabezados de esta biblioteca para que puedan ser incluidos en el proyecto de aplicación.

FLUJO DE DISEÑO DE PROYECTO DE APLICACIÓN MEDIANTE LÍNEA DE COMANDOS

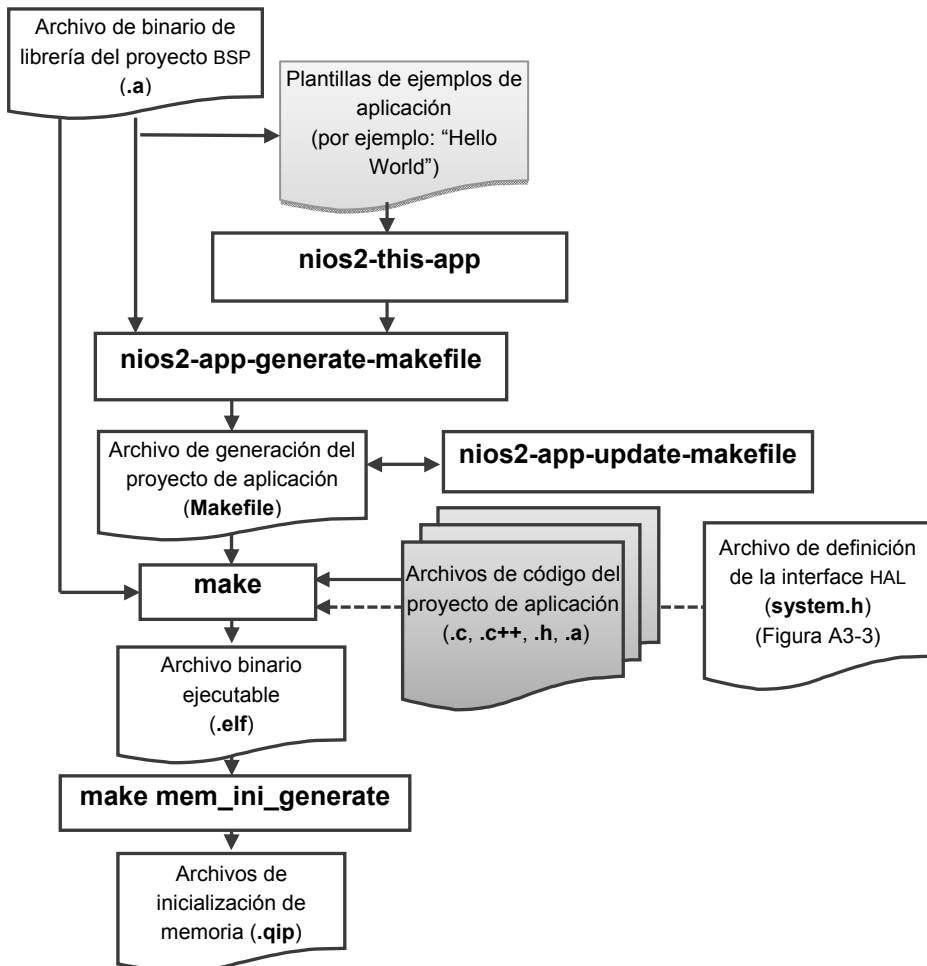


Figura A3-4: Flujo de diseño del proyecto de aplicación en línea de comandos.

La realización de estos procedimientos son necesarios para la implementación de un SOPC en un dispositivo FPGA de la firma Altera mediante utilidades de línea de comando y comandos Tcl. La utilización de estos procedimientos permite un eficiente, documentable y de fácil mantenimiento del flujo de diseño al independizarse de las variaciones que pueden tener diferentes versiones de las herramientas de desarrollo en sus interfaces gráficas. También permite una eficiente integración con herramientas de desarrollo de otros fabricantes.

ANEXO 4

COMANDOS TCL PARA LA UTILIDAD “QSYS-SCRIPT”

La utilidad `qsys-script` permite crear y manipular SOPC desarrollados en Qsys mediante comandos Tcl (9). Para la creación de un SOPC, se deberá realizar las siguientes acciones:

- Instanciar los diferentes componentes para confirmar los módulos del SOPC.
- Conectar los diferentes módulos de acuerdo a la arquitectura deseada del SOPC.
- Definir las diferentes interfaces entre los módulos de acuerdo a las características de las conexiones del SOPC.
- Definir los puertos del SOPC.
- Asignar las direcciones de memoria a los módulos del SOPC.
- Asignar las interrupciones a los módulos que poseen interrupciones.

Todas estas acciones, más las necesarias para definir los diferentes parámetros del proyecto, pueden realizarse mediante la utilidad `qsys-script` y los comandos Tcl. La utilidad `qsys-script` es copiada, durante la instalación de Quartus II, en la carpeta “<carpeta_de_instalacion>\quartus\sopc_builder\bin”.

Un proyecto de un SOPC en Qsys está conformado por los siguientes elementos:

Elemento	Descripción y comandos Tcl asociados
<archivo>.qsys	Contiene la descripción del sistema y del proyecto. • “create_system”: crea un sistema con un determinado nombre • “save_system”: guarda en archivo el sistema • “load_system” carga desde un archivo el sistema.
project	Contiene información sobre propiedades para la generación del sistema: (“DEVICE_FAMILY” y “DEVICE” en Qsys versión 13).

module	Determina la entidad de mayor jerarquía del sistema y módulo con que se instanciará en Quartus II el SOPC generado. Posee propiedades (“NAME” en Qsys versión 13).
instance	Es la instancia de un componente de librería para conformar un módulo del SOPC. Puede contener: - Interfaces para conectarse con otras instancias. - Propiedades para determinar sus características. - Asignaciones definidas según el componente. - Parámetros que permiten configurar la instancia. - Puerto contenido en alguna interfaz de la instancia.
interface	Permite la conexión de diferentes instancias.
connection	Determina la conexión entre interfaces de instancias.

EJEMPLO

El siguiente ejemplo ilustra la utilización de comandos Tcl de la utilidad `qsys-script` para listar el nombre de todas las instancias que componen un SOPC definido en el archivo “c:\n2sopc\sopcqsys.qsys”.

El archivo de comandos Tcl “QsysTcl.tcl” contendrá los siguientes comandos:

```
load_system "c:/n2sopc/sopcqsys.qsys"
puts "Propiedades del Proyecto"
foreach i [get_project_properties] {
    puts "$i [get_project_property $i]"
}
```

La ejecución de estos comandos será realizada mediante la utilidad `qsys-script`:

```
<carpeta>\qsys-script --script=QsysTcl.tcl --package-version=12.1
```

COMANDOS TCL DE QSYS-SCRIPT

A continuación, se detallan los comandos Tcl para la creación y manipulación de SOPC en Qsys (listado original de los comandos puede encontrarse en el Capítulo 6 de (9)):

add_connection

Conecta interfaces utilizando el tipo de conexión adecuado. Los nombres de interfaces consisten en el nombre de instancia del componente, seguido por el nombre de una interfaz proporcionada por ese módulo, por ejemplo, “mux0.out” es la interfaz “out” de la instancia “mux0”.

sintaxis: `add_connection <start> [<end>]`

```
ejemplo: add_connection dma.read_master sdram.s1
```

add_instance

Incorpora una instancia de un componente al sistema y le asigna un nombre a dicha instancia.

```
sintaxis: add_instance <name> <type> [<version>]
```

```
ejemplo: add_instance uart_0 altera_avalon_uart
```

add_interface

Incorpora una interfaz al sistema que puede ser utilizada para exportar señales fuera del soc. El comando **set_interface_property EXPORT_OF <instance.interface>** se utiliza para especificar las características de la interfaz exportada.

```
sintaxis: add_interface <name> <type> <direction>
```

```
ejemplo: add_interface my_export "Conduit" end
```

```
set_interface_property          my_export          EXPORT_OF
uart_0.external_connection
```

auto_assign_base_addresses

Asigna la dirección base a las interfaces en el mapa de memoria del sistema. Las interfaces que poseen su dirección fija mediante el comando **lock_avalon_base_address** mantienen sus direcciones durante el proceso de autoasignación.

```
sintaxis: auto_assign_base_addresses <instance>
```

```
ejemplo: auto_assign_base_addresses sdram
```

auto_assign_irqs

Asigna automáticamente los números de interrupción a todas las instancias del sistema que poseen interrupciones.

```
sintaxis: auto_assign_irqs <instance>
```

```
ejemplo: auto_assign_irqs sdram
```

auto_connect

Conecta una instancia o interfaz a las interfaces que concuerdan en el sistema. Por ejemplo, una interfaz Avalon-MM esclava es conectada a las interfaces Avalon-MM maestras.

```
sintaxis: auto_connect <element>
```

```
ejemplo: auto_connect sdram
```

```
auto_connect uart_0.s1
```

create_system

Remplaza al actual sistema por un sistema con el nombre dado en el comando.

```
sintaxis: create_system [<name>]
```

```
ejemplo: create_system my_new_system_name
```

get_composed_connection_parameter_value

Retorna el valor de un parámetro en una conexión en un subsistema, para una instancia que contiene a un subsistema.

```
sintaxis: get_composed_connection_parameter_value <instance>
<childConnection> <parameter>
```

```
ejemplo: get_composed_connection_parameter_value subsystem_0
cpu.data_master/memory.s0 baseAddress
```


get_composed_connection_parameters

Retorna una lista de parámetros de una conexión en un subsistema, para una instancia que contiene a un subsistema.

sintaxis: `get_composed_connection_parameters <instance>`
`<childConnection>`

ejemplo: `get_composed_connection_parameters subsystem_0`
`cpu.data_master/memory.s0`

get_composed_connections

Retorna una lista con todas las conexiones que posee una instancia que contiene un subsistema.

sintaxis: `get_composed_connections <instance>`

ejemplo: `get_composed_connections subsystem_0`

get_composed_instance_assignment

Retorna el valor de una asignación de una instancia de un subsistema.

sintaxis: `get_composed_instance_assignment <instance>`
`<childInstance> <key>`

ejemplo: `get_composed_instance_assignment subsystem_0`
`video_0 "embeddedsw.CMacro.colorSpace"`

get_composed_instance_assignments

Retorna una lista de asignaciones de una instancia en un subsistema.

sintaxis: `get_composed_instance_assignments <instance>`
`<childInstance>`

ejemplo: `get_composed_instance_assignments subsystem_0 cpu`

get_composed_instance_parameter_value

Retorna el valor de un parámetro de una instancia de un subsistema.

sintaxis: `get_composed_instance_parameter_value <instance>`
`<childInstance> <parameter>`

ejemplo: `get_composed_instance_parameter_value subsystem_0`
`cpu DATA_WIDTH`

get_composed_instance_parameters

Retorna una lista de parámetros de una instancia de un subsistema.

sintaxis: `get_composed_instance_parameters <instance>`
`<childInstance>`

ejemplo: `get_composed_instance_parameters subsystem_0 cpu`

get_composed_instances

Retorna una lista de las instancias que componen un subsistema.

sintaxis: `get_composed_instances <instance>`

ejemplo: `get_composed_instances subsystem_0`

get_connection_parameter_property

Retorna el valor de una propiedad de parámetro en una conexión.

sintaxis: `get_connection_parameter_property <connection>`
`<parameter> <property>`

```
ejemplo: get_connection_parameter_property
cpu.data_master/dma0.csr baseAddress UNITS
```

get_connection_parameter_value

Obtiene el valor de un parámetro de una conexión. Los parámetros representan aspectos de la conexión que pueden ser modificados una vez que la conexión es creada, tal como la dirección base para una conexión Avalon-MM.

```
sintaxis: get_connection_parameter_value <connection>
<parameter>
```

```
ejemplo: get_connection_parameter_value
cpu.data_master/dma0.csr baseAddress
```

get_connection_parameters

Retorna una lista de parámetros de una conexión. La lista de parámetros es la misma para conexiones del mismo tipo.

```
sintaxis: get_connection_parameters <connection>
```

```
ejemplo: get_connection_parameters cpu.data_master/dma0.csr
```

get_connection_properties

Retorna una lista de propiedades de una conexión. La lista de propiedades es la misma para todas las conexiones, independientemente del tipo.

```
sintaxis: get_connection_properties
```

```
ejemplo: get_connection_properties
```

get_connection_property

Retorna el valor de una propiedad de conexión.

```
sintaxis: get_connection_property <connection> <property>
```

```
ejemplo: get_connection_property cpu.data_master/dma0.csr
```

TYPE

get_connections

Retorna una lista de conexiones en un sistema si no fue especificado ningún elemento. Si una instancia es especificada, por ejemplo "cpu", entonces retorna una lista con todas las conexiones a cualquier interfaz de la instancia. Si una interfaz de una instancia es especificada, por ejemplo "cpu.instruction_master", entonces solo retornan las conexiones a dicha interfaz.

```
sintaxis: get_connections [<element>]
```

```
ejemplo: get_connections
```

```
get_connections cpu
```

```
get_connections cpu.instruction_master
```

get_instance_assignment

Retorna el valor de una asignación en una instancia.

```
sintaxis: get_instance_assignment <instance> <key>
```

```
ejemplo: get_instance_assignment video_processor
embeddedsw.CMacro.colorSpace
```

get_instance_assignments

Retorna una lista de claves de asignación para una determinada interfaz de una instancia.

```
sintaxis: get_instance_assignments <instance>
```

ejemplo: `get_instance_assignments sdr`

get_instance_interface_assignment

Retorna el valor de una clave de asignación de una interfaz de una instancia.

sintaxis: `get_instance_interface_assignment <instance>
<interface> <key>`

ejemplo: `get_instance_interface_assignment sdr s1
embeddedsw.configuration.isFlash`

get_instance_interface_assignments

Retorna el valor de una asignación de una interfaz de una instancia.

sintaxis: `get_instance_interface_assignments <instance>
<interface>`

ejemplo: `get_instance_interface_assignments sdr s1`

get_instance_interface_parameter_property

Retorna el valor de la propiedad de un parámetro en la interfaz de una instancia.

sintaxis: `get_instance_interface_parameter_property
<instance> <interface> <parameter> <property>`

ejemplo: `get_instance_interface_parameter_property uart_0 s0
setupTime ENABLED`

get_instance_interface_parameter_value

Retorna el valor de un parámetro de la interfaz de una instancia.

sintaxis: `get_instance_interface_parameter_value <instance>
<interface> <parameter>`

ejemplo: `get_instance_interface_parameter_value uart_0 s0
setupTime`

get_instance_interface_parameters

Retorna una lista de parámetros para la interfaz de una instancia.

sintaxis: `get_instance_interface_parameters <instance>
<interface>`

ejemplo: `get_instance_interface_parameters uart_0 s0`

get_instance_interface_port_property

Retorna el valor de la propiedad de un puerto de la interfaz de una instancia.

sintaxis: `get_instance_interface_port_property <instance>
<interface> <port> <property>`

ejemplo: `get_instance_interface_port_property uart_0 exports
tx WIDTH`

get_instance_interface_ports

Retorna una lista de puertos en una interfaz de una instancia.

sintaxis: `get_instance_interface_ports <instance> <interface>`

ejemplo: `get_instance_interface_ports uart_0 s0`

get_instance_interface_properties

Retorna una lista de propiedades que pueden ser consultadas de una interfaz de una instancia.

sintaxis: `get_instance_interface_properties`

ejemplo: `get_instance_interface_properties`

get_instance_interface_property

Retorna el valor de la propiedad de una interfaz de una instancia.

sintaxis: `get_instance_interface_property <instance>
<interface> <property>`

ejemplo: `get_instance_interface_property uart_0 s0
DESCRIPTION`

get_instance_interfaces

Retorna una lista de interfaces de una instancia.

sintaxis: `get_instance_interfaces <instance>`

ejemplo: `get_instance_interfaces uart_0`

get_instance_parameter_property

Retorna el valor de una propiedad en el parámetro de una instancia.

sintaxis: `get_instance_parameter_property <instance>
<parameter> <property>`

ejemplo: `get_instance_parameter_property uart_0 baudRate
ENABLED`

get_instance_parameter_value

Retorna el valor de una propiedad en una instancia.

sintaxis: `get_instance_parameter_value <instance> <parameter>`

ejemplo: `get_instance_parameter_value uart_0 baudRate`

get_instance_parameters

Retorna una lista de parámetros en una instancia.

sintaxis: `get_instance_parameters <instance>`

ejemplo: `get_instance_parameters uart_0`

get_instance_port_property

Retorna el valor de una propiedad de un puerto contenido en una interfaz de una instancia.

sintaxis: `get_instance_port_property <instance> <port>
<property>`

ejemplo: `get_instance_port_property uart_0 tx WIDTH`

get_instance_properties

Retorna una lista de propiedades para una instancia.

sintaxis: `get_instance_properties`

ejemplo: `get_instance_properties`

get_instance_property

Retorna el valor de una propiedad para una instancia.

sintaxis: `get_instance_property <instance> <property>`

ejemplo: `get_instance_property cpu ENABLED`

get_instances

Retorna una lista de nombres de instancias para todas las instancias en el sistema.

sintaxis: `get_instances`

ejemplo: `get_instances`

get_interface_port_property

Retorna el valor de una propiedad de un puerto contenido por una de las interfaces exportadas.

sintaxis: `get_interface_port_property <interface> <port> <property>`

ejemplo: `get_interface_port_property uart_exports tx DIRECTION`

get_interface_ports

Retorna los nombres de todos los puertos que han sido agregados a una interfaz.

sintaxis: `get_interface_ports <interface>`

ejemplo: `get_interface_ports export_clk_out`

get_interface_properties

Retorna el nombre de todas las propiedades disponibles de la interfaz. La lista de las propiedades es la misma para todas las interfaces del mismo tipo.

sintaxis: `get_interface_properties`

ejemplo: `get_interface_properties`

get_interface_property

Retorna el valor de una propiedad de una interfaz especificada.

sintaxis: `get_interface_property <interface> <property>`

ejemplo: `get_interface_property export_clk_out EXPORT_OF`

get_interfaces

Retorna una lista de las interfaces de mayor nivel del sistema.

sintaxis: `get_interfaces`

ejemplo: `get_interfaces`

get_module_properties

Retorna las propiedades que se pueden manipular para la entidad de mayor jerarquía del sistema.

sintaxis: `get_module_properties`

ejemplo: `get_module_properties`

get_module_property

Retorna el valor de una propiedad del sistema.

sintaxis: `get_module_property <property>`

ejemplo: `get_module_property NAME`

get_parameter_properties

Retorna una lista de propiedades que pueden encuestarse por sus parámetros.

sintaxis: `get_parameter_properties`

ejemplo: `get_parameter_properties`

get_port_properties

Retorna una lista de propiedades que pueden ser encuestadas por puertos.

sintaxis: `get_port_properties`

ejemplo: `get_port_properties`

get_project_properties

Retorna una lista de propiedades que pueden ser consultadas para el proyecto de Quartus II.

sintaxis: `get_project_properties`

ejemplo: `get_project_properties`

get_project_property

Retorna el valor de una propiedad del proyecto de Quartus II.

sintaxis: `get_project_property <property>`

ejemplo: `get_project_property DEVICE_FAMILY`

load_system

Carga un sopc a Qsys desde un archivo y utiliza dicho sistema como actual para la aplicación de los siguientes comandos Tcl.

sintaxis: `load_system <file>`

ejemplo: `load_system example.qsys`

lock_avalon_base_address

Previene la modificación o cambio de una dirección de memoria de una interfaz o una instancia cuando se aplica los comandos **auto_assign_base_addresses** o **auto_assign_system_base_addresses**.

sintaxis: `lock_avalon_base_address <instance.interface>`

ejemplo: `lock_avalon_base_address sdram.s1`

preview_insert_avalon_streaming_adapters

Ejecuta la inserción de adaptadores para conexiones Avalon-ST con lo cual adapta errores en la conexión, como ser diferentes anchos de datos.

sintaxis: `preview_insert_avalon_streaming_adapters`

ejemplo: `preview_insert_avalon_streaming_adapters`

remove_connection

Remueve una conexión del sistema.

sintaxis: `remove_connection <connection>`

ejemplo: `remove_connection cpu.data_master/sdram.s0`

remove_instance

Remueve una instancia del sistema.

sintaxis: `remove_instance <instance>`

ejemplo: `remove_instance cpu`

remove_interface

Remueve una interfaz exportada del sistema.

sintaxis: `remove_interface <interface>`

ejemplo: `remove_interface clk_out`

save_system

Guarda el actual sistema a un archivo. Si el nombre del archivo no es especificado, entonces se guarda con el mismo nombre con el que se abrió mediante el comando **load_system**.

sintaxis: `save_system [<file>]`

ejemplo: `save_system`

`save_system example.qsys`

send_message

Envía un mensaje al usuario. El mensaje es interpretado como HTML. El <nivel> (“<level>”) puede ser: “ERROR”, “WARNING”, “INFO”, “PROGRESS” o “DEBUG”.

sintaxis: send_message <level> <message>

ejemplo: send_message ERROR "The system is down!"

set_connection_parameter_value

Configura el valor de un parámetro para una conexión.

sintaxis: set_connection_parameter_value <connection>
<parameter> <value>

ejemplo: set_connection_parameter_value
cpu.data_master/dma0.csr baseAddress "0x000a0000"

set_instance_parameter_value

Configura el valor para un parámetro de una instancia.

sintaxis: set_instance_parameter_value <instance> <parameter>
<value>

ejemplo: set_instance_parameter_value uart_0 baudRate 9600

set_instance_property

Configura el valor de una propiedad de una instancia. Generalmente, es utilizado para modificar el parámetro “ENABLED” de la instancia que la incluye o excluye de la generación del sistema.

sintaxis: set_instance_property <instance> <property> <value>

ejemplo: set_instance_property cpu ENABLED false

set_interface_property

Configura el valor de una propiedad de una interfaz exportada. Este comando es utilizado para configurar la propiedad “EXPORT_OF” para especificar cuál interfaz de una instancia es exportada.

sintaxis: set_interface_property <interface> <property>
<value>

ejemplo: set_interface_property clk_out EXPORT_OF clk.clk_out

set_module_property

Configura el valor de una propiedad del sistema como ser el nombre modificando la propiedad “NAME”.

sintaxis: set_module_property <property> <value>

ejemplo: set_module_property NAME "new_system_name"

set_project_property

Configura propiedades del proyecto, como ser la familia del dispositivo utilizado.

sintaxis: set_project_property <property> <value>

ejemplo: set_project_property DEVICE_FAMILY "Cyclone IV GX"

set_validation_property

Configura la manera en que la validación del sistema es llevada a cabo. Si la propiedad “AUTOMATIC_VALIDATION” es asignada a “false”, entonces la validación no es llevada a cabo cada vez que se ejecuta un comando.

sintaxis: set_validation_property <property> <value>

```
ejemplo: set_validation_property AUTOMATIC_VALIDATION false
```

unlock_avalon_base_address

Permite que la dirección de memoria de las instancias pueda ser modificada por los comandos **auto_assign_base_addresses** o **auto_assign_system_base_addresses**.

```
sintaxis: unlock_avalon_base_address <instance.interface>
```

```
ejemplo: unlock_avalon_base_address sdram.s1
```

upgrade_sopc_system

Migra un SOPC realizado en *SOPC Builder* y almacenado en un archivo ".sopc" a un sistema compatible con Qsys. El sistema migrado y cargado debe ser almacenado a un archivo mediante el comando **save_system**.

```
sintaxis: upgrade_sopc_system <filename>
```

```
ejemplo: upgrade_sopc_system old_system.sopc
```

validate_connection

Valida una conexión específica y retorna un mensaje con el resultado de la validación.

```
sintaxis: validate_connection <connection>
```

```
ejemplo: validate_connection cpu.data_master/sdram.s1
```

validate_instance

Valida una instancia específica y retorna un mensaje con el resultado de la validación.

```
sintaxis: validate_instance <instance>
```

```
ejemplo: validate_instance cpu
```

validate_instance_interface

Valida una interfaz y retorna un mensaje con el resultado de la validación.

```
sintaxis: validate_instance_interface <instance> <interface>
```

```
ejemplo: validate_instance_interface cpu data_master
```

validate_system

Valida el sistema y retorna un mensaje con el resultado de la validación.

```
sintaxis: validate_system
```

```
ejemplo: validate_system
```


ANEXO 5

COMANDOS TCL PARA LA CONFIGURACIÓN DE PROYECTOS BSP

El presente anexo lista los diferentes comandos Tcl que pueden ser utilizados para la configuración de proyectos BSP. Estos comandos pueden ser ejecutados mediante las utilidades de línea de comandos **nios2-bsp-create-settings**, **nios2-bsp-update-settings** y **nios2-bsp-query-settings**. El listado original de los comandos puede encontrarse en el Capítulo 15 de (18):

add_memory_device

Permite definir un área de memoria definida por el usuario, fuera del hardware del sistema. Este comando agrega un área de memoria externa en el mapa de memoria del proyecto BSP, permitiendo la definición de regiones en tal dispositivo.

```
sintaxis: add_memory_device <device name> <base address>  
<span>
```

add_memory_region

Crea una nueva región de memoria para la compilación del proyecto. Esta región no debe solaparse con ninguna otra región del proyecto.

```
sintaxis: add_memory_region <name> <slave_desc> <offset>  
<span>
```

add_section_mapping

Permite asociar una determinada sección de compilación a una región específica de memoria. Si la sección no existe, el comando la crea. Si la sección existe, entonces es sobrescrita.

```
add_section_mapping <section_name> <memory_region_name>
```

are_same_resource

Retorna un valor que indica si dos descriptores esclavos están conectados al mismo recurso. Por ejemplo, una memoria *dual-port* posee dos puertos esclavos que acceden a la misma memoria y misma unidad de control.

```
sintaxis: are_same_resource <slave_desc1> <slave_desc2>
```

delete_memory_region

Elimina una determinada región de memoria. La región debe existir para evitar una condición de error.

sintaxis: delete_memory_region <region_name>

delete_section_mapping

Elimina un determinado mapeo de sección.

sintaxis: delete_section_mapping <section_name>

disable_sw_package

Deshabilita un determinado paquete de software. Las configuraciones realizadas a un paquete de software deshabilitado no estarán más disponibles en el proyecto BSP.

sintaxis: disable_sw_package <software_package_name>

enable_sw_package

Habilita un paquete de software. Si no se especifica la versión, se habilita la última versión disponible.

sintaxis: enable_sw_package <software_package_name>

get_addr_span

Retorna la longitud en bytes de la cantidad de dirección que posee un determinado descriptor de esclavo.

sintaxis: get_addr_span <slave_desc>

get_assignment

Retorna el nombre del valor de la asignación para un determinado nombre de instancia.

sintaxis: get_assignment <module_name> <assignment_name>

get_available_drivers

Retorna una lista de controladores de dispositivos disponibles, compatibles con la instancia del módulo especificado. Retorna una lista vacía, si no existen controladores para el descriptor esclavo especificado.

sintaxis: get_available_drivers <module_name>

get_available_sw_packages

Retorna la lista de nombres de los paquetes de software disponibles en el proyecto BSP actual.

sintaxis: get_available_sw_packages

get_base_addr

Retorna la dirección base de un descriptor esclavo.

sintaxis: get_base_addr <slave_desc>

get_break_offset

Retorna la dirección de *break* del procesador.

sintaxis: get_break_offset

get_break_slave_desc

Retorna el descriptor esclavo asociado con la dirección de *break* del procesador.

sintaxis: get_break_slave_desc

get_cpu_name

Retorna el nombre del procesador del proyecto BSP.

sintaxis: get_cpu_name

get_current_memory_regions

Retorna una lista ordenada de los registros que representan las regiones actuales de memoria para el compilador. Cada es una lista que contiene el nombre de la región, el descriptor esclavo asociado al dispositivo de memoria, el *offset* y el *span*.

sintaxis: `get_current_memory_regions`

get_current_section_mappings

Retorna una lista de listas que contienen todos los mapeos de secciones. Cada lista representa un mapeo de sección con el siguiente formato: {<Nombre_de_la_sección> <región_de_memoria>}.

sintaxis: `get_current_section_mappings`

get_default_memory_regions

Retorna una lista de registros representando las regiones de memoria del compilador por defecto. Estas opciones por defecto son la mejor opción para la configuración de los vectores de inicio y excepción.

sintaxis: `get_default_memory_regions`

get_driver

Retorna el nombre del controlador asociado con una determinada instancia de un modulo. Retorna *none*, si no existe controlador asociado.

sintaxis: `get_driver <module_name>`

get_enabled_sw_packages

Retorna una lista de los paquetes de software actualmente habilitados. Cada elemento de la lista contiene el nombre del paquete y su versión separados por ":".

sintaxis: `get_enabled_sw_packages`

get_exception_offset

Retorna la dirección de la excepción del procesador.

sintaxis: `get_exception_offset`

get_exception_slave_desc

Retorna el descriptor esclavo asociado con la dirección de excepción del procesador.

sintaxis: `get_exception_slave_desc`.

get_fast_tlb_miss_exception_offset

Retorna la dirección de excepción de la tabla rápida de translación (TLB). Solo los procesadores con MMU tienen tal excepción.

sintaxis: `get_fast_tlb_miss_exception_offset`

get_fast_tlb_miss_exception_slave_desc

Retorna el descriptor esclavo asociado con la tabla TLB de la dirección de excepción. Solo los procesadores con MMU tienen tal excepción.

sintaxis: `get_fast_tlb_miss_exception_slave_desc`

get_interrupt_controller_id

Retorna el ID del descriptor esclavo del controlador de interrupciones (-1 si no hay controlador de interrupciones conectado).

sintaxis: `get_interrupt_controller_id <slave_desc>`

get_irq_interrupt_controller_id

Retorna el ID del controlador de interrupción conectado a las IRQ asociada con el descriptor asociado (-1 si no existe controlador).

`get_irq_interrupt_controller_id <slave_desc>`

get_irq_number

Retorna el número de requerimiento de interrupción de un descriptor esclavo o -1, si el número de requerimiento de interrupción no es encontrado.

sintaxis: `get_irq_number <slave_desc>`

get_memory_region

Retorna la información sobre una determinada región de memoria definida en el compilador. El formato es una lista que contiene el nombre de la región, el descriptor asociado al dispositivo de memoria, el *offset* y el *span*.

sintaxis: `get_memory_region <name>`

get_module_class_name

Retorna el nombre de la clase de módulo asociado con una determinada instancia.

sintaxis: `get_module_class_name <module_name>`

get_module_name

Retorna el nombre de la instancia asociada con un determinado descriptor esclavo.

sintaxis: `get_module_name <slave_desc>`

get_reset_offset

Retorna la dirección de inicialización del procesador.

sintaxis: `get_reset_offset`

get_reset_slave_desc

Retorna el descriptor asociado con la dirección de inicialización del procesador.

sintaxis: `get_reset_slave_desc`

get_section_mapping

Retorna el nombre de la región de memoria para una determinada sección del compilador. Retorna *null*, si la sección no existe.

sintaxis: `get_section_mapping <section_name>`

get_setting

Retorna el valor de una determinada configuración del proyecto BSP.

sintaxis: `get_setting <name>`

get_setting_desc

Retorna un *string* describiendo una configuración del proyecto BSP.

sintaxis: `get_setting_desc <name>`

get_slave_descs

Retorna una lista ordenada de los descriptores esclavos conectados al procesador Nios II.

sintaxis: `get_slave_descs`

is_char_device

Retorna un valor booleano que indica si un descriptor esclavo es un *character device*.

sintaxis: `is_char_device <slave_desc>`

is_connected_interrupt_controller_device

Retorna un valor booleano que indica si un descriptor esclavo es un dispositivo controlador de interrupciones conectado al procesador.

sintaxis: `is_connected_interrupt_controller_device
<slave_desc>`

is_connected_to_data_master

Retorna un valor booleano que indica si el descriptor esclavo está conectado a un maestro de datos.

sintaxis: `is_connected_to_data_master <slave_desc>`

is_connected_to_instruction_master

Retorna un valor booleano que indica si un descriptor esclavo está conectado a un maestro de instrucciones.

sintaxis: `is_connected_to_instruction_master <slave_desc>`

is_ethernet_mac_device

Retorna un valor booleano que indica si un descriptor esclavo es un *Ethernet* MAC device.

sintaxis: `is_ethernet_mac_device <slave_desc>`

is_flash

Retorna un valor booleano que indica si un descriptor esclavo es un *flash memory device*.

sintaxis: `is_flash <slave_desc>`

is_memory_device

Retorna un valor booleano que indica si un descriptor esclavo es un *memory device*.

sintaxis: `is_memory_device <slave_desc>`

is_non_volatile_storage

Retorna un valor booleano que indica si un descriptor esclavo es un *non-volatile storage device*.

sintaxis: `is_non_volatile_storage <slave_desc>`

is_timer_device

Retorna un valor booleano que indica si un descriptor esclavo es un *timer device*.

sintaxis: `is_timer_device <slave_desc>`

log_debug

Muestra un mensaje en la consola de salida cuando el nivel de acceso es *debug*.

sintaxis: `log_debug <message>`

log_default

Muestra un mensaje en la consola de salida cuando el nivel de acceso es *default* o mayor.

sintaxis: `log_default <message>`

log_error

Muestra un mensaje en la consola de error **stderr**.

sintaxis: `log_error <message>`

log_verbose

Muestra un mensaje en la consola de salida cuando el nivel de acceso es *verbose* o mayor.

sintaxis: `log_verbose <message>`

set_driver

Selecciona un determinado controlador para una instancia.

sintaxis: `set_driver <driver_name> <module_name>`

set_ignore_file

Permite seleccionar un determinado archivo de proyecto bsp pero prevenir de almacenar permanentemente las modificaciones. Permite importar paquetes de software de otros proyectos.

sintaxis: `set_ignore_file <software_component_name>
<file_name> <ignore>`

set_setting

Establece el valor de una determinada configuración del proyecto BSP.

sintaxis: `set_setting <name> <value>`

update_memory_region

Actualiza una región de memoria en el compilador.

sintaxis: `update_memory_region <name> <slave_desc> <offset>
`

update_section_mapping

Actualiza una determinada sección en el compilador. El compilador crea un mapa de secciones de acuerdo al orden en que aparecen.

sintaxis: `update_section_mapping <section_name>
<memory_region_name>`

add_default_memory_regions

Configura las regiones de memoria del proyecto BSP en el compilador.

sintaxis: `add_default_memory_regions`

create_bsp

Crea un nuevo proyecto BSP.

sintaxis: `create_bsp <bspType> <bsp version> <processor name>
<sopcininfo>`

generate_bsp

Genera un nuevo proyecto BSP.

sintaxis: `generate_bsp <bspDir>`

get_available_bsp_type_versions

Obtiene versiones de los diferentes tipos de proyectos BSP disponibles (para utilización en el operando **bspType** del comando **create_bsp**).

sintaxis: `get_available_bsp_type_versions <bsp_type>
<sopcininfo_path>`

get_available_bsp_types

Obtiene versiones de los diferentes tipos de proyectos BSP disponibles (para utilización en el operando **bspType** del comando **create_bsp**).

sintaxis: `get_available_bsp_types <sopcininfo_path>`

get_available_cpu_architectures

Obtiene las arquitecturas de procesadores disponibles.

sintaxis: `get_available_cpu_architectures`

get_available_cpu_names

Obtiene los nombres de los procesadores para un determinado sopc.

sintaxis: `get_available_cpu_names <sopcininfo_path>`

get_available_software

Obtiene el software disponible para un determinado tipo de proyecto BSP (controladores, paquetes de software and componentes BSP).

sintaxis: `get_available_software <bsp_type> <filter>
<sopcininfo_path>`

get_available_software_setting_properties

Obtiene el nombre de las propiedades de configuraciones disponibles de un componente de software.

```
sintaxis: get_available_software_setting_properties  
<setting_name> <software_name> <software_version> <sopcinfol_path>
```

get_available_software_settings

Obtiene los nombres de las configuraciones disponibles de los componentes de software.

```
sintaxis: get_available_software_settings <software_name>  
<software_version> <sopcinfol_path>
```

get_bsp_version

Obtiene la versión del sistema operativo del proyecto BSP.

```
sintaxis: get_bsp_version
```

get_cpu_architecture

Obtiene la arquitectura del procesador (por ejemplo, Nios II) de un determinado SOPC.

```
sintaxis: get_cpu_architecture <processor_name>  
<sopcinfol_path>
```

get_nios2_dpx_thread_num

Si el proyecto BSP es manejado por un procesador Nios II DPX, entonces esta función retorna el número de *threads* que el procesador soporta. Retorna *null* en otro caso.

```
sintaxis: get_nios2_dpx_thread_num
```

get_sopcinfol_file

Retorna el directorio del archivo **.sopcinfol** del proyecto BSP actual.

```
sintaxis: get_sopcinfol_file
```

get_supported_bsp_types

Obtiene los tipos de proyectos BSP soportados para un determinado procesador y SOPC.

```
sintaxis: get_supported_bsp_types <processor_name>  
<sopcinfol_path>
```

is_bsp_hal_extension

Retorna un valor booleano que indica si el proyecto BSP está basado en la HAL de Altera.

```
sintaxis: is_bsp_hal_extension
```

is_bsp_lwhal_extension

Retorna un valor booleano que indica si el proyecto BSP está basado en la versión *Lightweight* de la HAL de Altera.

```
sintaxis: is_bsp_lwhal_extension
```

open_bsp

Abre un proyecto BSP existente.

```
sintaxis: open_bsp <settingsFile>
```

save_bsp

Guarda un proyecto BSP.

```
sintaxis: save_bsp <settingsFile>
```

set_bsp_version

Establece la versión del sistema operativo del proyecto BSP en un determinado valor.

```
sintaxis: set_bsp_version <version>
```

set_logging_mode

Establece el nivel de acceso.

sintaxis: `set_logging_mode <mode>`

ANEXO 6

REFERENCIA DE CONFIGURACIONES DE BSP, PAQUETES DE SOFTWARE Y CONTROLADORES

En esta anexo, se listan las diferentes configuraciones que pueden realizarse sobre el proyecto BSP, los paquetes de software y los controladores de dispositivos de un SOPC de Altera. Un detalle completo de estas configuraciones puede encontrarse en el Capítulo 15 de (18).

hal.enable_instruction_related_exceptions_api

Habilita el registro de *handlers* para el servicio de excepciones. Valor por defecto: *false*.

hal.max_file_descriptors

Determina el número de archivos de descriptores estáticamente alojados. Valor por defecto: 32.

hal.exclude_default_exception

Excluye el vector por defecto de excepción. Valor por defecto: *false*. Archivo destino: *system.h*.

hal.sys_clk_timer

Determina el descriptor esclavo del reloj del sistema (*system clock timer device*). Este dispositivo provee una interrupción periódica. Valor por defecto: ninguno.

hal.timestamp_timer

Determina el descriptor esclavo del dispositivo de temporizado de alta resolución. Valor por defecto: ninguno.

ucosii.os_max_tasks

Determina el máximo número de tareas. Valor por defecto: 10.

ucosii.os_lowest_prio

Determina el mínimo valor de prioridad asignable. Valor por defecto: 20.

ucosii.os_thread_safe_newlib

Determina la utilización de la biblioteca C *thread safe*. Valor por defecto: 1.

ucosii.miscellaneous.os_arg_chk_en

Habilita el chequeo de argumentos. Valor por defecto: 1.

ucosii.miscellaneous.os_cpu_hooks_en

Habilita la característica “MicroC/OS-II hooks”. Valor por defecto: 1.

ucosii.miscellaneous.os_debug_en

Habilita las variables para depuración. Valor por defecto: 1.

ucosii.miscellaneous.os_sched_lock_en

Incluye el código para las funciones OSSchedLock() and OSSchedUnlock(). Estas funciones permiten controlar la apropiación de las tareas por el *scheduler* del sistema operativo. Valor por defecto: 1.

ucosii.miscellaneous.os_task_stat_en

Habilita la tarea de estadísticas del sistema operativo. Valor por defecto: 1.

ucosii.miscellaneous.os_task_stat_stk_chk_en

Verifica la pila de las áreas desde la tarea de estadística. Valor por defecto: 1.

ucosii.miscellaneous.os_tick_step_en

Habilita la características *tick stepping* para uCOS-View. Valor por defecto: 1.

ucosii.miscellaneous.os_event_name_size

Tamaño del nombre de grupos *Event Control Block*. Valor por defecto: 32.

ucosii.miscellaneous.os_max_events

Máximo número de *Event Control Blocks*. Valor por defecto: 60.

ucosii.miscellaneous.os_task_idle_stk_size

Tamaño de la pila de la tarea *Idle*. Valor por defecto: 512.

ucosii.miscellaneous.os_task_stat_stk_size

Tamaño de la pila de la tarea de estadísticas. Valor por defecto: 512.

ucosii.task.os_task_change_prio_en

Incluye código para la función OSTaskChangePrio(). Valor por defecto: 1.

ucosii.task.os_task_create_en

Incluye código para la función OSTaskCreate(). Valor por defecto: 1.

ucosii.task.os_task_create_ext_en

Incluye código para la función OSTaskCreateExt(). Valor por defecto: 1.

ucosii.task.os_task_del_en

Incluye código para la función OSTaskDel(). Valor por defecto: 1.

ucosii.task.os_task_name_size

Tamaño del nombre de la tarea. Valor por defecto: 32.

ucosii.task.os_task_profile_en

Incluye estructura de datos para ejecutar el perfil de la tarea (*profiling*). Valor por defecto: 1.

ucosii.task.os_task_query_en

Incluye código para la función OSTaskQuery. Valor por defecto: 1.

ucosii.task.os_task_suspend_en

Incluye código par alas funciones OSTaskSuspend() y OSTaskResume(). Valor por defecto: 1.

ucosii.task.os_task_sw_hook_en

Incluye código para la función OSTaskSwHook(). Valor por defecto: 1.

ucosii.time.os_time_tick_hook_en

Incluye código para la función OSTimeTickHook(). Valor por defecto: 1.

ucosii.time.os_time_dly_resume_en

Incluye código para la función OSTimeDlyResume(). Valor por defecto: 1.

ucosii.time.os_time_dly_hmsm_en

Incluye código para la función OSTimeDlyHMSM(). Valor por defecto: 1.

ucosii.time.os_time_get_set_en

Incluye código para la función OSTimeGet y OSTimeSet(). Valor por defecto: 1.

ucosii.os_flag_en

Habilita el código para *Event Flags*. Esta configuración es requerida para el correcto funcionamiento de los controladores de la HAL de Altera en un ambiente *multithreaded*. Valor por defecto: 1.

ucosii.event_flag.os_flag_wait_clr_en

Incluye código para *Wait on Clear Event Flags*. Esta configuración es requerida para el correcto funcionamiento de los controladores de la HAL de Altera en un ambiente *multithreaded*. Valor por defecto: 1.

ucosii.event_flag.os_flag_accept_en

Incluye código para la función OSFlagAccept(). Esta configuración es requerida para el correcto funcionamiento de los controladores de la HAL de Altera en un ambiente *multithreaded*. Valor por defecto: 1.

ucosii.event_flag.os_flag_del_en

Incluye código para la función OSFlagDel(). Esta configuración es requerida para el correcto funcionamiento de los controladores de la HAL de Altera en un ambiente *multithreaded*. Valor por defecto: 1.

ucosii.event_flag.os_flag_query_en

Incluye código para la función OSFlagQuery(). Esta configuración es requerida para el correcto funcionamiento de los controladores de la HAL de Altera en un ambiente *multithreaded*. Valor por defecto: 1.

ucosii.event_flag.os_flag_name_size

Tamaño del nombre del grupo *Event Flags*. Valor por defecto: 32. Se recomienda no modificarlo.

ucosii.event_flag.os_flags_nbits

Bits (8,16,32) de *Event Flag*. Valor por defecto: 16. Se recomienda no modificarlo.

ucosii.event_flag.os_max_flags

Máximo número de grupos *Event Flags*. Valor por defecto: 20.

ucosii.os_mutex_en

Habilita código para semáforos Mutex. Valor por defecto: 1.

ucosii.mutex.os_mutex_accept_en

Incluye código para la función OSMutexAccept(). Valor por defecto: 1.

ucosii.mutex.os_mutex_del_en

Incluye código para la función OSMutexDel(). Valor por defecto: 1.

ucosii.mutex.os_mutex_query_en

Incluye código para la función OSMutexQuery. Valor por defecto: 1.

ucosii.os_sem_en

Habilita código para semáforos. Esta configuración es requerida para el correcto funcionamiento de los controladores de la HAL de Altera en un ambiente *multithreaded*. Valor por defecto: 1.

ucosii.semaphore.os_sem_accept_en

Incluye código para la función OSSemAccept(). Esta configuración es requerida para el correcto funcionamiento de los controladores de la HAL de Altera en un ambiente *multithreaded*. Valor por defecto: 1.

ucosii.semaphore.os_sem_set_en

Incluye código para la función OSSemSet(). Esta configuración es requerida para el correcto funcionamiento de los controladores de la HAL de Altera en un ambiente *multithreaded*. Valor por defecto: 1.

ucosii.semaphore.os_sem_del_en

Incluye código para la función OSSemDel(). Esta configuración es requerida para el correcto funcionamiento de los controladores de la HAL de Altera en un ambiente *multithreaded*. Valor por defecto: 1.

ucosii.semaphore.os_sem_query_en

Incluye código para la función OSSemQuery(). Esta configuración es requerida para el correcto funcionamiento de los controladores de la HAL de Altera en un ambiente *multithreaded*. Valor por defecto: 1.

ucosii.os_mbox_en

Habilita código para *mailboxes*. Valor por defecto: 1.

ucosii.mailbox.os_mbox_accept_en

Incluye código para la función OSMboxAccept(). Valor por defecto: 1.

ucosii.mailbox.os_mbox_del_en

Incluye código para la función OSMboxDel(). Valor por defecto: 1.

ucosii.mailbox.os_mbox_post_en

Incluye código para la función OSMboxPost(). Valor por defecto: 1.

ucosii.mailbox.os_mbox_post_opt_en

Incluye código para la función OSMboxPostOpt(). Valor por defecto: 1.

ucosii.mailbox.os_mbox_query_en

Incluye código para la función OSMboxQuery(). Valor por defecto: 1.

ucosii.os_q_en

Habilita código para *Queues*. Valor por defecto: 1.

ucosii.queue.os_q_accept_en

Incluye código para la función OSQAccept(). Valor por defecto: 1.

ucosii.queue.os_q_del_en

Incluye código para la función OSQDel(). Valor por defecto: 1.

ucosii.queue.os_q_flush_en

Incluye código para la función OSQFlush(). Valor por defecto: 1.

ucosii.queue.os_q_post_en

Incluye código para la función OSQFlush(). Valor por defecto: 1.

ucosii.queue.os_q_post_front_en

Incluye código para la función OSQPostFront(). Valor por defecto: 1.

ucosii.queue.os_q_post_opt_en

Incluye código para la función OSQPostOpt(). Valor por defecto: 1.

ucosii.queue.os_q_query_en

Incluye código para la función OSQQuery(). Valor por defecto: 1.

ucosii.queue.os_max_qs

Máximo número de *Queue Control Blocks*. Valor por defecto: 20.

ucosii.os_mem_en

Habilita el código para manejo de memoria. Valor por defecto: 1.

ucosii.memory.os_mem_query_en

Incluye código para la función *OSMemQuery()*. Valor por defecto: 1.

ucosii.memory.os_mem_name_size

Tamaño del nombre de la partición de memoria. Valor por defecto: 32.

ucosii.memory.os_max_mem_part

Máximo número de particiones de memoria. Valor por defecto: 60.

ucosii.os_tmr_en

Habilita código para temporizadores. Valor por defecto: 0.

ucosii.timer.os_task_tmr_stk_size

Tamaño de pila para tarea *timer*. Valor por defecto: 512.

ucosii.timer.os_task_tmr_prio

Prioridad de la tarea *timer2* (0 es la mayor prioridad). Valor por defecto: 2.

ucosii.timer.os_tmr_cfg_max

Máximo número de *timers*. Valor por defecto: 16.

ucosii.timer.os_tmr_cfg_name_size

Tamaño del nombre de *timer*. Valor por defecto: 16.

ucosii.timer.os_tmr_cfg_ticks_per_sec

Frecuencia a la que debe ejecutarse la tarea del *timer* (en Hz). Valor por defecto: 10.

ucosii.timer.os_tmr_cfg_wheel_size

Tamaño de la rueda del *timer* (*timer wheel*) en número de rayos (*spokes*). Valor por defecto: 2.

altera_avalon_uart_driver.enable_small_driver

Configuración versión reducida de los controladores UART (*small-footprint (polled mode driver)*). Valor por defecto: *false*.

altera_avalon_uart_driver.enable_ioctl

Habilitación del soporte de entrada/salida ("*ioctl()* support"). Valor por defecto: *false*.

altera_avalon_jtag_uart_driver.enable_small_driver

Configuración versión reducida de los controladores jtag-uart (*small-footprint (polled mode driver)*). Valor por defecto: *false*.

altera_hostfs.hostfs_name

Punto de montaje de sistemas de archivos. Valor por defecto: */mnt/host*.

altera_iniche.iniche_default_if

Obsoleto. Interfaz por defecto de control de acceso al medio (MAC). Valor por defecto: *NOT_USED*.

altera_iniche.enable_dhcp_client

Habilita DHCP para asignación automática de dirección IP. Valor por defecto: *true*.

altera_iniche.enable_ip_fragments

Rensamblar fragmentos de paquetes IP. Valor por defecto: *true*.

altera_iniche.enable_include_tcp

Habilitar el protocolo TCP. Valor por defecto: *true*.

altera_iniche.enable_tcp_zerocopy

Usar TCP zero-copy. Valor por defecto: *false*.

altera_iniche.enable_net_stats

Habilitar estadísticas de red. Valor por defecto: *false*.

altera_ro_zipfs.ro_zipfs_name

Punto de montaje de sistemas de archivos ZIP. Valor por defecto: */mnt/rozipfs*

altera_ro_zipfs.ro_zipfs_offset

Desplazamiento del sistema de archivos desde la base de la memoria *flash*. Valor por defecto: *0x100000*.

altera_ro_zipfs.ro_zipfs_base

Dirección base de la memoria *flash*. Valor por defecto: *0x0*.

hal.linker.allow_code_at_reset

Indica si el código de inicialización es permitido en la dirección de *reset*. Esta configuración es *false* si un programa de carga externo está presente. Valor por defecto: *0*.

hal.linker.enable_alt_load

Habilita la facilidad *alt_load()* que copia secciones *.text* a memoria RAM. Valor por defecto: *1*.

hal.linker.enable_alt_load_copy_exceptions

Causa que la facilidad *alt_load()* copia las secciones *.exceptions*. Valor por defecto: *0*.

hal.linker.enable_alt_load_copy_rodata

Causa que la facilidad *alt_load()* copia las secciones *.rodata*. Valor por defecto: *0*.

hal.linker.enable_alt_load_copy_rwdata

Causa que la facilidad *alt_load()* copia las secciones *.rwdata*. Valor por defecto: *0*.

hal.linker.enable_exception_stack

Habilita el uso de una pila separada para excepciones. Valor por defecto: *0*.

hal.linker.exception_stack_memory_region_name

Nombre de una región de memoria existente para ser dividida y crear la región de pila de excepción. Valor por defecto: *none*.

hal.linker.exception_stack_size

Tamaño de la pila de excepciones en *byte*. Valor por defecto: *1024*.

hal.linker.enable_interrupt_stack

Habilita el uso de una pila separada para interrupciones. Valor por defecto: *0*.

hal.linker.interrupt_stack_memory_region_name

Nombre de una región de memoria existente para ser dividida y crear la región de pila de interrupción. Valor por defecto: *none*.

hal.linker.interrupt_stack_size

Tamaño de la pila de interrupción en *byte*. Valor por defecto: *1024*.

hal.make.ar

Comando para la creación de archivos de bibliotecas. Valor por defecto: *nios2-elf-ar*.

hal.make.ar_post_process

Comando para ser ejecutado luego de la creación de archivos de bibliotecas. Valor por defecto: *none*.

hal.make.ar_pre_process

Comando para ser ejecutado antes de la creación de bibliotecas. Valor por defecto: *none*.

hal.make.as

Comando del ensamblador. Valor por defecto: nios2-elf-gcc.

hal.make.as_post_process

Comando para ser ejecutado luego de que cada archive ensamblador es ensamblado. Valor por defecto: *none*.

hal.make.as_pre_process

Comando para ser ejecutado antes de que cada archive ensamblador es ensamblado. Valor por defecto: *none*.

hal.make.bsp_arflags

Declaraciones pasadas al compilador de bibliotecas. Valor por defecto: -src.

hal.make.bsp_asflags

Definiciones del usuario pasadas directamente al ensamblador. Valor por defecto: -Wa,-gdwarf2.

hal.make.bsp_cflags_debug

Configura el nivel de depuración (*debug*) del compilador C/C++. Valor por defecto: -g.

hal.make.bsp_cflags_defined_symbols

Preprocesador de las definiciones de macros. Valor por defecto: *none*.

hal.make.bsp_cflags_optimization

Nivel de optimización del compilador C/C++. Valor por defecto: -O0.

hal.make.bsp_cflags_undefined_symbols

Preprocesador de las macros no definidas (en las cláusulas #undef).

hal.make.bsp_cflags_user_flags

Definiciones pasadas al compilador cuando compila archivos C, C++ y .S. Valor por defecto: *none*.

hal.make.bsp_cflags_warnings

Configuración del nivel de compilación del compilador C/C++. Valor por defecto: -Wall.

hal.make.bsp_cxx_flags

Definiciones pasadas al compilador C++. Valor por defecto: *none*.

hal.make.bsp_inc_dirs

Texto que contiene el *path* donde el compilador busca las definiciones de los encabezamientos de las funciones. Valor por defecto: *none*.

hal.make.build_post_process

Comando a ejecutar luego de que el proyecto BSP es construido. Valor por defecto: *none*.

hal.make.build_pre_process

Comando a ejecutar antes de que el proyecto BSP es construido. Valor por defecto: *none*.

hal.make.cc

Comando del compilador C. Valor por defecto: nios2-elf-gcc -xc.

hal.make.cc_post_process

Comando ejecutado luego de que cada archive .c o .S es compilado. Valor por defecto: *none*.

hal.make.cc_pre_process

Comando ejecutado antes de que cada archive .c o .S es compilado. Valor por defecto: *none*.

hal.make.cxx

Comando del compilador C++. Valor por defecto: nios2-elf-gcc -xc++.

hal.make.cxx_post_process

Comando ejecutado luego de que cada archive C++ es compilado. Valor por defecto: *none*.

hal.make.cxx_pre_process

Comando ejecutado antes de que cada archive C++ es compilado. Valor por defecto: *none*.

hal.make.ignore_system_derived.big_endian

Habilita la generación del proyecto BSP consultando, si el sistema SoPC tiene representación *big endian*. Valor por defecto: 0.

hal.make.ignore_system_derived.fpu_present

Habilita la generación del proyecto BSP consultando, si el sistema SoPC tiene unidad FPU presente. Valor por defecto: 0.

hal.make.ignore_system_derived.hardware_divide_present

Habilita la generación del proyecto BSP consultando, si el sistema SoPC tiene división por hardware presente. Valor por defecto: 0.

hal.make.ignore_system_derived.hardware_fp_cust_inst_divider_present

Habilita la generación del proyecto BSP consultando si el sistema SoPC tiene división en punto flotante presente. Valor por defecto: 0.

hal.make.ignore_system_derived.hardware_fp_cust_inst_no_divider_present

Habilita la generación del proyecto BSP consultando, si el sistema SoPC tiene unidad de punto flotante sin división presente. Valor por defecto: 0.

hal.make.ignore_system_derived.sopc_simulation_enabled

Habilita la generación del proyecto BSP consultando, si el sistema SoPC tiene simulación habilitada. Valor por defecto: 0.

hal.make.ignore_system_derived.debug_core_present

Habilita la generación del proyecto BSP consultando, si el sistema SoPC tiene unidad de depuración presente. Valor por defecto: 0.

hal.make.ignore_system_derived.hardware_multiplier_present

Habilita la generación del proyecto BSP consultando, si el sistema SoPC tiene multiplicador presente. Valor por defecto: 0.

hal.make.ignore_system_derived.hardware_mux_present

Habilita la generación del proyecto BSP consultando, si el sistema SoPC tiene multiplicador por hardware presente. Valor por defecto: 0.

hal.make.ignore_system_derived.sopc_system_base_address

Habilita la generación del proyecto BSP consultando, si el sistema SoPC por la identificación del sistema (*system id*). Valor por defecto: 0.

hal.make.ignore_system_derived.sopc_system_id

Habilita la generación del proyecto BSP consultando, si el sistema SoPC por la dirección base de la identificación del sistema (*system id*). Valor por defecto: 0.

hal.make.ignore_system_derived.sopc_system_timestamp

Habilita la generación del proyecto BSP consultando por la estampa de tiempo del sistema. Valor por defecto: 0.

hal.make.rm

Comando utilizado para eliminar archivos cuando se realiza una construcción limpia (*clean target*). Valor por defecto: `rm -f`.

hal.custom_newlib_flags

Construir una versión personalizada de newlib con definiciones separadas por espacios en el compilador. Valor por defecto: *none*.

hal.enable_c_plus_plus

Habilita soporte para el lenguaje C++. Valor por defecto: 1.

hal.enable_clean_exit

Habilita el *exit* limpio que destruye todas las llamadas C++, cierra todos los archivos, etc. Deshabilitando esta característica reduce el requerimiento de memoria del código generado. Valor por defecto: 1.

hal.enable_exit

Incluye soporte de la function *exit()*. Valor por defecto: 1.

hal.enable_gprof

Causa que el código compilado tenga habilitado el perfilado de código *gprof*". Valor por defecto: 0.

hal.enable_lightweight_device_driver_api

Habilita la versión de controladores *lightweight* de la interfaz API de la HAL de Altera. Valor por defecto: 0.

hal.enable_mul_div_emulation

Incluye código al proyecto BSP para emular las instrucciones de multiplicación y división. Valor por defecto: 0.

hal.enable_reduced_device_drivers

Habilita la versión reducida de los controladores de la HAL de Altera. Valor por defecto: 0.

hal.enable_runtime_stack_checking

Habilita la opción de la HAL de chequeo de la pila durante tiempo de corrida. Valor por defecto: 0.

hal.enable_sim_optimize

El proyecto BSP es compilado con optimizaciones para la simulación del HDL tales como inicialización de caché, borrado de la sección *.bss* y salto de los lazos de retardo largo. Valor por defecto: 0.

hal.enable_small_c_library

Causa que la versión pequeña de la biblioteca newlib sea utilizada. Valor por defecto: 0.

hal.enable_sopc_sysid_check

Habilita la verificación de la identificación del sistema (*ID check*). Valor por defecto: 1.

hal.log_port

Habilita el descriptor esclavo para que mensajes de la HAL sean mostrados en el dispositivo en modo de caracteres. Valor por defecto: *none*.

hal.log_flags

Valor asignado a la definición *ALT_LOG_FLAGS* en el archive generado *public.mk*. Valor por defecto: 0.

hal.stderr

Descriptor esclavo del dispositivo en modo-carácter para consola *STDERR*. Valor por defecto: *none*.

hal.stdin

Descriptor esclavo del dispositivo en modo-carácter para consola *STDIN*. Valor por defecto: *none*.

hal.stdout

Descriptor esclavo del dispositivo en modo-carácter para consola STDOUT. Valor por defecto: *none*.

ANEXO 7

FUNCIONES Y ESTRUCTURAS DE LA HAL DE ALTERA

En este anexo, se listan las diferentes funciones y estructuras definidas en la HAL de Altera accesibles desde el proyecto de aplicación. Estas funciones y estructuras definen la interfaz API (“Interface del programa de aplicación”, *Application Program Interface*, en inglés) que ofrece la HAL hacia el hardware del sistema. La descripción de estas funciones tiene el solo objetivo de cubrir las características que brinda la HAL de Altera mediante las diferentes funciones que ofrece y no una documentación detallada de cada una de ellas. Un detalle completo de estas funciones y estructuras puede encontrarse en el Capítulo 14 de (18). Existen otras funciones, como por ejemplo `printf`, que son provistas por la biblioteca *newlib* y que no son descriptas en este anexo.

FUNCIONES DE LA API DE LA HAL DE ALTERA

`_exit()`

La función `exit()` de la biblioteca *newlib* llama a la función `_exit()` para finalizar el proceso actual. Típicamente, la función `exit()` llama a esta función cuando la función `main()` se complete. La HAL de Altera es un solo proceso y se bloquea a su finalización para siempre, pero las interrupciones no se deshabilitan y los servicios de interrupción (ISR) continúan ejecutándose.

Prototipo: `void _exit (int exit_code)`
Archivo “include”: `<unistd.h>`
Retorna: -

`_rename()`

Esta función es provista para compatibilidad con la biblioteca *newlib*.

Prototipo: `int _rename(char *existing, char* new)`
Archivo “include”: `<stdio.h>`
Retorna: Siempre falla retornando -1, y con `errno` configurado a `ENOSYS`.

alt_alarm_start()

Esta función diagrama la llamada de una señal de alarma al pasar *ntick* cuentas del reloj. La información de la alarma está en la estructura **alarm*, y la función que es llamada está direccionada por **callback* que es llamada con el argumento *context*.

Prototipo: `int alt_alarm_start ( alt_alarm* alarm, alt_u32 nticks, alt_u32 (*callback) (void* context), void* context )`

Archivo "include": `<sys/alt_alarm.h>`

Retorna: 0 si fue sin error o negativo en caso contrario. Falla si no existe reloj del sistema disponible.

alt_alarm_stop()

Permite cancelar una alarma previamente diagramada.

Prototipo: `void alt_alarm_stop (alt_alarm* alarm)`

Archivo "include": `<sys/alt_alarm.h>`

Retorna: -

alt_dcache_flush()

Realiza un *flush* de una región de caché de longitud *len*.

Prototipo: `void alt_dcache_flush (void* start, alt_u32 len)`

Archivo "include": `<sys/alt_cache.h>`

Retorna: -

alt_dcache_flush_all()

Realiza un *flush* a toda la memoria caché de datos.

Prototipo: `void alt_dcache_flush_all (void)`

Archivo "include": `<sys/alt_cache.h>`

Retorna: -

alt_dev_reg()

Registra un dispositivo en el sistema. Después de ser registrado, se puede acceder al dispositivo mediante las funciones de entrada/salida.

Prototipo: `int alt_dev_reg(alt_dev* dev)`

Archivo "include": `<sys/alt_dev.h>`

Retorna: 0 si se complete con éxito, negativo en caso contrario.

alt_dma_rxchan_close()

Notifica al sistema que una aplicación ha finalizado la utilización del canal de acceso directo a memoria (DMA).

Prototipo: `int alt_dma_rxchan_close (alt_dma_rxchan rxchan)`

Archivo "include": `<sys/alt_dma.h>`

Retorna: 0 si se complete con éxito, negativo en caso contrario.

alt_dma_rxchan_depth()

Retorna el máximo número de requerimientos de recepción que pueden ser enviados a un canal de transmisión de DMA.

Prototipo: `alt_u32 alt_dma_rxchan_depth(alt_dma_rxchan dma)`

Archivo "include": `<sys/alt_dma.h>`

Retorna: Retorna el máximo número de requerimientos de lectura que pueden ser enviados.

alt_dma_rxchan_ioctl()

Realiza operaciones de entrada/salida con un canal de recepción de DMA.

Prototipo: `int alt_dma_rxchan_ioctl (alt_dma_rxchan dma, int req, void* arg)`

Archivo "include": `<sys/alt_dma.h>`

Retorna: Un número negativo si hubo error

alt_dma_rxchan_open()

Obtiene el descriptor para un determinado canal de recepción de DMA. Por ejemplo: `"/dev/dma_0"`.

Prototipo: `alt_dma_rxchan alt_dma_rxchan_open (const char* name)`

Archivo `"include": <sys/alt_dma.h>`

Retorna: nada si la operación fue exitosa

alt_dma_rxchan_prepare()

Envía un requerimiento de recepción a un canal de recepción de DMA.

Prototipo: `int alt_dma_rxchan_prepare (alt_dma_rxchan dma, void* data, alt_u32 length, alt_rxchan_done* done, void* handle)`

Archivo `"include": <sys/alt_dma.h>`

Retorna: 0 si la ejecución es exitosa, negativo en caso contrario.

alt_dma_rxchan_reg()

Registra un canal de recepción de DMA en el sistema.

Prototipo: `int alt_dma_rxchan_reg (alt_dma_rxchan_dev* dev)`

Archivo `"include": <sys/alt_dma_dev.h>`

Retorna: 0 si la ejecución es exitosa, negativo en caso contrario.

alt_dma_txchan_close()

Notifica al sistema que la aplicación finalice el uso de un canal de transmisión de DMA.

Prototipo: `int alt_dma_txchan_close (alt_dma_txchan txchan)`

Archivo `"include": <sys/alt_dma.h>`

Retorna: 0 si la ejecución es exitosa, negativo en caso contrario.

alt_dma_txchan_ioctl()

Realiza operación sobre dispositivos de entrada/salida específicos que involucran canales de transmisión de DMA.

Prototipo: `int alt_dma_txchan_ioctl (alt_dma_txchan dma, int req, void* arg)`

Archivo `"include": <sys/alt_dma.h>`

Retorna: Un valor negativo indica error.

alt_dma_txchan_open()

Obtiene el descriptor para un canal de transmisión de DMA. Por ejemplo: `"/dev/dma_0"`.

Prototipo: `alt_dma_txchan alt_dma_txchan_open (const char* name)`

Archivo `"include": <sys/alt_dma.h>`

Retorna: Nada en caso exitoso.

alt_dma_txchan_reg()

Registra un canal de transmisión de DMA en el sistema.

Prototipo: `int alt_dma_txchan_reg (alt_dma_txchan_dev* dev)`

Archivo `"include": <sys/alt_dma_dev.h>`.

Retorna: 0 en caso éxitos, negativo en caso contrario.

alt_dma_txchan_send()

Envía un requerimiento de transmisión a un canal de transmisión de DMA.

Prototipo: `int alt_dma_txchan_send (alt_dma_txchan dma, const void* from, alt_u32 length, alt_txchan_done* done, void* handle)`

Archivo `"include": <sys/alt_dma.h>`

Retorna: 0 si se ejecutó exitosamente, negativo en caso contrario.

alt_dma_txchan_space()

Retorna el número de requerimientos de transmisión que pueden ser enviados a un canal específico de transmisión de DMA.

Prototipo: `int alt_dma_txchan_space (alt_dma_txchan dma)`

Archivo `"include": <sys/alt_dma.h>`

Retorna: retorna el número de requerimientos de transmisión que pueden ser enviados.

alt_erase_flash_block()

Elimina un determinado bloque del dispositivo de memoria *flash*.

Prototipo: `int alt_erase_flash_block(alt_flash_fd* fd, int offset, int length)`

Archivo "include": `<sys/alt_flash.h>`

Retorna: 0 en caso exitoso, negativo en caso contrario.

alt_exception_cause_generated_bad_addr()

Valida el argumento **bad_addr** a un manejador de excepción.

Prototipo: `int alt_exception_cause_generated_bad_addr (alt_exception_cause cause)`

Archivo "include": `<sys/alt_exceptions.h>`

Retorna: Distinto de 0 significa que "bad_addr" contiene la dirección que causó la excepción. Cero significa que "bad_addr" debe ser ignorado.

alt_flash_close_dev()

Cierra un dispositivo de memoria *flash*.

Prototipo: `void alt_flash_close_dev(alt_flash_fd* fd)`

Archivo "include": `<sys/alt_flash.h>`

Retorna: -

alt_flash_open_dev()

Abre un dispositivo de memoria *flash*.

Prototipo: `alt_flash_fd* alt_flash_open_dev(const char* name)`

Archivo "include": `<sys/alt_flash.h>`

Retorna: 0 si existió fallo, cualquier otro valor en caso contrario.

alt_fs_reg()

Registra un sistema de archivos en la HAL del sistema.

Prototipo: `int alt_fs_reg (alt_dev* dev)`

Archivo "include": `<sys/alt_dev.h>`

Retorna: The return value is zero upon success. A negative return value indicates failure.

alt_get_flash_info()

Obtiene detalles de la región eliminada en parte del dispositivo de memoria *flash*.

Prototipo: `int alt_get_flash_info(alt_flash_fd* fd, flash_region** info, int* number_of_regions)`

Archivo "include": `<sys/alt_flash.h>`

Retorna: 0 si fue exitosa, negativo en otro caso.

alt_ic_irq_disable()

Deshabilita una interrupción simple.

Prototipo: `int alt_ic_irq_disable (alt_u32 ic_id, alt_u32 irq)`

Archivo "include": `<sys/alt_irq.h>`

Retorna: 0 si fue exitosa, distinto a cero en caso contrario.

alt_ic_irq_enable()

Habilita una interrupción simple.

Prototipo: `int alt_ic_irq_enable (alt_u32 ic_id, alt_u32 irq)`

Archivo "include": `<sys/alt_irq.h>`

Retorna: 0 si fue exitosa, distinto a cero en caso contrario.

alt_ic_irq_enabled()

Determina si una interrupción específica está habilitada.

Prototipo: int alt_ic_irq_enabled (alt_u32 ic_id, alt_u32 irq)
 Archivo "include": <sys/alt_irq.h>
 Retorna: 0 si fue exitosa, distinto a cero en caso contrario

alt_ic_isr_register()

Registra una rutina de servicio a interrupción (ISR).

Prototipo: int alt_ic_isr_register (alt_u32 ic_id, alt_u32 irq,
 alt_isr_func isr, void* isr_context, void* flags)
 Archivo "include": <sys/alt_irq.h>
 Retorna: 0 si fue exitosa, distinto a cero en caso contrario

alt_icache_flush()

Invalida una región de memoria de la caché de instrucciones.

Prototipo: void alt_icache_flush (void* start, alt_u32 len)
 Archivo "include": <sys/alt_cache.h>
 Retorna: -

alt_icache_flush_all()

Invalida toda la memoria de la caché de instrucciones.

Prototipo: void alt_icache_flush_all (void)
 Archivo "include": <sys/alt_cache.h>
 Retorna: -

alt_instruction_exception_register()

Registra un manejador de excepción de instrucción.

Prototipo: void alt_instruction_exception_register (alt_exception_result
 (*handler) (alt_exception_cause cause, alt_u32 exception_pc, alt_u32
 bad_addr))
 Archivo "include": <sys/alt_exceptions.h>
 Retorna: -

alt_irq_disable()

Deshabilita una interrupción simple.

Prototipo: int alt_irq_disable (alt_u32 id)
 Archivo "include": <sys/alt_irq.h>
 Retorna: 0.

alt_irq_disable_all()

Deshabilita todas las interrupciones enmascarables.

Prototipo: alt_irq_context alt_irq_disable_all (void)
 Archivo "include": <sys/alt_irq.h>
 Retorna: -

alt_irq_enable()

Habilita una interrupción específica.

Prototipo: int alt_irq_enable (alt_u32 id)
 Archivo "include": <sys/alt_irq.h>
 Retorna: 0.

alt_irq_enable_all()

Habilita todas las interrupciones previamente deshabilitadas con la función

alt_irq_enable_all().

Prototipo: void alt_irq_enable_all (alt_irq_context context)
 Archivo "include": <sys/alt_irq.h>
 Retorna: -

alt_irq_enabled()

Determina si una interrupción enmascarable está habilitada.

Prototipo: int alt_irq_enabled (void)

Archivo "include": <sys/alt_irq.h>

Retorna: 0 si la interrupción está deshabilitada y no cero en caso contrario.

alt_irq_register()

Registra una subrutina de servicio a la interrupción (ISR).

Prototipo: int alt_irq_register (alt_u32 id, void* context, void (*isr)(void*, alt_u32))

Archivo "include": <sys/alt_irq.h>

Retorna: 0 si se ejecutó exitosamente, no cero en caso contrario.

alt_llist_insert()

Inserta una entrada doblemente enlazada en la lista.

Prototipo: void alt_llist_insert(alt_llist* list, alt_llist* entry)

Archivo "include": <sys/alt_llist.h>

Retorna: -

alt_llist_remove()

Elimina una entrada doblemente enlazada en la lista.

Prototipo: void alt_llist_remove(alt_llist* entry)

Archivo "include": <sys/alt_llist.h>

Retorna: -

alt_load_section()

Permite cargar secciones manualmente en el arranque del sistema (*bootloading*).

Prototipo: void alt_load_section(alt_u32* from, alt_u32* to, alt_u32* end)

Archivo "include": <sys/alt_load.h>

Retorna: -

alt_nticks()

Retorna el número de *ticks* de reloj producidos desde la inicialización del sistema.

Prototipo: alt_u32 alt_nticks (void)

Archivo "include": <sys/alt_alarm.h>

Retorna: número de "ticks" del reloj desde la inicialización del sistema. 0 si no existe reloj de sistema

alt_read_flash()

Lee datos desde un dispositivo de memoria *flash*.

Prototipo: int alt_read_flash(alt_flash_fd* fd, int offset, void* dest_addr, int length)

Archivo "include": <sys/alt_flash.h>

Retorna: The return value is zero on success and nonzero otherwise.

alt_remap_cached()

Permite remapear una región de memoria para acceso de caché.

Prototipo: void* alt_remap_cached (volatile void* ptr, alt_u32 len);

Archivo "include": <sys/alt_cache.h>

Retorna: The return value for this function is the remapped memory region.

alt_remap_uncached()

Permite remapear una región de memoria para acceso no caché.

Prototipo: volatile void* alt_remap_uncached (void* ptr, alt_u32 len);

Archivo "include": <sys/alt_cache.h>

Retorna: The return value for this function is the remapped memory region.

alt_sysclk_init()

Registra la presencia de un controlador de reloj del sistema.

Prototipo: int alt_sysclk_init (alt_u32 nticks)
Archivo "include": <sys/alt_alarm.h>
Retorna: 0 si es exitosa, negativo en caso contrario

alt_tick()

Esta función es llamada por el reloj del sistema cada *tick*. Se ejecuta como rutina de servicio a la interrupción del *timer* del sistema.

Prototipo: void alt_tick (void)
Archivo "include": <sys/alt_alarm.h>
Retorna: -

alt_ticks_per_second()

Retorna la cantidad de *ticks* que el reloj del sistema produce por segundo.

Prototipo: alt_u32 alt_ticks_per_second (void)
Archivo "include": <sys/alt_alarm.h>
Retorna: Retorna la cantidad de "ticks" que el reloj del sistema produce por segundo, 0 si no existe reloj del sistema

alt_timestamp()

Retorna el valor actual de la estampilla de tiempo.

Prototipo: alt_u32 alt_timestamp (void)
Archivo "include": <sys/alt_timestamp.h>
Retorna: Retorna el valor actual de la estampilla de tiempo.

alt_timestamp_freq()

Retorna la frecuencia con que es incrementada la estampilla de tiempo del sistema.

Prototipo: alt_u32 alt_timestamp_freq (void)
Archivo "include": <sys/alt_timestamp.h>
Retorna: Retorna la frecuencia con que es incrementada la estampilla de tiempo del sistema.

alt_timestamp_start()

Inicia el temporizador de la estampilla de tiempo.

Prototipo: int alt_timestamp_start (void)
Archivo "include": <sys/alt_timestamp.h>
Retorna: 0 si se ejecutó exitosamente, negativo en caso contrario.

alt_uncached_free()

Libera memoria.

Prototipo: void alt_uncached_free (volatile void* ptr)
Archivo "include": <sys/alt_cache.h>
Retorna: -

alt_uncached_malloc()

Aloja una determinada región de memoria para su utilización.

Prototipo: volatile void* alt_uncached_malloc (size_t size)
Archivo "include": <sys/alt_cache.h>
Retorna: El puntero a la región de memoria asignada.

alt_write_flash()

Escribe datos al dispositivo de memoria *flash*.

Prototipo: int alt_write_flash(alt_flash_fd* fd, int offset, const void* src_addr, int length)
Archivo "include": <sys/alt_flash.h>
Retorna: The return value is zero on success and nonzero otherwise.

alt_write_flash_block()

Escribe un bloque de datos al dispositivo de memoria *flash*.

Prototipo: `int alt_write_flash_block(alt_flash_fd* fd, int block_offset, int data_offset, const void *data, int length)`

Archivo "include": `<sys/alt_flash.h>`

Retorna: 0 si se ejecutó exitosamente, negative caso contrario.

close()

Función `close()` estándar para el cierre de un determinado recurso abierto con la función `open()`.

Prototipo: `int close (int fd)`

Archivo "include": `<unistd.h>`

Retorna: 0 si se ejecutó exitosamente, -1 en caso contrario.

execve()

Función provista para compatibilidad con la biblioteca.

Prototipo: `int execve(const char *path, char *const argv[], char *const envp[])`

Archivo "include": `<unistd.h>`

Retorna: siempre falla y retorna -1.

fcntl()

Permite modificar los *flags* de un descriptor abierto.

Prototipo: `int fcntl(int fd, int cmd)`

Archivo "include": `<unistd.h>`

`<fcntl.h>`

Retorna: ver documentación.

fork()

Provista para compatibilidad con la biblioteca `newlib`.

Prototipo: `pid_t fork (void)`

Archivo "include": `<unistd.h>`

Retorna: siempre falla, retornando -1

fstat()

Obtiene información sobre un descriptor abierto.

Prototipo: `int fstat (int fd, struct stat *st)`

Archivo "include": `<sys/stat.h>`

Retorna: 0 si es exitosa, -1 en caso contrario.

getpid()

Provisto para compatibilidad con la biblioteca `newlib`.

Prototipo: `pid_t getpid (void)`

Archivo "include": `<unistd.h>`

Retorna: Puesto que la HAL de Altera no puede tener múltiples procesos, `getpid()` siempre retorna el mismo número de identificación ID.

gettimeofday()

Obtiene el tiempo actual en una estructura de tiempo.

Prototipo: `int gettimeofday(struct timeval *ptimeval, struct timezone *ptimezone)`

Archivo "include": `<sys/time.h>`

Retorna: 0 si se completó exitosamente, `-ENOTSUP` si no hay reloj disponible.

ioctl()

Permite manipular dispositivos de entrada/salida con características particulares. Es equivalente a la función `ioctl()` estándar de `UNIX`.

Prototipo: `int ioctl (int fd, int req, void* arg)`
Archivo "include": `<sys/ioctl.h>`
Retorna: Ver documentación.

isatty()

Determina si un dispositivo asociado con un descriptor de archivo es un dispositivo terminal.

Prototipo: `int isatty(int fd)`
Archivo "include": `<unistd.h>`
Retorna: 1 si el dispositivo es un dispositivo de carácter, 0 en caso contrario.

kill()

Envía señales a los procesos.

Prototipo: `int kill(int pid, int sig)`
Archivo "include": `<signal.h>`
Retorna: 0 si se ejecuta exitosamente, -1 en caso contrario.

link()

Provista para compatibilidad con la biblioteca newlib.

Prototipo: `int link(const char *_path1, const char *_path2)`
Archivo "include": `<unistd.h>`
Retorna: Siempre falla retornando -1.

lseek()

Mueve el puntero de lectura/escritura asociado con el descriptor de un archivo.

Prototipo: `off_t lseek(int fd, off_t ptr, int whence)`
Archivo "include": `<unistd.h>`
Retorna: positive o cero en caso exitoso, -1 en caso de error.

open()

Abre un archivo o dispositivo y devuelve el descriptor.

Prototipo: `int open (const char* pathname, int flags, mode_t mode)`
Archivo "include": `<unistd.h>`, `<fcntl.h>`
Retorna: el Nuevo descriptor del archive o dispositivo.

read()

Permite la lectura de datos desde un archivo o dispositivo abierto mediante la función `open()`.

Prototipo: `int read(int fd, void *ptr, size_t len)`
Archivo "include": `<unistd.h>`
Retorna: el número de bytes leídos.

sbrk()

Extiende dinámicamente el segmento de datos de una aplicación.

Prototipo: `caddr_t sbrk(int incr)`
Archivo "include": `<unistd.h>`
Retorna: -

settimeofday()

Estable el tiempo del sistema.

Prototipo: `int settimeofday (const struct timeval *t, const struct timezone *tz)`
Archivo "include": `<sys/time.h>`
Retorna: 0 si se ejecutó exitosamente, -1 si no existe reloj del sistema.

stat()

Similar a "`fstat()`", pero toma como argumento el nombre del archivo en lugar de su descriptor.

Prototipo: `int stat(const char *file_name, struct stat *buf);`

Archivo "include": <sys/stat.h>
 Retorna: -

times()

Provee compatibilidad con la función de la biblioteca newlib.

Prototipo: clock_t times (struct tms *buf)
 Archivo "include": <sys/times.h>
 Retorna: 0 si no existe reloj del sistema.

unlink()

Provee compatibilidad con la función de la biblioteca newlib.

Prototipo: int unlink(char *name)
 Archivo "include": <unistd.h>
 Retorna: Siempre falla, retornando -1.

usleep()

Bloquea la ejecución hasta que transcurran los milisegundos especificados.

Prototipo: int usleep (unsigned int us)
 Archivo "include": <unistd.h>
 Retorna: 0 si se ejecutó exitosamente, -1 en caso contrario.

wait()

Espera hasta que un proceso hijo (*child*) ejecuta la función exit(). Debido a que la HAL de Altera no soporta la ejecución de procesos hijos, esta función retorna inmediatamente.

Prototipo: int wait(int *status)
 Archivo "include": <sys/wait.h>
 Retorna: El contenido de "status" es siempre -1 que indica que no hay procesos hijos. La function retorna siempre -1 y el flag ECHILD en errno indicando que no existen procesos hijos a los cuales esperar.

write()

Escribe un bloque de datos a un archivo o a un dispositivo de escritura. Deben haber sido abierto con la función open().

Prototipo: int write(int fd, const void *ptr, size_t len)
 Archivo "include": <unistd.h>
 Retorna: el número de bytes escrito o -1 si existió error.

ESTRUCTURAS ESTÁNDAR DE LA HAL DE ALTERA

Altera define tipos de datos estándar en su interfaz HAL con el fin de mejorar la portabilidad. Estos tipos de datos son definidos en el archivo de encabezados `alt_types.h`. La Tabla A7. 1 lista estos tipos de datos de la HAL de Altera:

TABLA A7. 1: TIPOS DE DATOS ESTÁNDAR DE LA HAL DE ALTERA

Tipo de dato	Descripción
alt_8	Signed 8-bit integer.
alt_u8	Unsigned 8-bit integer.
alt_16	Signed 16-bit integer.
alt_u16	Unsigned 16-bit integer.
alt_32	Signed 32-bit integer.
alt_u32	Unsigned 32-bit integer.
alt_64	Signed 64-bit integer.
alt_u64	Unsigned 64-bit integer.

ANEXO 8

BIBLIOTECA DE CÓDIGO ABIERTO

NEWLIB

INTRODUCCIÓN

La plataforma Nios II SBT brinda el soporte para la creación de sistemas basados en la HAL de Altera que incluye compatibilidad con la biblioteca de código abierto newlib. En este anexo, se listan las diferentes funciones definidas en cada uno de los archivos de definiciones de la biblioteca newlib ((28)). Esta descripción tiene por objetivo enumerar las diferentes funciones para determinar las características que brinda esta biblioteca, pero un mayor detalle requerirá la documentación.

TIPOS DE DATOS EN C

Las diferentes funciones de la biblioteca newlib están definidas sobre el siguiente conjunto de tipos de datos estándar de ANSI C:

Tipo	Tamaño	Descripción	Nombres alternativos
<i>char</i>	1 byte	Caracteres en formato, por ejemplo, ASCII	
<i>signed char</i>	1 byte	Enteros entre -128 y 127	
<i>unsigned char</i>	1 byte	Enteros entre 0 y 255	
<i>short</i>	2 bytes	Enteros entre -32.768 y 32.767	<i>short int</i> , <i>signed short</i> <i>signed short int</i>
<i>unsigned short</i>	2 bytes	Enteros entre 0 y 65.535	<i>unsigned short int</i>
<i>int</i>	4 bytes	Entero entre -2.147.483.648 y +2.147.483.647	<i>signed</i> , <i>signed int</i>
<i>unsigned int</i>	4 bytes	Entero entre 0 y +4.294.967.295	<i>unsigned</i>

<i>long</i>	4 bytes	Entero entre -2.147.483.648 y +2.147.483.647	<i>long int, signed long, signed long int</i>
<i>unsigned long</i>	4 bytes	Entero entre 0 y +4,294,967,295	
<i>long long</i>	8 bytes	Entero entre -9.223.372.036.854.775.808 y 9.223.372.036.854.775.807	<i>long long int, signed long long, signed long long int</i>
<i>unsigned long long</i>	8 bytes	Entero entre 0 y 18.446.744.073.709.551.616	
<i>float</i>	4 bytes	Formato de punto flotante IEEE de simple-precisión	
<i>double</i>	8 bytes	Formato de punto flotante IEEE de doble-precisión	
<i>long double</i>	12 bytes	Formato de punto flotante IEEE de precisión <i>long</i>	

Existen definiciones de tipos de datos especiales. Por ejemplo, *wide-character*, adecuados para representar alfabetos como el Kanji y que consta de una secuencia de uno o más *bytes*.

FUNCIONES DE UTILIDADES ESTÁNDAR (STDLIB.H)

Esta biblioteca agrupa las funciones utilizadas en una gran variedad de programas. El archivo **stdlib.h** define los encabezados de estas funciones.

Función	Descripción
_Exit	Finaliza la ejecución de un programa sin liberar memoria.
a64l	Conversión de <i>string</i> a <i>long long</i> .
abort	Terminación anormal de un programa.
abs	Valor absoluto.
assert	Macro para el diagnóstico en depuración.
atexit	Requerir ejecución de funciones al salir del programa.
atof	Conversión de <i>string</i> a <i>double</i> o <i>float</i> .
atoi	Conversión de <i>string</i> a entero.
atoll	Conversión de <i>string</i> a <i>long long</i> .
bsearch	Búsqueda binaria.
calloc	Aloja espacio para arreglos.
div	Divide dos enteros.
ecvtbuf	Conversión <i>double</i> o <i>float</i> a <i>string</i> .
ecvt	Conversión <i>double</i> o <i>float</i> a <i>string</i> .
__env_lock	Apropiar variables de ambiente para su modificación.
gvcvt	Formatea <i>double</i> o <i>float</i> como <i>string</i> .
exit	Finaliza ejecución de programa.
getenv	Obtiene variable de ambiente.
labs	Valor absoluto de entero <i>long</i> .
ldiv	División de enteros <i>long</i> .

labs	Valor absoluto de <i>long long</i> .
ldiv	División de enteros <i>long long</i> .
malloc	Aloja y maneja memoria dinámicamente.
mallinfo	Obtiene información de memoria alojada.
__malloc_lock	Apropiación de memoria para <i>malloc</i> y <i>free</i> .
mbsrtowcs	Convierte un <i>string</i> a <i>wide-character string</i> .
mbstowcs	Convierte <i>multibyte string</i> a <i>wide string</i> .
mblen	Longitud de <i>string</i> .
mbtowc	Conversión de <i>multibyte</i> a <i>wide-character</i> .
on_exit	Requiere ejecución de funciones al salir del programa.
qsort	Ordena un arreglo.
rand	Número pseudoaleatorio.
rand48	Números pseudoaleatorio uniformemente distribuidos.
strtod	Conversión <i>string</i> a <i>double</i> o <i>float</i> .
strtol	Conversión <i>string</i> a <i>long</i> .
strtoll	Conversión <i>string</i> a <i>long long</i> .
strtoul	Conversión <i>string</i> a <i>unsigned long</i> .
strtoull	Conversión <i>string</i> a <i>unsigned long long</i> .
wcsrtombs	Conversión <i>wide-character string</i> a <i>character string</i> .
wctod	Conversión <i>wide string</i> a <i>double</i> o <i>float</i> .
wcstol	Conversión <i>wide string</i> a <i>long</i> .
wcstoll	Conversión <i>wide string</i> a <i>long long</i> .
wcstoul	Conversión <i>wide string</i> a <i>unsigned long</i> .
wcstoull	Conversión <i>wide string</i> a <i>unsigned long long</i> .
system	Ejecutar un comando.
wcstombs	Conversión <i>wide string</i> a <i>multibyte string</i> .
wctomb	Conversión <i>wide-character</i> a <i>multibyte</i> .

MACROS Y FUNCIONES PARA CARACTERES (CTYPE.H)

Esta biblioteca agrupa funciones y macros para la manipulación de caracteres y *strings*. El archivo **ctype.h** contiene el encabezado de todas las funciones y macros definidas en esta biblioteca.

Función	Descripción
isalnum	Predicado alfanumérico.
isalpha	Predicado alfabético.
isascii	Predicado carácter ASCII.
iscntrl	Predicado de carácter de control.
isdigit	Predicado de dígito decimal.
islower	Predicado de carácter en minúscula.
isprint	Predicado de carácter imprimible.
ispunct	Predicado de carácter de puntuación.
isspace	Predicado de carácter de espacio en blanco.
isupper	Predicado de carácter en mayúscula.

isxdigit	Predicado de dígito hexadecimal.
toascii	Fuerza enteros al rango ASCII.
tolower	Convierte a minúsculas.
toupper	Convierte a mayúsculas.
iswalnum	Predicado alfanumérico de <i>wide-character</i> .
iswalpha	Predicado alfabético de <i>wide-character</i> .
iswblank	Predicado en blanco de <i>wide-character</i> .
iswcntrl	Predicado de control de <i>wide-character</i> .
iswdigit	Predicado decimal de <i>wide-character</i> .
iswgraph	Predicado de símbolo gráfico en <i>wide-character</i> .
iswlower	Predicado minúscula de <i>wide-character</i> .
iswprint	Predicado imprimible en <i>wide-character</i> .
iswpunct	Predicado de puntuación en <i>wide-character</i> .
iswspace	Predicado de espacio en blanco en <i>wide-character</i> .
iswupper	Predicado mayúscula en <i>wide-character</i> .
iswxdigit	Predicado hexadecimal en <i>wide-character</i> .
iswctype	Verificar extensión de <i>wide-character</i> .
wctype	Verificar tipo de <i>wide-character</i> .
tolower	Convertir <i>wide-characters</i> a minúsculas.
toupper	Convertir <i>wide-characters</i> a mayúsculas.
towctrans	Convierte <i>wide-character</i> .
wctrans	Transforma tipo <i>wide-character</i> .

FUNCIONES DE ENTRADA/SALIDA (STDIO.H)

La biblioteca *stdio* agrupa todas las funciones de entrada y salida. Entre estas funciones existen las correspondientes para la generación o lectura de cadenas de caracteres desde los dispositivos de entrada. Estas cadenas de caracteres pueden provenir de un dispositivo que los envía individual y consecutivamente en lo que se define como *stream*. Los dispositivos globales que utiliza esta librería son:

- **stdin**: dispositivo estándar de entrada.
- **stdout**: dispositivo estándar de salida.
- **stderr**: dispositivo estándar para mensajes de error.

El archivo **stdio.h** contiene los encabezados de todas las funciones y macros que define esta biblioteca:

Función	Descripción
clearerr	Borrar indicador de error en archivo o <i>stream</i> .
diprintf	Imprimir a un descriptor de archivo (solamente entero).
dprintf	Imprimir a un descriptor de archivo.
fclose	Cerrar un archivo.
fcloseall	Cerrar todos los archivos.
fdopen	Cambiar un archivo abierto a <i>stream</i> .

feof	Verificar si es el final del archivo.
ferror	Comprobar si existe error de lectura o escritura.
fflush	Borrar el <i>buffer</i> de salida a archivo.
fgetc	Obtener un carácter desde archivo o <i>stream</i> .
fgetpos	Guardar la posición en un <i>stream</i> o archivo.
fgets	Obtener un <i>string</i> desde <i>stream</i> o archivo.
fgetwc	Obtener un <i>wide-character</i> desde <i>stream</i> o archivo.
fgetws	Obtener un <i>string</i> de <i>wide-character</i> <i>string</i> desde <i>stream</i> o archivo.
fileno	Obtener el descriptor de archivo asociado a un <i>stream</i> .
fmemopen	Abrir un <i>stream</i> con un <i>buffer</i> de longitud fija.
fopen	Abrir un archivo.
fopencookie	Abrir un <i>stream</i> con custom callbacks.
fpurge	Descartar todas las entradas/salidas pendientes en un <i>stream</i> .
fputc	Escribir en carácter a <i>stream</i> o archivo.
fputs	Escribir un <i>string</i> en a archivo o <i>string</i> .
fputwc	Escribir un <i>wide-character</i> a un archivo o <i>stream</i> .
fputws	Escribir un <i>string</i> <i>wide-character</i> a un archivo o <i>stream</i> .
fread	Leer un arreglo de elementos desde un archivo.
freopen	Abrir un archivo utilizando un descriptor existente.
fseek	Determinar la posición en un archivo.
fsetpos	Recomponer la posición de un <i>stream</i> o archivo.
ftell	Retorna la posición en un <i>stream</i> o <i>file</i> .
funopen	Abre un <i>stream</i> con custom callbacks.
fwide	Configura y determina la orientación de un <i>FILE stream</i> .
fwrite	Escribe elementos de arreglo desde memoria a un archivo o <i>stream</i> .
getc	Obtiene un carácter desde un archivo o <i>stream</i> (macro).
getc_unlocked	Obtiene un carácter desde un archivo o <i>stream</i> (macro).
getchar	Obtener una carácter de la entrada estándar (macro).
getchar_unlocked	Obtener un carácter de la entrada estándar (macro).
getdelim	Obtener un <i>string</i> desde un archivo o <i>stream</i> .
getline	Obtener un <i>string</i> desde un archivo o <i>stream</i> .
gets	Obtener un <i>string</i> desde la entrada estándar (obsoleto).
getw	Obtener un <i>word</i> (int) desde un archivo o <i>stream</i> .
getwchar	Obtener un <i>wide-character</i> desde la entrada estándar.
mktemp	Generara un nombre de archivo sin usar.
open_memstream	Abrir un archivo de escritura con una longitud de <i>buffer</i> arbitrario.
perror	Imprimir un mensaje de error en el dispositivo estándar de error.
putc	Escribir un carácter en un <i>stream</i> o archivo (macro).
putc_unlocked	Escribir un carácter en un <i>stream</i> o archivo (macro).
putchar	Escribir un carácter en la salida estándar (macro).
putchar_unlocked	Escribir un carácter en la salida estándar (macro).
puts	Escribir un <i>string</i> en la salida estándar.
putw	Escribir un <i>word</i> (int) a un archivo o <i>stream</i> .
putwchar	Escribir un <i>wide-character</i> a la salida estándar.
remove	Elimina un nombre de archivo.
rename	Renombra un archivo.

rewind	Reinicializa un archivo o <i>stream</i> .
setbuf	Especifica un <i>buffer</i> total para un archivo o <i>stream</i> .
setbuffer	Especifica un <i>buffer</i> total para un archivo o <i>stream</i> dimensionado.
setlinebuf	Especifica un <i>buffer</i> de línea para archivo o <i>stream</i> .
setvbuf	Especifica un <i>buffer</i> para archivo o <i>stream</i> .
siprintf	Escribe una salida con formato (solamente entero).
siscanf	Solicita y formatea una entrada (solamente entero).
sprintf	Escribe una salida con formato.
sscanf	Solicita y formatea una entrada.
swprintf	Escribe una salida <i>wide-character</i> con formato.
swscanf	Solicita y formatea una entrada <i>wide-character</i> .
tmpfile	Crea un archivo temporario.
tmpnam	Genera un nombre para un archivo temporario.
ungetc	Pone de vuelta el dato en un <i>stream</i> .
ungetwc	Pone de vuelta un <i>wide-character</i> en un <i>stream</i> .
vfprintf	Formatea una lista de argumentos variable.
vfscanf	Solicita una lista de argumentos variable.
vwprintf	Formatea una lista de argumentos <i>wide-character</i> variable.
vwscanf	Solicita y formatea una lista de argumentos desde una entrada <i>wide-character</i> .
viprintf	Formatea una lista de argumentos variable (solamente entero).
viscanf	Solicita una lista de formato variable (solamente entero).

MANIPULACIÓN DE MEMORIA Y STRING (STRING.H)

La biblioteca *string* ofrece funciones para la manipulación de áreas de memoria y *string*. Las encabezados de estas funciones se encuentran en el archivo **string.h**.

Función	Descripción
bcmp	Compara dos áreas de memoria.
bcopy	Copia regiones de memoria.
bzero	Inicializa memoria a cero.
index	Busca un carácter en un <i>string</i> .
memccpy	Copia regiones de memoria hasta un <i>token</i> final.
memchr	Encuentra un carácter en memoria.
memcmp	Compara dos áreas de memoria.
memcpy	Copia regiones de memoria.
memmem	Encuentra segmentos de memoria.
memmove	Mueve memoria posiblemente solapada.
mempcpy	Copia regiones de memoria.
memset	Configura un área de memoria.
rindex	Búsqueda en reversa de <i>character</i> en <i>string</i> .
stpcpy	Copia un <i>string</i> y retorna un puntero a su final.
stpncpy	Copia <i>string</i> retornando puntero a su final.
strcasecmp	Compara <i>string</i> ignorando mayúsculas o minúsculas.

strcasestr	Encuentra un segmento de <i>string</i> ignorando mayúsculas o minúsculas.
strcat	Concatena <i>string</i> .
strchr	Busca carácter en <i>string</i> .
strcmp	Compara <i>string</i> .
strcoll	Compara <i>string</i> según ubicación.
strcpy	Copia <i>string</i> .
strcspn	Cuenta caracteres en <i>string</i> .
strerror	Convierte número de error en <i>string</i> .
strerror_r	Convierte número de error en <i>string</i> .
strlen	Retorna longitud de <i>string</i> .
strlwr	Convierte <i>string</i> a minúsculas.
strncasemp	Compara <i>string</i> ignorando minúsculas y mayúsculas.
strncat	Concatena <i>string</i> .
strncmp	Compara <i>string</i> .
strncpy	Copia <i>string</i> .
strnlen	Retorna longitud de <i>string</i> .
strpbrk	Encuentra caracteres en <i>string</i> .
strrchr	Búsqueda reversa de carácter en <i>string</i> .
strsignal	Retorna <i>string</i> de mensaje de una señal.
strspn	Encuentra la coincidencia inicial.
strstr	Encuentra un segmento de <i>string</i> .
strtok	Obtiene el siguiente <i>token</i> en un <i>string</i> .
strupr	Convierte <i>string</i> a mayúscula.
strxfrm	Transforma <i>string</i> .
swab	Intercambia <i>bytes</i> adyacentes.
wscasemp	Compara <i>string</i> de wide-character strings.
wcsdup	Duplica <i>string</i> de wide-character.
wcsncasemp	Compara <i>string</i> de wide-character strings.

STRING DEL TIPO *WIDE CHARACTER* (WCHAR.H)

La biblioteca *wchar* agrupa funciones para la manipulación de datos del tipo *wide-character*. Debido a su rara utilización en sistemas embebidos, no se detallarán en este libro las funciones definidas en la biblioteca **wchar.h**.

MANEJO DE SEÑALES: SIGNAL HANDLING (SIGNAL.H)

Una señal es un evento que interrumpe el flujo normal de control de un programa. La biblioteca *signal* define un conjunto de señales que la mayoría de los sistemas soporta, como ser:

Señal	Descripción
SIGABRT	Terminación anormal de un programa, Producida por la function <<abort>>.
SIGFPE	Error aritmético, tal como desborde ("overflow") o división por cero.

SIGILL	Intento de ejecución de dato no ejecutable.
SIGINT	Interrupción: señal de atención interactiva.
SIGSEGV	Intento de acceso a memoria no permitido.
SIGTERM	Requerimiento de finalización del programa.

Existen dos funciones para el manejo asincrónico de las señales. La primera permite que el programa produzca (*raising*) una señal y la segunda la especificación de una función para el manejo de la señal cuando esta se produzca (*handler*).

Para el soporte de estas funciones, la biblioteca *signal.h* define tres macros:

Macro	Descripción
SIG_DFL	Utilizada para seleccionar el manejo por defecto del ambiente operativo de la señal.
SIG_IGN	Utilizada para ignorar la señal.
SIG_ERR	Retornada cuando el manejador configurado no pudo procesar la señal por algún motivo.

La biblioteca *signal.h* define las siguientes funciones asociadas con señales:

Función	Descripción
psignal	Imprime un mensaje de la señal a la consola de error estándar (stderr).
raise	Envía una señal.
signal	Especifica una subrutina para el manejo de una señal.

FUNCIONES DE TIEMPO (TIME.H)

La biblioteca *time.h* define funciones para el reporte del tiempo, transcurrido, actual y de cómputo, o para realizar cálculos basados en tiempo. Define las siguientes funciones:

Función	Descripción
asctime	Formatear tiempo como <i>string</i> .
clock	Tiempo acumulado de procesador.
ctime	Convertir tiempo en <i>string</i> con formato de la zona local.
difftime	Restar dos tiempos.
gmtime	Convertir tiempo a representación tradicional UTC (GMT).
localtime	Convertir tiempo a representación local.
mktime	Convertir tiempo a representación aritmética.
strftime	Convertir día y hora a <i>string</i> .
time	Obtener la fecha actual.
__tz_lock	Apropiarse las variables temporales de la región.
tzset	Configurar la información de la región.

CONFIGURACIÓN LOCAL (LOCALE.H)

La biblioteca *locale* agrupa definiciones para la configuración de las características locales de representación tales como: separador decimal y símbolo de moneda, principalmente. La función incluida en la biblioteca *locale.h* es:

Función	Descripción
setlocale	Selecciona o consulta sobre la configuración local

REENTRANCIA

Reentrancia es una característica de las funciones de las bibliotecas que permite que múltiples procesos utilicen las mismas funciones sin que esto corrompa el estado de ejecución de ninguno de ellos. La biblioteca *newlib* intenta asegurar que las funciones permitan reentrancia siempre que sea posible. Sin embargo, no todas las funciones permiten ser trivialmente implementadas con esta característica

La biblioteca *reent.h* define versiones reentrantes de funciones que se denominan *hooks*. Es por esto que existen varias funciones que presentan dos versiones: la primera es la versión definida con su nombre original y que no soporta reentrancia, mientras que la segunda es una versión que incluye esta característica y que posee el mismo nombre precedido por el símbolo “_” y finalizadas con “_r”. Por ejemplo: *_fopen_r* que permite abrir un archivo con reentrancia.

LLAMADAS AL SISTEMA (SYSTEM CALLS)

Las rutinas del lenguaje C dependen de un grupo de funciones que son implementadas por servicios del sistema operativo. En el caso de que la plataforma no cuente con un sistema operativo (como es el caso cuando se utiliza la HAL de Altera), deben existir funciones que replacen a los servicios para que las correspondiente llamadas no produzcan fallas en la ejecución del programa.

DEFINICIONES PARA LA INTERFAZ CON EL SISTEMA OPERATIVO (STUBS)

La especificación de las funciones de *newlib* puede requerir interacción con el sistema operativo. De esta manera, existen funciones que brindan una interfaz con el sistema operativo. En caso de que no exista un sistema operativo, entonces están funciones debieran “fallar” controladamente. La manera habitual de implementación de esta falla es mediante el retorno de un código de error, a través de la estructura global de error definida en la biblioteca *error.h* soportando reentrancia de las funciones.

Las funciones que implementan esta interfaz con el sistema operativo son denominadas funciones **stub**. La HAL de Altera define estas funciones para compatibilizar con la biblioteca *newlib*.

Las funciones **stub** definidas en la biblioteca newlib son:

Función	Descripción
_exit	Sale de un programa sin limpiar los archivos.
close	Cierra un archivo.
environ	Un puntero a la lista de argumentos con que fue ejecutado el programa.
execve	Transfiere el control a un proceso nuevo.
fork	Crea un proceso nuevo.
fstat	Estado de un archivo abierto.
getpid	Obtiene el Identificador de un proceso.
isatty	Consulta si un <i>stream</i> es una consola de terminal.
kill	Envía una señal.
link	Establece un nuevo nombre para un archivo existente.
lseek	Establece la posición en un archivo.
open	Abre un archivo.
read	Lee un archivo.
sbrk	Incrementa el espacio de datos de un programa.
stat	Estado de un archivo (por nombre).
times	Información de temporizado del proceso actual.
unlink	Elimina la entrada de un enlace de archivo.
wait	Espera por un proceso <i>child</i> .
write	Escribe a un archivo.